PyQt5: Dari Kepingan Menuju Karya

Muhammad Rizqi Sholahuddin

February 22, 2026

Contents

1 Pendahuluan **9**

2 Kepingan Pertama: Jendela Kosong **11**
2.1 Kode Program 11
2.2 Penjelasan Kode 11
2.3 Hasil Tampilan 12

3 Menambah Label: Jendela yang Bicara **13**
3.1 Kode Program 13
3.2 Penjelasan Kode 13
3.3 Hasil Tampilan 14

4 Tombol Interaktif **15**
4.1 Kode Program 15
4.2 Penjelasan Kode 16
4.3 Hasil Tampilan 16

5 Layout Manager: Menyusun Kepingan **17**
5.1 Kode Program 17
5.2 Penjelasan Kode 18
5.3 Hasil Tampilan 18

6 Input Pengguna: Formulir Sederhana **21**
6.1 Kode Program 21
6.2 Penjelasan Kode 22
6.3 Hasil Tampilan 23

7 Signal & Slot: Menghubungkan Kepingan **25**
7.1 Kode Program 25
7.2 Penjelasan Kode 26
7.3 Hasil Tampilan 27

8 Widget Keren **29**
8.1 Kode Program 29
8.2 Penjelasan Kode 31
8.3 Hasil Tampilan 31

9 Menu & Toolbar **33**
9.1 Kode Program 33
9.2 Penjelasan Kode 35
9.3 Hasil Tampilan 35

10 Styling: Mewarnai Karya **37**
10.1 Kode Program 37
10.2 Penjelasan Kode 39
10.3 Hasil Tampilan 40

11 Proyek Mini: Todo App **41**
11.1 Kode Program . . . 41
11.2 Penjelasan Kode . . . 45
11.3 Hasil Tampilan . . . 46

12 Input Angka: Spin dan Dial **47**
12.1 Kode Program . . . 47
12.2 Penjelasan Kode . . . 48
12.3 Hasil Tampilan . . . 48

13 Pilihan Tunggal: Radio Button **51**
13.1 Kode Program . . . 51
13.2 Penjelasan Kode . . . 52
13.3 Hasil Tampilan . . . 53

14 Pilihan Ganda: Checkbox **55**
14.1 Kode Program . . . 55
14.2 Penjelasan Kode . . . 56
14.3 Hasil Tampilan . . . 57

15 Dropdown: ComboBox **59**
15.1 Kode Program . . . 59
15.2 Penjelasan Kode . . . 60
15.3 Hasil Tampilan . . . 61

16 Tanggal dan Waktu **63**
16.1 Kode Program . . . 63
16.2 Penjelasan Kode . . . 64
16.3 Hasil Tampilan . . . 65

17 Daftar: ListWidget **67**
17.1 Kode Program . . . 67
17.2 Penjelasan Kode . . . 69
17.3 Hasil Tampilan . . . 69

18 Tabel: TableWidget **71**
18.1 Kode Program . . . 71
18.2 Penjelasan Kode . . . 74
18.3 Hasil Tampilan . . . 74

19 Tab Widget **75**
19.1 Kode Program . . . 75
19.2 Penjelasan Kode . . . 78
19.3 Hasil Tampilan . . . 78

20 Scroll Area **81**
20.1 Kode Program . . . 81
20.2 Penjelasan Kode . . . 82
20.3 Hasil Tampilan . . . 83

21 Splitter **85**
21.1 Kode Program . . . 85
21.2 Penjelasan Kode . . . 87
21.3 Hasil Tampilan . . . 87

22 Dialog Standar **89**
22.1 Kode Program . . . 89
22.2 Penjelasan Kode . . . 91
22.3 Hasil Tampilan . . . 91

23 Timer **93**
23.1 Kode Program . . . 93
23.2 Penjelasan Kode . . . 95
23.3 Hasil Tampilan . . . 95

24 Settings: Menyimpan Preferensi **97**
24.1 Kode Program . . . 97
24.2 Penjelasan Kode . . . 99
24.3 Hasil Tampilan . . . 100

25 Clipboard **101**
25.1 Kode Program . . . 101
25.2 Penjelasan Kode . . . 103
25.3 Hasil Tampilan . . . 103

26 Proyek Mini: Kalkulator **105**
26.1 Kode Program . . . 105
26.2 Penjelasan Kode . . . 108
26.3 Hasil Tampilan . . . 108

27 Threading Dasar **111**
27.1 Kode Program . . . 111
27.2 Penjelasan Kode . . . 113
27.3 Hasil Tampilan . . . 114

28 Mouse Events **115**
28.1 Kode Program . . . 115
28.2 Penjelasan Kode . . . 117
28.3 Hasil Tampilan . . . 117

29 Keyboard Events **119**
29.1 Kode Program . . . 119
29.2 Penjelasan Kode . . . 122
29.3 Hasil Tampilan . . . 122

30 Drag and Drop **125**
30.1 Kode Program . . . 125
30.2 Penjelasan Kode . . . 128
30.3 Hasil Tampilan . . . 129

31 TreeWidget **131**
31.1 Kode Program . . . 131
31.2 Penjelasan Kode . . . 134
31.3 Hasil Tampilan . . . 134

32 StackedWidget **135**
32.1 Kode Program . . . 135
32.2 Penjelasan Kode . . . 139
32.3 Hasil Tampilan . . . 139

33 Graphics View **141**
33.1 Kode Program . . . 141
33.2 Penjelasan Kode . . . 144
33.3 Hasil Tampilan . . . 144

34 Context Menu **147**
34.1 Kode Program . . . 147
34.2 Penjelasan Kode . . . 150
34.3 Hasil Tampilan . . . 150

35 System Tray **151**
35.1 Kode Program . . . 151
35.2 Penjelasan Kode . . . 153
35.3 Hasil Tampilan . . . 154

36 Custom Widget **155**
36.1 Kode Program . . . 155
36.2 Penjelasan Kode . . . 160
36.3 Hasil Tampilan . . . 160

37 Form Layout **163**
37.1 Kode Program . . . 163
37.2 Penjelasan Kode . . . 166
37.3 Hasil Tampilan . . . 166

38 Validator **169**
38.1 Kode Program . . . 169
38.2 Penjelasan Kode . . . 173
38.3 Hasil Tampilan . . . 173

39 Splash Screen **175**
39.1 Kode Program . . . 175
39.2 Penjelasan Kode . . . 178
39.3 Hasil Tampilan . . . 178

40 Status Bar **181**
40.1 Kode Program . . . 181
40.2 Penjelasan Kode . . . 184
40.3 Hasil Tampilan . . . 184

41 Notes App **187**
41.1 Kode Program . . . 187
41.2 Penjelasan Kode . . . 193
41.3 Hasil Tampilan . . . 194

42 Shortcut Keys **195**
42.1 Kode Program . . . 195
42.2 Penjelasan Kode . . . 198
42.3 Hasil Tampilan . . . 198

43 Internationalization (i18n) **201**
43.1 Kode Program . . . 201
43.2 Penjelasan Kode . . . 204
43.3 Hasil Tampilan . . . 205

44 Database SQLite **207**
44.1 Kode Program . . . 207
44.2 Penjelasan Kode . . . 213
44.3 Hasil Tampilan . . . 213

45 Network Request **215**
45.1 Kode Program . . . 215
45.2 Penjelasan Kode . . . 218
45.3 Hasil Tampilan . . . 218

46 Animation **221**
46.1 Kode Program . . . 221
46.2 Penjelasan Kode . . . 226
46.3 Hasil Tampilan . . . 226

47 Proyek Akhir: Address Book **229**
47.1 Kode Program . . . 229
47.2 Penjelasan Kode . . . 237
47.3 Hasil Tampilan . . . 237

48 Proyek Akhir: File Manager **239**
48.1 Kode Program . . . 239
48.2 Penjelasan Kode . . . 246
48.3 Hasil Tampilan . . . 246

49 Proyek Akhir: Unit Converter **247**
49.1 Kode Program . . . 247
49.2 Penjelasan Kode . . . 253
49.3 Hasil Tampilan . . . 253

50 Proyek Akhir: Pomodoro Timer **255**
50.1 Kode Program . . . 255
50.2 Penjelasan Kode . . . 263
50.3 Hasil Tampilan . . . 263

Chapter 1

Pendahuluan

Modul ini berisi **50 bab** yang akan memandu Anda membangun aplikasi GUI menggunakan PyQt5 secara bertahap.

Struktur Tiap Bab

1. **Kode Program** – potongan skrip Python siap dijalankan.
2. **Penjelasan Kode** – ulasan singkat struktur dan alur program.
3. **Hasil Tampilan** – screenshot GUI agar Anda tahu ekspektasi output.

Peta Perjalanan

1. Bab 2–10 : Widget dasar, layout, styling.
2. Bab 11–20 : Widget lanjutan dan mini–proyek.
3. Bab 21–40 : Fitur menengah (dialog, i18n, tray, dll.).
4. Bab 41–50 : Proyek akhir berskala kecil hingga menengah.

Nikmati proses belajar—mulai dari *jendela kosong* hingga aplikasi lengkap!

Chapter 2

Kepingan Pertama: Jendela Kosong

2.1 Kode Program

```python
# BAB 2: Kepingan Pertama - Jendela Kosong
# ================================
# Ini adalah baseplate kita - fondasi untuk semua aplikasi PyQt5

import sys
from PyQt5.QtWidgets import QApplication, QMainWindow

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Jendela Pertamaku - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 300) # x, y, lebar, tinggi

# Baris-baris ini SELALU ada di setiap aplikasi PyQt5
app = QApplication(sys.argv) # Membuat aplikasi
jendela = JendelaKu()      # Membuat jendela
jendela.show()             # Menampilkan jendela
sys.exit(app.exec_())      # Memulai event loop
```

2.2 Penjelasan Kode

Setiap aplikasi GUI dimulai dari satu hal: jendela kosong. Meski tampak sederhana, skrip 19 baris ini sudah memuat seluruh kerangka dasar PyQt5.

1. QApplication: Mesin Penggerak

`QApplication(sys.argv)` adalah objek pertama yang *wajib* dibuat. Ia menginisialisasi seluruh infrastruktur Qt—event loop, font system, clipboard, dan koneksi ke window manager OS. Tanpa baris ini, tidak ada widget yang bisa hidup.

2. Kelas JendelaKu Mewarisi QMainWindow

Dengan mewarisi `QMainWindow`, kelas kita langsung mendapat fitur bawaan: title bar, kemampuan menampung menu bar, toolbar, status bar, dan central widget. `super().__init__()` memanggil konstruktor induk agar semua fitur itu aktif.

3. Mengatur Identitas dan Ukuran

`setWindowTitle()` memberi nama pada title bar. `setGeometry(100, 100, 400, 300)` menentukan posisi (x=100, y=100 dari pojok kiri atas layar) dan ukuran (lebar 400, tinggi 300 piksel).

4. Menampilkan dan Menjalankan

`jendela.show()` meminta OS menggambar jendela di layar. `app.exec_()` memulai *event loop*—sebuah loop tak terbatas yang menunggu dan mendistribusikan event (klik, ketik, resize) ke widget yang tepat. Aplikasi baru berhenti ketika jendela ditutup.

Pelajaran Penting

• Pola empat langkah ini—buat `QApplication`, buat jendela, `show()`, `exec_()`—akan muncul di *setiap* program PyQt5.
• `QMainWindow` adalah pilihan utama untuk aplikasi dengan menu dan toolbar; untuk jendela sederhana, `QWidget` juga cukup.
• Event loop adalah jantung aplikasi GUI—tanpanya, jendela muncul lalu langsung menghilang.

2.3 Hasil Tampilan

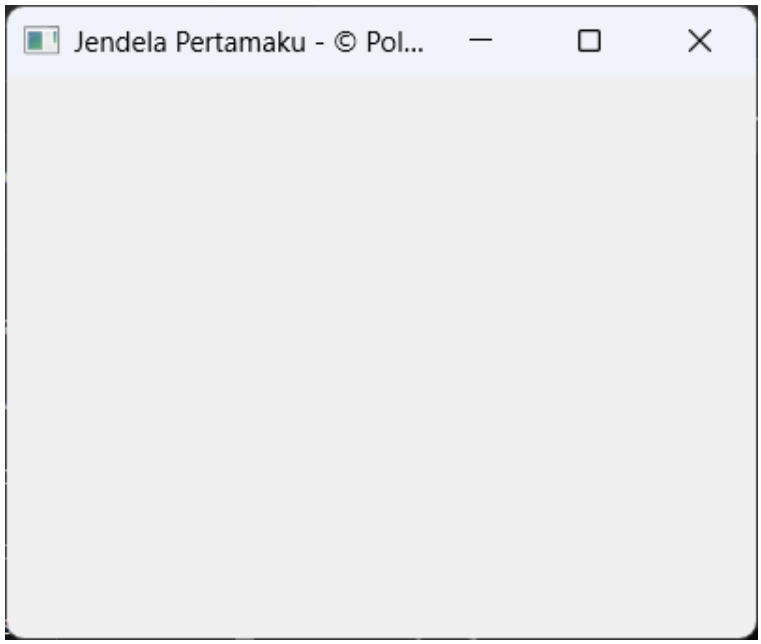

Figure 2.1: Jendela Kosong

Chapter 3

Menambah Label: Jendela yang Bicara

3.1 Kode Program

```python
# BAB 3: Menambah Label - Jendela yang Bicara
# ==========================================

import sys
from PyQt5.QtWidgets import QApplication, QMainWindow, QLabel
from PyQt5.QtGui import QFont
from PyQt5.QtCore import Qt

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Label Kustom - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 300)

        # Membuat label dengan styling
        label = QLabel("Halo, PyQt5!")
        label.setAlignment(Qt.AlignCenter) # Teks di tengah
        label.setFont(QFont("Arial", 24, QFont.Bold))
        label.setStyleSheet("color: blue; background-color: #f0f0f0;")

        # Set sebagai widget utama
        self.setCentralWidget(label)

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
sys.exit(app.exec_())
```

3.2 Penjelasan Kode

Jendela kosong belum berguna—ia perlu "berbicara". Widget pertama yang kita tambahkan adalah `QLabel`: kotak teks statis yang menampilkan informasi kepada pengguna.

1. Membuat dan Mengisi Label

`QLabel("Halo, PyQt5!")` membuat label dengan teks awal. Satu baris, langsung terlihat—tidak perlu konfigurasi rumit.

2. Perataan Teks

`setAlignment(Qt.AlignCenter)` menempatkan teks tepat di tengah widget, baik horizontal maupun vertikal. Konstanta `Qt.AlignCenter` adalah gabungan `AlignHCenter | AlignVCenter`.

3. Font dan Styling

`QFont("Arial", 24, QFont.Bold)` mengatur jenis huruf, ukuran, dan ketebalan. `setStyleSheet("color: blue; background-color: #f0f0f0;")` menambahkan warna teks dan latar—sintaks CSS yang sudah familiar bagi pengembang web.

4. Central Widget

`setCentralWidget(label)` menjadikan label sebagai widget utama jendela. `QMainWindow` hanya boleh punya satu central widget; ia akan mengisi seluruh area konten dan menyesuaikan ukuran saat jendela di-resize.

Pelajaran Penting

- `QLabel` adalah widget paling dasar—gunakan untuk menampilkan teks, gambar, bahkan HTML sederhana.
- `setStyleSheet()` membawa kekuatan CSS ke dunia desktop; hampir semua properti visual bisa diatur.
- `setCentralWidget()` memastikan widget mengisi area utama `QMainWindow` secara otomatis.

3.3 Hasil Tampilan

Figure 3.1: Label pada Jendela

Chapter 4

Tombol Interaktif

4.1 Kode Program

```python
# BAB 4: Tombol Interaktif
# ========================

import sys
from PyQt5.QtWidgets import QApplication, QMainWindow, QPushButton, QMessageBox

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Tombol dengan Aksi - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 300)

        # Tombol 1 - Sapa
        tombol1 = QPushButton("Klik Aku!", self)
        tombol1.move(150, 80)
        tombol1.resize(100, 40)
        tombol1.clicked.connect(self.tombol_diklik)

        # Tombol 2 - Info
        tombol2 = QPushButton("Info", self)
        tombol2.move(150, 130)
        tombol2.resize(100, 40)
        tombol2.clicked.connect(self.tampilkan_info)

        # Tombol 3 - Keluar
        tombol3 = QPushButton("Keluar", self)
        tombol3.move(150, 180)
        tombol3.resize(100, 40)
        tombol3.setStyleSheet("background-color: #e74c3c; color: white;")
        tombol3.clicked.connect(self.close)

    def tombol_diklik(self):
        QMessageBox.information(self, "Info", "Tombol berhasil diklik!")

    def tampilkan_info(self):
        QMessageBox.about(self, "Tentang", "Aplikasi Demo PyQt5\nVersi 1.0")

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
sys.exit(app.exec_())
```

4.2 Penjelasan Kode

Label hanya menampilkan—tombol memungkinkan pengguna *bertindak*. Bab ini memperkenalkan `QPushButton` dan konsep paling penting di Qt: **signal & slot**.

1. Tiga Tombol, Tiga Aksi

Skrip membuat tiga `QPushButton` yang diposisikan manual menggunakan `move()` dan `resize()`:

- **"Klik Aku!"**: menampilkan `QMessageBox.information`—dialog pesan sederhana.
- **"Info"**: menampilkan `QMessageBox.about`—dialog "Tentang Aplikasi".
- **"Keluar"**: di-style merah dengan CSS dan memanggil `self.close()` untuk menutup jendela.

2. Signal & Slot: Kabel Tak Terlihat

Setiap `QPushButton` memancarkan signal `clicked` saat ditekan. Dengan `.connect()`, signal itu dihubungkan ke sebuah *slot* (metode Python biasa). Contoh: `tombol1.clicked.connect(self.tombol_diklik)`. Qt mengurus perutean event—Anda cukup mendefinisikan "apa yang terjadi".

3. QMessageBox: Dialog Bawaan

`QMessageBox` menyediakan dialog standar (information, warning, question, about) tanpa perlu mendesain dari nol. Cukup panggil metode statis dengan judul dan pesan.

Pelajaran Penting

- Signal & slot menggantikan polling manual—kode lebih bersih dan responsif.
- Posisi manual (`move`/`resize`) berfungsi tapi tidak adaptif; bab berikutnya akan memperkenalkan layout yang lebih baik.
- `QMessageBox` adalah cara tercepat untuk berkomunikasi dengan pengguna.

4.3 Hasil Tampilan

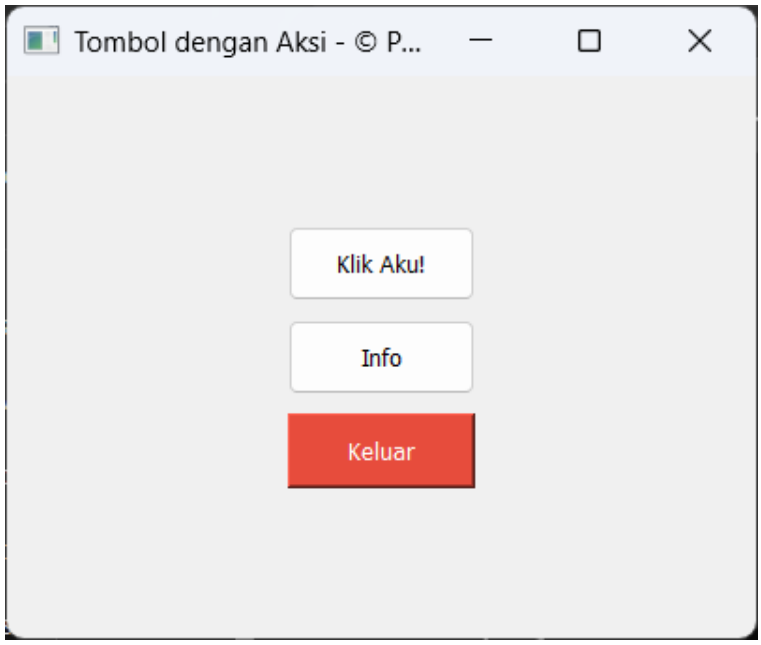

Figure 4.1: Tombol dengan Aksi

Chapter 5

Layout Manager: Menyusun Kepingan

5.1 Kode Program

```python
# BAB 5: Layout Manager - Menyusun Kepingan
# ========================================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QPushButton, QLabel, QVBoxLayout,
                            QHBoxLayout, QGridLayout, QWidget)
from PyQt5.QtCore import Qt

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Layout Campuran - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 350)

        widget = QWidget()
        layout_utama = QVBoxLayout()

        # HEADER
        header = QLabel("📊 DASHBOARD")
        header.setStyleSheet("""
            font-size: 20px;
            font-weight: bold;
            background-color: #3498db;
            color: white;
            padding: 10px;
        """)
        header.setAlignment(Qt.AlignCenter)
        layout_utama.addWidget(header)

        # Tombol horizontal
        layout_tombol = QHBoxLayout()

        btn1 = QPushButton("File")
        btn2 = QPushButton("Edit")
        btn3 = QPushButton("View")

        layout_tombol.addWidget(btn1)
        layout_tombol.addWidget(btn2)
        layout_tombol.addWidget(btn3)

        layout_utama.addLayout(layout_tombol)
```

```
        # Grid untuk konten
        layout_grid = QGridLayout()

        layout_grid.addWidget(QLabel("📈 Chart"), 0, 0)
        layout_grid.addWidget(QLabel("📋 Table"), 0, 1)
        layout_grid.addWidget(QLabel("📝 Notes"), 1, 0)
        layout_grid.addWidget(QLabel("⚙️ Settings"), 1, 1)

        layout_utama.addLayout(layout_grid)

        # FOOTER
        footer = QLabel("Status: Siap")
        footer.setStyleSheet("background-color: #2c3e50; color: white; padding: 5px;")
        layout_utama.addWidget(footer)

        widget.setLayout(layout_utama)
        self.setCentralWidget(widget)

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
sys.exit(app.exec_())
```

5.2 Penjelasan Kode

Di bab sebelumnya, widget diposisikan manual dengan `move()`—rapuh saat jendela di-resize. Layout manager menata widget secara otomatis dan adaptif.

1. Tiga Jenis Layout dalam Satu Skrip

Skrip ini mendemonstrasikan ketiganya sekaligus:

- `QVBoxLayout`: menyusun widget secara vertikal (atas ke bawah) sebagai layout utama.
- `QHBoxLayout`: menyusun tiga tombol (File, Edit, View) secara horizontal.
- `QGridLayout`: menata empat label konten (Chart, Table, Notes, Settings) dalam grid 2×2.

2. Nesting Layout

Layout bisa dimasukkan ke dalam layout lain via `addLayout()`. Struktur skrip: `QVBoxLayout` utama berisi header, lalu `QHBoxLayout` tombol, lalu `QGridLayout` konten, lalu footer. Nesting ini menghasilkan tata letak kompleks dari komponen sederhana.

3. Header dan Footer dengan Styling

Label "DASHBOARD" di-style dengan background biru, teks putih, dan padding. Footer menampilkan status dengan background gelap. CSS-like styling membuat UI tampak profesional tanpa gambar eksternal.

4. QWidget sebagai Kontainer

Karena `QMainWindow` membutuhkan `setCentralWidget()`, layout tidak bisa langsung dipasang. Solusinya: buat `QWidget()`, terapkan layout ke widget itu, lalu jadikan central widget.

Pelajaran Penting

- Selalu gunakan layout manager—jangan posisi manual—agar UI responsif saat di-resize.
- Nesting layout (`addLayout`) memungkinkan desain yang fleksibel dan modular.
- Pola `QWidget` + layout + `setCentralWidget` akan dipakai di hampir semua bab selanjutnya.

5.3 Hasil Tampilan

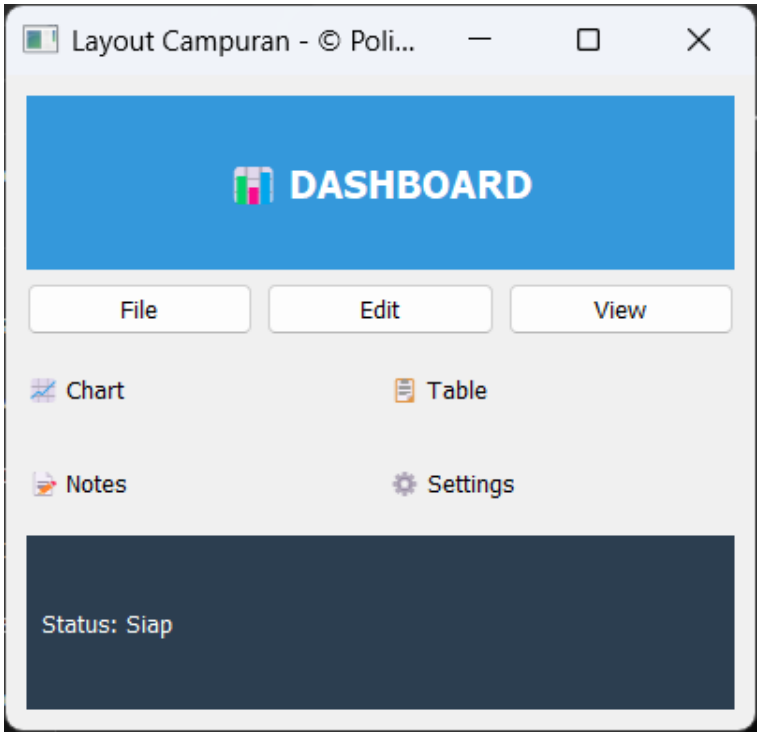

Figure 5.1: Contoh Layout

Chapter 6

Input Pengguna: Formulir Sederhana

6.1 Kode Program

```python
# BAB 6: Input Pengguna - Formulir Sederhana
# ==========================================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLineEdit, QLabel, QPushButton,
                            QVBoxLayout, QWidget, QMessageBox)

class FormRegistrasi(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Form Registrasi - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 350, 300)

        widget = QWidget()
        layout = QVBoxLayout()

        # Judul
        judul = QLabel("📝 FORM REGISTRASI")
        judul.setStyleSheet("font-size: 18px; font-weight: bold; color: #2c3e50;")
        layout.addWidget(judul)

        # Nama
        layout.addWidget(QLabel("Nama:"))
        self.input_nama = QLineEdit()
        self.input_nama.setPlaceholderText("Nama lengkap")
        layout.addWidget(self.input_nama)

        # Email
        layout.addWidget(QLabel("Email:"))
        self.input_email = QLineEdit()
        self.input_email.setPlaceholderText("email@contoh.com")
        layout.addWidget(self.input_email)

        # Telepon
        layout.addWidget(QLabel("Telepon:"))
        self.input_telepon = QLineEdit()
        self.input_telepon.setPlaceholderText("08xxxxxxxxxx")
        layout.addWidget(self.input_telepon)

        # Spacer
        layout.addSpacing(10)
```

```
        # Tombol
        tombol = QPushButton("DAFTAR")
        tombol.setStyleSheet("""
            QPushButton {
                background-color: #27ae60;
                color: white;
                padding: 10px;
                font-size: 14px;
                font-weight: bold;
            }
            QPushButton:hover {
                background-color: #2ecc71;
            }
        """)
        tombol.clicked.connect(self.daftar)
        layout.addWidget(tombol)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def daftar(self):
        nama = self.input_nama.text()
        email = self.input_email.text()
        telepon = self.input_telepon.text()

        if not nama or not email:
            QMessageBox.warning(self, "Peringatan", "Nama dan Email wajib diisi!")
            return

        pesan = f"Pendaftaran berhasil!\n\nNama: {nama}\nEmail: {email}\nTelepon: {telepon}"
        QMessageBox.information(self, "Sukses", pesan)

app = QApplication(sys.argv)
jendela = FormRegistrasi()
jendela.show()
sys.exit(app.exec_())
```

6.2 Penjelasan Kode

Aplikasi yang hanya menampilkan informasi belum lengkap—ia harus bisa *menerima* input. Bab ini membangun form registrasi sederhana dengan tiga field dan satu tombol.

1. QLineEdit: Kotak Input Teks

Tiga `QLineEdit` dibuat untuk nama, email, dan telepon. Masing-masing diberi `setPlaceholderText()` yang menampilkan teks panduan berwarna abu-abu saat field kosong—petunjuk halus tanpa memakan ruang.

2. Validasi Sederhana

Metode `daftar()` membaca isi ketiga field. Jika nama atau email kosong, `QMessageBox.warning()` menampilkan peringatan. Jika valid, `QMessageBox.information()` menampilkan data registrasi. Di aplikasi nyata, titik ini adalah tempat menyimpan ke database atau mengirim ke server.

3. Styling Tombol dengan Pseudo-State

Tombol "DAFTAR" menggunakan stylesheet dengan pseudo-state `:hover`—warna berubah saat kursor melayang. Ini memberikan feedback visual yang membuat UI terasa responsif.

4. Layout Vertikal

Semua widget ditata dalam `QVBoxLayout`: judul, label+input (berselang-seling), spacer, dan tombol. `addSpacing(10)` menambah jarak kosong antar elemen untuk kerapian.

Pelajaran Penting

- `QLineEdit` + `placeholderText` adalah kombinasi standar untuk input teks.

- Validasi di sisi klien mencegah data kosong masuk ke sistem.
- Pola “baca input → validasi → proses” adalah fondasi semua form di aplikasi mana pun.

6.3 Hasil Tampilan

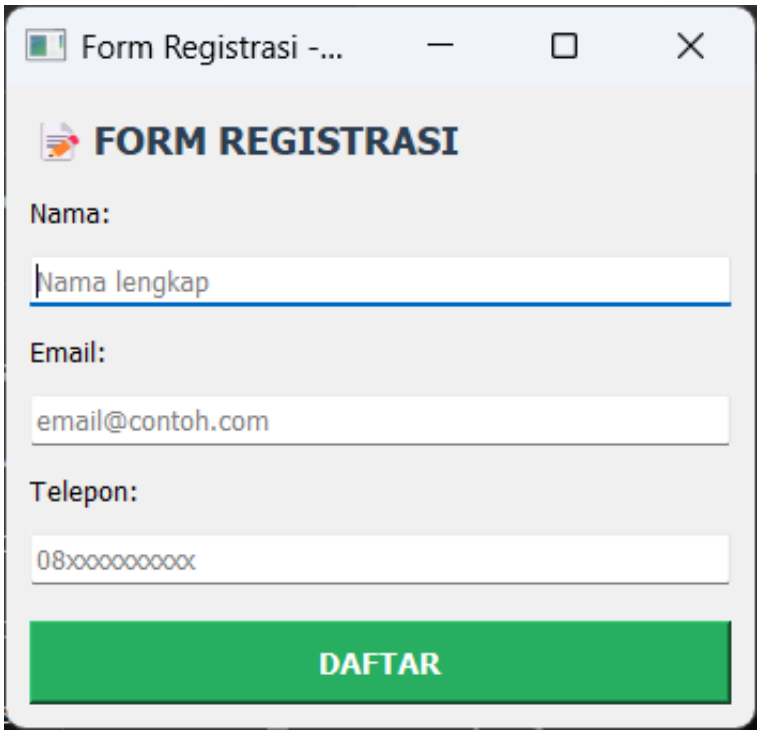

Figure 6.1: Form Input

Chapter 7

Signal & Slot: Menghubungkan Kepingan

7.1 Kode Program

```python
# BAB 7: Signal & Slot - Menghubungkan Kepingan
# ==============================================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLineEdit, QLabel, QPushButton,
                            QVBoxLayout, QWidget)
from PyQt5.QtCore import Qt

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Signal & Slot Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 300)

        widget = QWidget()
        layout = QVBoxLayout()

        # Label instruksi
        instruksi = QLabel("Ketik nama Anda, lalu klik Sapa atau tekan Enter:")
        layout.addWidget(instruksi)

        # Input
        self.input = QLineEdit()
        self.input.setPlaceholderText("Ketik nama Anda...")
        layout.addWidget(self.input)

        # Label untuk menampilkan salam
        self.label_salam = QLabel("Halo, ...")
        self.label_salam.setStyleSheet("font-size: 24px; color: #2c3e50;")
        self.label_salam.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.label_salam)

        # Tombol
        self.tombol = QPushButton("Sapa!")
        self.tombol.setStyleSheet("padding: 15px; font-size: 16px;")
        layout.addWidget(self.tombol)

        # Label status
```

```
        self.status = QLabel("Status: Menunggu...")
        self.status.setStyleSheet("color: gray; font-style: italic;")
        layout.addWidget(self.status)

        # === MENGHUBUNGKAN SIGNAL DENGAN SLOT ===

        # Signal: textChanged -> dipicu setiap kali teks berubah
        self.input.textChanged.connect(self.nama_berubah)

        # Signal: returnPressed -> dipicu ketika Enter ditekan
        self.input.returnPressed.connect(self.sapa)

        # Signal: clicked -> dipicu ketika tombol diklik
        self.tombol.clicked.connect(self.sapa)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def nama_berubah(self, teks):
        """Slot: dipanggil ketika teks berubah"""
        self.status.setText(f"Status: Mengetik '{teks}'")

    def sapa(self):
        """Slot: dipanggil ketika tombol diklik atau Enter ditekan"""
        nama = self.input.text()
        if nama:
            self.label_salam.setText(f"Halo, {nama}! 👋")
            self.status.setText("Status: Sudah disapa!")
        else:
            self.label_salam.setText("Halo, ...")

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
sys.exit(app.exec_())
```

7.2 Penjelasan Kode

Signal & slot adalah mekanisme komunikasi inti Qt—"kabel tak terlihat" yang menghubungkan aksi pengguna ke respons aplikasi. Bab ini mendemonstrasikannya lewat skenario interaktif: ketik nama, lalu lihat salam berubah secara real-time.

1. Tiga Signal, Tiga Perilaku

Skrip menghubungkan tiga signal berbeda:

- `textChanged`: dipancarkan setiap karakter diketik di `QLineEdit`. Slot `nama_berubah()` memperbarui label status secara real-time.
- `returnPressed`: dipancarkan saat pengguna menekan Enter. Slot `sapa()` menampilkan salam.
- `clicked`: dipancarkan saat tombol "Sapa!" ditekan. Dihubungkan ke slot yang sama: `sapa()`.

2. Satu Slot, Banyak Pemicu

Metode `sapa()` dipanggil baik oleh Enter maupun klik tombol—menunjukkan bahwa satu slot bisa melayani banyak signal. Sebaliknya, satu signal juga bisa dihubungkan ke banyak slot.

3. Parameter Signal

`textChanged` mengirimkan teks terkini sebagai parameter ke slot `nama_berubah(self, teks)`. Qt secara otomatis meneruskan data—Anda tidak perlu mengambilnya secara manual.

4. Validasi Ringan

Di `sapa()`, pengecekan `if nama:` memastikan label hanya diperbarui jika input tidak kosong—validasi minimal namun penting.

Pelajaran Penting

- Signal & slot memisahkan *pengirim* dan *penerima*—widget tidak perlu tahu siapa yang mendengarkan.
- Signal bisa membawa data (teks, angka, objek) ke slot secara otomatis.
- Pola ini adalah fondasi arsitektur event-driven di seluruh ekosistem Qt.

7.3 Hasil Tampilan

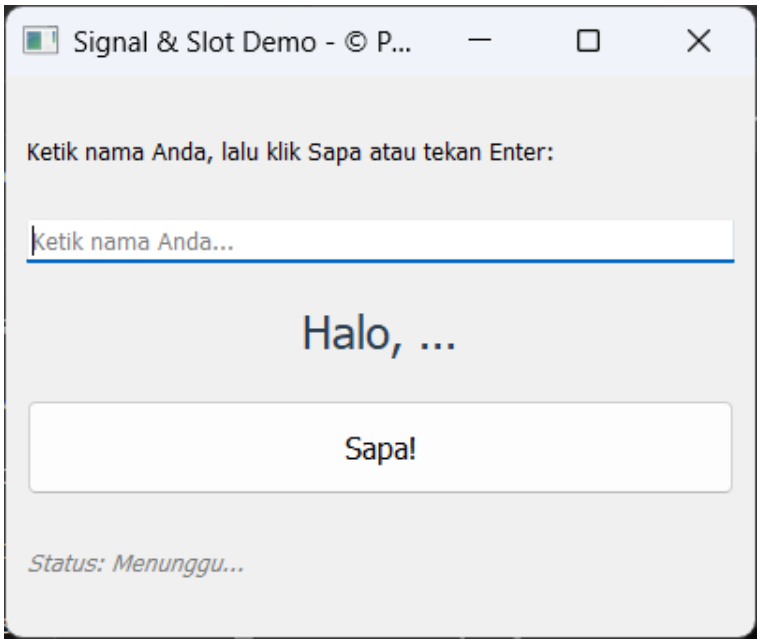

Figure 7.1: Demonstrasi Signal dan Slot

Chapter 8

Widget Keren

8.1 Kode Program

```python
# BAB 8: Widget Keren
# ===================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QComboBox, QSlider, QProgressBar,
                            QCheckBox, QRadioButton, QLabel, QPushButton,
                            QVBoxLayout, QHBoxLayout, QWidget, QGroupBox)
from PyQt5.QtCore import Qt

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Widget Keren - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 450, 500)

        widget = QWidget()
        layout = QVBoxLayout()

        # === COMBOBOX ===
        grup_combo = QGroupBox("Pilih Warna Favorit:")
        layout_combo = QVBoxLayout()

        self.combo = QComboBox()
        self.combo.addItems(["Merah", "Hijau", "Biru", "Kuning", "Ungu"])
        self.combo.currentTextChanged.connect(self.warna_dipilih)
        layout_combo.addWidget(self.combo)

        self.label_warna = QLabel("Warna: -")
        layout_combo.addWidget(self.label_warna)

        grup_combo.setLayout(layout_combo)
        layout.addWidget(grup_combo)

        # === SLIDER & PROGRESS ===
        grup_slider = QGroupBox("Slider & Progress:")
        layout_slider = QVBoxLayout()

        self.slider = QSlider(Qt.Horizontal)
        self.slider.setMinimum(0)
        self.slider.setMaximum(100)
        self.slider.setValue(50)
```

```
        self.slider.valueChanged.connect(self.nilai_berubah)
        layout_slider.addWidget(self.slider)

        self.label_nilai = QLabel("Nilai: 50")
        self.label_nilai.setStyleSheet("font-size: 20px; font-weight: bold;")
        layout_slider.addWidget(self.label_nilai)

        self.progress = QProgressBar()
        self.progress.setValue(50)
        layout_slider.addWidget(self.progress)

        grup_slider.setLayout(layout_slider)
        layout.addWidget(grup_slider)

        # === CHECKBOX & RADIO ===
        grup_pilihan = QGroupBox("Pilihan:")
        layout_pilihan = QHBoxLayout()

        # Checkbox
        layout_cb = QVBoxLayout()
        self.cb1 = QCheckBox("Membaca")
        self.cb2 = QCheckBox("Gaming")
        self.cb3 = QCheckBox("Olahraga")
        layout_cb.addWidget(self.cb1)
        layout_cb.addWidget(self.cb2)
        layout_cb.addWidget(self.cb3)

        # Radio
        layout_rb = QVBoxLayout()
        self.rb1 = QRadioButton("Laki-laki")
        self.rb2 = QRadioButton("Perempuan")
        layout_rb.addWidget(self.rb1)
        layout_rb.addWidget(self.rb2)

        layout_pilihan.addLayout(layout_cb)
        layout_pilihan.addLayout(layout_rb)

        grup_pilihan.setLayout(layout_pilihan)
        layout.addWidget(grup_pilihan)

        # Tombol tampilkan
        tombol = QPushButton("Tampilkan Pilihan")
        tombol.clicked.connect(self.tampilkan)
        layout.addWidget(tombol)

        self.hasil = QLabel("Hasil: -")
        self.hasil.setStyleSheet("font-weight: bold; color: #2c3e50;")
        layout.addWidget(self.hasil)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def warna_dipilih(self, warna):
        warna_map = {
            "Merah": "#e74c3c",
            "Hijau": "#27ae60",
            "Biru": "#3498db",
            "Kuning": "#f1c40f",
            "Ungu": "#9b59b6"
        }
        self.label_warna.setText(f"Warna: {warna}")
```

```
        self.label_warna.setStyleSheet(f"color: {warna_map.get(warna, 'black')}; font-weight: bold;
            font-size: 18px;")

    def nilai_berubah(self, nilai):
        self.label_nilai.setText(f"Nilai: {nilai}")
        self.progress.setValue(nilai)

    def tampilkan(self):
        hobi = []
        if self.cb1.isChecked(): hobi.append("Membaca")
        if self.cb2.isChecked(): hobi.append("Gaming")
        if self.cb3.isChecked(): hobi.append("Olahraga")

        gender = "-"
        if self.rb1.isChecked(): gender = "Laki-laki"
        elif self.rb2.isChecked(): gender = "Perempuan"

        self.hasil.setText(f"Gender: {gender} | Hobi: {', '.join(hobi) if hobi else 'Tidak ada'}")

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
sys.exit(app.exec_())
```

8.2 Penjelasan Kode

Qt menyediakan puluhan widget siap pakai. Bab ini mengumpulkan beberapa favorit dalam satu jendela: combo box, slider, progress bar, checkbox, dan radio button—masing-masing dikelompokkan dalam `QGroupBox`.

1. QComboBox: Dropdown Pilihan

Lima warna ditambahkan via `addItems()`. Signal `currentTextChanged` memanggil `warna_dipilih()` yang mengubah teks dan warna label secara dinamis menggunakan dictionary pemetaan warna ke kode hex.

2. QSlider + QProgressBar: Pasangan Serasi

`QSlider(Qt.Horizontal)` menghasilkan signal `valueChanged` setiap kali digeser. Slot `nilai_berubah()` memperbarui label angka *dan* `QProgressBar` secara bersamaan—demonstrasi satu signal mengendalikan dua widget.

3. QCheckBox dan QRadioButton

Tiga checkbox (Membaca, Gaming, Olahraga) memungkinkan pilihan ganda. Dua radio button (Laki-laki, Perempuan) otomatis saling eksklusif karena berada dalam satu parent. Tombol "Tampilkan Pilihan" membaca status semua widget lalu menampilkan ringkasan di label hasil.

4. QGroupBox: Pengelompokan Visual

Setiap kategori widget dibungkus `QGroupBox` dengan judul. Ini bukan hanya estetika—pengelompokan membantu pengguna memahami hubungan antar widget secara intuitif.

Pelajaran Penting

- Qt menyediakan widget untuk hampir semua kebutuhan input—kenali mereka sebelum membuat custom widget.
- `QGroupBox` mengelompokkan widget terkait secara visual dan semantik.
- Satu slot bisa membaca banyak widget sekaligus untuk menghasilkan output gabungan.

8.3 Hasil Tampilan

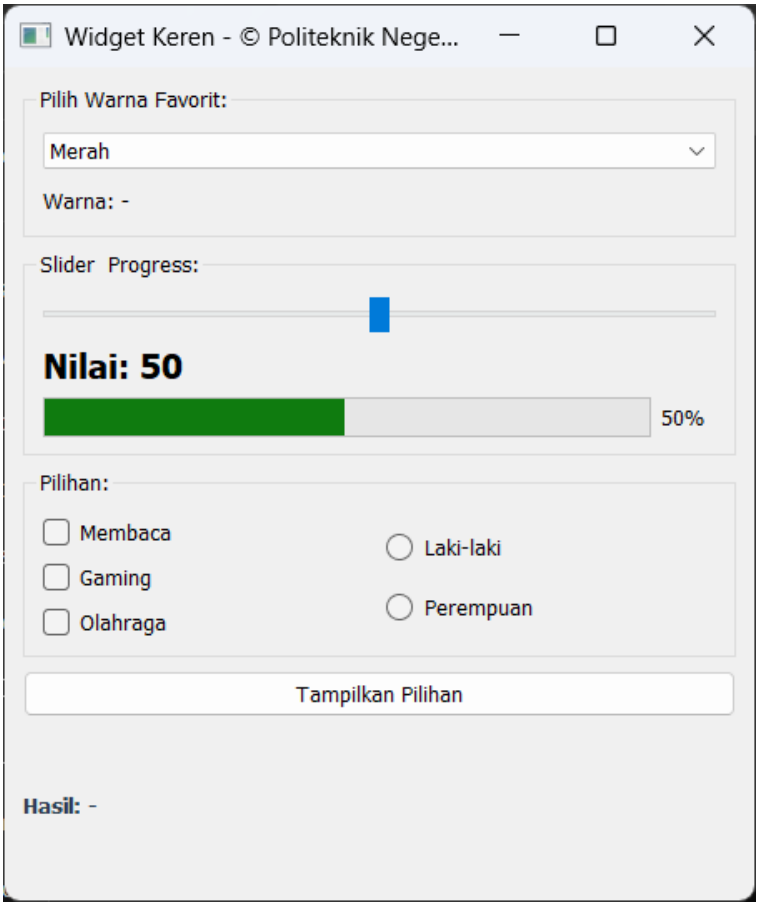

Figure 8.1: Berbagai Widget Menarik

Chapter 9

Menu & Toolbar

9.1 Kode Program

```python
# BAB 9: Menu & Toolbar
# ====================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLabel, QAction, QToolBar, QMessageBox)
from PyQt5.QtCore import Qt

class AplikasiKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Aplikasi dengan Menu & Toolbar - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 600, 400)

        # Label utama
        self.label = QLabel("Selamat datang! Pilih menu atau klik toolbar.")
        self.label.setStyleSheet("font-size: 18px; padding: 20px;")
        self.label.setAlignment(Qt.AlignCenter)
        self.setCentralWidget(self.label)

        # === MENU BAR ===
        menubar = self.menuBar()

        # Menu File
        menu_file = menubar.addMenu("File")

        aksi_baru = QAction("Baru", self)
        aksi_baru.setShortcut("Ctrl+N")
        aksi_baru.setStatusTip("Buat file baru")
        aksi_baru.triggered.connect(self.file_baru)
        menu_file.addAction(aksi_baru)

        aksi_buka = QAction("Buka", self)
        aksi_buka.setShortcut("Ctrl+O")
        aksi_buka.triggered.connect(self.file_buka)
        menu_file.addAction(aksi_buka)

        aksi_simpan = QAction("Simpan", self)
        aksi_simpan.setShortcut("Ctrl+S")
        aksi_simpan.triggered.connect(self.file_simpan)
        menu_file.addAction(aksi_simpan)
```

```
        menu_file.addSeparator()

        aksi_keluar = QAction("Keluar", self)
        aksi_keluar.setShortcut("Ctrl+Q")
        aksi_keluar.triggered.connect(self.close)
        menu_file.addAction(aksi_keluar)

        # Menu Edit
        menu_edit = menubar.addMenu("Edit")

        aksi_undo = QAction("Undo", self)
        aksi_undo.setShortcut("Ctrl+Z")
        menu_edit.addAction(aksi_undo)

        aksi_redo = QAction("Redo", self)
        aksi_redo.setShortcut("Ctrl+Y")
        menu_edit.addAction(aksi_redo)

        # Menu Bantuan
        menu_bantuan = menubar.addMenu("Bantuan")

        aksi_tentang = QAction("Tentang", self)
        aksi_tentang.triggered.connect(self.tentang)
        menu_bantuan.addAction(aksi_tentang)

        # === TOOLBAR ===
        toolbar = QToolBar("Toolbar Utama")
        toolbar.setStyleSheet("QToolBar { spacing: 5px; padding: 5px; }")
        self.addToolBar(toolbar)

        aksi_tb_baru = QAction("📄 Baru", self)
        aksi_tb_baru.triggered.connect(self.file_baru)
        toolbar.addAction(aksi_tb_baru)

        aksi_tb_buka = QAction("📂 Buka", self)
        aksi_tb_buka.triggered.connect(self.file_buka)
        toolbar.addAction(aksi_tb_buka)

        aksi_tb_simpan = QAction("💾 Simpan", self)
        aksi_tb_simpan.triggered.connect(self.file_simpan)
        toolbar.addAction(aksi_tb_simpan)

        toolbar.addSeparator()

        aksi_tb_keluar = QAction("❌ Keluar", self)
        aksi_tb_keluar.triggered.connect(self.close)
        toolbar.addAction(aksi_tb_keluar)

        # === STATUS BAR ===
        self.statusBar().showMessage("Siap")

    def file_baru(self):
        self.label.setText("📄 File baru dibuat!")
        self.statusBar().showMessage("File baru dibuat")

    def file_buka(self):
        self.label.setText("📂 Membuka file...")
        self.statusBar().showMessage("Membuka file")

    def file_simpan(self):
```

```
        self.label.setText("💾 File disimpan!")
        self.statusBar().showMessage("File disimpan", 3000)

    def tentang(self):
        QMessageBox.about(self, "Tentang",
            "Aplikasi Demo PyQt5\n"
            "Versi 1.0\n\n"
            "Dibuat untuk pembelajaran\n"
            "PyQt5 dari 0 to Hero!")

app = QApplication(sys.argv)
jendela = AplikasiKu()
jendela.show()
sys.exit(app.exec_())
```

9.2 Penjelasan Kode

Aplikasi desktop profesional memiliki menu bar di atas dan toolbar di bawahnya. Qt menyediakan keduanya sebagai fitur bawaan `QMainWindow`—tinggal isi dengan aksi.

1. Menu Bar dengan Tiga Menu

`self.menuBar()` mengembalikan menu bar bawaan. Tiga menu ditambahkan: File (Baru, Buka, Simpan, Keluar), Edit (Undo, Redo), dan Bantuan (Tentang). Setiap item adalah objek `QAction` dengan shortcut keyboard.

2. QAction: Satu Aksi, Banyak Tampilan

`QAction` bukan widget—ia adalah abstraksi *aksi*. Satu QAction bisa muncul di menu *dan* toolbar sekaligus. Signal `triggered` dihubungkan ke slot yang sama. Contoh: "Baru" muncul di menu File (Ctrl+N) dan toolbar (ikon "Baru").

3. Toolbar dengan Emoji

Alih-alih file ikon, skrip menggunakan emoji unicode (dokumen, folder, floppy disk) sebagai label toolbar. `addSeparator()` menambah garis pemisah antar kelompok aksi.

4. Status Bar

`self.statusBar().showMessage("Siap")` menampilkan pesan di bagian bawah jendela. Pesan bisa diperbarui setiap kali aksi dipicu—memberikan feedback instan kepada pengguna. Parameter timeout (misal 3000 ms) membuat pesan hilang otomatis.

Pelajaran Penting

- `QAction` memisahkan logika aksi dari tampilan—satu definisi, banyak penempatan.
- Shortcut keyboard (`setShortcut`) meningkatkan produktivitas power user.
- Menu bar, toolbar, dan status bar adalah "tiga serangkai" aplikasi desktop profesional.

9.3 Hasil Tampilan

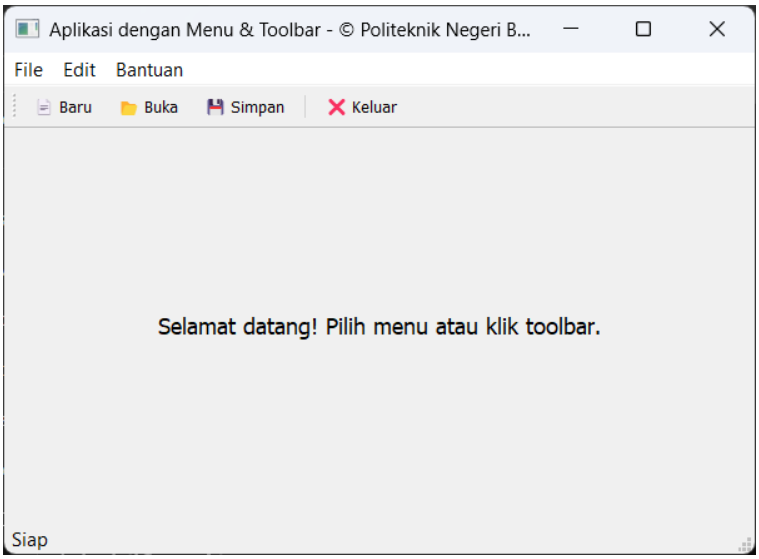

Figure 9.1: Menu dan Toolbar

Chapter 10

Styling: Mewarnai Karya

10.1 Kode Program

```python
# BAB 10: Styling - Mewarnai Karya Anda
# ====================================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLineEdit, QLabel, QPushButton,
                            QVBoxLayout, QWidget)
from PyQt5.QtCore import Qt

class FormStylish(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Form Stylish - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 400)

        # === GLOBAL STYLESHEET ===
        self.setStyleSheet("""
            QMainWindow {
                background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
                    stop:0 #EA6689, stop:1 #4BA29F);
            }
            QLabel {
                color: white;
                font-size: 14px;
                font-weight: bold;
            }
            QLabel#title {
                font-size: 24px;
                font-weight: bold;
                padding: 20px;
            }
            QLineEdit {
                padding: 12px;
                border: 2px solid rgba(255, 255, 255, 0.3);
                border-radius: 10px;
                font-size: 14px;
                background-color: rgba(255, 255, 255, 0.9);
            }
            QLineEdit:focus {
                border: 2px solid white;
                background-color: white;
            }
```

```
            QPushButton {
                padding: 15px;
                border: none;
                border-radius: 10px;
                font-size: 16px;
                font-weight: bold;
            }
            QPushButton#register {
                background-color: #e74c3c;
                color: white;
            }
            QPushButton#register:hover {
                background-color: #c0392b;
            }
            QPushButton#register:pressed {
                background-color: #a93226;
            }
            QPushButton#login {
                background-color: rgba(255, 255, 255, 0.2);
                color: white;
                border: 2px solid white;
            }
            QPushButton#login:hover {
                background-color: rgba(255, 255, 255, 0.3);
            }
        """)

        widget = QWidget()
        layout = QVBoxLayout()

        # Title
        title = QLabel("🚀 CREATE ACCOUNT")
        title.setObjectName("title")
        title.setAlignment(Qt.AlignCenter)
        layout.addWidget(title)

        layout.addSpacing(20)

        # Username
        layout.addWidget(QLabel("Username"))
        self.username = QLineEdit()
        self.username.setPlaceholderText("Masukkan username")
        layout.addWidget(self.username)

        # Email
        layout.addWidget(QLabel("Email"))
        self.email = QLineEdit()
        self.email.setPlaceholderText("email@contoh.com")
        layout.addWidget(self.email)

        # Password
        layout.addWidget(QLabel("Password"))
        self.password = QLineEdit()
        self.password.setPlaceholderText("Masukkan password")
        self.password.setEchoMode(QLineEdit.Password)
        layout.addWidget(self.password)

        # Confirm Password
        layout.addWidget(QLabel("Confirm Password"))
        self.confirm = QLineEdit()
        self.confirm.setPlaceholderText("Konfirmasi password")
```

```
        self.confirm.setEchoMode(QLineEdit.Password)
        layout.addWidget(self.confirm)

        layout.addSpacing(20)

        # Buttons
        btn_register = QPushButton("REGISTER")
        btn_register.setObjectName("register")
        btn_register.clicked.connect(self.register)
        layout.addWidget(btn_register)

        btn_login = QPushButton("Sudah punya akun? Login")
        btn_login.setObjectName("login")
        layout.addWidget(btn_login)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def register(self):
        username = self.username.text()
        email = self.email.text()
        password = self.password.text()
        confirm = self.confirm.text()

        if not username or not email or not password:
            self.statusBar().showMessage("⚠️ Semua field wajib diisi!")
            return

        if password != confirm:
            self.statusBar().showMessage("⚠️ Password tidak cocok!")
            return

        self.statusBar().showMessage("✅ Registrasi berhasil!")

app = QApplication(sys.argv)
jendela = FormStylish()
jendela.show()
sys.exit(app.exec_())
```

10.2 Penjelasan Kode

Fungsionalitas tanpa estetika terasa hambar. Bab ini membangun form "Create Account" dengan styling CSS lengkap—gradasi latar, border radius, efek hover, dan skema warna konsisten.

1. Global Stylesheet

Seluruh styling diterapkan sekaligus via `self.setStyleSheet()` dengan string CSS multi-baris. Keuntungannya: semua gaya terpusat di satu tempat, mudah diubah tanpa menyentuh logika widget.

2. Selector Berdasarkan Tipe dan ID

- `QMainWindow`: background gradasi ungu menggunakan `qlineargradient`.
- `QLabel`: teks putih, bold, 14px—berlaku untuk semua label.
- `QLabel#title`: selector ID (`setObjectName("title")`) memberi ukuran lebih besar khusus judul.
- `QPushButton#register` dan `QPushButton#login`: dua tombol dengan gaya berbeda meski tipe sama.

3. Pseudo-State: Interaksi Visual

`:hover` mengubah warna saat kursor melayang. `:pressed` mengubah warna saat diklik. `:focus` pada `QLineEdit` membuat border menjadi putih solid saat field aktif. Efek-efek ini membuat UI terasa *hidup*.

4. Password Field

`setEchoMode(QLineEdit.Password)` menyembunyikan karakter yang diketik—kebutuhan standar untuk field password.

5. Validasi Feedback via Status Bar

Alih-alih dialog, pesan error ditampilkan di status bar—lebih halus dan tidak mengganggu alur pengguna.

Pelajaran Penting

- `setStyleSheet()` mendukung hampir semua properti CSS: gradient, border-radius, padding, pseudo-state.
- Gunakan `setObjectName()` + selector `#id` untuk memberi gaya unik pada widget tertentu.
- Styling terpusat (satu string besar) lebih mudah dipelihara daripada styling per widget.

10.3 Hasil Tampilan

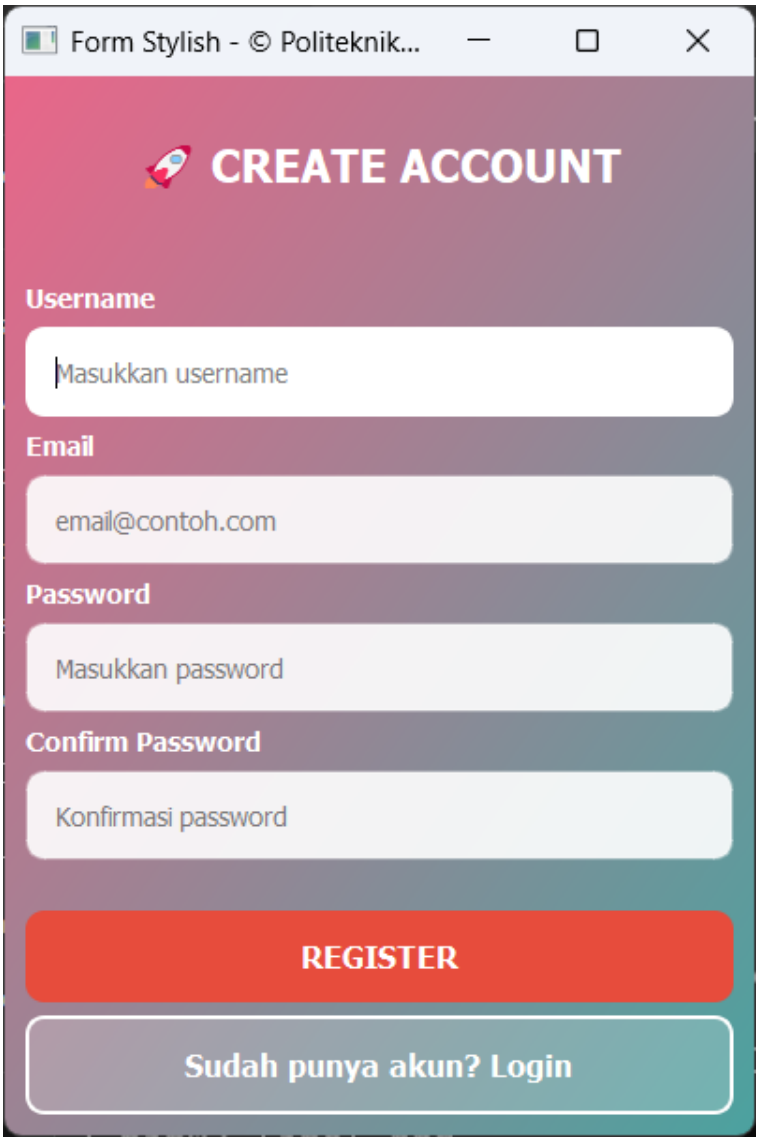

Figure 10.1: Contoh Styling Aplikasi

Chapter 11

Proyek Mini: Todo App

11.1 Kode Program

```
# BAB 11: Proyek Akhir - To-Do List App
# ====================================
# Aplikasi lengkap dengan fitur:
# - Menambah tugas
# - Menandai selesai
# - Menghapus tugas
# - Menyimpan ke file

import sys
import json
import os
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                         QLineEdit, QListWidget, QListWidgetItem,
                         QPushButton, QVBoxLayout, QHBoxLayout,
                         QWidget, QMessageBox, QCheckBox, QLabel)
from PyQt5.QtCore import Qt
from datetime import datetime

class TodoApp(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("✅ To-Do List App - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 450, 550)

        self.tasks = []
        self.file_path = os.path.join(os.path.dirname(__file__), "tasks.json")
        self.load_tasks()

        self.setup_ui()

    def setup_ui(self):
        self.setStyleSheet("""
            QMainWindow {
                background-color: #f8f9fa;
            }
            QLineEdit {
                padding: 15px;
                border: 2px solid #dee2e6;
                border-radius: 10px;
                font-size: 14px;
                background-color: white;
            }
```

```
43            QLineEdit:focus {
44                border: 2px solid #EA6689;
45            }
46            QPushButton {
47                padding: 12px 20px;
48                border: none;
49                border-radius: 8px;
50                font-size: 14px;
51                font-weight: bold;
52            }
53            QPushButton#add {
54                background-color: #EA6689;
55                color: white;
56            }
57            QPushButton#add:hover {
58                background-color: #5a6fd6;
59            }
60            QPushButton#done {
61                background-color: #28a745;
62                color: white;
63            }
64            QPushButton#done:hover {
65                background-color: #218838;
66            }
67            QPushButton#delete {
68                background-color: #dc3545;
69                color: white;
70            }
71            QPushButton#delete:hover {
72                background-color: #c82333;
73            }
74            QPushButton#clear {
75                background-color: #6c757d;
76                color: white;
77            }
78            QPushButton#clear:hover {
79                background-color: #5a6268;
80            }
81            QListWidget {
82                border: 2px solid #dee2e6;
83                border-radius: 10px;
84                background-color: white;
85                font-size: 14px;
86                padding: 5px;
87            }
88            QListWidget::item {
89                padding: 12px;
90                border-radius: 5px;
91                margin: 2px;
92            }
93            QListWidget::item:selected {
94                background-color: #e3f2fd;
95            }
96            QListWidget::item:hover {
97                background-color: #f8f9fa;
98            }
99            QLabel#stats {
100               color: #6c757d;
101               font-size: 12px;
102           }
103       """)
```

```python

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("📝 MY TO-DO LIST")
        header.setStyleSheet("""
            font-size: 24px;
            font-weight: bold;
            color: #2c3e50;
            padding: 10px;
        """)
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Input area
        input_layout = QHBoxLayout()

        self.input = QLineEdit()
        self.input.setPlaceholderText("Tambah tugas baru...")
        self.input.returnPressed.connect(self.add_task)
        input_layout.addWidget(self.input)

        btn_add = QPushButton("➕")
        btn_add.setObjectName("add")
        btn_add.clicked.connect(self.add_task)
        btn_add.setFixedWidth(50)
        input_layout.addWidget(btn_add)

        layout.addLayout(input_layout)

        # List widget
        self.list_widget = QListWidget()
        self.list_widget.itemDoubleClicked.connect(self.toggle_task)
        layout.addWidget(self.list_widget)

        # Stats
        self.label_stats = QLabel()
        self.label_stats.setObjectName("stats")
        self.update_stats()
        layout.addWidget(self.label_stats)

        # Button area
        btn_layout = QHBoxLayout()

        btn_done = QPushButton("  Selesai")
        btn_done.setObjectName("done")
        btn_done.clicked.connect(self.mark_done)
        btn_layout.addWidget(btn_done)

        btn_delete = QPushButton("🗑️ Hapus")
        btn_delete.setObjectName("delete")
        btn_delete.clicked.connect(self.delete_task)
        btn_layout.addWidget(btn_delete)

        btn_clear = QPushButton("Clear All")
        btn_clear.setObjectName("clear")
        btn_clear.clicked.connect(self.clear_all)
        btn_layout.addWidget(btn_clear)

        layout.addLayout(btn_layout)
```

```
165
166         widget.setLayout(layout)
167         self.setCentralWidget(widget)
168
169         self.refresh_list()
170
171     def add_task(self):
172         text = self.input.text().strip()
173         if text:
174             task = {
175                 "text": text,
176                 "done": False,
177                 "created": datetime.now().strftime("%Y-%m-%d %H:%M")
178             }
179             self.tasks.append(task)
180             self.save_tasks()
181             self.refresh_list()
182             self.input.clear()
183
184     def toggle_task(self, item):
185         index = self.list_widget.row(item)
186         if index >= 0:
187             self.tasks[index]["done"] = not self.tasks[index]["done"]
188             self.save_tasks()
189             self.refresh_list()
190
191     def mark_done(self):
192         current = self.list_widget.currentRow()
193         if current >= 0:
194             self.tasks[current]["done"] = True
195             self.save_tasks()
196             self.refresh_list()
197
198     def delete_task(self):
199         current = self.list_widget.currentRow()
200         if current >= 0:
201             del self.tasks[current]
202             self.save_tasks()
203             self.refresh_list()
204
205     def clear_all(self):
206         if not self.tasks:
207             return
208         reply = QMessageBox.question(self, "Konfirmasi",
209                                      "Hapus semua tugas?",
210                                      QMessageBox.Yes | QMessageBox.No)
211         if reply == QMessageBox.Yes:
212             self.tasks = []
213             self.save_tasks()
214             self.refresh_list()
215
216     def refresh_list(self):
217         self.list_widget.clear()
218         for task in self.tasks:
219             item = QListWidgetItem(task["text"])
220             if task["done"]:
221                 item.setForeground(Qt.gray)
222                 font = item.font()
223                 font.setStrikeOut(True)
224                 item.setFont(font)
225             self.list_widget.addItem(item)
```

```
        self.update_stats()

    def update_stats(self):
        total = len(self.tasks)
        done = sum(1 for t in self.tasks if t["done"])
        pending = total - done
        self.label_stats.setText(f"📊 Total: {total} | ✅Selesai: {done} | ⏳Pending: {pending}")

    def load_tasks(self):
        try:
            if os.path.exists(self.file_path):
                with open(self.file_path, "r") as f:
                    self.tasks = json.load(f)
        except:
            self.tasks = []

    def save_tasks(self):
        try:
            with open(self.file_path, "w") as f:
                json.dump(self.tasks, f, indent=2)
        except Exception as e:
            print(f"Error saving: {e}")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = TodoApp()
    window.show()
    sys.exit(app.exec_())
```

11.2 Penjelasan Kode

Proyek mini pertama ini menyatukan semua konsep dari bab sebelumnya—layout, widget, signal & slot, styling, dan penyimpanan data—menjadi aplikasi To-Do List yang benar-benar fungsional.

1. Persistensi Data dengan JSON

`load_tasks()` membaca file `tasks.json` saat startup; `save_tasks()` menulis ulang setiap ada perubahan. Jika file belum ada, list dimulai kosong. Modul `json` dan `os.path` bawaan Python membuat persistensi ini ringan tanpa dependensi tambahan.

2. Antarmuka Tiga Bagian

`setup_ui()` membangun UI dalam tiga blok vertikal:

- **Input**: `QLineEdit` + tombol tambah, mendukung Enter dan klik.
- **Daftar**: `QListWidget` menampilkan tugas; double-klik men-toggle status selesai.
- **Aksi**: tiga tombol horizontal—Selesai, Hapus, dan Clear All (dengan konfirmasi `QMessageBox`).

3. Styling Modern

CSS lengkap di `setStyleSheet()` memberikan tampilan modern: border radius, warna berbeda per tombol (`#add`, `#done`, `#delete`, `#clear`), efek hover, dan item list yang responsif. Hasilnya terasa seperti aplikasi mobile meski berjalan di desktop.

4. Alur Data Konsisten

Setiap aksi (tambah, toggle, hapus, clear) mengikuti pola yang sama: modifikasi `self.tasks` → `save_tasks()` → `refresh_list()`. Konsistensi ini mencegah bug dan membuat kode mudah dipahami.

5. Statistik Real-Time

`update_stats()` menghitung total, selesai, dan pending, lalu menampilkannya di label bawah. Diperbarui setiap kali list berubah—pengguna selalu tahu progress mereka.

Pelajaran Penting

- Proyek kecil yang fungsional adalah cara terbaik mengonsolidasi pengetahuan.
- Pola "modifikasi data → simpan → refresh UI" adalah arsitektur standar aplikasi CRUD.
- Meskipun hanya 250 baris, aplikasi ini sudah layak digunakan sehari-hari—bukti bahwa PyQt5 produktif.

11.3 Hasil Tampilan

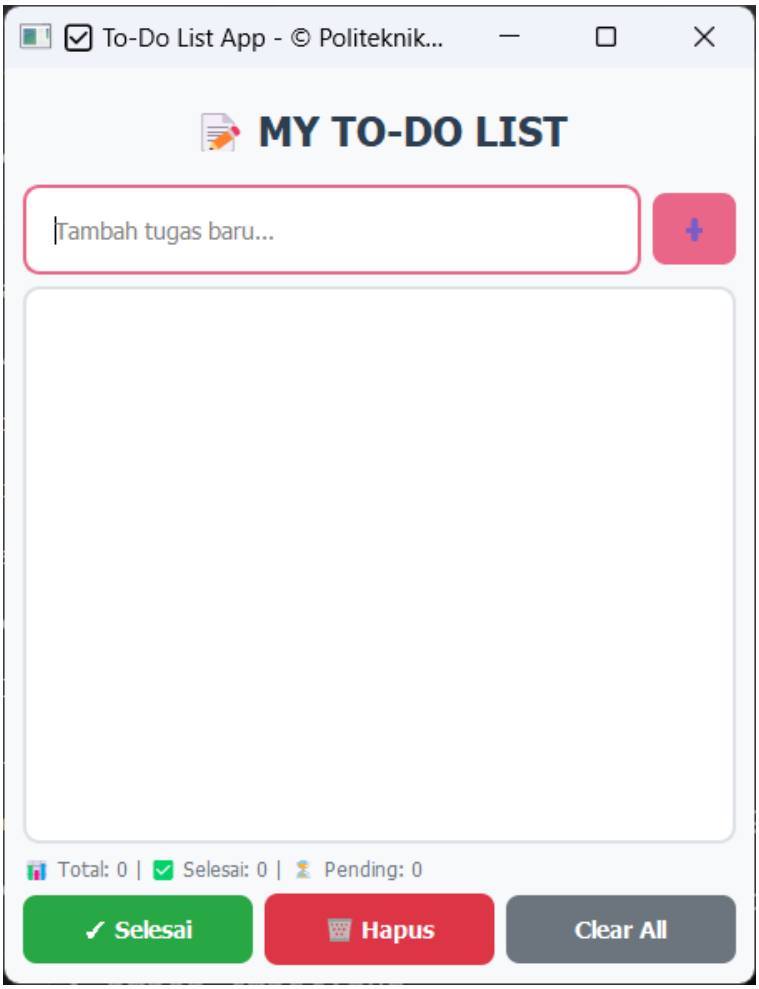

Figure 11.1: Aplikasi Todo Sederhana

Chapter 12

Input Angka: Spin dan Dial

12.1 Kode Program

```python
# BAB 12: Input Angka - Spin dan Dial
# ================================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QSpinBox, QDoubleSpinBox, QDial,
                            QLabel, QVBoxLayout, QWidget)

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Spin & Dial Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 350, 400)

        widget = QWidget()
        layout = QVBoxLayout()

        # SpinBox - Integer
        layout.addWidget(QLabel("Jumlah (Integer):"))
        self.spinbox = QSpinBox()
        self.spinbox.setRange(0, 100)
        self.spinbox.setValue(1)
        self.spinbox.setPrefix("Qty: ")
        self.spinbox.setSuffix(" pcs")
        self.spinbox.valueChanged.connect(self.spin_berubah)
        layout.addWidget(self.spinbox)

        # DoubleSpinBox - Decimal
        layout.addWidget(QLabel("Harga (Decimal):"))
        self.doublespin = QDoubleSpinBox()
        self.doublespin.setRange(0.0, 1000000.0)
        self.doublespin.setDecimals(2)
        self.doublespin.setPrefix("Rp ")
        self.doublespin.setSingleStep(1000)
        layout.addWidget(self.doublespin)

        # Dial
        layout.addWidget(QLabel("Volume:"))
        self.dial = QDial()
        self.dial.setRange(0, 100)
        self.dial.setValue(50)
        self.dial.setNotchesVisible(True)
```

```
43        self.dial.valueChanged.connect(self.dial_berubah)
44        layout.addWidget(self.dial)
45
46        # Label hasil
47        self.label = QLabel("Volume: 50%")
48        self.label.setStyleSheet("font-size: 18px; font-weight: bold;")
49        layout.addWidget(self.label)
50
51        widget.setLayout(layout)
52        self.setCentralWidget(widget)
53
54    def spin_berubah(self, nilai):
55        print(f"Qty: {nilai}")
56
57    def dial_berubah(self, nilai):
58        self.label.setText(f"Volume: {nilai}%")
59
60 app = QApplication(sys.argv)
61 jendela = JendelaKu()
62 jendela.show()
63 sys.exit(app.exec_())
```

12.2 Penjelasan Kode

Masukkan angka terlihat sederhana, tetapi antarmuka yang baik harus menyajikan kontrol tepat untuk tiap kebutuhan. Di bab ini Anda akan menjumpai `QSpinBox`, `QDoubleSpinBox`, dan `QDial`. Ketiganya berbeda rasa namun berbagi jiwa yang sama: mereka membatasi nilai agar selalu valid.

1. SpinBox Integer

Deretan baris 18–26 menciptakan kotak angka bulat (0–100). Fitur `prefix` dan `suffix` menambahkan teks "Qty:" dan "pcs" sehingga pengguna tak perlu menebak satuan. Signal `valueChanged(int)` dihubungkan ke slot `spin_berubah()`, yang sekadar mencetak nilai—cocok dijadikan titik integrasi logika stok nanti.

2. DoubleSpinBox Desimal

Input harga membutuhkan angka pecahan. `QDoubleSpinBox` mengizinkan 2 digit desimal, rentang besar, dan langkah kenaikan seribu rupiah. Semua ini dicapai hanya dengan `setDecimals()` dan `setSingleStep()`—ketimbang menulis validasi manual.

3. Dial untuk Nilai Proporsional

`QDial` di baris 38–44 ibarat kenop volume analog. Ketika diputar, ia memancarkan nilai 0–100 yang langsung ditampilkan label *Volume*. Metode `dial_berubah()` mendemontrasikan bagaimana menyesuaikan UI real-time tanpa tombol Apply.

4. Susun Layout

Semua kontrol dimasukkan ke `QVBoxLayout`. Kelebihannya: saat jendela di-resize, widget bergerak bersama secara proporsional.

Pelajaran Penting

- Gunakan widget khusus (`QSpinBox`, `QDial`) agar validasi input datang "gratis"—pengguna tidak bisa memasukkan nilai di luar rentang.
- Signal `valueChanged` membuat pembaruan UI reaktif tanpa loop polling.
- Prefiks, sufiks, dan `setSingleStep()` meningkatkan kejelasan sekaligus kecepatan input.

12.3 Hasil Tampilan

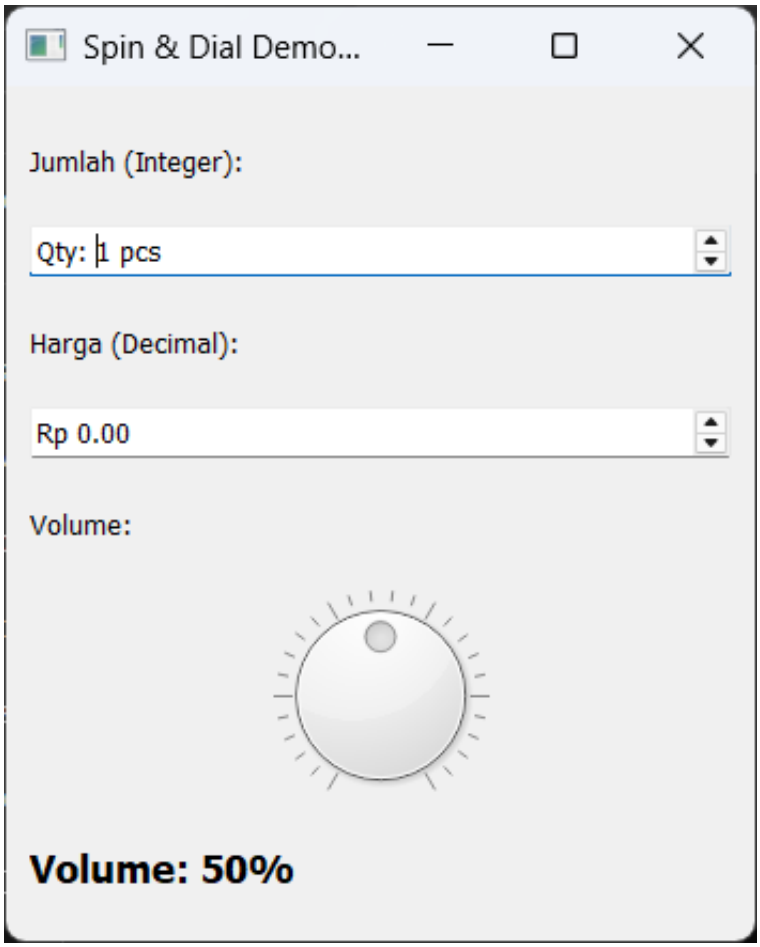

Figure 12.1: Widget SpinBox dan Dial

Chapter 13

Pilihan Tunggal: Radio Button

13.1 Kode Program

```
# BAB 13: Radio Button
# ====================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QRadioButton, QButtonGroup, QLabel,
                            QVBoxLayout, QHBoxLayout, QWidget, QGroupBox)

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("RadioButton Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 300)

        widget = QWidget()
        layout = QVBoxLayout()

        # Grup 1: Ukuran
        grup_ukuran = QGroupBox("Pilih Ukuran:")
        layout_ukuran = QVBoxLayout()

        self.bg_ukuran = QButtonGroup()
        self.rb_s = QRadioButton("S (Small)")
        self.rb_m = QRadioButton("M (Medium)")
        self.rb_l = QRadioButton("L (Large)")
        self.rb_xl = QRadioButton("XL (Extra Large)")

        self.bg_ukuran.addButton(self.rb_s, 1)
        self.bg_ukuran.addButton(self.rb_m, 2)
        self.bg_ukuran.addButton(self.rb_l, 3)
        self.bg_ukuran.addButton(self.rb_xl, 4)
        self.rb_m.setChecked(True) # Default

        layout_ukuran.addWidget(self.rb_s)
        layout_ukuran.addWidget(self.rb_m)
        layout_ukuran.addWidget(self.rb_l)
        layout_ukuran.addWidget(self.rb_xl)
        grup_ukuran.setLayout(layout_ukuran)

        # Grup 2: Warna
        grup_warna = QGroupBox("Pilih Warna:")
        layout_warna = QVBoxLayout()
```

```
        self.bg_warna = QButtonGroup()
        self.rb_merah = QRadioButton("🔴 Merah")
        self.rb_hijau = QRadioButton("🟢 Hijau")
        self.rb_biru = QRadioButton("🔵 Biru")
        self.rb_kuning = QRadioButton("🟡 Kuning")

        self.bg_warna.addButton(self.rb_merah, 1)
        self.bg_warna.addButton(self.rb_hijau, 2)
        self.bg_warna.addButton(self.rb_biru, 3)
        self.bg_warna.addButton(self.rb_kuning, 4)
        self.rb_merah.setChecked(True)

        layout_warna.addWidget(self.rb_merah)
        layout_warna.addWidget(self.rb_hijau)
        layout_warna.addWidget(self.rb_biru)
        layout_warna.addWidget(self.rb_kuning)
        grup_warna.setLayout(layout_warna)

        # Layout horizontal untuk 2 grup
        layout_h = QHBoxLayout()
        layout_h.addWidget(grup_ukuran)
        layout_h.addWidget(grup_warna)
        layout.addLayout(layout_h)

        # Label hasil
        self.hasil = QLabel("Ukuran: M | Warna: Merah")
        self.hasil.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px;")
        layout.addWidget(self.hasil)

        # Connect signals
        self.bg_ukuran.buttonClicked.connect(self.update_hasil)
        self.bg_warna.buttonClicked.connect(self.update_hasil)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def update_hasil(self):
        ukuran_map = {1: "S", 2: "M", 3: "L", 4: "XL"}
        warna_map = {1: "Merah", 2: "Hijau", 3: "Biru", 4: "Kuning"}

        ukuran = ukuran_map.get(self.bg_ukuran.checkedId(), "-")
        warna = warna_map.get(self.bg_warna.checkedId(), "-")

        self.hasil.setText(f"Ukuran: {ukuran} | Warna: {warna}")

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
sys.exit(app.exec_())
```

13.2 Penjelasan Kode

Bayangkan Anda sedang memesan kaos secara online: Anda hanya boleh memilih *satu* ukuran dan *satu* warna. Itulah inti dari `QRadioButton`—widget yang memastikan pengguna hanya bisa mencentang satu opsi dalam satu kelompok.

1. Mengelompokkan Pilihan dengan QButtonGroup

Skrip membuat dua grup terpisah menggunakan `QButtonGroup`: satu untuk ukuran (S, M, L, XL) dan

satu untuk warna (Merah, Hijau, Biru, Kuning). Setiap radio button didaftarkan ke grup dengan ID numerik melalui `addButton(rb, id)`. Berkat pemisahan ini, memilih "L" tidak akan membatalkan pilihan "Merah"—kedua grup bekerja secara independen.

2. Tampilan Rapi dengan QGroupBox

Agar pengguna tahu mana kelompok ukuran dan mana kelompok warna, masing-masing dibungkus `QGroupBox` bertajuk. Dua group box ini diletakkan berdampingan di dalam `QHBoxLayout`, sementara keseluruhan tata letak vertikal diatur oleh `QVBoxLayout`.

3. Reaksi Real-Time

Signal `buttonClicked` dari kedua grup dihubungkan ke satu slot `update_hasil()`. Setiap kali pengguna mengklik radio button mana pun, slot ini membaca `checkedId()` dari tiap grup, menerjemahkannya lewat dictionary, lalu menampilkan ringkasan di label bawah—misalnya "Ukuran: L | Warna: Biru". Tanpa tombol "OK", tanpa delay; umpan balik langsung terasa.

Pelajaran Penting

- `QRadioButton` cocok ketika pilihan bersifat *mutually exclusive* dalam satu grup.
- `QButtonGroup` memungkinkan banyak grup independen dalam satu jendela.
- Mapping ID ke teks melalui dictionary membuat kode lebih mudah dirawat ketimbang rangkaian `if-elif`.

13.3 Hasil Tampilan

Figure 13.1: Radio Button

Chapter 14

Pilihan Ganda: Checkbox

14.1 Kode Program

```python
# BAB 14: CheckBox
# ================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QCheckBox, QLabel, QPushButton,
                            QVBoxLayout, QWidget, QGroupBox)
from PyQt5.QtCore import Qt

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("CheckBox Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 350, 400)

        widget = QWidget()
        layout = QVBoxLayout()

        # Grup Topping
        grup_topping = QGroupBox("Pilih Topping Pizza:")
        layout_topping = QVBoxLayout()

        self.cb_keju = QCheckBox("🧀 Extra Keju (+Rp 10.000)")
        self.cb_jamur = QCheckBox("🍄 Jamur (+Rp 8.000)")
        self.cb_daging = QCheckBox("🥩 Daging Sapi (+Rp 20.000)")
        self.cb_sosis = QCheckBox("🌭 Sosis (+Rp 12.000)")
        self.cb_sayur = QCheckBox("🥬 Sayuran (+Rp 5.000)")

        layout_topping.addWidget(self.cb_keju)
        layout_topping.addWidget(self.cb_jamur)
        layout_topping.addWidget(self.cb_daging)
        layout_topping.addWidget(self.cb_sosis)
        layout_topping.addWidget(self.cb_sayur)
        grup_topping.setLayout(layout_topping)
        layout.addWidget(grup_topping)

        # Tristate checkbox
        self.cb_semua = QCheckBox("Pilih Semua (Tristate)")
        self.cb_semua.setTristate(True)
        self.cb_semua.stateChanged.connect(self.pilih_semua)
        layout.addWidget(self.cb_semua)
```

```
        # Tombol
        btn = QPushButton("Hitung Total")
        btn.clicked.connect(self.hitung_total)
        btn.setStyleSheet("padding: 10px; font-size: 14px;")
        layout.addWidget(btn)

        # Hasil
        self.hasil = QLabel("Total: Rp 0")
        self.hasil.setStyleSheet("font-size: 18px; font-weight: bold; color: #27ae60;")
        layout.addWidget(self.hasil)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def pilih_semua(self, state):
        checked = state == Qt.Checked
        self.cb_keju.setChecked(checked)
        self.cb_jamur.setChecked(checked)
        self.cb_daging.setChecked(checked)
        self.cb_sosis.setChecked(checked)
        self.cb_sayur.setChecked(checked)

    def hitung_total(self):
        total = 50000 # Harga dasar pizza

        if self.cb_keju.isChecked():
            total += 10000
        if self.cb_jamur.isChecked():
            total += 8000
        if self.cb_daging.isChecked():
            total += 20000
        if self.cb_sosis.isChecked():
            total += 12000
        if self.cb_sayur.isChecked():
            total += 5000

        self.hasil.setText(f"Total: Rp {total:,}")

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
sys.exit(app.exec_())
```

14.2 Penjelasan Kode

Kalau radio button memaksa satu pilihan, `QCheckBox` justru mempersilakan pengguna mencentang sebanyak mungkin—persis seperti memilih topping pizza. Skrip ini mensimulasikan hal itu: lima topping tersedia, masing-masing punya harga berbeda.

1. Deretan Checkbox di Dalam QGroupBox

Lima `QCheckBox` (Extra Keju, Jamur, Daging Sapi, Sosis, Sayuran) dikelompokkan dalam `QGroupBox` bertajuk "Pilih Topping Pizza:". Setiap checkbox berdiri sendiri—bisa dicentang atau tidak tanpa memengaruhi tetangganya.

2. Tristate Checkbox: Pilih Semua

Ada satu checkbox spesial: "Pilih Semua (Tristate)". Dengan `setTristate(True)`, ia memiliki tiga kondisi: unchecked, partially checked, dan fully checked. Signal `stateChanged` dihubungkan ke slot `pilih_semua()`, yang mencentang atau menghapus centang seluruh topping sekaligus—fitur kenyamanan yang sering dijumpai di aplikasi nyata.

3. Menghitung Total Harga

Tombol "Hitung Total" memanggil `hitung_total()`. Di dalamnya, harga dasar pizza (Rp 50.000) ditambahkan sesuai topping yang tercentang menggunakan `isChecked()`. Hasilnya ditampilkan di label dengan format ribuan (`:,`)—sentuhan kecil yang membuat angka lebih terbaca manusia.

Pelajaran Penting

- Gunakan `QCheckBox` ketika pengguna boleh memilih nol, satu, atau banyak opsi.
- Mode tristate berguna untuk "Select All" / "Deselect All".
- Kalkulasi real-time bisa diimplementasikan hanya dengan membaca status boolean setiap checkbox.

14.3 Hasil Tampilan

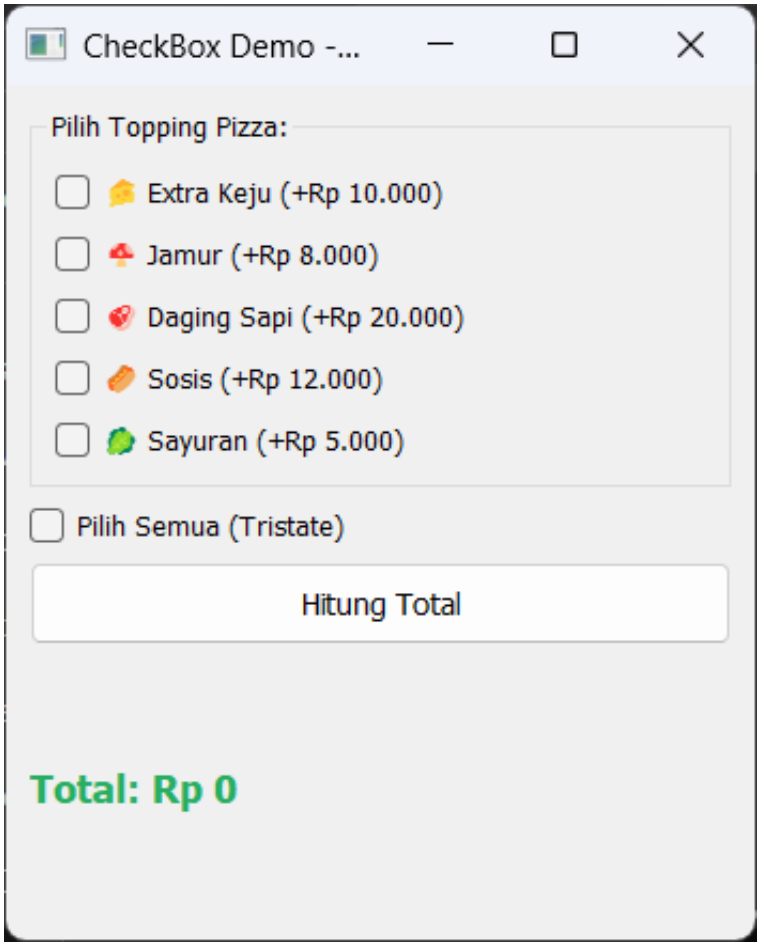

Figure 14.1: Checkbox

Chapter 15

Dropdown: ComboBox

15.1 Kode Program

```python
# BAB 15: ComboBox (Dropdown)
# ============================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QComboBox, QLabel, QPushButton,
                            QVBoxLayout, QHBoxLayout, QWidget)

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("ComboBox Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 300)

        widget = QWidget()
        layout = QVBoxLayout()

        # ComboBox dengan teks saja
        layout.addWidget(QLabel("Pilih Kota:"))
        self.combo_kota = QComboBox()
        self.combo_kota.addItems([
            "Jakarta",
            "Bandung",
            "Surabaya",
            "Yogyakarta",
            "Semarang",
            "Malang",
            "Bali"
        ])
        self.combo_kota.currentTextChanged.connect(self.kota_dipilih)
        layout.addWidget(self.combo_kota)

        # ComboBox dengan data tersembunyi
        layout.addWidget(QLabel("\nPilih Produk:"))
        self.combo_produk = QComboBox()

        # Format: (nama, harga)
        produk = [
            ("Laptop ASUS ROG", 15000000),
            ("MacBook Air M2", 18000000),
            ("Dell XPS 15", 20000000),
            ("Lenovo ThinkPad", 12000000),
```

```
43            ("HP Spectre", 16500000),
44        ]
45
46        for nama, harga in produk:
47            self.combo_produk.addItem(f"{nama}", harga)
48
49        self.combo_produk.currentIndexChanged.connect(self.produk_dipilih)
50        layout.addWidget(self.combo_produk)
51
52        # Label harga
53        self.label_harga = QLabel("Harga: Rp 15,000,000")
54        self.label_harga.setStyleSheet("font-size: 16px; font-weight: bold; color: #2980b9;")
55        layout.addWidget(self.label_harga)
56
57        # ComboBox editable
58        layout.addWidget(QLabel("\nComboBox Editable:"))
59        self.combo_edit = QComboBox()
60        self.combo_edit.setEditable(True)
61        self.combo_edit.addItems(["Java", "Python", "JavaScript", "C++", "Go"])
62        self.combo_edit.setInsertPolicy(QComboBox.InsertAtTop)
63        layout.addWidget(self.combo_edit)
64
65        # Tombol tambah
66        btn = QPushButton("Tambah Bahasa Baru")
67        btn.clicked.connect(lambda: self.combo_edit.addItem(self.combo_edit.currentText()))
68        layout.addWidget(btn)
69
70        widget.setLayout(layout)
71        self.setCentralWidget(widget)
72
73    def kota_dipilih(self, kota):
74        print(f"Kota dipilih: {kota}")
75
76    def produk_dipilih(self, index):
77        harga = self.combo_produk.itemData(index)
78        nama = self.combo_produk.itemText(index)
79        self.label_harga.setText(f"Harga: Rp {harga:,}")
80        print(f"Produk: {nama}, Harga: {harga}")
81
82 app = QApplication(sys.argv)
83 jendela = JendelaKu()
84 jendela.show()
85 sys.exit(app.exec_())
```

15.2 Penjelasan Kode

Pernahkah Anda mengisi formulir online dan menjumpai daftar yang muncul saat diklik? Itulah `QComboBox`—dropdown hemat ruang yang menyembunyikan puluhan opsi di balik satu baris.

1. ComboBox Sederhana: Pilih Kota

Combo box pertama diisi tujuh nama kota lewat `addItems()`. Signal `currentTextChanged` langsung terhubung ke slot `kota_dipilih()` yang mencetak nama kota ke konsol. Pengguna cukup klik, pilih, dan data langsung tersedia—tanpa tombol konfirmasi.

2. ComboBox dengan Data Tersembunyi

Combo box kedua lebih canggih: setiap item menyimpan data tambahan (harga) melalui parameter kedua `addItem(teks, data)`. Saat pengguna memilih produk laptop, slot `produk_dipilih()` mengambil harga lewat `itemData(index)` dan menampilkannya di label. Teknik ini sangat berguna ketika Anda ingin menampilkan nama ramah di UI namun menyimpan nilai teknis (ID database, harga, kode) di belakang layar.

3. ComboBox Editable

Combo box ketiga diatur `setEditable(True)` sehingga pengguna boleh mengetik item baru selain memilih yang tersedia. Kebijakan `InsertAtTop` memastikan item baru selalu muncul di posisi teratas daftar.

Pelajaran Penting

- `QComboBox` ideal ketika pilihan banyak namun ruang layar terbatas.
- Simpan data tersembunyi via `addItem(text, userData)` untuk memisahkan tampilan dan logika.
- Mode editable mengubah combo box menjadi hybrid input + dropdown.

15.3 Hasil Tampilan

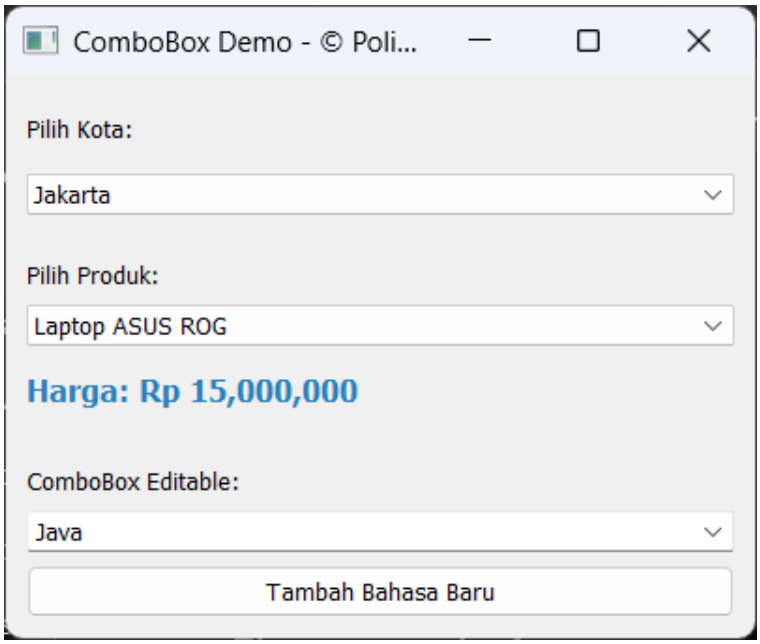

Figure 15.1: ComboBox

Chapter 16

Tanggal dan Waktu

16.1 Kode Program

```
# BAB 16: Tanggal dan Waktu
# ==========================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QDateEdit, QTimeEdit, QDateTimeEdit,
                            QCalendarWidget, QLabel, QPushButton,
                            QVBoxLayout, QHBoxLayout, QWidget, QGroupBox)
from PyQt5.QtCore import QDate, QTime, QDateTime, Qt

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Date & Time Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 500)

        widget = QWidget()
        layout = QVBoxLayout()

        # Date Edit
        grup_date = QGroupBox("Pilih Tanggal:")
        layout_date = QVBoxLayout()

        self.date_edit = QDateEdit()
        self.date_edit.setCalendarPopup(True)
        self.date_edit.setDate(QDate.currentDate())
        self.date_edit.setDisplayFormat("dddd, dd MMMM yyyy")
        self.date_edit.dateChanged.connect(self.tanggal_berubah)
        layout_date.addWidget(self.date_edit)

        self.label_date = QLabel(f"Tanggal: {QDate.currentDate().toString('dddd, dd MMMM yyyy')}")
        layout_date.addWidget(self.label_date)

        grup_date.setLayout(layout_date)
        layout.addWidget(grup_date)

        # Time Edit
        grup_time = QGroupBox("Pilih Waktu:")
        layout_time = QVBoxLayout()

        self.time_edit = QTimeEdit()
        self.time_edit.setTime(QTime.currentTime())
```

```
        self.time_edit.setDisplayFormat("HH:mm:ss")
        self.time_edit.timeChanged.connect(self.waktu_berubah)
        layout_time.addWidget(self.time_edit)

        self.label_time = QLabel(f"Waktu: {QTime.currentTime().toString('HH:mm:ss')}")
        layout_time.addWidget(self.label_time)

        grup_time.setLayout(layout_time)
        layout.addWidget(grup_time)

        # DateTime Edit
        grup_datetime = QGroupBox("Tanggal & Waktu:")
        layout_datetime = QVBoxLayout()

        self.datetime_edit = QDateTimeEdit()
        self.datetime_edit.setCalendarPopup(True)
        self.datetime_edit.setDateTime(QDateTime.currentDateTime())
        self.datetime_edit.setDisplayFormat("dd/MM/yyyy HH:mm")
        layout_datetime.addWidget(self.datetime_edit)

        grup_datetime.setLayout(layout_datetime)
        layout.addWidget(grup_datetime)

        # Calendar Widget
        layout.addWidget(QLabel("\nKalender:"))
        self.calendar = QCalendarWidget()
        self.calendar.setGridVisible(True)
        self.calendar.clicked.connect(self.tanggal_kalender_dipilih)
        layout.addWidget(self.calendar)

        self.label_calendar = QLabel("Klik tanggal di kalender...")
        self.label_calendar.setStyleSheet("font-weight: bold;")
        layout.addWidget(self.label_calendar)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def tanggal_berubah(self, date):
        self.label_date.setText(f"Tanggal: {date.toString('dddd, dd MMMM yyyy')}")

    def waktu_berubah(self, time):
        self.label_time.setText(f"Waktu: {time.toString('HH:mm:ss')}")

    def tanggal_kalender_dipilih(self, date):
        self.label_calendar.setText(f"Dipilih: {date.toString('dddd, dd MMMM yyyy')}")

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
sys.exit(app.exec_())
```

16.2 Penjelasan Kode

Mengetik tanggal secara manual rawan salah format. Qt menyediakan keluarga widget khusus—`QDateEdit`, `QTimeEdit`, `QDateTimeEdit`, dan `QCalendarWidget`—yang memastikan input selalu valid.

1. QDateEdit dengan Kalender Pop-Up

Widget pertama menampilkan tanggal hari ini dalam format lokal "dddd, dd MMMM yyyy" (misal: Senin, 03 Februari 2026). Berkat `setCalendarPopup(True)`, pengguna bisa mengklik ikon kalender kecil lalu memilih tanggal secara visual. Signal `dateChanged` memperbarui label di bawahnya secara real-time.

2. QTimeEdit

Widget kedua menangkap waktu (jam:menit:detik). Formatnya dikendalikan oleh `setDisplayFormat("HH:mm:ss")`. Signal `timeChanged` bekerja serupa: setiap perubahan langsung terlihat di label.

3. QDateTimeEdit

Kombinasi tanggal dan waktu dalam satu widget. Cocok untuk skenario seperti penjadwalan rapat atau pemesanan tiket.

4. QCalendarWidget

Kalender penuh satu halaman dengan grid terlihat (`setGridVisible(True)`). Signal `clicked` mengirim tanggal yang dipilih ke slot `tanggal_kalender_dipilih()`, yang memformat hasilnya menjadi teks ramah.

Pelajaran Penting

- Gunakan widget tanggal/waktu bawaan Qt agar validasi format otomatis.
- `setDisplayFormat()` memungkinkan format tampilan disesuaikan tanpa mengubah data internal.
- Kalender pop-up dan `QCalendarWidget` membuat pengalaman memilih tanggal seintuitif aplikasi ponsel.

16.3 Hasil Tampilan

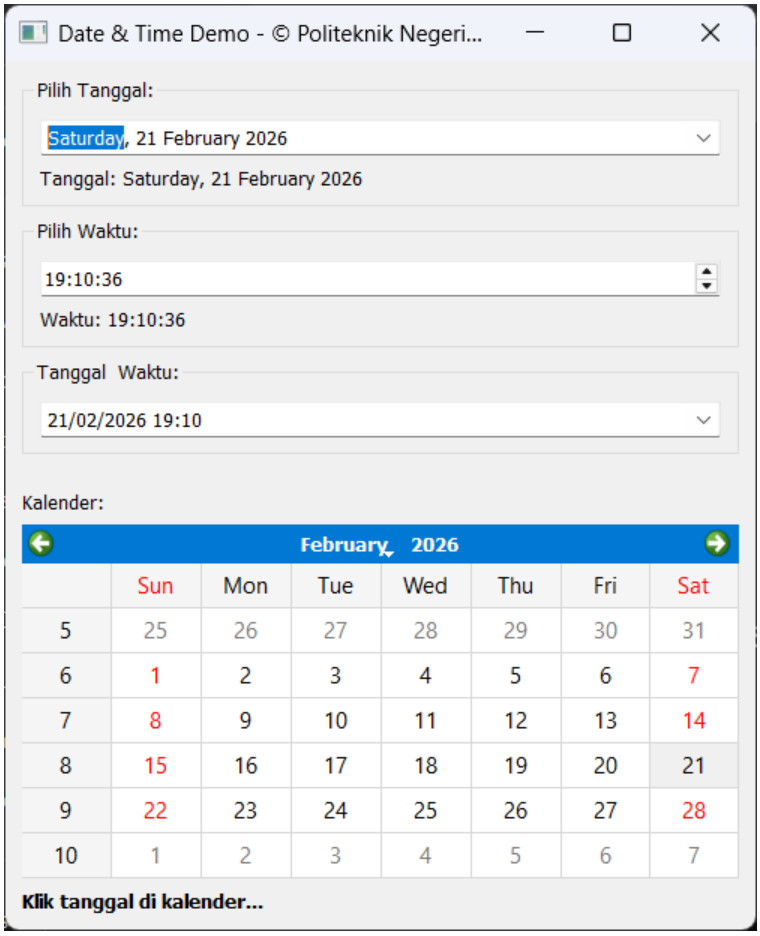

Figure 16.1: Widget DateTime

Chapter 17

Daftar: ListWidget

17.1 Kode Program

```python
# BAB 17: ListWidget (Daftar)
# ===========================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QListWidget, QListWidgetItem, QLineEdit,
                            QPushButton, QLabel, QVBoxLayout,
                            QHBoxLayout, QWidget)

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("ListWidget Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 450)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("📝 DAFTAR TUGAS")
        header.setStyleSheet("font-size: 18px; font-weight: bold; padding: 10px;")
        layout.addWidget(header)

        # Input area
        input_layout = QHBoxLayout()

        self.input = QLineEdit()
        self.input.setPlaceholderText("Tambah tugas baru...")
        self.input.returnPressed.connect(self.tambah_item)
        input_layout.addWidget(self.input)

        btn_add = QPushButton("➕")
        btn_add.setFixedWidth(50)
        btn_add.clicked.connect(self.tambah_item)
        input_layout.addWidget(btn_add)

        layout.addLayout(input_layout)

        # List widget
        self.list = QListWidget()
        self.list.addItems([
            "Belajar PyQt5",
```

```
43            "Kerjakan tugas matematika",
44            "Meeting jam 14:00",
45            "Review kode project"
46        ])
47        self.list.currentItemChanged.connect(self.item_dipilih)
48        self.list.itemDoubleClicked.connect(self.item_double_click)
49        layout.addWidget(self.list)
50
51        # Info label
52        self.label_info = QLabel("Total: 4 item | Klik item untuk memilih")
53        layout.addWidget(self.label_info)
54
55        # Button area
56        btn_layout = QHBoxLayout()
57
58        btn_edit = QPushButton("✏️ Edit")
59        btn_edit.clicked.connect(self.edit_item)
60        btn_layout.addWidget(btn_edit)
61
62        btn_delete = QPushButton("🗑️ Hapus")
63        btn_delete.clicked.connect(self.hapus_item)
64        btn_layout.addWidget(btn_delete)
65
66        btn_clear = QPushButton("🧹 Clear")
67        btn_clear.clicked.connect(self.clear_all)
68        btn_layout.addWidget(btn_clear)
69
70        layout.addLayout(btn_layout)
71
72        # Selection mode
73        btn_select_all = QPushButton("Pilih Semua")
74        btn_select_all.clicked.connect(lambda: self.list.selectAll())
75        layout.addWidget(btn_select_all)
76
77        widget.setLayout(layout)
78        self.setCentralWidget(widget)
79
80        self.update_info()
81
82    def tambah_item(self):
83        text = self.input.text().strip()
84        if text:
85            self.list.addItem(text)
86            self.input.clear()
87            self.update_info()
88
89    def item_dipilih(self, current, previous):
90        if current:
91            self.label_info.setText(f"Dipilih: {current.text()}")
92
93    def item_double_click(self, item):
94        self.label_info.setText(f"Double click: {item.text()}")
95
96    def edit_item(self):
97        current = self.list.currentItem()
98        if current:
99            self.input.setText(current.text())
100           self.list.takeItem(self.list.row(current))
101           self.update_info()
102
103   def hapus_item(self):
```

```
        current = self.list.currentRow()
        if current >= 0:
            self.list.takeItem(current)
            self.update_info()

    def clear_all(self):
        self.list.clear()
        self.update_info()

    def update_info(self):
        count = self.list.count()
        self.label_info.setText(f"Total: {count} item")

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
sys.exit(app.exec_())
```

17.2 Penjelasan Kode

`QListWidget` adalah papan tulis digital tempat Anda menampilkan daftar item yang bisa ditambah, dihapus, dan diedit secara interaktif. Skrip ini menyajikan daftar tugas mini.

1. Input dan Penambahan Item

`QLineEdit` di atas list menerima teks baru. Pengguna bisa menekan Enter (signal `returnPressed`) atau mengklik tombol + untuk memanggil `tambah_item()`. Teks ditambahkan ke list dengan `addItem()`, lalu input dikosongkan—alur yang terasa alami.

2. Interaksi: Klik dan Double-Klik

Signal `currentItemChanged` dipanggil setiap kali item berbeda dipilih, menampilkan nama item di label info. Signal `itemDoubleClicked` menangkap klik ganda—pola umum untuk membuka atau mengedit detail item.

3. Edit, Hapus, dan Clear

Tombol Edit memindahkan teks item terpilih ke input field dan menghapusnya dari list (`takeItem`), sehingga pengguna bisa memperbaiki lalu menambahkan kembali. Tombol Hapus menghilangkan item saat ini, sedangkan Clear mengosongkan seluruh daftar. Setiap aksi diakhiri `update_info()` yang memperbarui jumlah total item.

4. Pilih Semua

Tombol terakhir memanggil `selectAll()`—berguna saat pengguna ingin melakukan aksi massal.

Pelajaran Penting

- `QListWidget` cocok untuk data satu dimensi yang dinamis.
- Pisahkan logika CRUD ke metode tersendiri agar kode tetap bersih.
- Selalu perbarui statistik (counter) setelah setiap mutasi data agar UI konsisten.

17.3 Hasil Tampilan

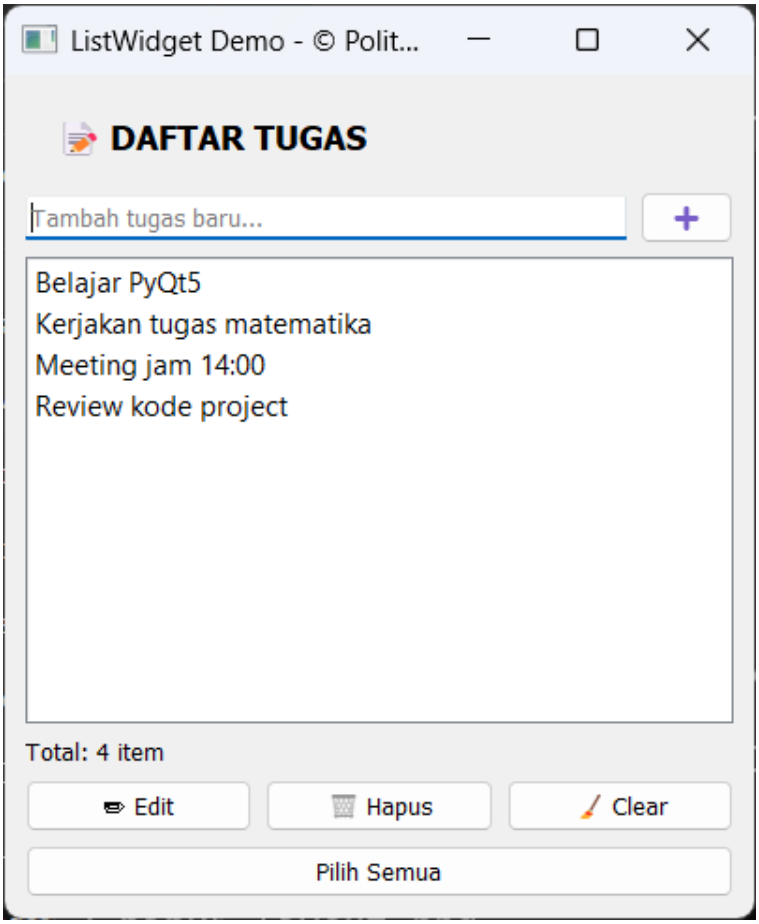

Figure 17.1: ListWidget

Chapter 18

Tabel: TableWidget

18.1 Kode Program

```python
# BAB 18: TableWidget (Tabel)
# ============================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QTableWidget, QTableWidgetItem, QLineEdit,
                            QPushButton, QLabel, QVBoxLayout,
                            QHBoxLayout, QWidget, QHeaderView)
from PyQt5.QtCore import Qt

class JendelaKu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TableWidget Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 600, 450)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("📊 DATA MAHASISWA")
        header.setStyleSheet("font-size: 18px; font-weight: bold; padding: 10px;")
        layout.addWidget(header)

        # Input form
        input_layout = QHBoxLayout()

        self.input_nim = QLineEdit()
        self.input_nim.setPlaceholderText("NIM")
        input_layout.addWidget(self.input_nim)

        self.input_nama = QLineEdit()
        self.input_nama.setPlaceholderText("Nama")
        input_layout.addWidget(self.input_nama)

        self.input_nilai = QLineEdit()
        self.input_nilai.setPlaceholderText("Nilai (0-100)")
        self.input_nilai.setFixedWidth(100)
        input_layout.addWidget(self.input_nilai)

        btn_add = QPushButton("➕ Tambah")
        btn_add.clicked.connect(self.tambah_baris)
```

```
        input_layout.addWidget(btn_add)

        layout.addLayout(input_layout)

        # Table
        self.table = QTableWidget()
        self.table.setColumnCount(4)
        self.table.setHorizontalHeaderLabels(["NIM", "Nama", "Nilai", "Grade"])

        # Header stretch
        header = self.table.horizontalHeader()
        header.setSectionResizeMode(0, QHeaderView.ResizeToContents)
        header.setSectionResizeMode(1, QHeaderView.Stretch)
        header.setSectionResizeMode(2, QHeaderView.ResizeToContents)
        header.setSectionResizeMode(3, QHeaderView.ResizeToContents)

        # Sample data
        data = [
            ("12345", "Andi Pratama", 85),
            ("12346", "Budi Santoso", 75),
            ("12347", "Caca Wijaya", 90),
            ("12348", "Doni Putra", 65),
        ]

        for row, (nim, nama, nilai) in enumerate(data):
            self.add_row(row, nim, nama, nilai)

        layout.addWidget(self.table)

        # Stats
        self.label_stats = QLabel()
        self.update_stats()
        layout.addWidget(self.label_stats)

        # Buttons
        btn_layout = QHBoxLayout()

        btn_delete = QPushButton("🗑️ Hapus Baris")
        btn_delete.clicked.connect(self.hapus_baris)
        btn_layout.addWidget(btn_delete)

        btn_clear = QPushButton("🧹 Clear All")
        btn_clear.clicked.connect(self.clear_table)
        btn_layout.addWidget(btn_clear)

        layout.addLayout(btn_layout)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def add_row(self, row, nim, nama, nilai):
        self.table.insertRow(row)

        grade = self.hitung_grade(nilai)

        self.table.setItem(row, 0, QTableWidgetItem(nim))
        self.table.setItem(row, 1, QTableWidgetItem(nama))

        item_nilai = QTableWidgetItem(str(nilai))
        # Warna berdasarkan nilai
        if nilai >= 85:
```

```
            item_nilai.setBackground(Qt.green)
        elif nilai >= 70:
            item_nilai.setBackground(Qt.yellow)
        else:
            item_nilai.setBackground(Qt.red)
        self.table.setItem(row, 2, item_nilai)

        self.table.setItem(row, 3, QTableWidgetItem(grade))

    def tambah_baris(self):
        nim = self.input_nim.text().strip()
        nama = self.input_nama.text().strip()
        nilai_text = self.input_nilai.text().strip()

        if nim and nama and nilai_text:
            try:
                nilai = int(nilai_text)
                if 0 <= nilai <= 100:
                    row = self.table.rowCount()
                    self.add_row(row, nim, nama, nilai)

                    self.input_nim.clear()
                    self.input_nama.clear()
                    self.input_nilai.clear()
                    self.update_stats()
            except ValueError:
                pass

    def hitung_grade(self, nilai):
        if nilai >= 85: return "A"
        elif nilai >= 70: return "B"
        elif nilai >= 55: return "C"
        elif nilai >= 40: return "D"
        else: return "E"

    def hapus_baris(self):
        current = self.table.currentRow()
        if current >= 0:
            self.table.removeRow(current)
            self.update_stats()

    def clear_table(self):
        self.table.setRowCount(0)
        self.update_stats()

    def update_stats(self):
        count = self.table.rowCount()
        total_nilai = 0
        for row in range(count):
            try:
                nilai = int(self.table.item(row, 2).text())
                total_nilai += nilai
            except:
                pass

        rata = total_nilai / count if count > 0 else 0
        self.label_stats.setText(f"📊 Total: {count} mahasiswa | Rata-rata: {rata:.1f}")

app = QApplication(sys.argv)
jendela = JendelaKu()
jendela.show()
```

```
sys.exit(app.exec_())
```

18.2 Penjelasan Kode

Data dua dimensi—baris dan kolom—paling nyaman ditampilkan dalam tabel. `QTableWidget` menyediakan spreadsheet mini di dalam aplikasi Anda. Skrip ini mengelola data mahasiswa: NIM, Nama, Nilai, dan Grade.

1. Menyiapkan Tabel

Tabel diinisialisasi dengan empat kolom via `setColumnCount(4)` dan diberi header. Mode resize kolom diatur lewat `QHeaderView`: kolom NIM dan Grade menyesuaikan isi (`ResizeToContents`), sedangkan kolom Nama meregang mengisi sisa ruang (`Stretch`).

2. Menambah Baris dengan Warna Kontekstual

Metode `add_row()` menyisipkan baris, menghitung grade otomatis, lalu memberi warna latar pada sel nilai: hijau untuk ≥ 85, kuning untuk ≥ 70, dan merah di bawahnya. Efek visual ini membantu pembaca menangkap informasi hanya dengan sekali lirik—prinsip *progressive disclosure* dalam desain UX.

3. Input dari Pengguna

Tiga `QLineEdit` di atas tabel menerima NIM, nama, dan nilai. Tombol Tambah memanggil `tambah_baris()` yang memvalidasi tipe data, membatasi rentang 0–100, lalu membersihkan form setelah berhasil.

4. Statistik Real-Time

Metode `update_stats()` menghitung jumlah mahasiswa dan rata-rata nilai, lalu menampilkannya di label bawah. Setiap kali baris ditambah atau dihapus, statistik langsung diperbarui.

Pelajaran Penting

- `QTableWidget` ideal untuk data tabular kecil hingga menengah.
- Warna latar sel (`setBackground`) meningkatkan keterbacaan data.
- Validasi input sebelum menyisipkan baris mencegah data kotor masuk ke tabel.

18.3 Hasil Tampilan

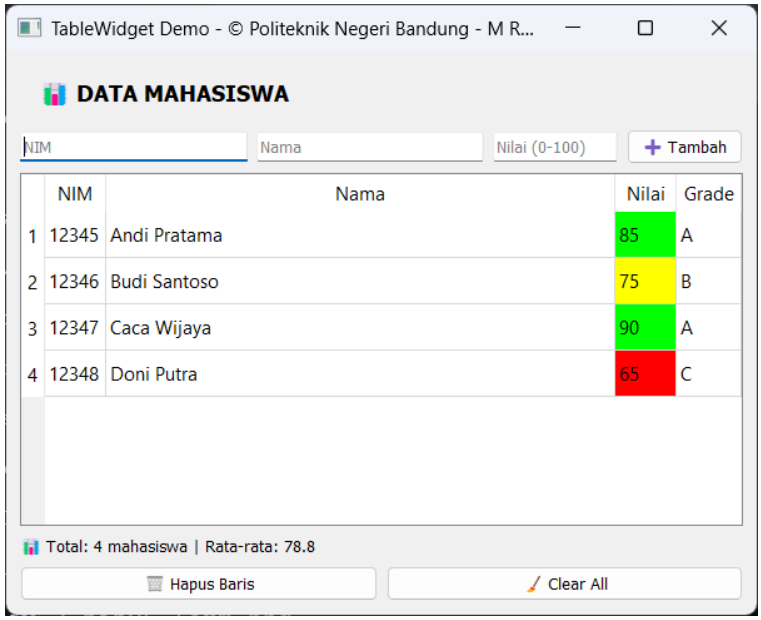

Figure 18.1: TableWidget

Chapter 19

Tab Widget

19.1 Kode Program

```python
# BAB 19: Tab Widget
# ===================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QTabWidget, QWidget, QLabel, QLineEdit,
                            QTextEdit, QPushButton, QSpinBox,
                            QVBoxLayout, QHBoxLayout, QFormLayout)
from PyQt5.QtCore import Qt

class TabApp(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Tab Widget Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 400)

        self.tabs = QTabWidget()
        self.tabs.setTabsClosable(False)
        self.setCentralWidget(self.tabs)

        # Tab 1: Home
        self.create_home_tab()

        # Tab 2: Calculator
        self.create_calc_tab()

        # Tab 3: Notes
        self.create_notes_tab()

        # Tab 4: Settings
        self.create_settings_tab()

    def create_home_tab(self):
        tab = QWidget()
        layout = QVBoxLayout()

        label = QLabel("🏠 WELCOME")
        label.setStyleSheet("font-size: 24px; font-weight: bold;")
        label.setAlignment(Qt.AlignCenter)
        layout.addWidget(label)

```

```
        info = QLabel("Ini adalah aplikasi demo Tab Widget.\n\nKlik tab di atas untuk berpindah
            halaman.")
        info.setAlignment(Qt.AlignCenter)
        layout.addWidget(info)

        tab.setLayout(layout)
        self.tabs.addTab(tab, "Home")

    def create_calc_tab(self):
        tab = QWidget()
        layout = QVBoxLayout()

        label = QLabel("🔢 KALKULATOR SEDERHANA")
        label.setStyleSheet("font-size: 18px; font-weight: bold;")
        layout.addWidget(label)

        form = QFormLayout()

        self.input_a = QSpinBox()
        self.input_a.setRange(-10000, 10000)
        form.addRow("Angka 1:", self.input_a)

        self.input_b = QSpinBox()
        self.input_b.setRange(-10000, 10000)
        form.addRow("Angka 2:", self.input_b)

        layout.addLayout(form)

        # Buttons
        btn_layout = QHBoxLayout()

        btn_add = QPushButton("+")
        btn_add.clicked.connect(lambda: self.calculate("+"))
        btn_layout.addWidget(btn_add)

        btn_sub = QPushButton("-")
        btn_sub.clicked.connect(lambda: self.calculate("-"))
        btn_layout.addWidget(btn_sub)

        btn_mul = QPushButton("×")
        btn_mul.clicked.connect(lambda: self.calculate("*"))
        btn_layout.addWidget(btn_mul)

        btn_div = QPushButton("÷")
        btn_div.clicked.connect(lambda: self.calculate("/"))
        btn_layout.addWidget(btn_div)

        layout.addLayout(btn_layout)

        self.result = QLabel("Hasil: -")
        self.result.setStyleSheet("font-size: 20px; font-weight: bold; color: #27ae60;")
        layout.addWidget(self.result)

        layout.addStretch()
        tab.setLayout(layout)
        self.tabs.addTab(tab, "Calculator")

    def create_notes_tab(self):
        tab = QWidget()
        layout = QVBoxLayout()

```

```
        label = QLabel("📝 CATATAN")
        label.setStyleSheet("font-size: 18px; font-weight: bold;")
        layout.addWidget(label)

        self.notes = QTextEdit()
        self.notes.setPlaceholderText("Tulis catatan di sini...")
        layout.addWidget(self.notes)

        btn_layout = QHBoxLayout()

        btn_clear = QPushButton("Clear")
        btn_clear.clicked.connect(self.notes.clear)
        btn_layout.addWidget(btn_clear)

        btn_copy = QPushButton("Copy All")
        btn_copy.clicked.connect(lambda: self.notes.selectAll())
        btn_layout.addWidget(btn_copy)

        layout.addLayout(btn_layout)

        tab.setLayout(layout)
        self.tabs.addTab(tab, "Notes")

    def create_settings_tab(self):
        tab = QWidget()
        layout = QVBoxLayout()

        label = QLabel("⚙️ PENGATURAN")
        label.setStyleSheet("font-size: 18px; font-weight: bold;")
        layout.addWidget(label)

        form = QFormLayout()

        self.input_name = QLineEdit()
        self.input_name.setPlaceholderText("Nama Anda")
        form.addRow("Nama:", self.input_name)

        self.input_email = QLineEdit()
        self.input_email.setPlaceholderText("email@contoh.com")
        form.addRow("Email:", self.input_email)

        layout.addLayout(form)

        btn_save = QPushButton("Simpan Pengaturan")
        btn_save.clicked.connect(self.save_settings)
        layout.addWidget(btn_save)

        self.status = QLabel("")
        layout.addWidget(self.status)

        layout.addStretch()
        tab.setLayout(layout)
        self.tabs.addTab(tab, "Settings")

    def calculate(self, op):
        a = self.input_a.value()
        b = self.input_b.value()

        try:
            if op == "+":
                result = a + b
```

```
            elif op == "-":
                result = a - b
            elif op == "*":
                result = a * b
            elif op == "/":
                result = a / b if b != 0 else "Error: Div by 0"

            self.result.setText(f"Hasil: {result}")
        except Exception as e:
            self.result.setText(f"Error: {e}")

    def save_settings(self):
        name = self.input_name.text()
        email = self.input_email.text()
        self.status.setText(f"✅ Tersimpan! Nama: {name}, Email: {email}")

app = QApplication(sys.argv)
window = TabApp()
window.show()
sys.exit(app.exec_())
```

19.2 Penjelasan Kode

Aplikasi yang memiliki banyak fitur perlu cara elegan untuk menyembunyikan kompleksitas. `QTabWidget` menyediakan tab ala browser: konten berbeda, satu jendela.

1. Empat Tab, Empat Dunia

Skrip ini membuat empat tab: Home (sambutan), Calculator (kalkulator sederhana), Notes (area tulis), dan Settings (formulir pengaturan). Setiap tab dibuat oleh metode tersendiri (`create_home_tab()`, dst.) yang mengembalikan `QWidget` berisi layout lengkap. Tab ditambahkan via `self.tabs.addTab(widget, "Label")`.

2. Kalkulator di Tab Calculator

Dua `QSpinBox` menerima angka, lalu empat tombol operasi (+, -, ×, ÷) masing-masing terhubung ke `calculate(op)` melalui lambda. Pembagian nol ditangani secara eksplisit—kebiasaan baik yang mencegah crash.

3. Catatan dan Pengaturan

Tab Notes menyediakan `QTextEdit` dengan tombol Clear dan Copy All. Tab Settings menggunakan `QFormLayout` untuk menata pasangan label-input secara rapi. Tombol Simpan memanggil `save_settings()` yang menampilkan konfirmasi di label status.

Pelajaran Penting

- `QTabWidget` memecah UI kompleks menjadi halaman-halaman yang mudah dijelajahi pengguna.
- Pisahkan pembuatan setiap tab ke metode terpisah agar konstruktor tidak membengkak.
- Setiap tab adalah `QWidget` biasa—Anda bebas mengisinya dengan layout dan widget apa pun.

19.3 Hasil Tampilan

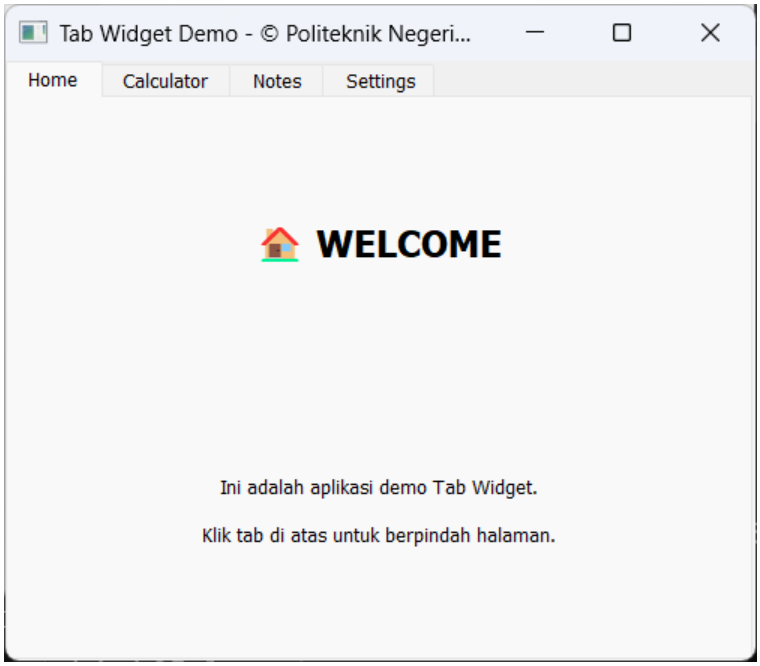

Figure 19.1: TabWidget

Chapter 20

Scroll Area

20.1 Kode Program

```python
# BAB 20: Scroll Area
# ====================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QScrollArea, QWidget, QLabel,
                            QPushButton, QVBoxLayout, QHBoxLayout)
from PyQt5.QtCore import Qt

class ScrollDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Scroll Area Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 500)

        # Scroll Area
        scroll = QScrollArea()
        scroll.setWidgetResizable(True)
        scroll.setStyleSheet("QScrollArea { border: none; }")

        # Container
        container = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("📜 KONTEN PANJANG")
        header.setStyleSheet("font-size: 24px; font-weight: bold; padding: 20px;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Banyak konten untuk demo scroll
        for i in range(50):
            item = self.create_item(i)
            layout.addWidget(item)

        container.setLayout(layout)
        scroll.setWidget(container)

        self.setCentralWidget(scroll)

    def create_item(self, index):
        widget = QWidget()
```

```
        layout = QHBoxLayout()

        # Label
        label = QLabel(f"Item #{index + 1}")
        label.setStyleSheet("font-size: 14px; font-weight: bold;")
        layout.addWidget(label)

        # Spacer
        layout.addStretch()

        # Buttons
        btn1 = QPushButton("View")
        btn1.setFixedWidth(60)
        btn1.clicked.connect(lambda checked, i=index: print(f"View item {i+1}"))
        layout.addWidget(btn1)

        btn2 = QPushButton("Delete")
        btn2.setFixedWidth(60)
        btn2.setStyleSheet("background-color: #e74c3c; color: white;")
        layout.addWidget(btn2)

        widget.setLayout(layout)
        widget.setStyleSheet("""
            QWidget {
                background-color: white;
                border: 1px solid #ddd;
                border-radius: 5px;
                padding: 5px;
                margin: 2px;
            }
            QWidget:hover {
                background-color: #f0f0f0;
            }
        """)

        return widget

app = QApplication(sys.argv)
window = ScrollDemo()
window.show()
sys.exit(app.exec_())
```

20.2 Penjelasan Kode

Bagaimana jika konten Anda lebih panjang dari tinggi jendela? Daripada memperbesar jendela tanpa batas, gunakan `QScrollArea`—bingkai yang membuat isinya bisa di-scroll vertikal maupun horizontal.

1. Membungkus Konten Panjang

Skrip membuat 50 item kartu di dalam sebuah container. Container tersebut dimasukkan ke `QScrollArea` dengan `setWidgetResizable(True)`, sehingga lebar isi mengikuti lebar scroll area. Tanpa pengaturan ini, konten bisa terpotong.

2. Kartu Item dengan Styling

Setiap item dibuat oleh `create_item(index)`: sebuah widget berisi label nomor, spacer, serta dua tombol (View dan Delete). CSS-style diterapkan langsung—border tipis, sudut membulat, dan efek hover yang mengubah warna latar. Hasilnya menyerupai daftar kartu di aplikasi modern.

3. Interaksi Tetap Responsif

Meskipun ada 50 kartu, scroll area hanya merender yang terlihat di layar (lazy rendering oleh Qt). Tombol View dihubungkan ke lambda yang mencetak indeks item—titik awal untuk menambahkan logika detail nanti.

Pelajaran Penting

- `QScrollArea` wajib dipakai ketika konten berpotensi melebihi ukuran jendela.
- Selalu aktifkan `setWidgetResizable(True)` agar konten beradaptasi.
- Styling per-item menciptakan tampilan list yang modern dan mudah dibaca.

20.3 Hasil Tampilan

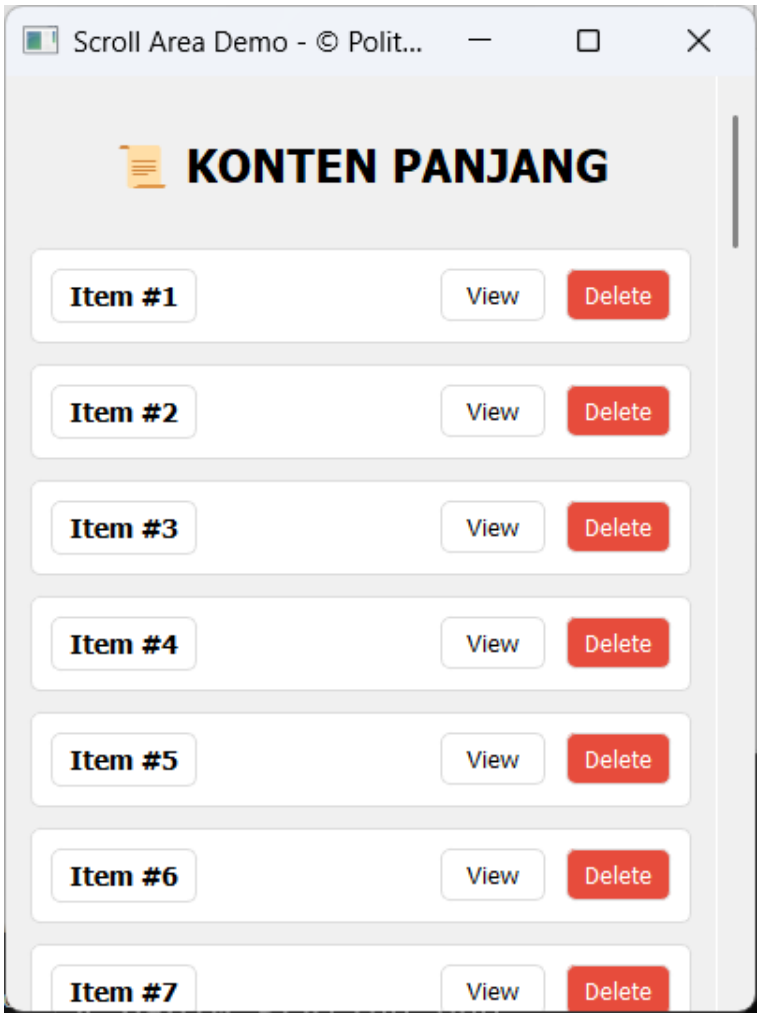

Figure 20.1: ScrollArea

Chapter 21

Splitter

21.1 Kode Program

```python
# BAB 21: Splitter
# ================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QSplitter, QListWidget, QTextEdit,
                            QTreeView, QFileSystemModel, QLabel,
                            QVBoxLayout, QWidget)
from PyQt5.QtCore import Qt, QDir

class SplitterDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Splitter Demo - File Explorer - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 800, 500)

        # Main splitter (horizontal)
        main_splitter = QSplitter(Qt.Horizontal)

        # Panel kiri: File tree
        self.tree = QTreeView()
        self.model = QFileSystemModel()
        self.model.setRootPath(QDir.rootPath())
        self.tree.setModel(self.model)
        self.tree.setRootIndex(self.model.index(QDir.homePath()))
        self.tree.setColumnHidden(1, True) # Hide size column
        self.tree.setColumnHidden(2, True) # Hide type column
        self.tree.setColumnHidden(3, True) # Hide date column
        self.tree.clicked.connect(self.file_clicked)

        main_splitter.addWidget(self.tree)

        # Panel kanan: Vertical splitter
        right_splitter = QSplitter(Qt.Vertical)

        # Preview area
        self.preview = QTextEdit()
        self.preview.setReadOnly(True)
        self.preview.setPlaceholderText("Pilih file untuk melihat preview...")
        right_splitter.addWidget(self.preview)

```

```
        # Info area
        info_widget = QWidget()
        info_layout = QVBoxLayout()

        info_label = QLabel("📁 INFO FILE")
        info_label.setStyleSheet("font-weight: bold;")
        info_layout.addWidget(info_label)

        self.info = QLabel("Pilih file di panel kiri")
        self.info.setWordWrap(True)
        info_layout.addWidget(self.info)

        info_widget.setLayout(info_layout)
        right_splitter.addWidget(info_widget)

        # Set ukuran proporsi
        right_splitter.setSizes([350, 100])

        main_splitter.addWidget(right_splitter)

        # Set ukuran proporsi
        main_splitter.setSizes([200, 600])

        self.setCentralWidget(main_splitter)

    def file_clicked(self, index):
        path = self.model.filePath(index)
        is_dir = self.model.isDir(index)

        if is_dir:
            self.preview.setPlainText(f"📁 Folder: {path}")
            self.info.setText(f"Jenis: Folder\nPath: {path}")
        else:
            # Coba baca file teks
            try:
                with open(path, 'r', encoding='utf-8', errors='ignore') as f:
                    content = f.read(5000) # Limit 5000 chars
                    self.preview.setPlainText(content)
            except:
                self.preview.setPlainText(f"❌ Tidak dapat membaca file: {path}")

            # Info file
            file_info = self.model.fileInfo(index)
            size = file_info.size()
            ext = file_info.suffix()

            self.info.setText(f"Nama: {file_info.fileName()}\n"
                              f"Ukuran: {size:,} bytes\n"
                              f"Ekstensi: {ext if ext else '-'}\n"
                              f"Path: {path}")

app = QApplication(sys.argv)
window = SplitterDemo()
window.show()
sys.exit(app.exec_())
```

21.2 Penjelasan Kode

Bayangkan jendela yang bisa Anda bagi dua dengan menyeret garis pembatas—itulah `QSplitter`. Pengguna bebas mengatur proporsi panel kiri dan kanan sesuai kebutuhan, tanpa harus me-resize seluruh jendela.

1. Splitter Horizontal: Tree + Preview

Splitter utama membagi jendela secara horizontal. Panel kiri berisi `QTreeView` yang menampilkan struktur file sistem menggunakan `QFileSystemModel`. Kolom ukuran, tipe, dan tanggal disembunyikan agar tampilan ringkas. Signal `clicked` menghubungkan klik pada file ke metode `file_clicked()`.

2. Splitter Vertikal di Dalam Splitter

Panel kanan sendiri dibagi lagi secara vertikal: area atas untuk preview isi file (`QTextEdit` read-only) dan area bawah untuk informasi file (nama, ukuran, ekstensi, path). Teknik *nested splitter* ini lazim ditemukan di IDE dan file manager profesional.

3. Membaca File Teks

Saat pengguna mengklik file, `file_clicked()` mencoba membacanya dengan encoding UTF-8 (dibatasi 5000 karakter). Folder hanya menampilkan path. Pendekatan `try-except` memastikan file biner atau terproteksi tidak menyebabkan crash.

4. Proporsi Awal

`setSizes([200, 600])` mengatur lebar awal panel. Pengguna bisa mengubahnya kapan saja dengan menyeret garis splitter.

Pelajaran Penting

- `QSplitter` memberi pengguna kendali atas tata letak tanpa kode tambahan.
- Splitter bisa di-nest—horizontal di dalam vertikal, atau sebaliknya.
- `QFileSystemModel` + `QTreeView` adalah cara tercepat menampilkan isi disk.

21.3 Hasil Tampilan

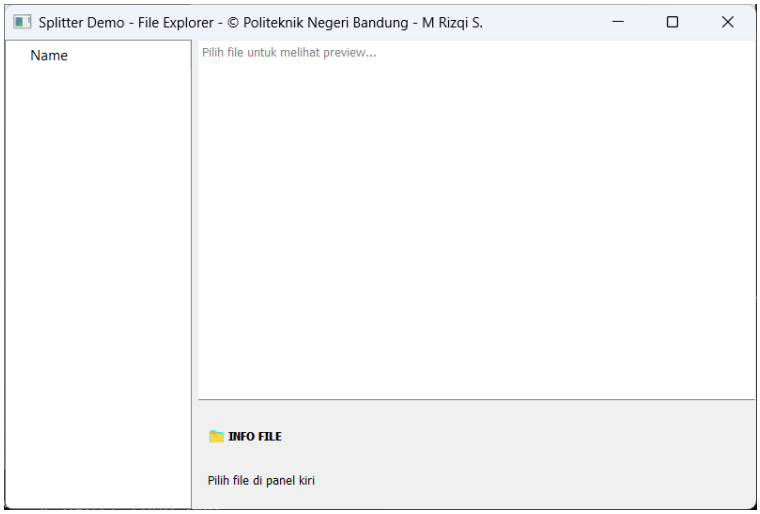

Figure 21.1: Splitter

Chapter 22

Dialog Standar

22.1 Kode Program

```python
# BAB 22: Dialog Standar
# =======================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QPushButton, QLabel, QVBoxLayout,
                            QWidget, QFileDialog, QColorDialog,
                            QFontDialog, QInputDialog, QMessageBox)
from PyQt5.QtCore import Qt

class DialogDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Dialog Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 450)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("💬 DIALOG STANDAR")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # File Dialog
        layout.addWidget(QLabel("\📁n File Dialog:"))

        btn_open = QPushButton("Buka File...")
        btn_open.clicked.connect(self.open_file)
        layout.addWidget(btn_open)

        btn_save = QPushButton("Simpan File...")
        btn_save.clicked.connect(self.save_file)
        layout.addWidget(btn_save)

        btn_folder = QPushButton("Pilih Folder...")
        btn_folder.clicked.connect(self.select_folder)
        layout.addWidget(btn_folder)

        # Color Dialog
        layout.addWidget(QLabel("\🎨n Color Dialog:"))
```

```
        btn_color = QPushButton("Pilih Warna...")
        btn_color.clicked.connect(self.select_color)
        layout.addWidget(btn_color)

        # Font Dialog
        layout.addWidget(QLabel("\🔤n Font Dialog:"))
        btn_font = QPushButton("Pilih Font...")
        btn_font.clicked.connect(self.select_font)
        layout.addWidget(btn_font)

        # Input Dialog
        layout.addWidget(QLabel("\✏️n Input Dialog:"))
        btn_text = QPushButton("Input Teks...")
        btn_text.clicked.connect(self.input_text)
        layout.addWidget(btn_text)

        btn_int = QPushButton("Input Angka...")
        btn_int.clicked.connect(self.input_int)
        layout.addWidget(btn_int)

        btn_item = QPushButton("Pilih Item...")
        btn_item.clicked.connect(self.select_item)
        layout.addWidget(btn_item)

        # Result
        self.result = QLabel("Hasil akan muncul di sini")
        self.result.setStyleSheet("padding: 10px; background: #f0f0f0; border-radius: 5px;")
        self.result.setWordWrap(True)
        layout.addWidget(self.result)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def open_file(self):
        file_path, _ = QFileDialog.getOpenFileName(
            self, "Buka File", "",
            "Text Files (*.txt);;Python Files (*.py);;All Files (*)"
        )
        if file_path:
            self.result.setText(f"📁 File dipilih:\n{file_path}")

    def save_file(self):
        file_path, _ = QFileDialog.getSaveFileName(
            self, "Simpan File", "",
            "Text Files (*.txt);;All Files (*)"
        )
        if file_path:
            self.result.setText(f"💾 Simpan ke:\n{file_path}")

    def select_folder(self):
        folder = QFileDialog.getExistingDirectory(self, "Pilih Folder")
        if folder:
            self.result.setText(f"📂 Folder dipilih:\n{folder}")

    def select_color(self):
        color = QColorDialog.getColor()
        if color.isValid():
            self.result.setText(f"🎨 Warna: RGB({color.red()}, {color.green()}, {color.blue()})")
            self.result.setStyleSheet(f"background-color: {color.name()}; padding: 10px;")

    def select_font(self):
```

```
        font, ok = QFontDialog.getFont()
        if ok:
            self.result.setText(f"🔤 Font: {font.family()}, Size: {font.pointSize()}")
            self.result.setFont(font)

    def input_text(self):
        text, ok = QInputDialog.getText(self, "Input Teks", "Masukkan nama Anda:")
        if ok:
            self.result.setText(f"✏️ Nama: {text}")

    def input_int(self):
        num, ok = QInputDialog.getInt(self, "Input Angka", "Masukkan umur:", min=0, max=100)
        if ok:
            self.result.setText(f"🔢 Umur: {num}")

    def select_item(self):
        items = ["Java", "Python", "JavaScript", "C++", "Go", "Rust"]
        item, ok = QInputDialog.getItem(self, "Pilih Item", "Bahasa favorit:", items, 0, False)
        if ok:
            self.result.setText(f"📝 Bahasa: {item}")

app = QApplication(sys.argv)
window = DialogDemo()
window.show()
sys.exit(app.exec_())
```

22.2 Penjelasan Kode

Dialog adalah jendela pop-up yang meminta perhatian pengguna sebelum melanjutkan. Qt menyediakan banyak dialog bawaan sehingga Anda tidak perlu membangun dari nol.

1. File Dialog

Tiga tombol mendemonstrasikan `QFileDialog`: `getOpenFileName` untuk memilih file, `getSaveFileName` untuk menentukan lokasi simpan, dan `getExistingDirectory` untuk memilih folder. Masing-masing mengembalikan path yang dipilih beserta filter yang digunakan.

2. Color Dialog

`QColorDialog.getColor()` membuka roda warna standar OS. Jika pengguna memilih warna valid, label hasil berubah warna latar sesuai pilihan—demonstrasi visual yang langsung terasa.

3. Font Dialog

`QFontDialog.getFont()` membuka dialog pemilihan font. Label hasil kemudian menampilkan nama dan ukuran font, sekaligus menerapkan font tersebut ke teksnya sendiri.

4. Input Dialog

Tiga varian `QInputDialog`: `getText` untuk teks bebas, `getInt` untuk bilangan bulat (dengan batas min/max), dan `getItem` untuk memilih dari daftar. Semua mengembalikan tuple `(value, ok)` sehingga Anda bisa mengecek apakah pengguna menekan OK atau Cancel.

Pelajaran Penting

- Dialog bawaan Qt konsisten dengan tampilan OS—pengguna langsung familiar.
- Selalu periksa nilai `ok` sebelum memproses hasil dialog.
- Filter file (`"Text Files (*.txt);;All Files (*)"`) membantu pengguna menemukan file yang relevan.

22.3 Hasil Tampilan

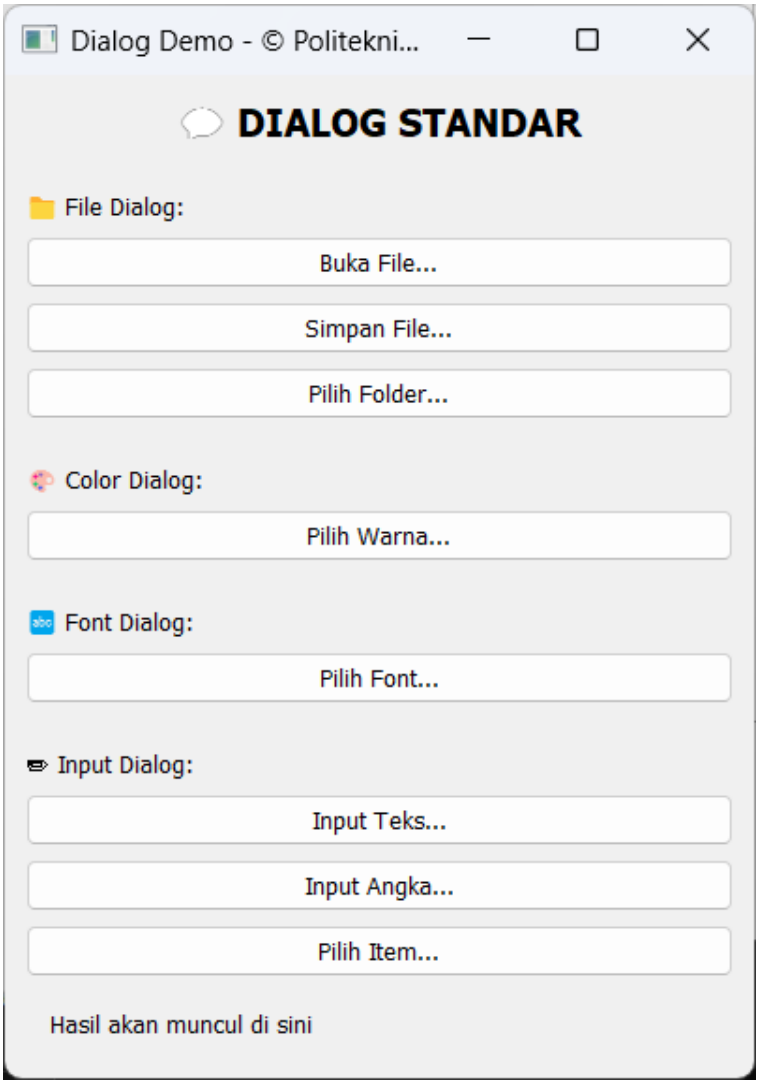

Figure 22.1: Dialog Standar

Chapter 23

Timer

23.1 Kode Program

```python
# BAB 23: Timer
# ==============

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLabel, QPushButton, QVBoxLayout,
                            QHBoxLayout, QWidget)
from PyQt5.QtCore import QTimer, QDateTime, Qt

class TimerDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Timer Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 350, 350)

        self.stopwatch_running = False
        self.stopwatch_ms = 0

        widget = QWidget()
        layout = QVBoxLayout()

        # ===== CLOCK (Timer periodik) =====
        clock_label = QLabel("⏰ REALTIME CLOCK")
        clock_label.setStyleSheet("font-size: 16px; font-weight: bold;")
        layout.addWidget(clock_label)

        self.clock = QLabel("00:00:00")
        self.clock.setStyleSheet("font-size: 36px; font-weight: bold; color: #2c3e50;")
        self.clock.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.clock)

        # Timer untuk clock (update setiap 1 detik)
        self.clock_timer = QTimer()
        self.clock_timer.timeout.connect(self.update_clock)
        self.clock_timer.start(1000)
        self.update_clock()

        layout.addSpacing(20)

        # ===== STOPWATCH =====
        sw_label = QLabel("⏱️ STOPWATCH")
        sw_label.setStyleSheet("font-size: 16px; font-weight: bold;")
```

```
        layout.addWidget(sw_label)

        self.stopwatch = QLabel("00:00:00.000")
        self.stopwatch.setStyleSheet("font-size: 36px; font-weight: bold; color: #e74c3c;")
        self.stopwatch.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.stopwatch)

        # Stopwatch buttons
        sw_btn_layout = QHBoxLayout()

        self.btn_start = QPushButton("Start")
        self.btn_start.clicked.connect(self.toggle_stopwatch)
        sw_btn_layout.addWidget(self.btn_start)

        self.btn_reset = QPushButton("Reset")
        self.btn_reset.clicked.connect(self.reset_stopwatch)
        sw_btn_layout.addWidget(self.btn_reset)

        layout.addLayout(sw_btn_layout)

        # Timer untuk stopwatch (update setiap 1 ms)
        self.sw_timer = QTimer()
        self.sw_timer.timeout.connect(self.update_stopwatch)

        layout.addSpacing(20)

        # ===== COUNTDOWN =====
        cd_label = QLabel("⏳ COUNTDOWN (Single Shot)")
        cd_label.setStyleSheet("font-size: 16px; font-weight: bold;")
        layout.addWidget(cd_label)

        self.countdown = QLabel("Klik tombol untuk mulai")
        self.countdown.setStyleSheet("font-size: 18px; color: #27ae60;")
        self.countdown.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.countdown)

        btn_countdown = QPushButton("Mulai Countdown 5 Detik")
        btn_countdown.clicked.connect(self.start_countdown)
        layout.addWidget(btn_countdown)

        self.countdown_value = 0

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def update_clock(self):
        now = QDateTime.currentDateTime()
        self.clock.setText(now.toString("HH:mm:ss"))

    def toggle_stopwatch(self):
        if self.stopwatch_running:
            self.sw_timer.stop()
            self.btn_start.setText("Start")
            self.stopwatch_running = False
        else:
            self.sw_timer.start(1) # Update setiap 1ms
            self.btn_start.setText("Stop")
            self.stopwatch_running = True

    def update_stopwatch(self):
        self.stopwatch_ms += 1
```

```
        ms = self.stopwatch_ms % 1000
        total_sec = self.stopwatch_ms // 1000
        sec = total_sec % 60
        min = (total_sec // 60) % 60
        hour = total_sec // 3600

        self.stopwatch.setText(f"{hour:02d}:{min:02d}:{sec:02d}.{ms:03d}")

    def reset_stopwatch(self):
        self.sw_timer.stop()
        self.stopwatch_running = False
        self.stopwatch_ms = 0
        self.stopwatch.setText("00:00:00.000")
        self.btn_start.setText("Start")

    def start_countdown(self):
        self.countdown_value = 5
        self.countdown.setText(f"Countdown: {self.countdown_value}")
        self.do_countdown()

    def do_countdown(self):
        if self.countdown_value > 0:
            self.countdown.setText(f"Countdown: {self.countdown_value}")
            self.countdown_value -= 1
            QTimer.singleShot(1000, self.do_countdown)
        else:
            self.countdown.setText("🎉 SELESAI!")

app = QApplication(sys.argv)
window = TimerDemo()
window.show()
sys.exit(app.exec_())
```

23.2 Penjelasan Kode

Banyak aplikasi membutuhkan sesuatu yang terjadi secara berkala—jam digital, animasi, auto-save. `QTimer` adalah detak jantung yang menggerakkan semua itu tanpa memblokir UI.

1. Realtime Clock (Timer Periodik)

Timer pertama (`clock_timer`) diatur dengan interval 1000 ms. Setiap detik, signal `timeout` memanggil `update_clock()` yang membaca waktu saat ini via `QDateTime.currentDateTime()` dan menampilkannya di label besar. Hasilnya: jam digital yang hidup.

2. Stopwatch (Timer Cepat)

Timer kedua berjalan setiap 1 ms untuk mensimulasikan stopwatch presisi tinggi. Tombol Start/Stop mentoggle timer, sementara Reset mengembalikan hitungan ke nol. Variabel `stopwatch_ms` dikonversi ke format `HH:MM:SS.mmm`—aritmetika sederhana namun hasilnya mengesankan.

3. Countdown (Single Shot)

`QTimer.singleShot(ms, callback)` menjalankan fungsi tepat sekali setelah delay tertentu. Skrip menggunakannya secara rekursif: setiap detik memanggil `do_countdown()` yang mengurangi penghitung, lalu menjadwalkan dirinya sendiri 1 detik kemudian. Saat mencapai nol, label berubah menjadi "🎉 SELESAI!".

Pelajaran Penting

- `QTimer` periodik cocok untuk update terus-menerus (jam, animasi).
- `singleShot` cocok untuk delay satu kali (splash screen, notifikasi tertunda).
- Timer Qt berjalan di event loop utama—UI tetap responsif tanpa threading.

23.3 Hasil Tampilan

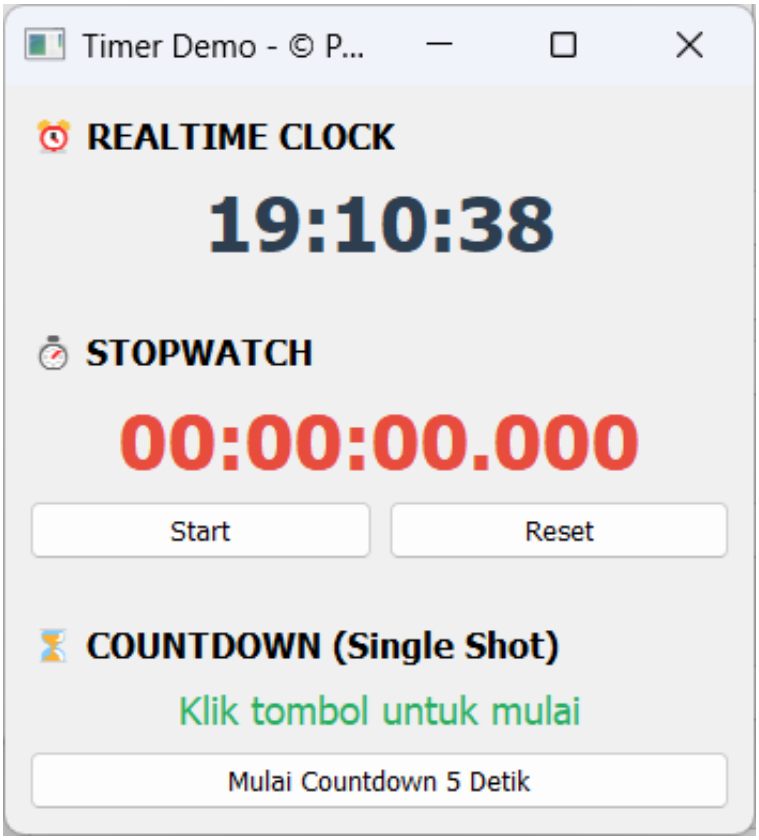

Figure 23.1: QTimer

Chapter 24

Settings: Menyimpan Preferensi

24.1 Kode Program

```python
# BAB 24: Settings
# =================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLineEdit, QCheckBox, QSpinBox,
                            QComboBox, QPushButton, QLabel,
                            QVBoxLayout, QWidget, QFormLayout)
from PyQt5.QtCore import QSettings

class SettingsDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Settings Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 400)

        # QSettings menyimpan ke registry (Windows) atau file config (Linux/Mac)
        self.settings = QSettings("MyCompany", "MyApp")

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("⚙️ PENGATURAN APLIKASI")
        header.setStyleSheet("font-size: 18px; font-weight: bold;")
        layout.addWidget(header)

        layout.addWidget(QLabel("Pengaturan akan tersimpan secara otomatis\ndan dimuat saat
            aplikasi dibuka kembali."))

        # Form
        form = QFormLayout()

        # Username
        self.input_username = QLineEdit()
        self.input_username.setPlaceholderText("Masukkan username")
        form.addRow("Username:", self.input_username)

        # Email
        self.input_email = QLineEdit()
        self.input_email.setPlaceholderText("email@contoh.com")
        form.addRow("Email:", self.input_email)
```

```
        # Theme
        self.combo_theme = QComboBox()
        self.combo_theme.addItems(["Light", "Dark", "System"])
        form.addRow("Theme:", self.combo_theme)

        # Font size
        self.spin_fontsize = QSpinBox()
        self.spin_fontsize.setRange(8, 32)
        self.spin_fontsize.setValue(12)
        form.addRow("Font Size:", self.spin_fontsize)

        # Remember me
        self.cb_remember = QCheckBox("Ingat saya")
        form.addRow("", self.cb_remember)

        # Notifications
        self.cb_notify = QCheckBox("Aktifkan notifikasi")
        self.cb_notify.setChecked(True)
        form.addRow("", self.cb_notify)

        layout.addLayout(form)

        # Buttons
        btn_layout = QVBoxLayout()

        btn_save = QPushButton("💾 Simpan Pengaturan")
        btn_save.setStyleSheet("background-color: #27ae60; color: white; padding: 10px;")
        btn_save.clicked.connect(self.save_settings)
        btn_layout.addWidget(btn_save)

        btn_load = QPushButton("📂 Muat Pengaturan")
        btn_load.setStyleSheet("background-color: #3498db; color: white; padding: 10px;")
        btn_load.clicked.connect(self.load_settings)
        btn_layout.addWidget(btn_load)

        btn_clear = QPushButton("🗑️ Hapus Semua Pengaturan")
        btn_clear.setStyleSheet("background-color: #e74c3c; color: white; padding: 10px;")
        btn_clear.clicked.connect(self.clear_settings)
        btn_layout.addWidget(btn_clear)

        layout.addLayout(btn_layout)

        # Status
        self.status = QLabel("")
        self.status.setStyleSheet("padding: 10px; background: #f0f0f0;")
        layout.addWidget(self.status)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

        # Auto load saat startup
        self.load_settings()

    def save_settings(self):
        self.settings.setValue("username", self.input_username.text())
        self.settings.setValue("email", self.input_email.text())
        self.settings.setValue("theme", self.combo_theme.currentText())
        self.settings.setValue("fontsize", self.spin_fontsize.value())
        self.settings.setValue("remember", self.cb_remember.isChecked())
        self.settings.setValue("notify", self.cb_notify.isChecked())
```

```
        self.status.setText("✅ Pengaturan berhasil disimpan!")
        print(f"Settings saved to: {self.settings.fileName()}")

    def load_settings(self):
        username = self.settings.value("username", "", type=str)
        email = self.settings.value("email", "", type=str)
        theme = self.settings.value("theme", "Light", type=str)
        fontsize = self.settings.value("fontsize", 12, type=int)
        remember = self.settings.value("remember", False, type=bool)
        notify = self.settings.value("notify", True, type=bool)

        self.input_username.setText(username)
        self.input_email.setText(email)

        index = self.combo_theme.findText(theme)
        if index >= 0:
            self.combo_theme.setCurrentIndex(index)

        self.spin_fontsize.setValue(fontsize)
        self.cb_remember.setChecked(remember)
        self.cb_notify.setChecked(notify)

        self.status.setText("✅ Pengaturan berhasil dimuat!")

    def clear_settings(self):
        self.settings.clear()

        # Reset form
        self.input_username.clear()
        self.input_email.clear()
        self.combo_theme.setCurrentIndex(0)
        self.spin_fontsize.setValue(12)
        self.cb_remember.setChecked(False)
        self.cb_notify.setChecked(True)

        self.status.setText("🗑️ Semua pengaturan telah dihapus!")

app = QApplication(sys.argv)
window = SettingsDemo()
window.show()
sys.exit(app.exec_())
```

24.2 Penjelasan Kode

Aplikasi profesional mengingat preferensi pengguna: username, tema, ukuran font. `QSettings` menyimpan data ini secara persisten—di registry Windows atau file config di Linux/Mac—tanpa Anda perlu mengelola file secara manual.

1. Inisialisasi QSettings

`QSettings("MyCompany", "MyApp")` membuat namespace unik. Qt otomatis memilih lokasi penyimpanan sesuai OS. Anda tidak perlu tahu di mana file disimpan; cukup baca dan tulis.

2. Formulir Pengaturan

Form terdiri dari `QLineEdit` (username, email), `QComboBox` (tema), `QSpinBox` (ukuran font), dan dua `QCheckBox` (ingat saya, notifikasi). Semua ditata rapi dengan `QFormLayout`.

3. Simpan, Muat, dan Hapus

`save_settings()` menulis setiap nilai dengan `setValue(key, value)`. `load_settings()` membacanya kembali dengan `value(key, default, type=...)`; parameter `type` penting agar Qt mengembalikan tipe data yang benar (bukan string mentah). `clear_settings()` menghapus semua data dan mereset form ke

kondisi awal.

4. Auto-Load Saat Startup

Di akhir konstruktor, `load_settings()` dipanggil otomatis. Artinya, begitu aplikasi dibuka, semua preferensi terisi tanpa pengguna menyentuh apa pun—pengalaman yang mulus dan profesional.

Pelajaran Penting

- `QSettings` abstrak penyimpanan lintas platform; Anda tulis sekali, jalan di mana saja.
- Selalu sediakan nilai default agar aplikasi tidak error saat pertama kali dijalankan.
- Tombol reset/hapus pengaturan adalah praktik UX yang baik untuk mengembalikan kondisi awal.

24.3 Hasil Tampilan

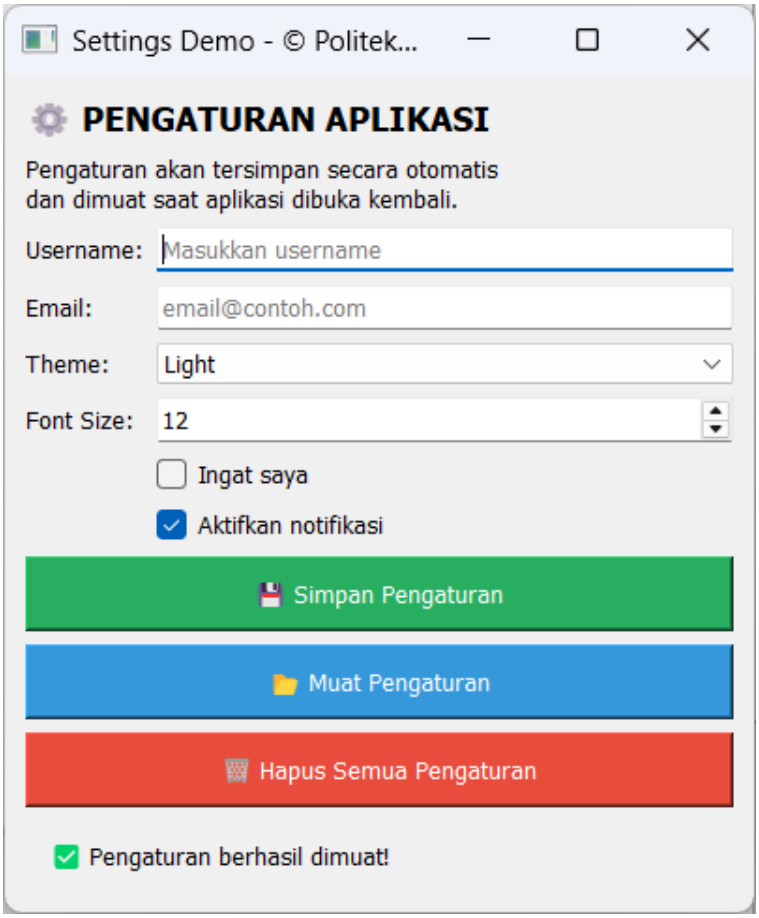

Figure 24.1: QSettings

Chapter 25

Clipboard

25.1 Kode Program

```python
# BAB 25: Clipboard
# ==================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QTextEdit, QPushButton, QLabel,
                            QVBoxLayout, QHBoxLayout, QWidget)
from PyQt5.QtCore import Qt

class ClipboardDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Clipboard Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 400)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("📋 CLIPBOARD MANAGER")
        header.setStyleSheet("font-size: 18px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Source text
        layout.addWidget(QLabel("Teks Sumber:"))
        self.source = QTextEdit()
        self.source.setPlaceholderText("Ketik atau paste teks di sini...")
        self.source.setText("Halo! Ini adalah contoh teks untuk dicopy ke clipboard.")
        layout.addWidget(self.source)

        # Buttons
        btn_layout = QHBoxLayout()

        btn_copy = QPushButton("📋 Copy")
        btn_copy.clicked.connect(self.copy_text)
        btn_copy.setStyleSheet("padding: 10px;")
        btn_layout.addWidget(btn_copy)

        btn_cut = QPushButton("✂️ Cut")
        btn_cut.clicked.connect(self.cut_text)
        btn_cut.setStyleSheet("padding: 10px;")
```

```
        btn_layout.addWidget(btn_cut)

        btn_paste = QPushButton("📥 Paste")
        btn_paste.clicked.connect(self.paste_text)
        btn_paste.setStyleSheet("padding: 10px;")
        btn_layout.addWidget(btn_paste)

        btn_clear = QPushButton("🗑️ Clear")
        btn_clear.clicked.connect(self.clear_text)
        btn_clear.setStyleSheet("padding: 10px;")
        btn_layout.addWidget(btn_clear)

        layout.addLayout(btn_layout)

        # Destination text
        layout.addWidget(QLabel("Teks Tujuan (Paste di sini):"))
        self.dest = QTextEdit()
        self.dest.setPlaceholderText("Paste teks di sini...")
        layout.addWidget(self.dest)

        # Clipboard info
        self.info = QLabel("Info: -")
        self.info.setStyleSheet("padding: 10px; background: #f0f0f0;")
        layout.addWidget(self.info)

        # Monitor clipboard
        btn_monitor = QPushButton("🔍 Cek Clipboard Saat Ini")
        btn_monitor.clicked.connect(self.check_clipboard)
        layout.addWidget(btn_monitor)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

        # Connect clipboard changes
        clipboard = QApplication.clipboard()
        clipboard.dataChanged.connect(self.on_clipboard_change)

    def copy_text(self):
        clipboard = QApplication.clipboard()
        text = self.source.textCursor().selectedText()
        if not text:
            text = self.source.toPlainText()
        clipboard.setText(text)
        self.info.setText(f"✅ Copied: {len(text)} karakter")

    def cut_text(self):
        clipboard = QApplication.clipboard()
        text = self.source.textCursor().selectedText()
        if text:
            clipboard.setText(text)
            self.source.textCursor().removeSelectedText()
        else:
            text = self.source.toPlainText()
            clipboard.setText(text)
            self.source.clear()
        self.info.setText(f"✂️ Cut: {len(text)} karakter")

    def paste_text(self):
        clipboard = QApplication.clipboard()
        text = clipboard.text()
        self.dest.setPlainText(text)
```

```
        self.info.setText(f"📥 Pasted: {len(text)} karakter")

    def clear_text(self):
        self.source.clear()
        self.dest.clear()
        self.info.setText("🗑️ Cleared")

    def check_clipboard(self):
        clipboard = QApplication.clipboard()
        text = clipboard.text()
        if text:
            preview = text[:100] + "..." if len(text) > 100 else text
            self.info.setText(f"📋 Clipboard: {len(text)} karakter\nPreview: {preview}")
        else:
            self.info.setText("📋 Clipboard kosong")

    def on_clipboard_change(self):
        self.info.setText("📋 Clipboard berubah!")

app = QApplication(sys.argv)
window = ClipboardDemo()
window.show()
sys.exit(app.exec_())
```

25.2 Penjelasan Kode

Copy-paste adalah fitur yang begitu fundamental sehingga sering dianggap remeh. Di baliknya, sistem operasi menyediakan *clipboard*—papan klip virtual. Qt mengaksesnya melalui `QApplication.clipboard()`.

1. Dua Area Teks

Skrip menyiapkan dua `QTextEdit`: sumber (tempat menulis/menempelkan teks) dan tujuan (tempat paste). Pemisahan ini memperjelas alur data: dari sumber → clipboard → tujuan.

2. Copy, Cut, Paste, Clear

`copy_text()` mengambil teks terseleksi (atau seluruh teks jika tidak ada seleksi) lalu menaruhnya di clipboard via `setText()`. `cut_text()` melakukan hal serupa namun juga menghapus teks dari sumber. `paste_text()` membaca clipboard dan menampilkannya di area tujuan. Label info menunjukkan jumlah karakter yang dioperasikan.

3. Memonitor Perubahan Clipboard

Signal `dataChanged` dari clipboard dihubungkan ke `on_clipboard_change()`. Setiap kali clipboard berubah—entah dari aplikasi ini atau aplikasi lain—label info akan memperbarui pesannya. Tombol "Cek Clipboard" membaca isi clipboard saat itu dan menampilkan preview hingga 100 karakter pertama.

Pelajaran Penting

- `QApplication.clipboard()` memberi akses ke clipboard sistem—lintas aplikasi.
- Selalu cek apakah ada teks terseleksi sebelum copy/cut untuk perilaku yang intuitif.
- Signal `dataChanged` memungkinkan aplikasi bereaksi terhadap perubahan clipboard dari luar.

25.3 Hasil Tampilan

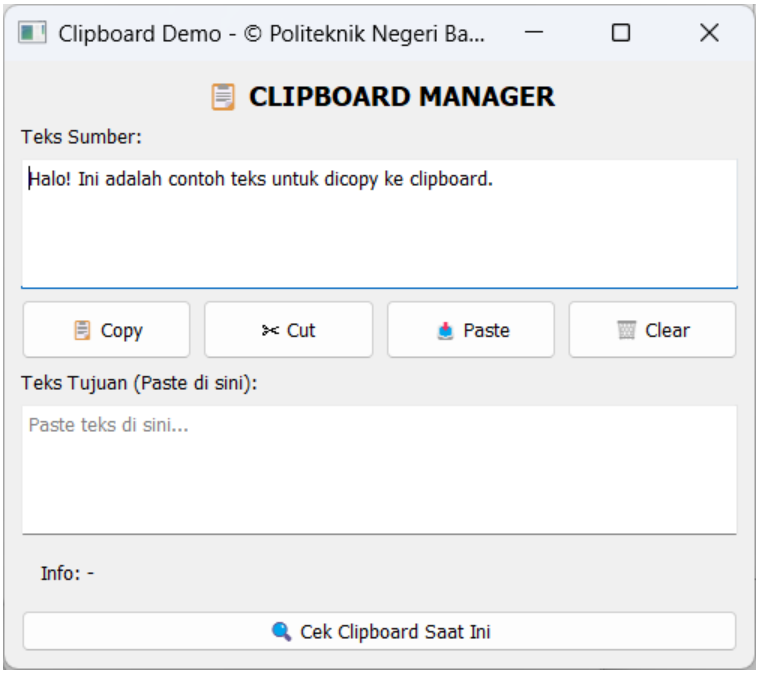

Figure 25.1: Clipboard

Chapter 26

Proyek Mini: Kalkulator

26.1 Kode Program

```
# BAB 26: Calculator App
# =======================
# Aplikasi kalkulator lengkap dengan CSS styling

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLineEdit, QPushButton, QGridLayout,
                            QWidget, QVBoxLayout)
from PyQt5.QtCore import Qt

class Calculator(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Calculator - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 320, 450)

        self.expression = ""

        # CSS Styling
        self.setStyleSheet("""
            QMainWindow {
                background-color: #1a1a2e;
            }
            QLineEdit {
                background-color: #16213e;
                color: #eee;
                font-size: 32px;
                padding: 15px;
                border: none;
                border-radius: 10px;
            }
            QPushButton {
                font-size: 22px;
                border-radius: 35px;
                min-height: 70px;
                min-width: 70px;
            }
            QPushButton#number {
                background-color: #0f3460;
                color: white;
            }
            QPushButton#number:hover {
```

```
            background-color: #1a4a7a;
        }
        QPushButton#op {
            background-color: #e94560;
            color: white;
        }
        QPushButton#op:hover {
            background-color: #ff6b6b;
        }
        QPushButton#func {
            background-color: #34836F;
            color: white;
        }
        QPushButton#func:hover {
            background-color: #4A91A8;
        }
        QPushButton#eq {
            background-color: #00b894;
            color: white;
        }
        QPushButton#eq:hover {
            background-color: #00d9a5;
        }
        QPushButton#clear {
            background-color: #d63031;
            color: white;
        }
        QPushButton#clear:hover {
            background-color: #ff4757;
        }
    """)

    widget = QWidget()
    layout = QVBoxLayout()

    # Display
    self.display = QLineEdit()
    self.display.setAlignment(Qt.AlignRight)
    self.display.setReadOnly(True)
    self.display.setText("0")
    layout.addWidget(self.display)

    # Buttons grid
    grid = QGridLayout()
    grid.setSpacing(10)

    # Row 0: Clear and operations
    buttons = [
        # (text, row, col, id)
        ("C", 0, 0, "clear"),
        ("±", 0, 1, "func"),
        ("%", 0, 2, "func"),
        ("÷", 0, 3, "op"),

        # Row 1
        ("7", 1, 0, "number"),
        ("8", 1, 1, "number"),
        ("9", 1, 2, "number"),
        ("×", 1, 3, "op"),

        # Row 2
```

```
104             ("4", 2, 0, "number"),
105             ("5", 2, 1, "number"),
106             ("6", 2, 2, "number"),
107             ("-", 2, 3, "op"),
108 
109             # Row 3
110             ("1", 3, 0, "number"),
111             ("2", 3, 1, "number"),
112             ("3", 3, 2, "number"),
113             ("+", 3, 3, "op"),
114 
115             # Row 4
116             ("0", 4, 0, "number"),
117             (".", 4, 1, "number"),
118             (" ", 4, 2, "func"),
119             ("=", 4, 3, "eq"),
120         ]
121 
122         for text, row, col, btn_id in buttons:
123             btn = QPushButton(text)
124             btn.setObjectName(btn_id)
125             btn.clicked.connect(lambda checked, t=text: self.on_click(t))
126             grid.addWidget(btn, row, col)
127 
128         layout.addLayout(grid)
129 
130         widget.setLayout(layout)
131         self.setCentralWidget(widget)
132 
133     def on_click(self, text):
134         if text == "C":
135             self.expression = ""
136             self.display.setText("0")
137 
138         elif text == " ": # Backspace
139             self.expression = self.expression[:-1]
140             self.display.setText(self.expression if self.expression else "0")
141 
142         elif text == "=":
143             try:
144                 # Replace symbols untuk eval
145                 expr = self.expression.replace("×", "*").replace("÷", "/")
146                 result = eval(expr)
147 
148                 # Format result
149                 if isinstance(result, float):
150                     if result.is_integer():
151                         result = int(result)
152                     else:
153                         result = round(result, 8)
154 
155                 self.display.setText(str(result))
156                 self.expression = str(result)
157             except ZeroDivisionError:
158                 self.display.setText("Error: Div by 0")
159                 self.expression = ""
160             except Exception as e:
161                 self.display.setText("Error")
162                 self.expression = ""
163 
164         elif text == "±": # Plus/minus
```

```
            if self.expression:
                if self.expression[0] == "-":
                    self.expression = self.expression[1:]
                else:
                    self.expression = "-" + self.expression
                self.display.setText(self.expression)

        elif text == "%": # Percent
            try:
                result = float(self.expression) / 100
                self.display.setText(str(result))
                self.expression = str(result)
            except:
                pass

        else:
            # Angka atau operator
            self.expression += text
            self.display.setText(self.expression)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    calc = Calculator()
    calc.show()
    sys.exit(app.exec_())
```

26.2 Penjelasan Kode

Kalkulator adalah proyek klasik untuk menguji pemahaman layout dan logika event. Skrip ini membangun kalkulator bergaya modern dengan `QGridLayout`, lengkap dengan styling CSS.

1. Display dengan QLineEdit

`QLineEdit` diatur read-only dan rata kanan sehingga berperilaku seperti layar kalkulator sungguhan. Font besar (28px) dan latar gelap memberikan kesan profesional.

2. Tombol Grid

Semua tombol (angka 0–9, operator +–*/%, titik desimal, backspace, clear, equal) ditata dalam `QGridLayout`. Setiap tombol dibuat lewat helper `create_button(text, row, col)` yang langsung menghubungkan signal `clicked` ke `on_click()` melalui lambda. Hasilnya: 20+ tombol hanya dalam beberapa baris kode.

3. Mesin Kalkulasi

Logika inti berada di `on_click()`. Tombol angka dan titik desimal menambahkan karakter ke `current_input`. Operator (+, –, *, /) menyimpan angka pertama dan operator, lalu mereset display. Tombol "=" memanggil `calculate()` yang menjalankan operasi dan menampilkan hasil. Pembagian nol ditangani secara eksplisit dengan pesan "Error".

4. Fitur Tambahan

- `%` mengonversi angka saat ini menjadi persentase (dibagi 100).
- `+/-` mengubah tanda positif/negatif.
- Backspace menghapus karakter terakhir; C mereset seluruh state.

Pelajaran Penting

- `QGridLayout` sempurna untuk tata letak berbasis baris-kolom seperti keypad.
- Lambda di `connect()` memungkinkan satu slot menangani banyak tombol berbeda.
- Pisahkan state (`current_input`, `first_number`, `operator`) dari UI agar logika lebih testable.

26.3 Hasil Tampilan

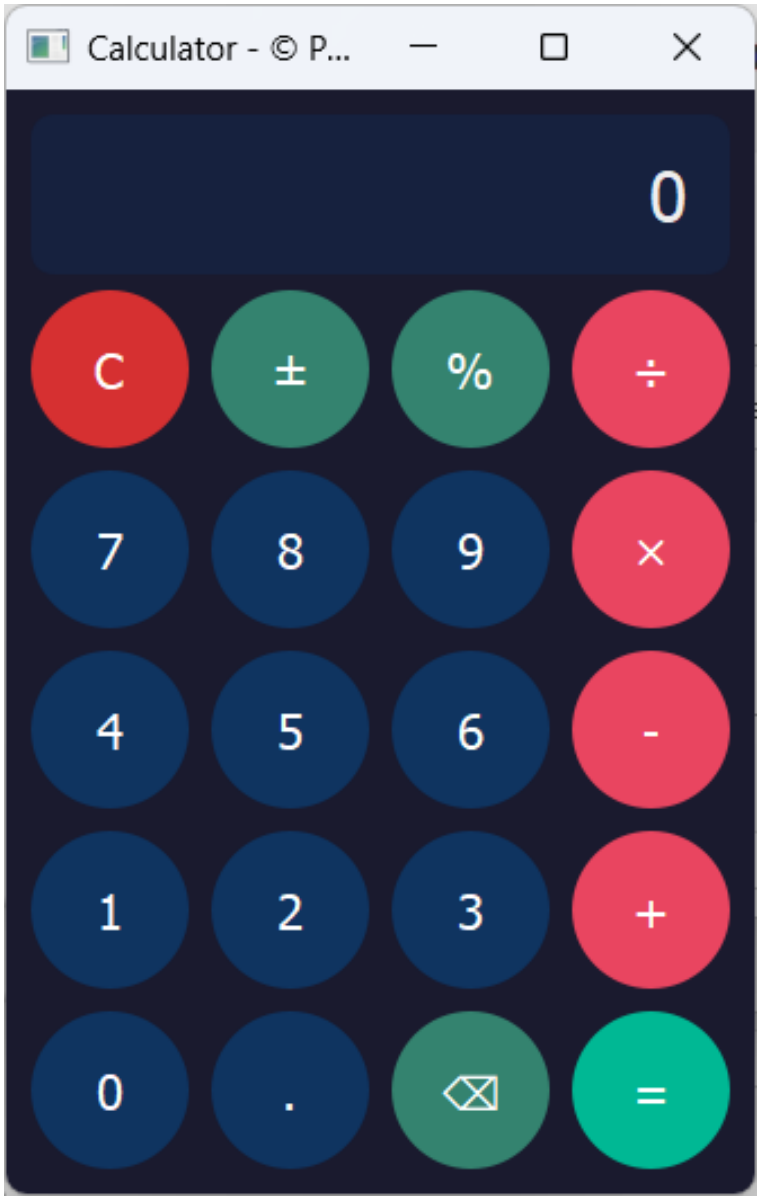

Figure 26.1: Kalkulator

Chapter 27

Threading Dasar

27.1 Kode Program

```python
# BAB 27: Threading Dasar
# ========================
# QThread untuk operasi berat tanpa freeze UI

import sys
import time
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QPushButton, QLabel, QProgressBar,
                            QVBoxLayout, QWidget)
from PyQt5.QtCore import QThread, pyqtSignal, Qt

class WorkerThread(QThread):
    """Worker thread untuk tugas berat"""
    # Signals untuk komunikasi ke main thread
    progress = pyqtSignal(int)  # Kirim progress (0-100)
    finished = pyqtSignal(str)  # Kirim hasil saat selesai
    error = pyqtSignal(str)     # Kirim error jika ada

    def __init__(self, duration=10):
        super().__init__()
        self.duration = duration
        self._is_running = True

    def run(self):
        """Method yang dijalankan di thread terpisah"""
        try:
            for i in range(self.duration + 1):
                if not self._is_running:
                    self.finished.emit("Dibatalkan!")
                    return

                # Simulasi tugas berat
                time.sleep(1)

                # Kirim progress ke main thread
                progress = int((i / self.duration) * 100)
                self.progress.emit(progress)

            self.finished.emit(f"Selesai dalam {self.duration} detik!")

        except Exception as e:
            self.error.emit(str(e))
```

```python

    def stop(self):
        """Hentikan thread"""
        self._is_running = False


class ThreadingDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Threading Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 300)

        self.worker = None

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("🧵 THREADING")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Info
        info = QLabel("Contoh QThread untuk operasi berat.\nUI tidak freeze saat thread berjalan!")
        info.setAlignment(Qt.AlignCenter)
        info.setStyleSheet("color: gray;")
        layout.addWidget(info)

        # Progress bar
        self.progress = QProgressBar()
        self.progress.setValue(0)
        self.progress.setStyleSheet("QProgressBar { height: 30px; }")
        layout.addWidget(self.progress)

        # Status
        self.status = QLabel("Status: Siap")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold;")
        self.status.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.status)

        # Buttons
        btn_start = QPushButton("▶️ Mulai Tugas (10 detik)")
        btn_start.setStyleSheet("padding: 15px; font-size: 14px;")
        btn_start.clicked.connect(self.start_task)
        layout.addWidget(btn_start)

        self.btn_stop = QPushButton("⏹ Batalkan")
        self.btn_stop.setStyleSheet("padding: 15px; font-size: 14px; background-color: #e74c3c;
            color: white;")
        self.btn_stop.clicked.connect(self.stop_task)
        self.btn_stop.setEnabled(False)
        layout.addWidget(self.btn_stop)

        # Counter untuk demo UI responsive
        self.counter_label = QLabel("Counter: 0 (klik untuk tambah)")
        self.counter_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.counter_label)

        btn_counter = QPushButton("➕ Tambah Counter")
        btn_counter.clicked.connect(self.increment_counter)
```

```
        layout.addWidget(btn_counter)

        self.counter = 0

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def start_task(self):
        self.progress.setValue(0)
        self.status.setText("Status: Sedang berjalan...")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; color: #3498db;")
        self.btn_stop.setEnabled(True)

        # Buat dan jalankan worker thread
        self.worker = WorkerThread(duration=10)
        self.worker.progress.connect(self.on_progress)
        self.worker.finished.connect(self.on_finished)
        self.worker.error.connect(self.on_error)
        self.worker.start()

    def stop_task(self):
        if self.worker:
            self.worker.stop()

    def on_progress(self, value):
        self.progress.setValue(value)

    def on_finished(self, message):
        self.status.setText(f"Status: {message}")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; color: #27ae60;")
        self.btn_stop.setEnabled(False)

    def on_error(self, error_msg):
        self.status.setText(f"Error: {error_msg}")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; color: #e74c3c;")
        self.btn_stop.setEnabled(False)

    def increment_counter(self):
        self.counter += 1
        self.counter_label.setText(f"Counter: {self.counter} (UI tetap responsive!)")


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = ThreadingDemo()
    window.show()
    sys.exit(app.exec_())
```

27.2 Penjelasan Kode

Jika Anda menjalankan proses berat (download file, query database besar) di thread utama, UI akan membeku—tombol tidak bisa diklik, jendela tampak "Not Responding". Solusinya: pindahkan pekerjaan berat ke thread terpisah menggunakan `QThread`.

1. Kelas WorkerThread

Kelas ini mewarisi `QThread` dan mendefinisikan tiga signal: `progress(int)` untuk mengirim persentase, `finished(str)` saat selesai, dan `error(str)` jika gagal. Metode `run()` berisi loop yang mensimulasikan tugas berat dengan `time.sleep(1)` selama 10 detik. Setiap iterasi mengirimkan progress ke main thread.

2. Komunikasi Thread → UI via Signal

Anda *tidak boleh* mengubah widget dari thread lain. Sebagai gantinya, worker mengirim signal yang dihubungkan ke slot di main thread: `progress` → update progress bar, `finished` → tampilkan pesan sukses, `error` → tampilkan pesan error. Pola ini menjaga keamanan thread.

3. Membatalkan Thread

Flag `_is_running` diperiksa setiap iterasi. Metode `stop()` mengubah flag menjadi `False`, sehingga loop berhenti secara kooperatif—lebih aman daripada `terminate()` yang memaksa thread mati.

4. Bukti UI Tetap Responsif

Tombol "Tambah Counter" di bawah progress bar membuktikan bahwa UI tetap bisa diklik meskipun worker sedang bekerja keras di belakang layar.

Pelajaran Penting

- Jangan pernah memblokir thread utama—gunakan `QThread` untuk operasi lama.
- Komunikasi thread ke UI harus melalui signal, bukan akses langsung widget.
- Gunakan flag kooperatif untuk menghentikan thread, bukan `terminate()`.

27.3 Hasil Tampilan

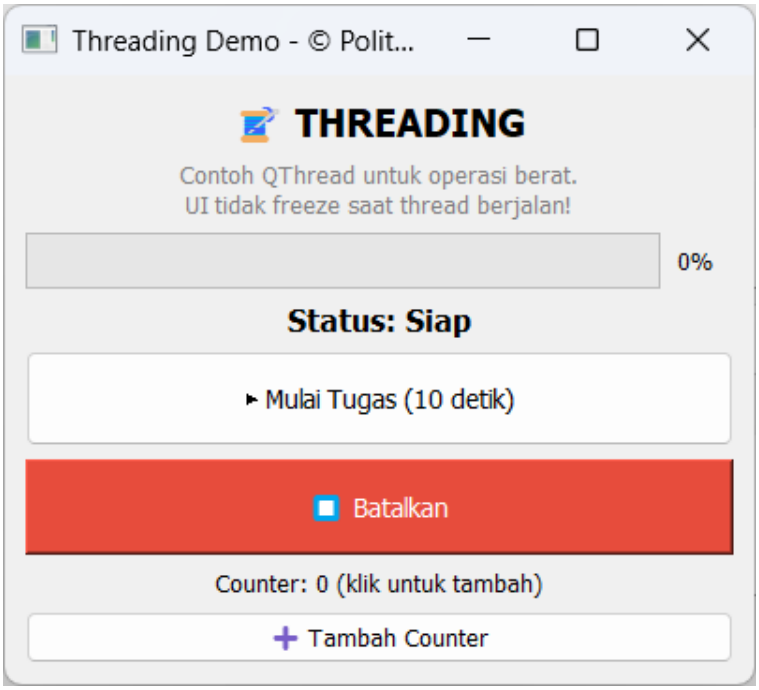

Figure 27.1: Threading Dasar

Chapter 28

Mouse Events

28.1 Kode Program

```python
# BAB 28: Mouse Events
# ====================
# Handling mouse click, move, enter, leave

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                             QLabel, QVBoxLayout, QWidget)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QMouseEvent

class MouseDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Mouse Events Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 400)

        self.setStyleSheet("background-color: #2c3e50;")

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("🖱 MOUSE EVENTS")
        header.setStyleSheet("font-size: 24px; font-weight: bold; color: white;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Instructions
        info = QLabel("Klik, geser, atau arahkan mouse di area bawah")
        info.setStyleSheet("color: #bdc3c7;")
        info.setAlignment(Qt.AlignCenter)
        layout.addWidget(info)

        # Event display
        self.event_label = QLabel("Event: -")
        self.event_label.setStyleSheet("""
            font-size: 18px;
            color: #2ecc71;
            padding: 10px;
            background-color: #34495e;
            border-radius: 5px;
        """)
```

```
        self.event_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.event_label)

        # Position display
        self.pos_label = QLabel("Position: (0, 0)")
        self.pos_label.setStyleSheet("""
            font-size: 16px;
            color: #3498db;
            padding: 10px;
            background-color: #34495e;
            border-radius: 5px;
        """)
        self.pos_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.pos_label)

        # Drawing area
        self.draw_label = QLabel("Area Gambar (klik dan drag)")
        self.draw_label.setStyleSheet("""
            background-color: white;
            border: 3px solid #3498db;
            border-radius: 10px;
            min-height: 150px;
            font-size: 14px;
            color: gray;
        """)
        self.draw_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.draw_label)

        self.last_pos = None
        self.drawing = False

        widget.setLayout(layout)
        self.setCentralWidget(widget)

        # Enable mouse tracking
        self.setMouseTracking(True)

    def mousePressEvent(self, event: QMouseEvent):
        x, y = event.x(), event.y()
        button = event.button()

        if button == Qt.LeftButton:
            self.event_label.setText("Event: Left Click Pressed")
            self.drawing = True
            self.last_pos = (x, y)
        elif button == Qt.RightButton:
            self.event_label.setText("Event: Right Click Pressed")
        elif button == Qt.MiddleButton:
            self.event_label.setText("Event: Middle Click Pressed")

        self.pos_label.setText(f"Position: ({x}, {y})")

    def mouseReleaseEvent(self, event: QMouseEvent):
        x, y = event.x(), event.y()
        button = event.button()

        if button == Qt.LeftButton:
            self.event_label.setText("Event: Left Click Released")
            self.drawing = False

        self.pos_label.setText(f"Position: ({x}, {y})")
```

```python

    def mouseMoveEvent(self, event: QMouseEvent):
        x, y = event.x(), event.y()
        self.pos_label.setText(f"Position: ({x}, {y})")

        if self.drawing:
            self.event_label.setText("Event: Dragging...")

    def enterEvent(self, event):
        self.event_label.setText("Event: Mouse Entered Window")

    def leaveEvent(self, event):
        self.event_label.setText("Event: Mouse Left Window")

    def mouseDoubleClickEvent(self, event: QMouseEvent):
        self.event_label.setText("Event: Double Click!")


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MouseDemo()
    window.show()
    sys.exit(app.exec_())
```

28.2 Penjelasan Kode

Mouse adalah perangkat input utama di aplikasi desktop. Qt memungkinkan Anda menangkap setiap gerakan, klik, dan bahkan arah scroll melalui *event handler* yang di-override.

1. Override Event Handler

Skrip meng-override enam metode bawaan `QMainWindow`: `mousePressEvent`, `mouseReleaseEvent`, `mouseMoveEvent`, `enterEvent`, `leaveEvent`, dan `mouseDoubleClickEvent`. Setiap handler menerima objek `QMouseEvent` yang membawa informasi posisi (`x()`, `y()`) dan tombol (`button()`).

2. Membedakan Tombol

Di `mousePressEvent`, kode memeriksa `event.button()` terhadap `Qt.LeftButton`, `Qt.RightButton`, dan `Qt.MiddleButton`. Label event diperbarui sesuai tombol yang ditekan—umpan balik langsung yang membantu memahami apa yang terjadi.

3. Tracking Posisi Real-Time

`setMouseTracking(True)` di konstruktor mengaktifkan `mouseMoveEvent` bahkan tanpa tombol ditekan. Posisi (x, y) ditampilkan di label biru, memberikan koordinat kursor secara real-time.

4. Deteksi Drag

Variabel `self.drawing` di-set `True` saat tombol kiri ditekan dan `False` saat dilepas. Di `mouseMoveEvent`, jika `drawing` aktif, label menunjukkan "Dragging..."—pondasi untuk fitur gambar atau seleksi area.

Pelajaran Penting

- Override event handler adalah cara Qt menangani input level rendah.
- `setMouseTracking(True)` diperlukan agar `mouseMoveEvent` aktif tanpa klik.
- Pola press → move → release adalah dasar fitur drag, drawing, dan selection.

28.3 Hasil Tampilan

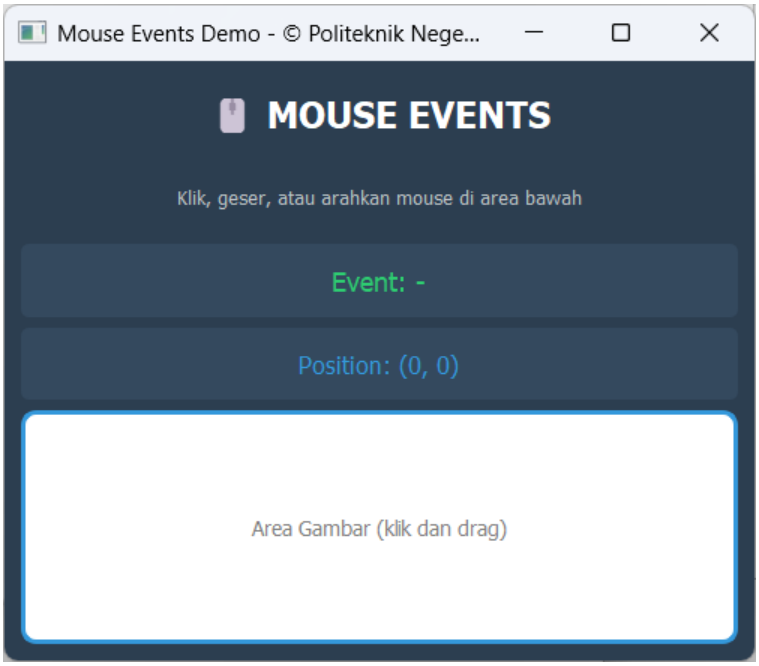

Figure 28.1: Mouse Events

Chapter 29

Keyboard Events

29.1 Kode Program

```python
# BAB 29: Keyboard Events
# ========================
# Handling key press, release, shortcuts

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                             QLabel, QVBoxLayout, QWidget)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QKeyEvent

class KeyboardDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Keyboard Events Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 400)

        self.setStyleSheet("background-color: #1a1a2e;")

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("⌨ KEYBOARD EVENTS")
        header.setStyleSheet("font-size: 24px; font-weight: bold; color: white;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Instructions
        info = QLabel("Tekan tombol keyboard untuk melihat event")
        info.setStyleSheet("color: #bdc3c7;")
        info.setAlignment(Qt.AlignCenter)
        layout.addWidget(info)

        # Key display
        self.key_label = QLabel("Key: -")
        self.key_label.setStyleSheet("""
            font-size: 28px;
            font-weight: bold;
            color: #e94560;
            padding: 20px;
            background-color: #16213e;
            border-radius: 10px;
```

```
        """)
        self.key_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.key_label)

        # Key code display
        self.code_label = QLabel("Key Code: -")
        self.code_label.setStyleSheet("""
            font-size: 18px;
            color: #0f3460;
            padding: 10px;
            background-color: #eee;
            border-radius: 5px;
        """)
        self.code_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.code_label)

        # Modifiers display
        self.mod_label = QLabel("Modifiers: None")
        self.mod_label.setStyleSheet("""
            font-size: 16px;
            color: #3498db;
            padding: 10px;
            background-color: #16213e;
            border-radius: 5px;
        """)
        self.mod_label.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.mod_label)

        # History
        self.history = QLabel("History: -")
        self.history.setStyleSheet("color: #95a5a6; font-size: 12px;")
        self.history.setAlignment(Qt.AlignCenter)
        self.history.setWordWrap(True)
        layout.addWidget(self.history)

        self.key_history = []

        # Shortcuts info
        shortcuts = QLabel("""
Shortcuts:•
 Ctrl+Q = Keluar•
 Ctrl+C = Copy message•
 Arrow keys = Gerak•
 Space = Pause
        """)
        shortcuts.setStyleSheet("color: #7f8c8d; padding: 20px;")
        layout.addWidget(shortcuts)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def keyPressEvent(self, event: QKeyEvent):
        key = event.key()
        modifiers = event.modifiers()
        text = event.text()

        # Get key name
        key_name = self.get_key_name(key)

        # Update display
        self.key_label.setText(f"Key: {key_name}")
```

```
        self.code_label.setText(f"Key Code: {key}")

        # Check modifiers
        mods = []
        if modifiers & Qt.ControlModifier:
            mods.append("Ctrl")
        if modifiers & Qt.ShiftModifier:
            mods.append("Shift")
        if modifiers & Qt.AltModifier:
            mods.append("Alt")

        self.mod_label.setText(f"Modifiers: {' + '.join(mods) if mods else 'None'}")

        # Add to history
        self.key_history.append(key_name)
        if len(self.key_history) > 10:
            self.key_history.pop(0)
        self.history.setText(f"History: {' →'.join(self.key_history)}")

        # Handle shortcuts
        if modifiers & Qt.ControlModifier:
            if key == Qt.Key_Q:
                self.close()
            elif key == Qt.Key_C:
                self.key_label.setText("Key: Copied!")

        # Special keys
        if key == Qt.Key_Space:
            self.key_label.setText("Key: SPACE (Pause)")
        elif key == Qt.Key_Return:
            self.key_label.setText("Key: ENTER")
        elif key == Qt.Key_Escape:
            self.key_label.setText("Key: ESCAPE")

    def keyReleaseEvent(self, event: QKeyEvent):
        key_name = self.get_key_name(event.key())
        self.code_label.setText(f"Released: {key_name}")

    def get_key_name(self, key):
        key_names = {
            Qt.Key_Space: "Space",
            Qt.Key_Return: "Enter",
            Qt.Key_Enter: "Enter (Numpad)",
            Qt.Key_Escape: "Escape",
            Qt.Key_Tab: "Tab",
            Qt.Key_Backspace: "Backspace",
            Qt.Key_Delete: "Delete",
            Qt.Key_Insert: "Insert",
            Qt.Key_Home: "Home",
            Qt.Key_End: "End",
            Qt.Key_PageUp: "Page Up",
            Qt.Key_PageDown: "Page Down",
            Qt.Key_Up: "↑",
            Qt.Key_Down: "↓",
            Qt.Key_Left: "←",
            Qt.Key_Right: "→",
            Qt.Key_Shift: "Shift",
            Qt.Key_Control: "Ctrl",
            Qt.Key_Alt: "Alt",
            Qt.Key_F1: "F1",
            Qt.Key_F2: "F2",
```

```
            Qt.Key_F3: "F3",
            Qt.Key_F4: "F4",
            Qt.Key_F5: "F5",
            Qt.Key_F6: "F6",
            Qt.Key_F7: "F7",
            Qt.Key_F8: "F8",
            Qt.Key_F9: "F9",
            Qt.Key_F10: "F10",
            Qt.Key_F11: "F11",
            Qt.Key_F12: "F12",
        }
        return key_names.get(key, chr(key) if 32 <= key <= 126 else f"Key({key})")


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = KeyboardDemo()
    window.show()
    sys.exit(app.exec_())
```

29.2 Penjelasan Kode

Selain mouse, keyboard adalah alat interaksi utama. Qt menangkapnya lewat `keyPressEvent` dan `keyReleaseEvent` yang masing-masing membawa objek `QKeyEvent`.

1. Menampilkan Tombol yang Ditekan

Setiap kali tombol ditekan, `keyPressEvent` membaca `event.key()` dan menerjemahkannya menjadi nama ramah via dictionary `get_key_name()`—misalnya `Qt.Key_Space` menjadi "Space", `Qt.Key_Up` menjadi "↑". Nama dan kode numerik ditampilkan di dua label terpisah.

2. Mendeteksi Modifier

`event.modifiers()` mengembalikan flag yang bisa dicek dengan bitwise AND: `Qt.ControlModifier`, `Qt.ShiftModifier`, `Qt.AltModifier`. Skrip mengumpulkan modifier aktif ke list dan menampilkannya—misalnya "Ctrl + Shift".

3. Shortcut Sederhana

Kombinasi Ctrl+Q menutup jendela, Ctrl+C menampilkan pesan "Copied!". Deteksinya cukup dengan memeriksa modifier dan key secara bersamaan di dalam `keyPressEvent`.

4. Riwayat Tombol

Setiap tombol yang ditekan ditambahkan ke list `key_history` (maks 10 item). Riwayat ditampilkan di label bawah dengan panah pemisah (→), memberikan visualisasi urutan input.

Pelajaran Penting

- `keyPressEvent` dan `keyReleaseEvent` menangkap semua input keyboard.
- Gunakan `event.modifiers()` untuk mendeteksi kombinasi Ctrl/Shift/Alt.
- Untuk shortcut aplikasi yang lebih formal, pertimbangkan `QShortcut` (dibahas di bab selanjutnya).

29.3 Hasil Tampilan

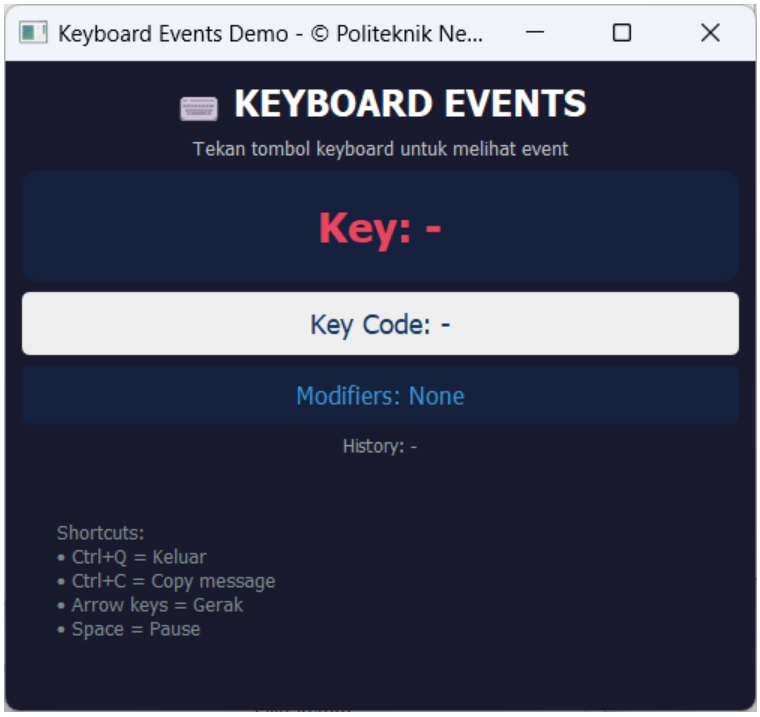

Figure 29.1: Keyboard Events

Chapter 30

Drag and Drop

30.1 Kode Program

```python
# BAB 30: Drag and Drop
# ======================
# Drag files, text, and custom data

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QListWidget, QListWidgetItem, QLabel,
                            QLineEdit, QPushButton, QVBoxLayout,
                            QHBoxLayout, QWidget)
from PyQt5.QtCore import Qt, QMimeData, pyqtSignal
from PyQt5.QtGui import QDragEnterEvent, QDropEvent

class DraggableListWidget(QListWidget):
    """ListWidget yang bisa di-drag item-nya"""

    def __init__(self):
        super().__init__()
        self.setDragEnabled(True)
        self.setAcceptDrops(True)
        self.setDropIndicatorShown(True)
        self.setDragDropMode(QListWidget.InternalMove)

    def startDrag(self, supportedActions):
        item = self.currentItem()
        if item:
            # Create drag data
            drag = QDrag(self)
            mime = QMimeData()
            mime.setText(item.text())
            drag.setMimeData(mime)
            drag.exec_(Qt.MoveAction)


class DropArea(QLabel):
    """Area yang menerima drop"""

    dropped = pyqtSignal(str) # Signal saat ada item di-drop

    def __init__(self):
        super().__init__()
        self.setText("📁 Drop files or text here\n\nDrag file dari file manager\natau drag item
            dari list kiri")
```

```
        self.setAlignment(Qt.AlignCenter)
        self.setStyleSheet("""
            QLabel {
                background-color: #34495e;
                color: #bdc3c7;
                border: 3px dashed #3498db;
                border-radius: 10px;
                padding: 20px;
                font-size: 14px;
            }
            QLabel:hover {
                background-color: #2c3e50;
                border-color: #e74c3c;
            }
        """)
        self.setAcceptDrops(True)
        self.setFixedHeight(150)

    def dragEnterEvent(self, event: QDragEnterEvent):
        if event.mimeData().hasText() or event.mimeData().hasUrls():
            event.acceptProposedAction()
            self.setStyleSheet(self.styleSheet().replace("#34495e", "#27ae60").replace("#3498db",
                "#27ae60"))

    def dragLeaveEvent(self, event):
        self.setStyleSheet("""
            QLabel {
                background-color: #34495e;
                color: #bdc3c7;
                border: 3px dashed #3498db;
                border-radius: 10px;
                padding: 20px;
                font-size: 14px;
            }
            QLabel:hover {
                background-color: #2c3e50;
                border-color: #e74c3c;
            }
        """)

    def dropEvent(self, event: QDropEvent):
        mime = event.mimeData()

        # Check for URLs (files)
        if mime.hasUrls():
            files = [url.toLocalFile() for url in mime.urls()]
            self.dropped.emit("\n".join(files))
        # Check for text
        elif mime.hasText():
            self.dropped.emit(mime.text())

        event.acceptProposedAction()

        # Reset style
        self.dragLeaveEvent(None)


class DragDropDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Drag & Drop Demo - © Politeknik Negeri Bandung - M Rizqi S.")
```

```
        self.setGeometry(100, 100, 600, 500)

        self.setAcceptDrops(True)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("🎯 DRAG AND DROP")
        header.setStyleSheet("font-size: 24px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Main content
        content_layout = QHBoxLayout()

        # Left: Draggable list
        left_widget = QWidget()
        left_layout = QVBoxLayout()

        left_layout.addWidget(QLabel("Draggable Items:"))

        self.drag_list = DraggableListWidget()
        self.drag_list.addItems([
            "🍎 Apple",
            "🍊 Orange",
            "🍇 Grape",
            "🍌 Banana",
            "🥝 Kiwi"
        ])
        left_layout.addWidget(self.drag_list)

        # Add new item
        add_layout = QHBoxLayout()
        self.input = QLineEdit()
        self.input.setPlaceholderText("Add item...")
        self.input.returnPressed.connect(self.add_item)
        add_layout.addWidget(self.input)

        btn_add = QPushButton("➕")
        btn_add.setFixedWidth(40)
        btn_add.clicked.connect(self.add_item)
        add_layout.addWidget(btn_add)

        left_layout.addLayout(add_layout)
        left_widget.setLayout(left_layout)

        content_layout.addWidget(left_widget)

        # Right: Drop area
        right_widget = QWidget()
        right_layout = QVBoxLayout()

        right_layout.addWidget(QLabel("Drop Area:"))

        self.drop_area = DropArea()
        self.drop_area.dropped.connect(self.on_drop)
        right_layout.addWidget(self.drop_area)

        right_layout.addWidget(QLabel("Dropped Items:"))

```

```
        self.dropped_list = QListWidget()
        right_layout.addWidget(self.dropped_list)

        right_widget.setLayout(right_layout)

        content_layout.addWidget(right_widget)

        layout.addLayout(content_layout)

        # Clear button
        btn_clear = QPushButton("Clear Dropped Items")
        btn_clear.clicked.connect(self.dropped_list.clear)
        layout.addWidget(btn_clear)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def add_item(self):
        text = self.input.text().strip()
        if text:
            self.drag_list.addItem(text)
            self.input.clear()

    def on_drop(self, data):
        items = data.split("\n")
        for item in items:
            if item.strip():
                self.dropped_list.addItem(f"📥 {item}")

        # Update drop area
        self.drop_area.setText(f"✅ Received {len(items)} item(s)\n\nDrop more here...")

    def dragEnterEvent(self, event):
        if event.mimeData().hasUrls():
            event.acceptProposedAction()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = DragDropDemo()
    window.show()
    sys.exit(app.exec_())
```

30.2 Penjelasan Kode

Drag-and-drop memungkinkan pengguna memindahkan data hanya dengan menyeret objek—gesture yang terasa alami. Qt mendukungnya melalui sistem `QMimeData` dan event handler drag/drop.

1. DraggableListWidget: Sumber Drag

Kelas ini mewarisi `QListWidget` dengan `setDragEnabled(True)` dan mode `InternalMove`. Saat item diseret, `startDrag()` membuat objek `QDrag` berisi `QMimeData` yang menyimpan teks item. Pengguna bisa menyusun ulang item dalam list ini.

2. DropArea: Penerima Drop

Kelas `DropArea` (turunan `QLabel`) menerima data yang di-drop. Tiga event handler di-override:

- `dragEnterEvent`: memeriksa apakah data berisi teks atau URL, lalu mengubah border menjadi hijau sebagai visual feedback.
- `dragLeaveEvent`: mengembalikan style ke semula saat kursor keluar area.
- `dropEvent`: memproses data—jika URL (file), ekstrak path lokal; jika teks, ambil langsung. Signal custom `dropped(str)` dikirim ke main window.

3. Menghubungkan Sumber dan Tujuan

Di main window, `DropArea.dropped` terhubung ke `on_drop()` yang menambahkan setiap item ke `dropped_list`. Pengguna juga bisa menambah item baru ke draggable list melalui input field.

Pelajaran Penting

• Drag-and-drop Qt bekerja lewat `QMimeData`—format data universal yang mendukung teks, URL, HTML, dan custom data.
• Visual feedback saat drag (warna berubah) penting agar pengguna tahu area mana yang menerima drop.
• `setAcceptDrops(True)` wajib dipanggil pada widget penerima.

30.3 Hasil Tampilan

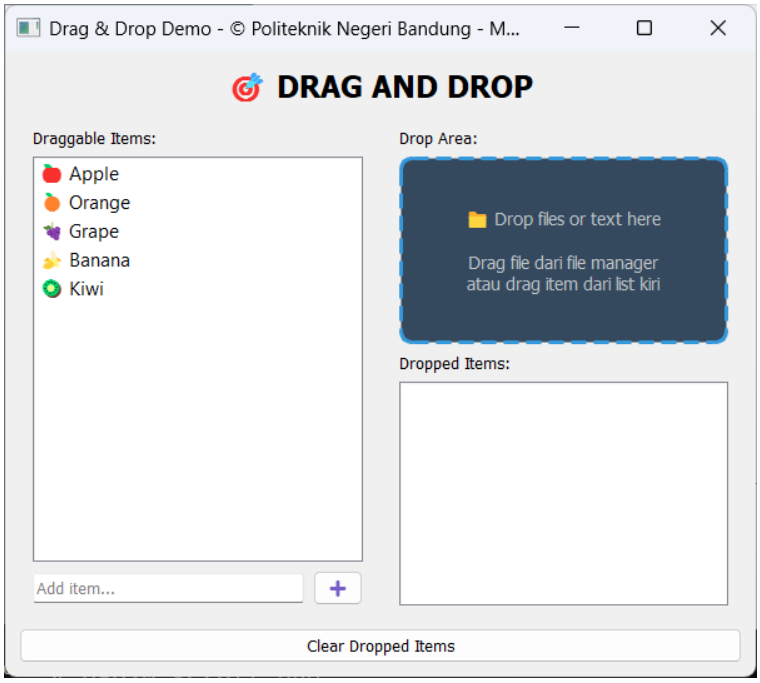

Figure 30.1: Drag and Drop

Chapter 31

TreeWidget

31.1 Kode Program

```python
# BAB 31: Tree Widget
# ====================
# Hierarchical data display

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QTreeWidget, QTreeWidgetItem, QPushButton,
                            QLineEdit, QLabel, QVBoxLayout, QHBoxLayout,
                            QWidget)
from PyQt5.QtCore import Qt

class TreeWidgetDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Tree Widget Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 500)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("🌳 TREE WIDGET")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Input for adding items
        input_layout = QHBoxLayout()

        self.input = QLineEdit()
        self.input.setPlaceholderText("New item name...")
        input_layout.addWidget(self.input)

        btn_add = QPushButton("Add Child")
        btn_add.clicked.connect(self.add_child)
        input_layout.addWidget(btn_add)

        btn_add_root = QPushButton("Add Root")
        btn_add_root.clicked.connect(self.add_root)
        input_layout.addWidget(btn_add_root)

        layout.addLayout(input_layout)
```

```python

        # Tree widget
        self.tree = QTreeWidget()
        self.tree.setHeaderLabels(["Name", "Type", "Size"])
        self.tree.setColumnWidth(0, 200)

        # Add sample data
        self.add_sample_data()

        self.tree.itemClicked.connect(self.on_item_click)
        self.tree.itemDoubleClicked.connect(self.on_item_double_click)
        layout.addWidget(self.tree)

        # Info label
        self.info = QLabel("Click an item to see info")
        self.info.setStyleSheet("padding: 10px; background: #f0f0f0;")
        layout.addWidget(self.info)

        # Buttons
        btn_layout = QHBoxLayout()

        btn_expand = QPushButton("Expand All")
        btn_expand.clicked.connect(self.tree.expandAll)
        btn_layout.addWidget(btn_expand)

        btn_collapse = QPushButton("Collapse All")
        btn_collapse.clicked.connect(self.tree.collapseAll)
        btn_layout.addWidget(btn_collapse)

        btn_delete = QPushButton("Delete Selected")
        btn_delete.clicked.connect(self.delete_selected)
        btn_delete.setStyleSheet("background-color: #e74c3c; color: white;")
        btn_layout.addWidget(btn_delete)

        layout.addLayout(btn_layout)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def add_sample_data(self):
        """Add sample hierarchical data"""

        # Root: Documents
        docs = QTreeWidgetItem(self.tree, ["📁 Documents", "Folder", ""])
        QTreeWidgetItem(docs, ["📄 resume.pdf", "PDF", "256 KB"])
        QTreeWidgetItem(docs, ["📄 letter.docx", "Word", "128 KB"])

        # Subfolder
        work = QTreeWidgetItem(docs, ["📁 Work", "Folder", ""])
        QTreeWidgetItem(work, ["📄 report.xlsx", "Excel", "512 KB"])
        QTreeWidgetItem(work, ["📄 presentation.pptx", "PowerPoint", "2 MB"])

        # Root: Pictures
        pics = QTreeWidgetItem(self.tree, ["📁 Pictures", "Folder", ""])
        QTreeWidgetItem(pics, ["🖼️ photo1.jpg", "Image", "3 MB"])
        QTreeWidgetItem(pics, ["🖼️ photo2.png", "Image", "5 MB"])

        # Subfolder
        vacation = QTreeWidgetItem(pics, ["📁 Vacation", "Folder", ""])
        QTreeWidgetItem(vacation, ["🖼️ beach.jpg", "Image", "4 MB"])
```

```
        QTreeWidgetItem(vacation, ["🖼️ mountain.jpg", "Image", "6 MB"])

        # Root: Music
        music = QTreeWidgetItem(self.tree, ["📁 Music", "Folder", ""])
        QTreeWidgetItem(music, ["🎵 song1.mp3", "Audio", "5 MB"])
        QTreeWidgetItem(music, ["🎵 song2.mp3", "Audio", "4 MB"])

        # Expand first level
        self.tree.expandToDepth(0)

    def add_root(self):
        text = self.input.text().strip()
        if text:
            item = QTreeWidgetItem(self.tree, [f"📁 {text}", "Folder", ""])
            self.tree.addTopLevelItem(item)
            self.input.clear()

    def add_child(self):
        text = self.input.text().strip()
        if text:
            current = self.tree.currentItem()
            if current:
                QTreeWidgetItem(current, [f"📄 {text}", "File", "New"])
                current.setExpanded(True)
            self.input.clear()

    def on_item_click(self, item, column):
        path = self.get_item_path(item)
        self.info.setText(f"Selected: {path}")

    def on_item_double_click(self, item, column):
        path = self.get_item_path(item)
        self.info.setText(f"Double-clicked: {path}")

    def get_item_path(self, item):
        """Get full path of item"""
        path = [item.text(0)]
        parent = item.parent()
        while parent:
            path.insert(0, parent.text(0))
            parent = parent.parent()
        return " / ".join(path)

    def delete_selected(self):
        item = self.tree.currentItem()
        if item:
            parent = item.parent()
            if parent:
                parent.removeChild(item)
            else:
                index = self.tree.indexOfTopLevelItem(item)
                self.tree.takeTopLevelItem(index)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = TreeWidgetDemo()
    window.show()
    sys.exit(app.exec_())
```

31.2 Penjelasan Kode

Data hierarkis—folder berisi subfolder, organisasi punya divisi—membutuhkan tampilan pohon. `QTreeWidget` menyediakan struktur induk-anak yang bisa di-expand dan di-collapse.

1. Data Sampel Hierarkis

Metode `add_sample_data()` membangun pohon file virtual: Documents (berisi file dan subfolder Work), Pictures (berisi foto dan subfolder Vacation), dan Music. Setiap node adalah `QTreeWidgetItem` dengan tiga kolom: Name, Type, dan Size. Child ditambahkan cukup dengan menyebut parent sebagai argumen pertama.

2. Tiga Kolom Header

`setHeaderLabels(["Name", "Type", "Size"])` memberi label kolom. Kolom pertama diperlebar via `setColumnWidth(0, 200)` agar nama file terbaca jelas.

3. Navigasi: Klik dan Double-Klik

Signal `itemClicked` memanggil `on_item_click()` yang membangun full path item (dari root ke child) menggunakan rekursi `get_item_path()`. Double-klik memicu pesan berbeda di label info.

4. Manipulasi Tree

Tombol Add Root menambah node top-level baru. Add Child menambah anak ke node yang sedang dipilih dan langsung meng-expand-nya. Delete Selected menghapus node terpilih beserta semua child-nya. Expand All dan Collapse All mengontrol tampilan keseluruhan.

Pelajaran Penting

- `QTreeWidget` ideal untuk data hierarkis dengan kedalaman tak terbatas.
- Setiap item bisa memiliki banyak kolom—cocok untuk menampilkan metadata.
- Rekursi pada `parent()` berguna untuk membangun path lengkap dari leaf ke root.

31.3 Hasil Tampilan

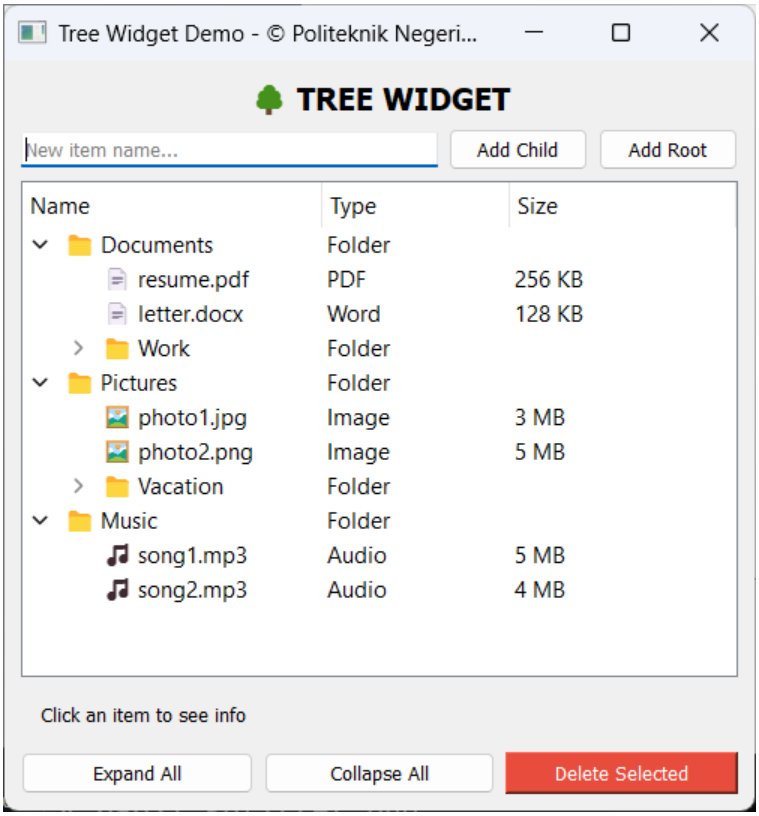

Figure 31.1: TreeWidget

Chapter 32

StackedWidget

32.1 Kode Program

```python
# BAB 32: Stacked Widget
# ========================
# Multiple pages, one visible at a time

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QStackedWidget, QWidget, QPushButton,
                            QLabel, QLineEdit, QTextEdit, QComboBox,
                            QVBoxLayout, QHBoxLayout, QFormLayout,
                            QListWidget, QListWidgetItem)
from PyQt5.QtCore import Qt

class PageHome(QWidget):
    def __init__(self, stacked):
        super().__init__()
        self.stacked = stacked

        layout = QVBoxLayout()

        header = QLabel("🏠 HOME")
        header.setStyleSheet("font-size: 24px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        welcome = QLabel("Welcome to the App!\n\nUse the menu on the left to navigate.")
        welcome.setAlignment(Qt.AlignCenter)
        welcome.setStyleSheet("font-size: 14px; color: gray;")
        layout.addWidget(welcome)

        # Quick actions
        layout.addWidget(QLabel("\nQuick Actions:"))

        btn_profile = QPushButton("Go to Profile")
        btn_profile.clicked.connect(lambda: self.stacked.setCurrentIndex(1))
        layout.addWidget(btn_profile)

        btn_settings = QPushButton("Go to Settings")
        btn_settings.clicked.connect(lambda: self.stacked.setCurrentIndex(2))
        layout.addWidget(btn_settings)

        layout.addStretch()
        self.setLayout(layout)
```

```python


class PageProfile(QWidget):
    def __init__(self, stacked):
        super().__init__()
        self.stacked = stacked

        layout = QVBoxLayout()

        header = QLabel("👤 PROFILE")
        header.setStyleSheet("font-size: 24px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Form
        form = QFormLayout()

        self.name = QLineEdit()
        self.name.setPlaceholderText("Your name")
        form.addRow("Name:", self.name)

        self.email = QLineEdit()
        self.email.setPlaceholderText("email@example.com")
        form.addRow("Email:", self.email)

        self.bio = QTextEdit()
        self.bio.setPlaceholderText("Tell us about yourself...")
        self.bio.setMaximumHeight(100)
        form.addRow("Bio:", self.bio)

        layout.addLayout(form)

        # Save button
        btn_save = QPushButton("Save Profile")
        btn_save.setStyleSheet("background-color: #27ae60; color: white; padding: 10px;")
        btn_save.clicked.connect(self.save)
        layout.addWidget(btn_save)

        # Status
        self.status = QLabel("")
        layout.addWidget(self.status)

        # Back button
        btn_back = QPushButton("← Back to Home")
        btn_back.clicked.connect(lambda: self.stacked.setCurrentIndex(0))
        layout.addWidget(btn_back)

        layout.addStretch()
        self.setLayout(layout)

    def save(self):
        name = self.name.text()
        email = self.email.text()
        self.status.setText(f"✅ Saved! Name: {name}, Email: {email}")


class PageSettings(QWidget):
    def __init__(self, stacked):
        super().__init__()
        self.stacked = stacked

```

```
        layout = QVBoxLayout()

        header = QLabel("⚙️ SETTINGS")
        header.setStyleSheet("font-size: 24px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Settings form
        form = QFormLayout()

        self.theme = QComboBox()
        self.theme.addItems(["Light", "Dark", "System"])
        form.addRow("Theme:", self.theme)

        self.language = QComboBox()
        self.language.addItems(["English", "Indonesia", "  ", " "])
        form.addRow("Language:", self.language)

        layout.addLayout(form)

        # Status
        self.status = QLabel("")
        layout.addWidget(self.status)

        # Back button
        btn_back = QPushButton("← Back to Home")
        btn_back.clicked.connect(lambda: self.stacked.setCurrentIndex(0))
        layout.addWidget(btn_back)

        layout.addStretch()
        self.setLayout(layout)


class PageAbout(QWidget):
    def __init__(self, stacked):
        super().__init__()
        self.stacked = stacked

        layout = QVBoxLayout()

        header = QLabel("ℹ️ ABOUT")
        header.setStyleSheet("font-size: 24px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        info = QLabel("""
Stacked Widget Demo
Version 1.0

This app demonstrates QStackedWidget
for creating multi-page applications.

Features:•
 Multiple pages•
 Navigation via sidebar•
 Each page is a separate class

Built with PyQt5 ❤️
        """)
        info.setAlignment(Qt.AlignCenter)
        layout.addWidget(info)
```

```python

        # Back button
        btn_back = QPushButton("← Back to Home")
        btn_back.clicked.connect(lambda: self.stacked.setCurrentIndex(0))
        layout.addWidget(btn_back)

        layout.addStretch()
        self.setLayout(layout)


class StackedDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Stacked Widget Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 600, 400)

        # Main layout
        main_widget = QWidget()
        main_layout = QHBoxLayout()

        # Sidebar (navigation)
        sidebar = QListWidget()
        sidebar.setFixedWidth(150)
        sidebar.setStyleSheet("""
            QListWidget {
                background-color: #2c3e50;
                color: white;
                border: none;
                font-size: 14px;
            }
            QListWidget::item {
                padding: 15px;
            }
            QListWidget::item:selected {
                background-color: #3498db;
            }
            QListWidget::item:hover {
                background-color: #34495e;
            }
        """)

        sidebar.addItem("🏠 Home")
        sidebar.addItem("👤 Profile")
        sidebar.addItem("⚙️ Settings")
        sidebar.addItem("ℹ️ About")
        sidebar.setCurrentRow(0)

        main_layout.addWidget(sidebar)

        # Stacked widget (pages)
        self.stack = QStackedWidget()

        # Add pages
        self.stack.addWidget(PageHome(self.stack)) # Index 0
        self.stack.addWidget(PageProfile(self.stack)) # Index 1
        self.stack.addWidget(PageSettings(self.stack)) # Index 2
        self.stack.addWidget(PageAbout(self.stack)) # Index 3

        main_layout.addWidget(self.stack)

        # Connect sidebar to stack
```

```
        sidebar.currentRowChanged.connect(self.stack.setCurrentIndex)

        main_widget.setLayout(main_layout)
        self.setCentralWidget(main_widget)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = StackedDemo()
    window.show()
    sys.exit(app.exec_())
```

32.2 Penjelasan Kode

`QStackedWidget` menampilkan satu halaman pada satu waktu—seperti tumpukan kartu yang hanya memperlihatkan kartu teratas. Dikombinasikan dengan sidebar navigasi, ia menjadi fondasi aplikasi multi-halaman.

1. Empat Halaman, Empat Kelas

Setiap halaman adalah kelas tersendiri (`PageHome`, `PageProfile`, `PageSettings`, `PageAbout`), masing-masing mewarisi `QWidget`. Konstruktor menerima referensi ke stacked widget agar bisa berpindah halaman dari dalam (misalnya tombol "Go to Profile" memanggil `stacked.setCurrentIndex(1)`).

2. Sidebar dengan QListWidget

Sidebar kiri berisi empat item menu. Signal `currentRowChanged` langsung dihubungkan ke `stack.setCurrentIndex`—satu baris kode yang menghubungkan navigasi ke konten. Styling CSS memberikan efek hover dan highlight pada item aktif.

3. Halaman Interaktif

Profile memiliki form (nama, email, bio) dengan tombol Save. Settings berisi combo box untuk tema dan bahasa. About menampilkan informasi statis. Setiap halaman punya tombol "Back to Home" untuk navigasi mundur.

Pelajaran Penting

- `QStackedWidget` ideal untuk wizard, dashboard, atau aplikasi dengan banyak view.
- Pisahkan setiap halaman ke kelas terpisah agar kode modular dan mudah dikelola.
- Menghubungkan sidebar ke stacked widget cukup satu baris: signal row changed ke setCurrentIndex.

32.3 Hasil Tampilan

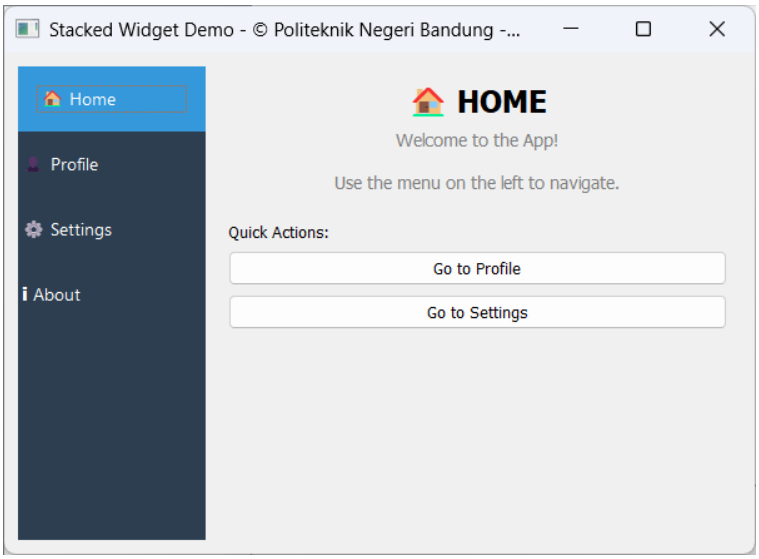

Figure 32.1: StackedWidget

Chapter 33

Graphics View

33.1 Kode Program

```python
# BAB 33: Graphics View (Simple)
# ==============================
# Basic drawing with QGraphicsView

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QGraphicsScene, QGraphicsView,
                            QGraphicsRectItem, QGraphicsEllipseItem,
                            QGraphicsLineItem, QGraphicsTextItem,
                            QPushButton, QVBoxLayout, QHBoxLayout,
                            QWidget, QColorDialog, QSpinBox, QLabel)
from PyQt5.QtCore import Qt, QRectF
from PyQt5.QtGui import QPen, QBrush, QColor, QFont, QPainter

class GraphicsDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Graphics View Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 800, 600)

        self.current_color = QColor("#3498db")

        widget = QWidget()
        layout = QVBoxLayout()

        # Toolbar
        toolbar = QHBoxLayout()

        toolbar.addWidget(QLabel("Color:"))
        btn_color = QPushButton("🎨")
        btn_color.clicked.connect(self.choose_color)
        toolbar.addWidget(btn_color)

        toolbar.addWidget(QLabel("Size:"))
        self.size_spin = QSpinBox()
        self.size_spin.setRange(20, 200)
        self.size_spin.setValue(50)
        toolbar.addWidget(self.size_spin)

        btn_rect = QPushButton("Add Rectangle")
        btn_rect.clicked.connect(self.add_rect)
        toolbar.addWidget(btn_rect)
```

```python

        btn_ellipse = QPushButton("Add Ellipse")
        btn_ellipse.clicked.connect(self.add_ellipse)
        toolbar.addWidget(btn_ellipse)

        btn_line = QPushButton("Add Line")
        btn_line.clicked.connect(self.add_line)
        toolbar.addWidget(btn_line)

        btn_text = QPushButton("Add Text")
        btn_text.clicked.connect(self.add_text)
        toolbar.addWidget(btn_text)

        btn_clear = QPushButton("Clear All")
        btn_clear.setStyleSheet("background-color: #e74c3c; color: white;")
        btn_clear.clicked.connect(self.clear_scene)
        toolbar.addWidget(btn_clear)

        toolbar.addStretch()
        layout.addLayout(toolbar)

        # Graphics view
        self.scene = QGraphicsScene()
        self.scene.setSceneRect(0, 0, 750, 500)

        self.view = QGraphicsView(self.scene)
        self.view.setStyleSheet("background-color: white; border: 2px solid #ccc;")
        self.view.setRenderHint(QPainter.Antialiasing)
        layout.addWidget(self.view)

        # Info
        self.info = QLabel("Click buttons to add shapes. Drag shapes to move them.")
        self.info.setStyleSheet("padding: 5px; background: #f0f0f0;")
        layout.addWidget(self.info)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

        # Add some initial shapes
        self.add_sample_shapes()

    def add_sample_shapes(self):
        """Add some sample shapes"""
        # Rectangle
        rect = QGraphicsRectItem(100, 100, 100, 80)
        rect.setBrush(QBrush(QColor("#3498db")))
        rect.setPen(QPen(QColor("#2980b9"), 2))
        rect.setFlag(rect.ItemIsMovable)
        rect.setFlag(rect.ItemIsSelectable)
        self.scene.addItem(rect)

        # Ellipse
        ellipse = QGraphicsEllipseItem(300, 150, 120, 80)
        ellipse.setBrush(QBrush(QColor("#e74c3c")))
        ellipse.setPen(QPen(QColor("#c0392b"), 2))
        ellipse.setFlag(ellipse.ItemIsMovable)
        ellipse.setFlag(ellipse.ItemIsSelectable)
        self.scene.addItem(ellipse)

        # Line
        line = QGraphicsLineItem(50, 300, 200, 350)
```

```
        line.setPen(QPen(QColor("#27ae60"), 4))
        self.scene.addItem(line)

        # Text
        text = self.scene.addText("Hello Graphics!")
        text.setDefaultTextColor(QColor("#2c3e50"))
        text.setFont(QFont("Arial", 16, QFont.Bold))
        text.setPos(400, 100)
        text.setFlag(text.ItemIsMovable)

    def choose_color(self):
        color = QColorDialog.getColor(self.current_color)
        if color.isValid():
            self.current_color = color

    def add_rect(self):
        size = self.size_spin.value()
        x = 200
        y = 200

        rect = QGraphicsRectItem(x, y, size, size * 0.8)
        rect.setBrush(QBrush(self.current_color))
        rect.setPen(QPen(self.current_color.darker(), 2))
        rect.setFlag(rect.ItemIsMovable)
        rect.setFlag(rect.ItemIsSelectable)
        self.scene.addItem(rect)

        self.info.setText(f"Added rectangle at ({x}, {y})")

    def add_ellipse(self):
        size = self.size_spin.value()
        x = 300
        y = 200

        ellipse = QGraphicsEllipseItem(x, y, size, size * 0.6)
        ellipse.setBrush(QBrush(self.current_color))
        ellipse.setPen(QPen(self.current_color.darker(), 2))
        ellipse.setFlag(ellipse.ItemIsMovable)
        ellipse.setFlag(ellipse.ItemIsSelectable)
        self.scene.addItem(ellipse)

        self.info.setText(f"Added ellipse at ({x}, {y})")

    def add_line(self):
        size = self.size_spin.value()
        x1, y1 = 100, 300
        x2, y2 = x1 + size * 2, y1 + size // 2

        line = QGraphicsLineItem(x1, y1, x2, y2)
        line.setPen(QPen(self.current_color, 4))
        self.scene.addItem(line)

        self.info.setText(f"Added line from ({x1}, {y1}) to ({x2}, {y2})")

    def add_text(self):
        text = self.scene.addText("Text")
        text.setDefaultTextColor(self.current_color)
        text.setFont(QFont("Arial", self.size_spin.value() // 3))
        text.setPos(250, 250)
        text.setFlag(text.ItemIsMovable)
        text.setFlag(text.ItemIsSelectable)
```

```python

        self.info.setText("Added text (double-click to edit)")

    def clear_scene(self):
        """Clear all items from the scene"""
        self.scene.clear()
        self.info.setText("Scene cleared")


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = GraphicsDemo()
    window.show()
    sys.exit(app.exec_())
```

33.2 Penjelasan Kode

Untuk menggambar bentuk, teks, atau diagram yang bisa digeser pengguna, Qt menyediakan framework `QGraphicsScene` + `QGraphicsView`. Scene adalah kanvas virtual; view adalah jendela yang menampilkannya.

1. Scene dan View

`QGraphicsScene` diinisialisasi dengan ukuran 750×500 piksel. `QGraphicsView` menampilkan scene dengan anti-aliasing aktif (`QPainter.Antialiasing`) agar garis terlihat halus.

2. Bentuk Bawaan

Qt menyediakan item grafis siap pakai: `QGraphicsRectItem` (persegi panjang), `QGraphicsEllipseItem` (elips), `QGraphicsLineItem` (garis), dan `addText()` untuk teks. Setiap item diberi warna fill (`setBrush`) dan outline (`setPen`). Flag `ItemIsMovable` dan `ItemIsSelectable` memungkinkan pengguna menyeret dan memilih item.

3. Toolbar Interaktif

Tombol Add Rectangle, Add Ellipse, Add Line, dan Add Text masing-masing memanggil metode yang membuat item baru di posisi tetap dengan ukuran dari `QSpinBox`. Tombol warna membuka `QColorDialog` untuk memilih warna aktif. Clear All menghapus seluruh scene.

Pelajaran Penting

- Framework Graphics View cocok untuk editor diagram, game 2D, atau visualisasi data interaktif.
- Item yang movable dan selectable membuat pengguna bisa mengatur tata letak secara bebas.
- Scene dan view dipisahkan—satu scene bisa ditampilkan oleh beberapa view sekaligus.

33.3 Hasil Tampilan

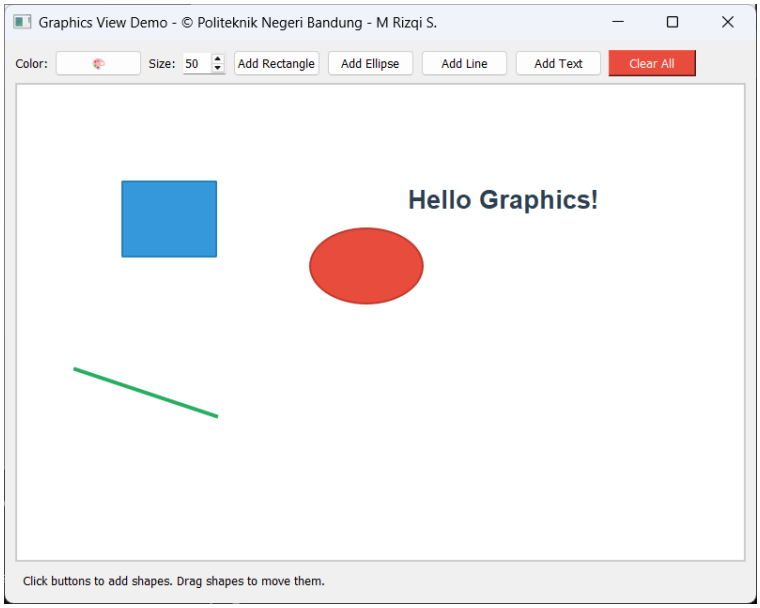

Figure 33.1: Graphics View

Chapter 34

Context Menu

34.1 Kode Program

```python
# BAB 34: Context Menu (Right-Click Menu)
# ========================================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QListWidget, QListWidgetItem, QLabel,
                            QMenu, QAction, QMessageBox,
                            QVBoxLayout, QWidget)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QCursor

class ContextMenuDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Context Menu Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 400)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("📋 CONTEXT MENU (Right-Click)")
        header.setStyleSheet("font-size: 18px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        info = QLabel("Right-click on the list or anywhere in the window")
        info.setStyleSheet("color: gray;")
        info.setAlignment(Qt.AlignCenter)
        layout.addWidget(info)

        # List with context menu
        self.list = QListWidget()
        self.list.addItems([
            "📄 Document 1",
            "📄 Document 2",
            "📄 Document 3",
            "📁 Folder 1",
            "📁 Folder 2"
        ])

```

```
        # Enable custom context menu
        self.list.setContextMenuPolicy(Qt.CustomContextMenu)
        self.list.customContextMenuRequested.connect(self.show_list_context_menu)

        layout.addWidget(self.list)

        # Status
        self.status = QLabel("Right-click to see context menu")
        self.status.setStyleSheet("padding: 10px; background: #f0f0f0;")
        layout.addWidget(self.status)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

        # Enable context menu for main window
        self.setContextMenuPolicy(Qt.CustomContextMenu)
        self.customContextMenuRequested.connect(self.show_window_context_menu)

    def show_list_context_menu(self, position):
        """Show context menu for list item"""
        item = self.list.itemAt(position)

        if item is None:
            return

        # Create menu
        menu = QMenu()

        # Actions
        action_open = QAction("📂 Open", self)
        action_open.triggered.connect(lambda: self.open_item(item))
        menu.addAction(action_open)

        action_edit = QAction("✏️ Edit", self)
        action_edit.triggered.connect(lambda: self.edit_item(item))
        menu.addAction(action_edit)

        menu.addSeparator()

        action_copy = QAction("📋 Copy", self)
        action_copy.triggered.connect(lambda: self.copy_item(item))
        menu.addAction(action_copy)

        action_cut = QAction("✂️ Cut", self)
        menu.addAction(action_cut)

        menu.addSeparator()

        action_delete = QAction("🗑️ Delete", self)
        action_delete.triggered.connect(lambda: self.delete_item(item))
        menu.addAction(action_delete)

        # Show menu at cursor position
        menu.exec_(self.list.mapToGlobal(position))

    def show_window_context_menu(self, position):
        """Show context menu for the window"""
        menu = QMenu()

        action_about = QAction("ℹ️ About", self)
        action_about.triggered.connect(self.show_about)
```

```
        menu.addAction(action_about)

        action_help = QAction("❓ Help", self)
        menu.addAction(action_help)

        menu.addSeparator()

        action_new = QAction("➕ New Item", self)
        action_new.triggered.connect(self.add_new_item)
        menu.addAction(action_new)

        # Show menu
        menu.exec_(self.mapToGlobal(position))

    def open_item(self, item):
        self.status.setText(f"Opened: {item.text()}")

    def edit_item(self, item):
        self.status.setText(f"Editing: {item.text()}")
        self.list.editItem(item)

    def copy_item(self, item):
        self.status.setText(f"Copied: {item.text()}")
        # Clone item
        new_item = item.clone()
        new_item.setText(item.text() + " (copy)")
        self.list.addItem(new_item)

    def delete_item(self, item):
        reply = QMessageBox.question(
            self, "Confirm Delete",
            f"Delete '{item.text()}'?",
            QMessageBox.Yes | QMessageBox.No
        )
        if reply == QMessageBox.Yes:
            self.list.takeItem(self.list.row(item))
            self.status.setText(f"Deleted: {item.text()}")

    def add_new_item(self):
        count = self.list.count() + 1
        self.list.addItem(f"📄 New Document {count}")
        self.status.setText("Added new item")

    def show_about(self):
        QMessageBox.about(
            self, "About",
            "Context Menu Demo\n\nRight-click to see context menus!"
        )


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = ContextMenuDemo()
    window.show()
    sys.exit(app.exec_())
```

34.2 Penjelasan Kode

Klik kanan pada file di desktop pasti memunculkan menu konteks. Fitur ini bisa Anda buat sendiri di PyQt5 menggunakan `QMenu` dan kebijakan `CustomContextMenu`.

1. Mengaktifkan Context Menu

Pada list widget, `setContextMenuPolicy(Qt.CustomContextMenu)` mengganti menu bawaan. Signal `customContextMenuRequested` dihubungkan ke `show_list_context_menu(pos)` yang menerima posisi kursor. Langkah serupa dilakukan untuk window utama.

2. Menu untuk Item List

Jika klik kanan tepat pada item, menu menampilkan Open, Edit, Copy, Cut, dan Delete. Setiap `QAction` dihubungkan ke metode terkait via lambda yang mengirim referensi item. Separator (`addSeparator()`) membagi aksi menjadi kelompok logis.

3. Menu untuk Window

Klik kanan di area kosong memunculkan menu berbeda: About, Help, dan New Item. Pemisahan ini membuat context menu terasa cerdas—aksi relevan muncul sesuai konteks.

4. Aksi CRUD

Open menampilkan nama item di status. Edit memanggil `editItem()` bawaan Qt. Copy menggandakan item dengan suffix "(copy)". Delete menampilkan dialog konfirmasi (`QMessageBox.question`) sebelum menghapus—keamanan penting agar data tidak hilang tanpa sengaja.

Pelajaran Penting

- Gunakan `CustomContextMenu` + `customContextMenuRequested` untuk menu kustom.
- `menu.exec_(mapToGlobal(pos))` menampilkan menu di posisi kursor yang benar.
- Selalu tampilkan dialog konfirmasi untuk aksi destruktif seperti delete.

34.3 Hasil Tampilan

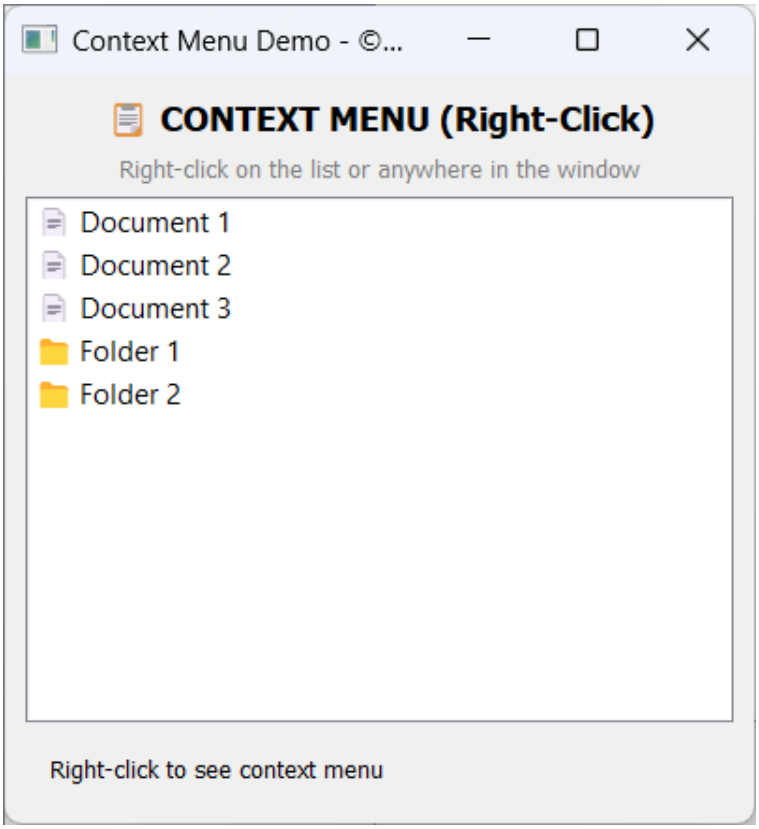

Figure 34.1: Context Menu

Chapter 35

System Tray

35.1 Kode Program

```python
# BAB 35: System Tray Icon
# ========================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QPushButton, QLabel, QVBoxLayout,
                            QWidget, QSystemTrayIcon, QMenu, QAction,
                            QSpinBox, QMessageBox)
from PyQt5.QtGui import QIcon
from PyQt5.QtCore import Qt

class SystemTrayDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("System Tray Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 400, 350)

        # Create system tray icon
        self.tray_icon = QSystemTrayIcon(self)

        # Use built-in icon (in real app, use custom icon)
        self.tray_icon.setIcon(self.style().standardIcon(self.style().SP_ComputerIcon))
        self.tray_icon.setToolTip("System Tray Demo")

        # Tray menu
        tray_menu = QMenu()

        action_show = QAction("Show Window", self)
        action_show.triggered.connect(self.show)
        tray_menu.addAction(action_show)

        action_hide = QAction("Hide Window", self)
        action_hide.triggered.connect(self.hide)
        tray_menu.addAction(action_hide)

        tray_menu.addSeparator()

        action_notify = QAction("Send Notification", self)
        action_notify.triggered.connect(self.send_notification)
        tray_menu.addAction(action_notify)

        tray_menu.addSeparator()
```

```
        action_quit = QAction("Quit", self)
        action_quit.triggered.connect(self.close)
        tray_menu.addAction(action_quit)

        self.tray_icon.setContextMenu(tray_menu)

        # Handle tray icon click
        self.tray_icon.activated.connect(self.on_tray_activated)

        # Show tray icon
        self.tray_icon.show()

        # Main window UI
        widget = QWidget()
        layout = QVBoxLayout()

        header = QLabel("🖥️ SYSTEM TRAY ICON")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        info = QLabel("This app can minimize to system tray.\n\nTry:\•n Click 'Hide' to minimize to
            tray\•n Click tray icon to restore\•n Right-click tray icon for menu")
        info.setAlignment(Qt.AlignCenter)
        info.setStyleSheet("padding: 20px;")
        layout.addWidget(info)

        # Notification settings
        layout.addWidget(QLabel("Notification Title:"))
        self.title = QLabel("Title")
        self.title.setStyleSheet("padding: 5px; background: #f0f0f0;")
        layout.addWidget(self.title)

        layout.addWidget(QLabel("Notification Message:"))
        self.message = QLabel("This is a notification message!")
        self.message.setStyleSheet("padding: 5px; background: #f0f0f0;")
        layout.addWidget(self.message)

        # Duration
        duration_layout = QVBoxLayout()
        duration_layout.addWidget(QLabel("Duration (ms):"))
        self.duration = QSpinBox()
        self.duration.setRange(1000, 30000)
        self.duration.setValue(3000)
        self.duration.setSingleStep(1000)
        layout.addWidget(self.duration)

        # Buttons
        btn_notify = QPushButton("📩 Send Notification")
        btn_notify.setStyleSheet("padding: 15px;")
        btn_notify.clicked.connect(self.send_notification)
        layout.addWidget(btn_notify)

        btn_hide = QPushButton("👁️ Hide to Tray")
        btn_hide.setStyleSheet("padding: 15px;")
        btn_hide.clicked.connect(self.hide)
        layout.addWidget(btn_hide)

        widget.setLayout(layout)
        self.setCentralWidget(widget)
```

```python

    def on_tray_activated(self, reason):
        """Handle tray icon activation"""
        if reason == QSystemTrayIcon.DoubleClick:
            self.show()
            self.activateWindow()
        elif reason == QSystemTrayIcon.Trigger:
            # Single click - show message
            self.tray_icon.showMessage(
                "Clicked!",
                "Double-click to show window",
                QSystemTrayIcon.Information,
                2000
            )

    def send_notification(self):
        """Send system notification"""
        self.tray_icon.showMessage(
            "Notification Title",
            "This is a notification from the system tray app!",
            QSystemTrayIcon.Information,
            self.duration.value()
        )

    def closeEvent(self, event):
        """Handle window close - minimize to tray instead of closing"""
        if self.tray_icon.isVisible():
            QMessageBox.information(
                self, "Minimizing to Tray",
                "The app will minimize to the system tray.\n"
                "Right-click the tray icon to quit."
            )
            self.hide()
            event.ignore()
        else:
            event.accept()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    app.setQuitOnLastWindowClosed(False) # Don't quit when window closes

    # Check if system tray is available
    if not QSystemTrayIcon.isSystemTrayAvailable():
        QMessageBox.critical(None, "Error", "System tray is not available on this system")
        sys.exit(1)

    window = SystemTrayDemo()
    window.show()

    sys.exit(app.exec_())
```

35.2 Penjelasan Kode

Beberapa aplikasi perlu tetap berjalan di latar belakang meskipun jendela ditutup—misalnya antivirus, chat, atau download manager. `QSystemTrayIcon` mewujudkannya dengan ikon kecil di system tray.

1. Membuat Tray Icon

`QSystemTrayIcon` diinisialisasi dengan ikon bawaan OS (`SP_ComputerIcon`) dan tooltip. Memanggil `show()` membuatnya muncul di area notifikasi taskbar.

2. Menu Tray
Menu klik kanan tray berisi Show Window, Hide Window, Send Notification, dan Quit. Masing-masing dihubungkan ke metode sederhana: `self.show()`, `self.hide()`, dll.

3. Interaksi Tray
Signal `activated` mendeteksi cara pengguna berinteraksi: double-click menampilkan jendela (textttDoubleClick), single-click menampilkan pesan singkat via `showMessage()`.

4. Minimize ke Tray, Bukan Close
Override `closeEvent` mengganti perilaku tombol X: alih-alih menutup aplikasi, jendela disembunyikan dan pesan informatif ditampilkan. Di `main()`, `setQuitOnLastWindowClosed(False)` memastikan aplikasi tidak mati saat jendela hilang.

5. Notifikasi Sistem
`showMessage(title, body, icon, duration)` menampilkan balloon notification bawaan OS. Durasi bisa diatur via `QSpinBox` di UI.

Pelajaran Penting
- System tray ideal untuk aplikasi yang perlu berjalan terus di background.
- Selalu cek `isSystemTrayAvailable()` sebelum membuat tray icon.
- Override `closeEvent` untuk mengubah perilaku tombol X menjadi minimize-to-tray.

35.3 Hasil Tampilan

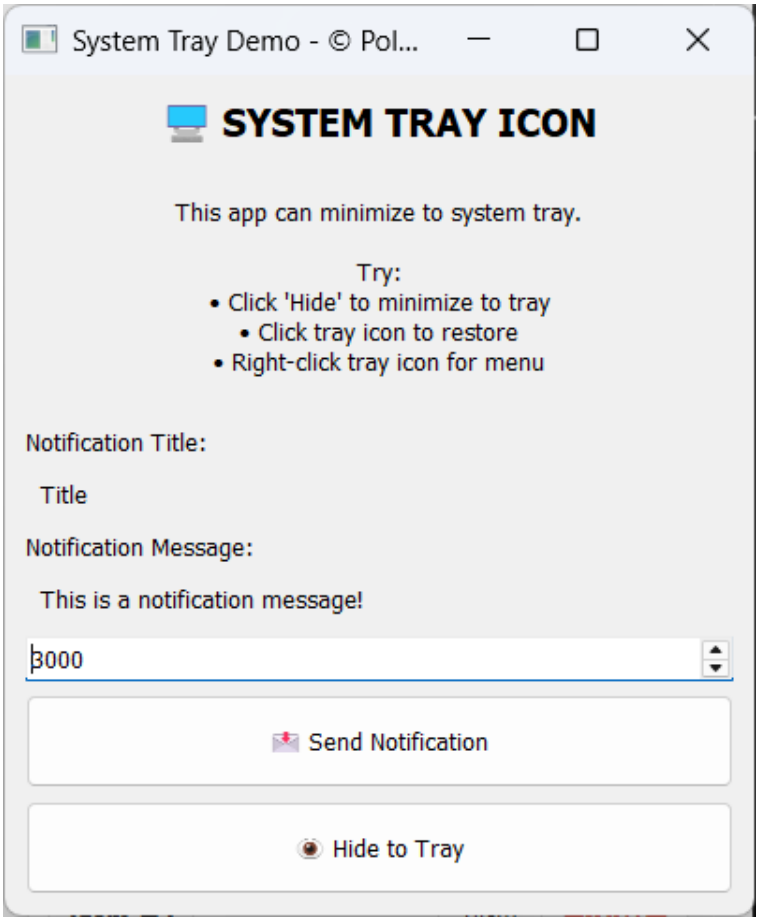

Figure 35.1: System Tray

Chapter 36

Custom Widget

36.1 Kode Program

```python
# BAB 36: Custom Widget
# ======================
# Membuat widget sendiri

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QWidget, QLabel, QVBoxLayout,
                            QHBoxLayout, QPushButton)
from PyQt5.QtCore import Qt, pyqtSignal
from PyQt5.QtGui import QPainter, QColor, QPen, QBrush, QFont


class CircularProgress(QWidget):
    """Custom widget: Progress circular"""

    def __init__(self, parent=None):
        super().__init__(parent)
        self._value = 0
        self._max_value = 100
        self._color = QColor("#3498db")
        self._bg_color = QColor("#ecf0f1")
        self._text_color = QColor("#2c3e50")

        self.setMinimumSize(120, 120)

    def setValue(self, value):
        self._value = min(max(value, 0), self._max_value)
        self.update()

    def value(self):
        return self._value

    def setColor(self, color):
        self._color = QColor(color)
        self.update()

    def paintEvent(self, event):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.Antialiasing)

        # Calculate dimensions
        size = min(self.width(), self.height())
```

```
        margin = 10
        pen_width = 15

        # Background circle
        painter.setPen(QPen(QBrush(self._bg_color), pen_width, Qt.SolidLine, Qt.RoundCap))
        center_x = self.width() // 2
        center_y = self.height() // 2
        radius = (size // 2) - margin - (pen_width // 2)

        painter.drawEllipse(center_x - radius, center_y - radius, radius * 2, radius * 2)

        # Progress arc
        if self._value > 0:
            progress = self._value / self._max_value
            angle = int(progress * 360 * 16) # Qt uses 1/16th degree

            painter.setPen(QPen(QBrush(self._color), pen_width, Qt.SolidLine, Qt.RoundCap))
            painter.drawArc(
                center_x - radius, center_y - radius,
                radius * 2, radius * 2,
                90 * 16, -angle # Start from top, go clockwise
            )

        # Text
        painter.setPen(self._text_color)
        painter.setFont(QFont("Arial", 20, QFont.Bold))
        painter.drawText(self.rect(), Qt.AlignCenter, f"{self._value}%")


class ToggleSwitch(QWidget):
    """Custom widget: Toggle switch"""

    toggled = pyqtSignal(bool)

    def __init__(self, parent=None):
        super().__init__(parent)
        self._checked = False
        self._color_on = QColor("#27ae60")
        self._color_off = QColor("#bdc3c7")

        self.setFixedSize(60, 30)
        self.setCursor(Qt.PointingHandCursor)

    def isChecked(self):
        return self._checked

    def setChecked(self, checked):
        self._checked = checked
        self.toggled.emit(checked)
        self.update()

    def paintEvent(self, event):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.Antialiasing)

        # Background
        color = self._color_on if self._checked else self._color_off
        painter.setBrush(QBrush(color))
        painter.setPen(Qt.NoPen)
        painter.drawRoundedRect(0, 0, self.width(), self.height(), 15, 15)

```

```
        # Circle
        circle_size = self.height() - 6
        circle_x = self.width() - circle_size - 3 if self._checked else 3
        painter.setBrush(QBrush(QColor("white")))
        painter.drawEllipse(circle_x, 3, circle_size, circle_size)

    def mousePressEvent(self, event):
        self.setChecked(not self._checked)


class StarRating(QWidget):
    """Custom widget: Star rating"""

    ratingChanged = pyqtSignal(int)

    def __init__(self, max_stars=5, parent=None):
        super().__init__(parent)
        self._rating = 0
        self._max_stars = max_stars
        self._hover = -1

        self.setMouseTracking(True)
        self.setFixedHeight(40)

    def rating(self):
        return self._rating

    def setRating(self, rating):
        self._rating = min(max(rating, 0), self._max_stars)
        self.ratingChanged.emit(self._rating)
        self.update()

    def paintEvent(self, event):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.Antialiasing)

        star_size = 30
        spacing = 5
        x = 0

        for i in range(self._max_stars):
            # Determine if star is filled
            if i < self._rating or i <= self._hover:
                color = QColor("#f1c40f") # Gold
            else:
                color = QColor("#bdc3c7") # Gray

            self.draw_star(painter, x, 5, star_size, color)
            x += star_size + spacing

    def draw_star(self, painter, x, y, size, color):
        painter.setBrush(QBrush(color))
        painter.setPen(Qt.NoPen)

        # Simple star using polygon
        from PyQt5.QtGui import QPolygonF
        from PyQt5.QtCore import QPointF

        points = []
        for i in range(10):
            angle = i * 36 - 90 # degrees
```

```
            import math
            rad = math.radians(angle)

            if i % 2 == 0:
                r = size // 2
            else:
                r = size // 4

            px = x + size // 2 + r * math.cos(rad)
            py = y + size // 2 + r * math.sin(rad)
            points.append(QPointF(px, py))

        painter.drawPolygon(QPolygonF(points))

    def mouseMoveEvent(self, event):
        star_size = 30
        spacing = 5
        x = event.x()

        self._hover = -1
        for i in range(self._max_stars):
            star_x = i * (star_size + spacing)
            if star_x <= x <= star_x + star_size:
                self._hover = i
                break

        self.update()

    def leaveEvent(self, event):
        self._hover = -1
        self.update()

    def mousePressEvent(self, event):
        if self._hover >= 0:
            self.setRating(self._hover + 1)


class CustomWidgetDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Custom Widget Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 500)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("🎨 CUSTOM WIDGETS")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Circular Progress
        layout.addWidget(QLabel("\n1. Circular Progress:"))

        progress_layout = QHBoxLayout()

        self.progress = CircularProgress()
        progress_layout.addWidget(self.progress)

        progress_btns = QVBoxLayout()
```

```python

        btn_0 = QPushButton("0%")
        btn_0.clicked.connect(lambda: self.progress.setValue(0))
        progress_btns.addWidget(btn_0)

        btn_50 = QPushButton("50%")
        btn_50.clicked.connect(lambda: self.progress.setValue(50))
        progress_btns.addWidget(btn_50)

        btn_100 = QPushButton("100%")
        btn_100.clicked.connect(lambda: self.progress.setValue(100))
        progress_btns.addWidget(btn_100)

        progress_layout.addLayout(progress_btns)
        layout.addLayout(progress_layout)

        # Toggle Switch
        layout.addWidget(QLabel("\n2. Toggle Switch:"))

        toggle_layout = QHBoxLayout()
        self.toggle = ToggleSwitch()
        self.toggle.toggled.connect(self.on_toggle)
        toggle_layout.addWidget(self.toggle)

        self.toggle_label = QLabel("OFF")
        toggle_layout.addWidget(self.toggle_label)
        toggle_layout.addStretch()

        layout.addLayout(toggle_layout)

        # Star Rating
        layout.addWidget(QLabel("\n3. Star Rating:"))

        rating_layout = QHBoxLayout()
        self.stars = StarRating()
        self.stars.ratingChanged.connect(self.on_rating_change)
        rating_layout.addWidget(self.stars)

        self.rating_label = QLabel("Rating: 0/5")
        rating_layout.addWidget(self.rating_label)
        rating_layout.addStretch()

        layout.addLayout(rating_layout)

        # Buttons
        btn_set_rating = QPushButton("Set Rating to 4")
        btn_set_rating.clicked.connect(lambda: self.stars.setRating(4))
        layout.addWidget(btn_set_rating)

        layout.addStretch()

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def on_toggle(self, checked):
        self.toggle_label.setText("ON" if checked else "OFF")

    def on_rating_change(self, rating):
        self.rating_label.setText(f"Rating: {rating}/5")


```

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = CustomWidgetDemo()
    window.show()
    sys.exit(app.exec_())
```

36.2 Penjelasan Kode

Widget bawaan Qt sudah lengkap, tapi kadang Anda butuh sesuatu yang unik—progress melingkar, toggle switch ala iOS, atau rating bintang. Solusinya: buat widget sendiri dengan meng-override `paintEvent()`.

1. CircularProgress: Menggambar Sendiri

Kelas ini mewarisi `QWidget` dan menyimpan nilai progres (0–100). Di `paintEvent()`, `QPainter` menggambar dua elemen: lingkaran latar (abu-abu) dan busur progres (biru) yang panjangnya proporsional terhadap nilai. Teks persentase ditulis di tengah. Memanggil `setValue()` memperbarui nilai lalu memanggil `update()` untuk me-repaint.

2. ToggleSwitch: Klik untuk On/Off

Widget ini menggambar rounded rectangle (latar) dan lingkaran (knob). Posisi knob bergeser ke kanan saat `_checked` bernilai `True`. Override `mousePressEvent` men-toggle state dan mengirim signal custom `toggled(bool)`—persis seperti signal bawaan Qt.

3. StarRating: Hover dan Klik

Lima bintang digambar menggunakan polygon. `mouseMoveEvent` mendeteksi bintang mana yang di-hover (highlight kuning), sementara `mousePressEvent` mengunci rating. Signal `ratingChanged(int)` dikirim setiap kali rating berubah. `setMouseTracking(True)` memastikan hover berfungsi tanpa klik.

Pelajaran Penting

- Override `paintEvent()` + `QPainter` adalah kunci membuat widget visual kustom.
- Definisikan signal kustom (`pyqtSignal`) agar widget Anda bisa berkomunikasi seperti widget bawaan.
- Selalu panggil `update()` setelah mengubah state agar Qt menjadwalkan repaint.

36.3 Hasil Tampilan

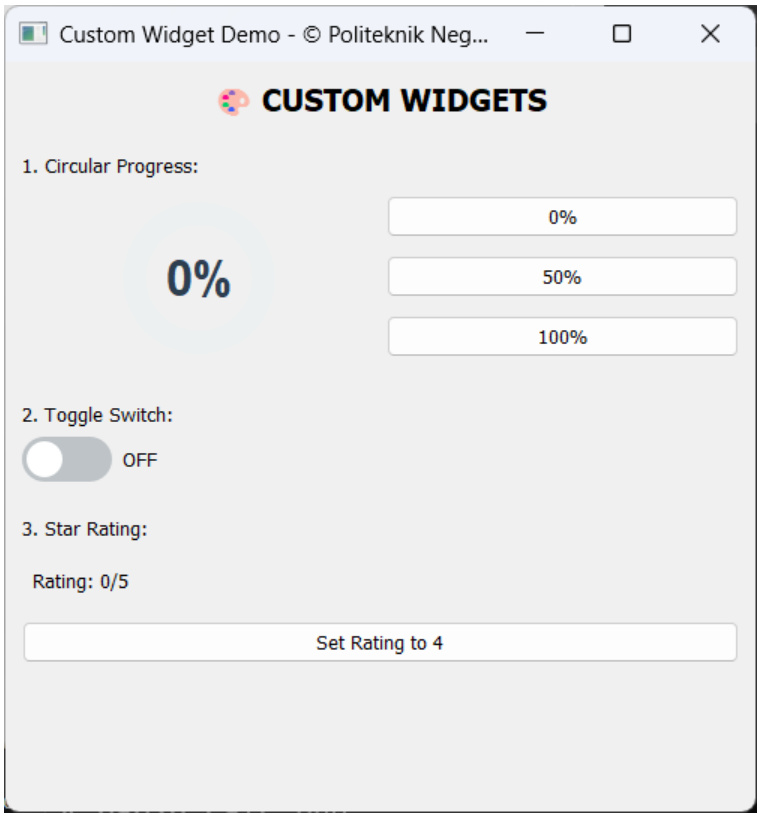

Figure 36.1: Custom Widget

Chapter 37

Form Layout

37.1 Kode Program

```python
# BAB 37: Form Layout
# ====================

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QWidget, QLabel, QLineEdit,
                            QSpinBox, QDoubleSpinBox, QComboBox,
                            QCheckBox, QDateEdit, QTextEdit,
                            QPushButton, QFormLayout, QVBoxLayout,
                            QGroupBox, QMessageBox)
from PyQt5.QtCore import Qt, QDate

class FormLayoutDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Form Layout Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 600)

        widget = QWidget()
        main_layout = QVBoxLayout()

        # Header
        header = QLabel("📝 FORM LAYOUT")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        main_layout.addWidget(header)

        # Personal Info Group
        group_personal = QGroupBox("Informasi Pribadi")
        form_personal = QFormLayout()

        # Name
        self.name = QLineEdit()
        self.name.setPlaceholderText("Nama lengkap")
        form_personal.addRow("Nama:", self.name)

        # Email
        self.email = QLineEdit()
        self.email.setPlaceholderText("email@contoh.com")
        form_personal.addRow("Email:", self.email)

        # Phone
```

```
        self.phone = QLineEdit()
        self.phone.setPlaceholderText("08xxxxxxxxxx")
        form_personal.addRow("Telepon:", self.phone)

        # Gender
        self.gender = QComboBox()
        self.gender.addItems(["Laki-laki", "Perempuan"])
        form_personal.addRow("Jenis Kelamin:", self.gender)

        # Birth Date
        self.birthdate = QDateEdit()
        self.birthdate.setCalendarPopup(True)
        self.birthdate.setDate(QDate(2000, 1, 1))
        self.birthdate.setDisplayFormat("dd/MM/yyyy")
        form_personal.addRow("Tanggal Lahir:", self.birthdate)

        group_personal.setLayout(form_personal)
        main_layout.addWidget(group_personal)

        # Address Group
        group_address = QGroupBox("Alamat")
        form_address = QFormLayout()

        # Street
        self.street = QLineEdit()
        self.street.setPlaceholderText("Nama jalan dan nomor")
        form_address.addRow("Jalan:", self.street)

        # City
        self.city = QComboBox()
        self.city.addItems(["Jakarta", "Bandung", "Surabaya", "Yogyakarta", "Lainnya"])
        self.city.setEditable(True)
        form_address.addRow("Kota:", self.city)

        # Postal Code
        self.postal = QSpinBox()
        self.postal.setRange(10000, 99999)
        self.postal.setPrefix("Kode Pos: ")
        form_address.addRow("", self.postal)

        # Full Address
        self.full_address = QTextEdit()
        self.full_address.setPlaceholderText("Alamat lengkap...")
        self.full_address.setMaximumHeight(80)
        form_address.addRow("Alamat Lengkap:", self.full_address)

        group_address.setLayout(form_address)
        main_layout.addWidget(group_address)

        # Preferences Group
        group_prefs = QGroupBox("Preferensi")
        form_prefs = QFormLayout()

        # Experience
        self.experience = QSpinBox()
        self.experience.setRange(0, 50)
        self.experience.setSuffix(" tahun")
        form_prefs.addRow("Pengalaman:", self.experience)

        # Expected Salary
        self.salary = QDoubleSpinBox()
```

```
        self.salary.setRange(0, 1000000000)
        self.salary.setPrefix("Rp ")
        self.salary.setDecimals(0)
        self.salary.setSingleStep(1000000)
        form_prefs.addRow("Gaji yang Diharapkan:", self.salary)

        # Agreement
        self.agree = QCheckBox("Saya setuju dengan syarat dan ketentuan")
        form_prefs.addRow("", self.agree)

        # Newsletter
        self.newsletter = QCheckBox("Berlangganan newsletter")
        self.newsletter.setChecked(True)
        form_prefs.addRow("", self.newsletter)

        group_prefs.setLayout(form_prefs)
        main_layout.addWidget(group_prefs)

        # Buttons
        btn_layout = QVBoxLayout()

        btn_submit = QPushButton("📝 Submit")
        btn_submit.setStyleSheet("background-color: #27ae60; color: white; padding: 15px;
            font-size: 14px;")
        btn_submit.clicked.connect(self.submit_form)
        btn_layout.addWidget(btn_submit)

        btn_clear = QPushButton("🗑️ Clear Form")
        btn_clear.clicked.connect(self.clear_form)
        btn_layout.addWidget(btn_clear)

        main_layout.addLayout(btn_layout)

        widget.setLayout(main_layout)
        self.setCentralWidget(widget)

    def submit_form(self):
        if not self.name.text().strip():
            QMessageBox.warning(self, "Error", "Nama harus diisi!")
            return

        if not self.email.text().strip():
            QMessageBox.warning(self, "Error", "Email harus diisi!")
            return

        if not self.agree.isChecked():
            QMessageBox.warning(self, "Error", "Anda harus menyetujui syarat dan ketentuan!")
            return

        # Success
        data = f"""
Nama: {self.name.text()}
Email: {self.email.text()}
Telepon: {self.phone.text()}
Gender: {self.gender.currentText()}
Tanggal Lahir: {self.birthdate.text()}
Kota: {self.city.currentText()}
Pengalaman: {self.experience.value()} tahun
Gaji: Rp {self.salary.value():,.0f}
Newsletter: {'Ya' if self.newsletter.isChecked() else 'Tidak'}
        """
```

```
        QMessageBox.information(self, "Data Submitted", f"Data berhasil disimpan!{data}")

    def clear_form(self):
        self.name.clear()
        self.email.clear()
        self.phone.clear()
        self.gender.setCurrentIndex(0)
        self.birthdate.setDate(QDate(2000, 1, 1))
        self.street.clear()
        self.city.setCurrentIndex(0)
        self.postal.setValue(10000)
        self.full_address.clear()
        self.experience.setValue(0)
        self.salary.setValue(0)
        self.agree.setChecked(False)
        self.newsletter.setChecked(True)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = FormLayoutDemo()
    window.show()
    sys.exit(app.exec_())
```

37.2 Penjelasan Kode

Formulir pendaftaran, profil pengguna, pengaturan—semua butuh pasangan label dan input yang rapi. `QFormLayout` menata semuanya secara otomatis dalam dua kolom: label di kiri, widget di kanan.

1. Tiga GroupBox, Tiga Kategori

Form dibagi menjadi tiga `QGroupBox`: Informasi Pribadi (nama, email, telepon, gender, tanggal lahir), Alamat (jalan, kota, kode pos, alamat lengkap), dan Preferensi (pengalaman, gaji, persetujuan). Pengelompokan ini membuat form panjang terasa terorganisir.

2. Ragam Widget Input

Skrip mendemonstrasikan hampir semua widget input: `QLineEdit` (teks bebas), `QComboBox` (pilihan, bahkan editable untuk kota), `QDateEdit` (tanggal lahir dengan kalender pop-up), `QSpinBox` (angka bulat untuk pengalaman dan kode pos), `QDoubleSpinBox` (angka desimal untuk gaji dengan prefix "Rp"), `QCheckBox` (persetujuan), dan `QTextEdit` (alamat panjang).

3. Validasi Saat Submit

Tombol Submit memanggil `submit_form()` yang memeriksa: nama harus diisi, email harus diisi, dan checkbox persetujuan harus tercentang. Jika gagal, `QMessageBox.warning` menampilkan pesan error. Jika sukses, semua data ditampilkan dalam dialog informasi.

4. Clear Form

Tombol Clear mereset semua widget ke nilai default—menunjukkan cara mengosongkan setiap tipe widget secara programatik.

Pelajaran Penting

- `QFormLayout` menghilangkan kerumitan menata label dan input secara manual.
- `QGroupBox` membagi form panjang menjadi seksi yang mudah dipahami.
- Validasi sebelum submit mencegah data tidak lengkap masuk ke sistem.

37.3 Hasil Tampilan

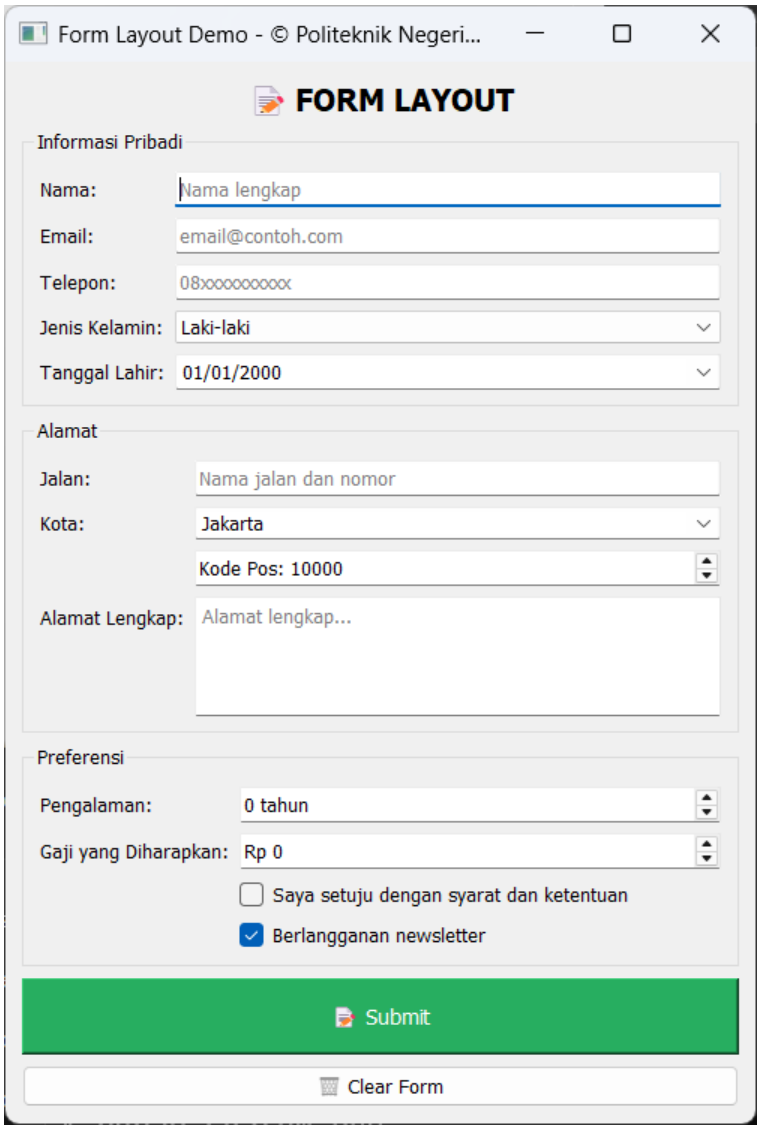

Figure 37.1: Form Layout

Chapter 38

Validator

38.1 Kode Program

```python
# BAB 38: Validator
# =================
# Input validation dengan QValidator

import sys
import re
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QWidget, QLabel, QLineEdit,
                            QPushButton, QVBoxLayout, QFormLayout,
                            QGroupBox)
from PyQt5.QtCore import Qt, QRegularExpression
from PyQt5.QtGui import QValidator, QIntValidator, QDoubleValidator, QRegularExpressionValidator


class EmailValidator(QValidator):
    """Custom validator untuk email"""

    def validate(self, input_str, pos):
        # Simple email regex
        pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

        if not input_str:
            return (QValidator.Intermediate, input_str, pos)

        if re.match(pattern, input_str):
            return (QValidator.Acceptable, input_str, pos)
        else:
            return (QValidator.Intermediate, input_str, pos)

    def fixup(self, input_str):
        return input_str.lower()


class PhoneValidator(QValidator):
    """Custom validator untuk nomor telepon Indonesia"""

    def validate(self, input_str, pos):
        # Format: 08xxxxxxxxxx (10-13 digit)
        pattern = r'^08[0-9]{0,11}$'

        if not input_str:
            return (QValidator.Intermediate, input_str, pos)
```

```
        if re.match(pattern, input_str) and len(input_str) >= 10:
            return (QValidator.Acceptable, input_str, pos)
        elif re.match(pattern, input_str):
            return (QValidator.Intermediate, input_str, pos)
        else:
            return (QValidator.Invalid, input_str, pos)


class ValidatorDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Validator Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 500)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("✅ INPUT VALIDATOR")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        info = QLabel("Validator membatasi input yang bisa dimasukkan user")
        info.setStyleSheet("color: gray;")
        info.setAlignment(Qt.AlignCenter)
        layout.addWidget(info)

        # Integer validation
        group_int = QGroupBox("Integer Validator (QIntValidator)")
        form_int = QFormLayout()

        self.int_input = QLineEdit()
        self.int_input.setPlaceholderText("Hanya angka 0-100")

        # Set validator
        int_validator = QIntValidator(0, 100)
        self.int_input.setValidator(int_validator)

        form_int.addRow("Angka (0-100):", self.int_input)

        self.int_status = QLabel("Status: -")
        form_int.addRow("", self.int_status)

        self.int_input.textChanged.connect(self.check_int)
        group_int.setLayout(form_int)
        layout.addWidget(group_int)

        # Double validation
        group_double = QGroupBox("Double Validator (QDoubleValidator)")
        form_double = QFormLayout()

        self.double_input = QLineEdit()
        self.double_input.setPlaceholderText("Angka desimal 0.0 - 1000.0")

        # Set validator
        double_validator = QDoubleValidator(0.0, 1000.0, 2) # 2 decimal places
        double_validator.setNotation(QDoubleValidator.StandardNotation)
        self.double_input.setValidator(double_validator)

```

```
        form_double.addRow("Harga (0-1000):", self.double_input)

        self.double_status = QLabel("Status: -")
        form_double.addRow("", self.double_status)

        self.double_input.textChanged.connect(self.check_double)
        group_double.setLayout(form_double)
        layout.addWidget(group_double)

        # Email validation
        group_email = QGroupBox("Custom Email Validator")
        form_email = QFormLayout()

        self.email_input = QLineEdit()
        self.email_input.setPlaceholderText("email@contoh.com")
        self.email_input.setValidator(EmailValidator())

        form_email.addRow("Email:", self.email_input)

        self.email_status = QLabel("Status: -")
        form_email.addRow("", self.email_status)

        self.email_input.textChanged.connect(self.check_email)
        group_email.setLayout(form_email)
        layout.addWidget(group_email)

        # Phone validation
        group_phone = QGroupBox("Custom Phone Validator (Indonesia)")
        form_phone = QFormLayout()

        self.phone_input = QLineEdit()
        self.phone_input.setPlaceholderText("08xxxxxxxxxx")
        self.phone_input.setValidator(PhoneValidator())

        form_phone.addRow("Telepon:", self.phone_input)

        self.phone_status = QLabel("Status: -")
        form_phone.addRow("", self.phone_status)

        self.phone_input.textChanged.connect(self.check_phone)
        group_phone.setLayout(form_phone)
        layout.addWidget(group_phone)

        # Regex validation
        group_regex = QGroupBox("Regex Validator (Kode Produk: ABC-123)")
        form_regex = QFormLayout()

        self.regex_input = QLineEdit()
        self.regex_input.setPlaceholderText("Format: ABC-123")

        # Regex: 3 huruf besar, strip, 3 digit
        regex = QRegularExpression(r'^[A-Z]{0,3}-?[0-9]{0,3}$')
        self.regex_input.setValidator(QRegularExpressionValidator(regex))

        form_regex.addRow("Kode Produk:", self.regex_input)

        self.regex_status = QLabel("Status: -")
        form_regex.addRow("", self.regex_status)

        self.regex_input.textChanged.connect(self.check_regex)
        group_regex.setLayout(form_regex)
```

```
        layout.addWidget(group_regex)

        # Submit button
        btn = QPushButton("Validate All")
        btn.setStyleSheet("padding: 15px;")
        btn.clicked.connect(self.validate_all)
        layout.addWidget(btn)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def get_status_color(self, validator, text):
        if not text:
            return "gray", "Empty"

        state, _, _ = validator.validate(text, 0)

        if state == QValidator.Acceptable:
            return "#27ae60", "✅ Valid"
        elif state == QValidator.Intermediate:
            return "#f39c12", "⏳ Incomplete"
        else:
            return "#e74c3c", "❌ Invalid"

    def check_int(self):
        validator = self.int_input.validator()
        color, status = self.get_status_color(validator, self.int_input.text())
        self.int_status.setStyleSheet(f"color: {color};")
        self.int_status.setText(f"Status: {status}")

    def check_double(self):
        validator = self.double_input.validator()
        color, status = self.get_status_color(validator, self.double_input.text())
        self.double_status.setStyleSheet(f"color: {color};")
        self.double_status.setText(f"Status: {status}")

    def check_email(self):
        validator = self.email_input.validator()
        color, status = self.get_status_color(validator, self.email_input.text())
        self.email_status.setStyleSheet(f"color: {color};")
        self.email_status.setText(f"Status: {status}")

    def check_phone(self):
        validator = self.phone_input.validator()
        color, status = self.get_status_color(validator, self.phone_input.text())
        self.phone_status.setStyleSheet(f"color: {color};")
        self.phone_status.setText(f"Status: {status}")

    def check_regex(self):
        validator = self.regex_input.validator()
        color, status = self.get_status_color(validator, self.regex_input.text())
        self.regex_status.setStyleSheet(f"color: {color};")
        self.regex_status.setText(f"Status: {status}")

    def validate_all(self):
        results = []

        for field, name in [
            (self.int_input, "Integer"),
            (self.double_input, "Double"),
            (self.email_input, "Email"),
```

```
            (self.phone_input, "Phone"),
            (self.regex_input, "Regex")
        ]:
            validator = field.validator()
            text = field.text()

            if validator:
                state, _, _ = validator.validate(text, 0)
                if state == QValidator.Acceptable:
                    results.append(f"✅ {name}: Valid")
                elif state == QValidator.Intermediate:
                    results.append(f"⏳ {name}: Incomplete")
                else:
                    results.append(f"❌ {name}: Invalid")

        from PyQt5.QtWidgets import QMessageBox
        QMessageBox.information(self, "Validation Results", "\n".join(results))


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = ValidatorDemo()
    window.show()
    sys.exit(app.exec_())
```

38.2 Penjelasan Kode

Daripada memvalidasi input saat tombol Submit ditekan, lebih baik mencegah input salah sejak awal. `QValidator` membatasi apa yang bisa diketik pengguna di `QLineEdit`—karakter yang tidak valid bahkan tidak bisa dimasukkan.

1. QIntValidator dan QDoubleValidator

Validator bawaan Qt: `QIntValidator(0, 100)` hanya mengizinkan angka bulat 0–100. `QDoubleValidator(0.0, 1000.0, 2)` mengizinkan angka desimal dengan maksimal 2 angka di belakang koma. Cukup panggil `setValidator()` pada `QLineEdit`.

2. Custom Validator: Email dan Telepon

Untuk aturan yang lebih kompleks, buat kelas turunan `QValidator` dan override `validate(input, pos)`. Metode ini mengembalikan salah satu dari tiga status:

- `Acceptable`: input valid dan lengkap.
- `Intermediate`: input belum lengkap tapi masih mungkin valid (sedang diketik).
- `Invalid`: input pasti salah—karakter ditolak.

`EmailValidator` menggunakan regex untuk memeriksa format email. `PhoneValidator` memastikan nomor dimulai "08" dan panjang 10–13 digit.

3. QRegularExpressionValidator

Untuk pola sederhana, cukup gunakan regex langsung: `QRegularExpressionValidator(QRegularExpression(pattern))`. Contoh: kode produk format "ABC-123" (3 huruf besar, strip, 3 digit).

4. Status Visual Real-Time

Setiap `textChanged` signal memanggil fungsi pengecekan yang mengubah warna label status: hijau (valid), kuning (belum lengkap), merah (invalid). Pengguna langsung tahu apakah inputnya benar tanpa menunggu submit.

Pelajaran Penting

- Validator mencegah input salah *sebelum* masuk ke logika aplikasi.
- Tiga status (Acceptable, Intermediate, Invalid) membuat UX terasa responsif.
- Custom validator memberi fleksibilitas penuh untuk aturan bisnis apa pun.

38.3 Hasil Tampilan

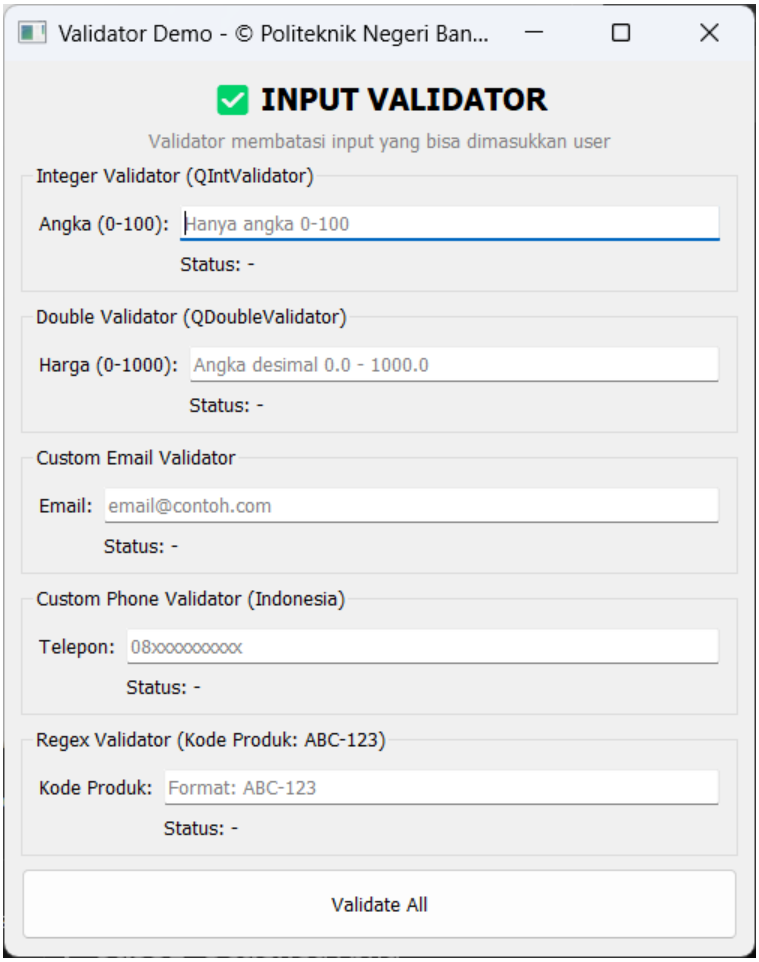

Figure 38.1: Validator

Chapter 39

Splash Screen

39.1 Kode Program

```python
# BAB 39: Splash Screen
# ====================
# Loading screen saat app startup

import sys
import time
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QSplashScreen, QLabel, QVBoxLayout,
                            QWidget, QProgressBar, QPushButton)
from PyQt5.QtCore import Qt, QTimer
from PyQt5.QtGui import QPixmap, QPainter, QColor, QFont


class CustomSplashScreen(QSplashScreen):
    """Custom splash screen dengan progress bar"""

    def __init__(self):
        # Create a pixmap for splash
        pixmap = QPixmap(400, 300)
        pixmap.fill(QColor("#2c3e50"))

        # Draw on pixmap
        painter = QPainter(pixmap)

        # Title
        painter.setPen(QColor("white"))
        painter.setFont(QFont("Arial", 28, QFont.Bold))
        painter.drawText(pixmap.rect(), Qt.AlignHCenter | Qt.AlignTop, "\n\nMyApp")

        # Subtitle
        painter.setFont(QFont("Arial", 12))
        painter.setPen(QColor("#bdc3c7"))
        painter.drawText(pixmap.rect(), Qt.AlignHCenter | Qt.AlignTop, "\n\n\n\n\nVersion 1.0.0")

        # Loading text placeholder
        painter.setFont(QFont("Arial", 10))
        painter.setPen(QColor("#3498db"))
        painter.drawText(20, 250, "Loading...")

        painter.end()

        super().__init__(pixmap)
```

```python

        self.progress = 0
        self.messages = [
            "Initializing...",
            "Loading modules...",
            "Setting up database...",
            "Loading preferences...",
            "Preparing interface...",
            "Almost ready...",
            "Welcome!"
        ]
        self.message_index = 0

    def update_progress(self, value):
        self.progress = value
        self.update() # Trigger repaint

    def drawContents(self, painter):
        # Draw progress bar
        bar_width = 360
        bar_height = 10
        bar_x = 20
        bar_y = 270

        # Background
        painter.setBrush(QColor("#34495e"))
        painter.setPen(Qt.NoPen)
        painter.drawRoundedRect(bar_x, bar_y, bar_width, bar_height, 5, 5)

        # Progress
        progress_width = int(bar_width * self.progress / 100)
        painter.setBrush(QColor("#3498db"))
        painter.drawRoundedRect(bar_x, bar_y, progress_width, bar_height, 5, 5)

        # Message
        if self.message_index < len(self.messages):
            painter.setPen(QColor("#3498db"))
            painter.drawText(20, 250, self.messages[self.message_index])

    def next_message(self):
        if self.message_index < len(self.messages) - 1:
            self.message_index += 1


class SplashScreenDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("MyApp - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 600, 400)

        widget = QWidget()
        layout = QVBoxLayout()

        # Welcome message
        header = QLabel("🎉 Welcome to MyApp!")
        header.setStyleSheet("font-size: 24px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        info = QLabel("The splash screen showed while this app was loading.\n\n"
                      "In a real app, the splash screen would show during:\n"
```

```
                    "• Database initialization\n"
                    "• Loading configuration\n"
                    "• Preparing resources")
        info.setAlignment(Qt.AlignCenter)
        info.setStyleSheet("color: gray; padding: 20px;")
        layout.addWidget(info)

        # Show splash again button
        btn = QPushButton("Show Splash Screen Again")
        btn.clicked.connect(self.show_splash_again)
        btn.setStyleSheet("padding: 15px;")
        layout.addWidget(btn)

        layout.addStretch()

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def show_splash_again(self):
        # Create and show splash
        splash = CustomSplashScreen()
        splash.show()

        # Simulate loading
        def update():
            current = splash.progress
            if current < 100:
                splash.update_progress(current + 10)
                if current % 15 == 0:
                    splash.next_message()
                QTimer.singleShot(200, update)
            else:
                splash.finish(self)

        update()


def main():
    app = QApplication(sys.argv)

    # Create splash screen
    splash = CustomSplashScreen()
    splash.show()

    # Create main window
    window = SplashScreenDemo()

    # Simulate loading process
    def update_splash():
        current = splash.progress
        if current < 100:
            splash.update_progress(current + 10)
            if current % 15 == 0:
                splash.next_message()
            QTimer.singleShot(200, update_splash)
        else:
            # Loading complete, show main window
            splash.finish(window)
            window.show()

    # Start loading simulation
```

```
    QTimer.singleShot(100, update_splash)

    sys.exit(app.exec_())


if __name__ == "__main__":
    main()
```

39.2 Penjelasan Kode

Saat aplikasi besar memuat resource—database, konfigurasi, plugin—pengguna melihat layar kosong. Splash screen mengisi kekosongan itu dengan branding dan progress bar.

1. CustomSplashScreen dengan QPainter
Kelas ini mewarisi `QSplashScreen`. Di konstruktor, `QPixmap` 400×300 piksel dibuat dan diisi background gelap. `QPainter` menggambar judul "MyApp" dan subtitle versi langsung ke pixmap. Hasilnya: gambar splash tanpa file eksternal.

2. Progress Bar Kustom di drawContents()
Override `drawContents()` menggambar progress bar dan pesan loading langsung di atas pixmap. Setiap kali `update_progress()` dipanggil, `update()` memicu repaint sehingga bar tumbuh dan pesan berubah ("Initializing...", "Loading modules...", dll.).

3. Simulasi Loading dengan QTimer
Di `main()`, splash ditampilkan sebelum main window dibuat. `QTimer.singleShot(200, update_splash)` dipanggil berulang—setiap 200ms progress bertambah 10%. Saat mencapai 100%, `splash.finish(window)` menutup splash dan menampilkan jendela utama.

4. Menampilkan Ulang
Tombol "Show Splash Screen Again" di main window membuktikan bahwa splash screen bisa ditampilkan kapan saja, bukan hanya saat startup.

Pelajaran Penting
• Splash screen meningkatkan *perceived performance*—pengguna merasa aplikasi responsif meski sedang loading.
• Gambar splash bisa dibuat secara programatik dengan `QPainter`, tanpa file gambar.
• `splash.finish(mainWindow)` otomatis menutup splash saat jendela utama muncul.

39.3 Hasil Tampilan

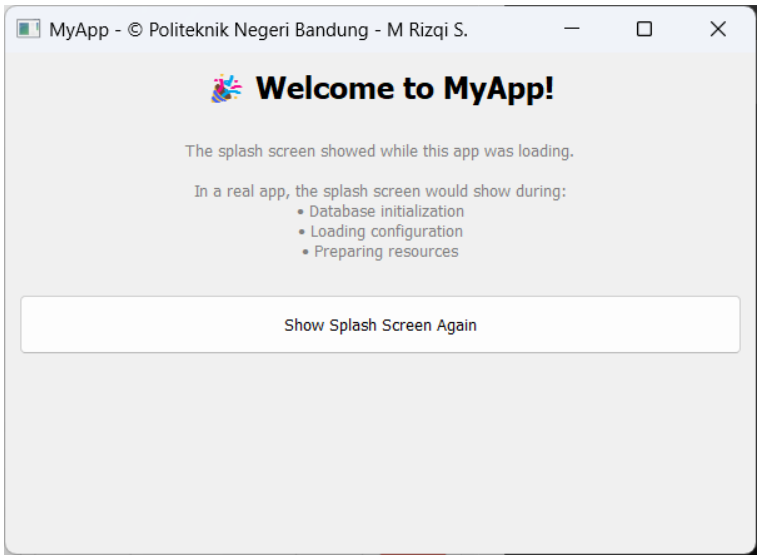

Figure 39.1: Splash Screen

Chapter 40

Status Bar

40.1 Kode Program

```python
# BAB 40: Status Bar
# ===================
# Status bar dengan informasi dan widget

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLabel, QPushButton, QLineEdit,
                            QProgressBar, QWidget, QVBoxLayout,
                            QHBoxLayout)
from PyQt5.QtCore import Qt, QTimer


class StatusBarDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Status Bar Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 600, 400)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("📊 STATUS BAR")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Instructions
        info = QLabel("Lihat status bar di bawah untuk informasi.\n"
                      "Status bar bisa menampilkan teks, widget, dan progress.")
        info.setAlignment(Qt.AlignCenter)
        info.setStyleSheet("color: gray; padding: 20px;")
        layout.addWidget(info)

        # Buttons to interact with status bar
        btn_layout = QVBoxLayout()

        btn_msg = QPushButton("Show Temporary Message")
        btn_msg.clicked.connect(lambda: self.statusBar().showMessage("Hello! This message will
            disappear in 3 seconds", 3000))
        btn_layout.addWidget(btn_msg)

```

```
        btn_perm = QPushButton("Show Permanent Message")
        btn_perm.clicked.connect(lambda: self.status_label.setText("Permanent message"))
        btn_layout.addWidget(btn_perm)

        btn_clear = QPushButton("Clear Status")
        btn_clear.clicked.connect(lambda: self.status_label.setText("Ready"))
        btn_layout.addWidget(btn_clear)

        btn_progress = QPushButton("Show Progress")
        btn_progress.clicked.connect(self.show_progress)
        btn_layout.addWidget(btn_progress)

        btn_coords = QPushButton("Toggle Coordinates Display")
        btn_coords.clicked.connect(self.toggle_coords)
        btn_layout.addWidget(btn_coords)

        layout.addLayout(btn_layout)

        # Input to demonstrate
        input_layout = QHBoxLayout()
        input_layout.addWidget(QLabel("Type here:"))

        self.input = QLineEdit()
        self.input.textChanged.connect(self.on_text_changed)
        input_layout.addWidget(self.input)

        layout.addLayout(input_layout)

        # Mouse position display
        self.mouse_label = QLabel("Mouse: (0, 0)")
        layout.addWidget(self.mouse_label)

        layout.addStretch()

        widget.setLayout(layout)
        self.setCentralWidget(widget)

        # Setup status bar
        self.setup_status_bar()

        # Timer for clock
        self.timer = QTimer()
        self.timer.timeout.connect(self.update_clock)
        self.timer.start(1000)

        self.show_coords = True

    def setup_status_bar(self):
        status = self.statusBar()

        # Normal message
        status.showMessage("Ready")

        # Permanent widgets (from right to left)

        # Progress bar
        self.progress = QProgressBar()
        self.progress.setFixedWidth(150)
        self.progress.setValue(0)
        status.addPermanentWidget(self.progress)

```

```
        # Status label
        self.status_label = QLabel(" Ready ")
        self.status_label.setStyleSheet("padding: 0 10px;")
        status.addPermanentWidget(self.status_label)

        # Coordinates
        self.coords_label = QLabel(" (0, 0) ")
        status.addPermanentWidget(self.coords_label)

        # Clock
        self.clock_label = QLabel(" 00:00:00 ")
        self.clock_label.setStyleSheet("padding: 0 10px; background: #3498db; color: white;
            border-radius: 3px;")
        status.addPermanentWidget(self.clock_label)

        # Status bar style
        status.setStyleSheet("""
            QStatusBar {
                background-color: #f8f9fa;
                border-top: 1px solid #dee2e6;
            }
            QStatusBar::item {
                border: none;
            }
        """)

    def update_clock(self):
        from datetime import datetime
        now = datetime.now()
        self.clock_label.setText(f" {now.strftime('%H:%M:%S')} ")

    def show_progress(self):
        self.progress.setValue(0)

        def update():
            current = self.progress.value()
            if current < 100:
                self.progress.setValue(current + 5)
                QTimer.singleShot(100, update)

        update()

    def toggle_coords(self):
        self.show_coords = not self.show_coords
        self.coords_label.setVisible(self.show_coords)

    def on_text_changed(self, text):
        self.status_label.setText(f" Typing... ({len(text)} chars) ")

    def mouseMoveEvent(self, event):
        x, y = event.x(), event.y()
        self.mouse_label.setText(f"Mouse: ({x}, {y})")

        if self.show_coords:
            self.coords_label.setText(f" ({x}, {y}) ")

        super().mouseMoveEvent(event)

    def enterEvent(self, event):
        self.statusBar().showMessage("Mouse entered window")
        super().enterEvent(event)
```

```
    def leaveEvent(self, event):
        self.statusBar().showMessage("Mouse left window")
        super().leaveEvent(event)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = StatusBarDemo()
    window.setMouseTracking(True)
    window.show()
    sys.exit(app.exec_())
```

40.2 Penjelasan Kode

Status bar—strip tipis di bagian bawah jendela—adalah tempat ideal untuk menampilkan informasi sekunder: status koneksi, posisi kursor, jam, atau progress background task.

1. Pesan Sementara dan Permanen

`statusBar().showMessage(text, timeout)` menampilkan pesan yang hilang setelah `timeout` milidetik. Widget permanen ditambahkan via `addPermanentWidget()`—mereka tetap terlihat meskipun pesan sementara muncul.

2. Widget di Status Bar

Skrip menambahkan empat widget permanen: `QProgressBar` (150px lebar), label status, label koordinat mouse, dan label jam digital. Jam diperbarui setiap detik oleh `QTimer`. Masing-masing di-style dengan CSS agar tampak rapi.

3. Interaksi Real-Time

- Mengetik di input field memperbarui label status dengan jumlah karakter.
- Menggerakkan mouse memperbarui label koordinat (x, y).
- Tombol "Show Progress" menjalankan animasi progress bar dari 0 ke 100.
- Tombol "Toggle Coordinates" menyembunyikan/menampilkan label koordinat.

4. Mouse Enter/Leave

Override `enterEvent` dan `leaveEvent` menampilkan pesan saat mouse masuk atau keluar jendela—demonstrasi bahwa status bar bisa merespons event apa pun.

Pelajaran Penting

- Status bar bawaan `QMainWindow` langsung tersedia via `statusBar()`.
- Gunakan pesan sementara untuk notifikasi singkat, widget permanen untuk info terus-menerus.
- CSS pada `QStatusBar` dan child widget-nya membuat status bar terlihat profesional.

40.3 Hasil Tampilan

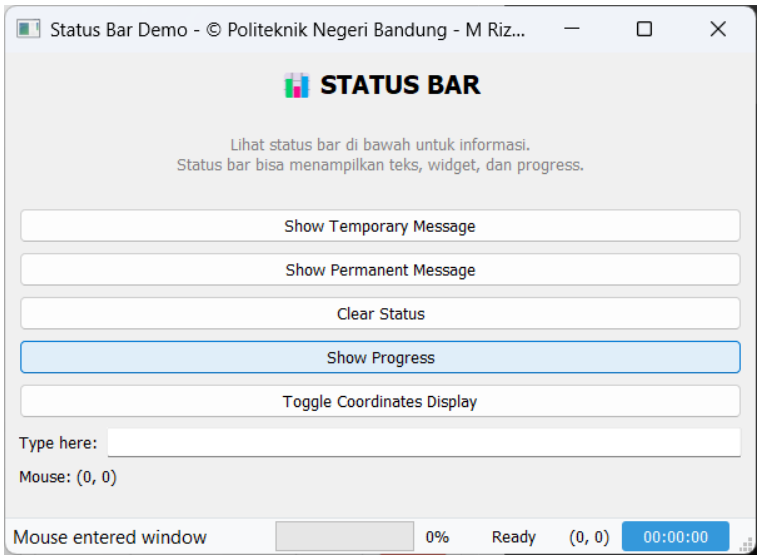

Figure 40.1: Status Bar

Chapter 41

Notes App

41.1 Kode Program

```
# BAB 41: Proyek Akhir - Notes App
# ================================
# Aplikasi catatan lengkap dengan fitur CRUD

import sys
import json
import os
from datetime import datetime
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QTextEdit, QListWidget, QListWidgetItem,
                            QLineEdit, QPushButton, QLabel,
                            QVBoxLayout, QHBoxLayout, QWidget,
                            QSplitter, QMessageBox, QInputDialog,
                            QFileDialog)
from PyQt5.QtCore import Qt, QSize
from PyQt5.QtGui import QFont, QIcon

class Note:
    """Model untuk satu catatan"""
    def __init__(self, title="", content="", created=None, modified=None):
        self.title = title
        self.content = content
        self.created = created or datetime.now().isoformat()
        self.modified = modified or datetime.now().isoformat()

    def to_dict(self):
        return {
            "title": self.title,
            "content": self.content,
            "created": self.created,
            "modified": self.modified
        }

    @classmethod
    def from_dict(cls, data):
        return cls(
            title=data.get("title", ""),
            content=data.get("content", ""),
            created=data.get("created"),
            modified=data.get("modified")
        )

```

```python

class NotesApp(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("📝 Notes App - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 800, 600)

        self.notes = []
        self.current_note = None
        self.file_path = os.path.join(os.path.dirname(__file__), "notes.json")

        self.setStyleSheet("""
            QMainWindow {
                background-color: #f5f5f5;
            }
            QListWidget {
                background-color: white;
                border: 1px solid #ddd;
                border-radius: 5px;
            }
            QListWidget::item {
                padding: 10px;
                border-bottom: 1px solid #eee;
            }
            QListWidget::item:selected {
                background-color: #3498db;
                color: white;
            }
            QTextEdit {
                background-color: white;
                border: 1px solid #ddd;
                border-radius: 5px;
                padding: 10px;
                font-size: 14px;
            }
            QLineEdit {
                padding: 10px;
                border: 1px solid #ddd;
                border-radius: 5px;
                font-size: 14px;
                font-weight: bold;
            }
            QPushButton {
                padding: 8px 15px;
                border-radius: 5px;
                font-weight: bold;
            }
        """)

        self.setup_ui()
        self.load_notes()

    def setup_ui(self):
        # Main widget
        main_widget = QWidget()
        main_layout = QVBoxLayout()

        # Header
        header = QLabel("📝 MY NOTES")
        header.setStyleSheet("font-size: 24px; font-weight: bold; padding: 10px;")
        header.setAlignment(Qt.AlignCenter)
```

```
        main_layout.addWidget(header)

        # Toolbar
        toolbar = QHBoxLayout()

        btn_new = QPushButton("➕ New")
        btn_new.setStyleSheet("background-color: #27ae60; color: white;")
        btn_new.clicked.connect(self.new_note)
        toolbar.addWidget(btn_new)

        btn_delete = QPushButton("🗑️ Delete")
        btn_delete.setStyleSheet("background-color: #e74c3c; color: white;")
        btn_delete.clicked.connect(self.delete_note)
        toolbar.addWidget(btn_delete)

        btn_save = QPushButton("💾 Save")
        btn_save.setStyleSheet("background-color: #3498db; color: white;")
        btn_save.clicked.connect(self.save_current_note)
        toolbar.addWidget(btn_save)

        toolbar.addStretch()

        btn_export = QPushButton("📤 Export")
        btn_export.clicked.connect(self.export_note)
        toolbar.addWidget(btn_export)

        btn_import = QPushButton("📥 Import")
        btn_import.clicked.connect(self.import_notes)
        toolbar.addWidget(btn_import)

        main_layout.addLayout(toolbar)

        # Splitter
        splitter = QSplitter(Qt.Horizontal)

        # Left panel - Notes list
        left_panel = QWidget()
        left_layout = QVBoxLayout()

        # Search
        self.search = QLineEdit()
        self.search.setPlaceholderText("🔍 Search notes...")
        self.search.textChanged.connect(self.filter_notes)
        left_layout.addWidget(self.search)

        # Notes list
        self.notes_list = QListWidget()
        self.notes_list.currentItemChanged.connect(self.on_note_selected)
        left_layout.addWidget(self.notes_list)

        # Stats
        self.stats = QLabel("0 notes")
        self.stats.setStyleSheet("color: gray; padding: 5px;")
        left_layout.addWidget(self.stats)

        left_panel.setLayout(left_layout)
        splitter.addWidget(left_panel)

        # Right panel - Note editor
        right_panel = QWidget()
        right_layout = QVBoxLayout()
```

```python

        # Title
        self.title = QLineEdit()
        self.title.setPlaceholderText("Note title...")
        self.title.textChanged.connect(self.mark_modified)
        right_layout.addWidget(self.title)

        # Meta info
        self.meta = QLabel("Created: - | Modified: -")
        self.meta.setStyleSheet("color: gray; font-size: 11px;")
        right_layout.addWidget(self.meta)

        # Editor
        self.editor = QTextEdit()
        self.editor.setPlaceholderText("Write your note here...")
        self.editor.textChanged.connect(self.mark_modified)
        right_layout.addWidget(self.editor)

        # Status
        self.status = QLabel("Ready")
        self.status.setStyleSheet("color: #27ae60;")
        right_layout.addWidget(self.status)

        right_panel.setLayout(right_layout)
        splitter.addWidget(right_panel)

        # Set splitter sizes
        splitter.setSizes([200, 600])

        main_layout.addWidget(splitter)

        main_widget.setLayout(main_layout)
        self.setCentralWidget(main_widget)

    def new_note(self):
        # Save current note first
        if self.current_note is not None:
            self.save_current_note()

        # Create new note
        note = Note(title="Untitled", content="")
        self.notes.insert(0, note)
        self.refresh_list()

        # Select new note
        self.notes_list.setCurrentRow(0)
        self.title.setFocus()
        self.title.selectAll()

    def delete_note(self):
        if self.current_note is None:
            return

        reply = QMessageBox.question(
            self, "Delete Note",
            f"Delete '{self.notes[self.current_note].title}'?",
            QMessageBox.Yes | QMessageBox.No
        )

        if reply == QMessageBox.Yes:
            del self.notes[self.current_note]
```

```
            self.current_note = None
            self.refresh_list()
            self.clear_editor()
            self.save_notes_to_file()

    def save_current_note(self):
        if self.current_note is None:
            return

        note = self.notes[self.current_note]
        note.title = self.title.text() or "Untitled"
        note.content = self.editor.toPlainText()
        note.modified = datetime.now().isoformat()

        self.refresh_list()
        self.update_meta()
        self.save_notes_to_file()

        self.status.setText(f"✅ Saved at {datetime.now().strftime('%H:%M:%S')}")

    def save_notes_to_file(self):
        try:
            data = [note.to_dict() for note in self.notes]
            with open(self.file_path, 'w') as f:
                json.dump(data, f, indent=2)
        except Exception as e:
            QMessageBox.warning(self, "Error", f"Could not save: {e}")

    def load_notes(self):
        try:
            if os.path.exists(self.file_path):
                with open(self.file_path, 'r') as f:
                    data = json.load(f)
                self.notes = [Note.from_dict(d) for d in data]
        except:
            self.notes = []

        self.refresh_list()

    def refresh_list(self):
        self.notes_list.clear()

        for note in self.notes:
            item = QListWidgetItem(note.title)
            item.setData(Qt.UserRole, note.created)

            # Preview
            preview = note.content[:50] + "..." if len(note.content) > 50 else note.content
            item.setToolTip(preview)

            self.notes_list.addItem(item)

        self.stats.setText(f"{len(self.notes)} notes")

    def on_note_selected(self, current, previous):
        if current is None:
            self.current_note = None
            self.clear_editor()
            return

        row = self.notes_list.row(current)
```

```
        self.current_note = row

        note = self.notes[row]
        self.title.blockSignals(True)
        self.title.setText(note.title)
        self.title.blockSignals(False)

        self.editor.blockSignals(True)
        self.editor.setPlainText(note.content)
        self.editor.blockSignals(False)

        self.update_meta()

    def clear_editor(self):
        self.title.clear()
        self.editor.clear()
        self.meta.setText("Created: - | Modified: -")

    def update_meta(self):
        if self.current_note is None:
            return

        note = self.notes[self.current_note]

        created = datetime.fromisoformat(note.created).strftime("%d/%m/%Y %H:%M")
        modified = datetime.fromisoformat(note.modified).strftime("%d/%m/%Y %H:%M")

        self.meta.setText(f"Created: {created} | Modified: {modified}")

    def mark_modified(self):
        if self.current_note is not None:
            self.status.setText("⚠️ Unsaved changes")

    def filter_notes(self, text):
        text = text.lower()

        for i in range(self.notes_list.count()):
            item = self.notes_list.item(i)
            note = self.notes[i]

            if text in note.title.lower() or text in note.content.lower():
                item.setHidden(False)
            else:
                item.setHidden(True)

    def export_note(self):
        if self.current_note is None:
            return

        file_path, _ = QFileDialog.getSaveFileName(
            self, "Export Note",
            f"{self.notes[self.current_note].title}.txt",
            "Text Files (*.txt)"
        )

        if file_path:
            note = self.notes[self.current_note]
            with open(file_path, 'w') as f:
                f.write(f"Title: {note.title}\n")
                f.write(f"Created: {note.created}\n")
                f.write(f"Modified: {note.modified}\n")
```

```
            f.write("-" * 40 + "\n\n")
            f.write(note.content)

        self.status.setText(f"✅ Exported to {file_path}")

    def import_notes(self):
        file_path, _ = QFileDialog.getOpenFileName(
            self, "Import Notes",
            "",
            "JSON Files (*.json);;Text Files (*.txt)"
        )

        if file_path:
            if file_path.endswith('.json'):
                with open(file_path, 'r') as f:
                    data = json.load(f)
                imported = [Note.from_dict(d) for d in data]
                self.notes = imported + self.notes
            else:
                with open(file_path, 'r') as f:
                    content = f.read()

                note = Note(
                    title=os.path.basename(file_path),
                    content=content
                )
                self.notes.insert(0, note)

            self.refresh_list()
            self.save_notes_to_file()
            self.status.setText(f"✅ Imported {file_path}")

    def closeEvent(self, event):
        # Auto-save on close
        self.save_current_note()
        event.accept()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = NotesApp()
    window.show()
    sys.exit(app.exec_())
```

41.2 Penjelasan Kode

Proyek ini menggabungkan banyak konsep sebelumnya menjadi aplikasi catatan fungsional—lengkap dengan CRUD, pencarian, dan penyimpanan ke file JSON.

1. Arsitektur Tiga Panel

Layout utama menggunakan `QSplitter` yang membagi jendela menjadi: sidebar kiri (daftar catatan via `QListWidget`), dan area kanan (
texttt`QLineEdit` untuk judul + `QTextEdit` untuk konten). Di bawah list, tombol New, Delete, dan Search tersedia. Pemisahan ini mengikuti pola
emphmaster-detail yang lazim di aplikasi catatan modern.

2. CRUD Catatan

Setiap catatan disimpan sebagai dictionary dengan key `title`, `content`, dan `timestamp`. Metode `new_note()` menambahkan catatan kosong, `save_current_note()` menyimpan perubahan, dan `delete_note()` menghapus dengan konfirmasi. Saat item list diklik, `on_note_selected()` memuat judul dan konten ke area

edit.

3. Pencarian

`search_notes()` memfilter list berdasarkan kata kunci—mencocokkan judul maupun konten. Item yang tidak cocok disembunyikan via `setHidden(True)`, memberikan pengalaman pencarian instan tanpa reload.

4. Persistensi JSON

`save_notes_to_file()` menyimpan seluruh list catatan ke file `notes.json`. `load_notes_from_file()` memuatnya saat startup. Format JSON dipilih karena sederhana dan mudah dibaca manusia.

5. Ekspor dan Impor

Tombol Export menyimpan catatan ke lokasi pilihan pengguna via `QFileDialog.getSaveFileName`. Import membaca file JSON eksternal dan menambahkannya ke koleksi.

Pelajaran Penting

- Pola master-detail (list + editor) adalah fondasi banyak aplikasi produktivitas.
- JSON adalah format penyimpanan ringan yang cocok untuk data non-relasional.
- Pencarian instan meningkatkan UX secara signifikan dibanding pencarian dengan tombol Submit.

41.3 Hasil Tampilan

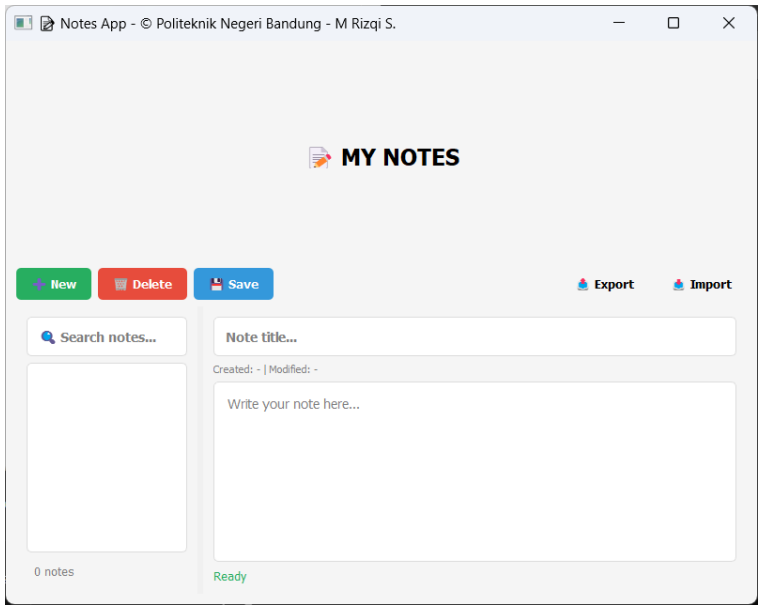

Figure 41.1: Notes App

Chapter 42

Shortcut Keys

42.1 Kode Program

```
# BAB 42: Shortcut Keys
# =====================
# Global dan local keyboard shortcuts

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLabel, QPushButton, QTextEdit,
                            QVBoxLayout, QWidget, QShortcut,
                            QMessageBox)
from PyQt5.QtGui import QKeySequence
from PyQt5.QtCore import Qt

class ShortcutDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Shortcut Keys Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 400)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("⌨ KEYBOARD SHORTCUTS")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        info = QLabel("Try the keyboard shortcuts below:")
        info.setStyleSheet("color: gray;")
        info.setAlignment(Qt.AlignCenter)
        layout.addWidget(info)

        # Shortcuts list
        shortcuts_list = """
Available Shortcuts:

Ctrl+N →  New
Ctrl+S →  Save
Ctrl+O →  Open
Ctrl+Q →  Quit
Ctrl+Z →  Undo
Ctrl+Y →  Redo
```

```
Ctrl+Shift+S →Save As
F1 →       Help
F5 →       Refresh
Esc →      Clear
Space →    Show Message
        """

        shortcuts_label = QLabel(shortcuts_list)
        shortcuts_label.setStyleSheet("font-family: monospace; padding: 20px; background: #f0f0f0;
            border-radius: 10px;")
        layout.addWidget(shortcuts_label)

        # Status
        self.status = QLabel("Press a shortcut key...")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px;")
        self.status.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.status)

        # Text edit for undo/redo demo
        self.text_edit = QTextEdit()
        self.text_edit.setPlaceholderText("Type here and try Ctrl+Z / Ctrl+Y for undo/redo...")
        self.text_edit.setMaximumHeight(100)
        layout.addWidget(self.text_edit)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

        # Setup shortcuts
        self.setup_shortcuts()

    def setup_shortcuts(self):
        # Ctrl+N - New
        shortcut_new = QShortcut(QKeySequence("Ctrl+N"), self)
        shortcut_new.activated.connect(self.action_new)

        # Ctrl+S - Save
        shortcut_save = QShortcut(QKeySequence("Ctrl+S"), self)
        shortcut_save.activated.connect(self.action_save)

        # Ctrl+O - Open
        shortcut_open = QShortcut(QKeySequence("Ctrl+O"), self)
        shortcut_open.activated.connect(self.action_open)

        # Ctrl+Q - Quit
        shortcut_quit = QShortcut(QKeySequence("Ctrl+Q"), self)
        shortcut_quit.activated.connect(self.close)

        # Ctrl+Z - Undo (using QAction would be better for this)
        shortcut_undo = QShortcut(QKeySequence("Ctrl+Z"), self)
        shortcut_undo.activated.connect(self.action_undo)

        # Ctrl+Y - Redo
        shortcut_redo = QShortcut(QKeySequence("Ctrl+Y"), self)
        shortcut_redo.activated.connect(self.action_redo)

        # Ctrl+Shift+S - Save As
        shortcut_saveas = QShortcut(QKeySequence("Ctrl+Shift+S"), self)
        shortcut_saveas.activated.connect(self.action_save_as)

        # F1 - Help
        shortcut_help = QShortcut(QKeySequence("F1"), self)
```

```
        shortcut_help.activated.connect(self.action_help)

        # F5 - Refresh
        shortcut_refresh = QShortcut(QKeySequence("F5"), self)
        shortcut_refresh.activated.connect(self.action_refresh)

        # Esc - Clear
        shortcut_clear = QShortcut(QKeySequence("Esc"), self)
        shortcut_clear.activated.connect(self.action_clear)

        # Space - Show message
        shortcut_space = QShortcut(QKeySequence("Space"), self)
        shortcut_space.activated.connect(self.action_show_message)

    def action_new(self):
        self.status.setText("📄 Action: New File")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px; color:
            #27ae60;")

    def action_save(self):
        self.status.setText("💾 Action: Save")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px; color:
            #3498db;")

    def action_save_as(self):
        self.status.setText("💾 Action: Save As...")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px; color:
            #9b59b6;")

    def action_open(self):
        self.status.setText("📂 Action: Open File")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px; color:
            #e67e22;")

    def action_undo(self):
        self.text_edit.undo()
        self.status.setText("↩️ Action: Undo")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px; color:
            #f39c12;")

    def action_redo(self):
        self.text_edit.redo()
        self.status.setText("↪️ Action: Redo")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px; color:
            #1abc9c;")

    def action_help(self):
        QMessageBox.information(self, "Help", "This is a demo of keyboard shortcuts in PyQt5!")

    def action_refresh(self):
        self.status.setText("🔄 Action: Refresh")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px; color:
            #3498db;")

    def action_clear(self):
        self.text_edit.clear()
        self.status.setText("🗑️ Action: Clear")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px; color:
            #e74c3c;")

    def action_show_message(self):
```

```
        self.status.setText("␣ Space pressed!")
        self.status.setStyleSheet("font-size: 16px; font-weight: bold; padding: 10px; color:
            #2ecc71;")


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = ShortcutDemo()
    window.show()
    sys.exit(app.exec_())
```

42.2 Penjelasan Kode

Pengguna berpengalaman mengharapkan shortcut keyboard: Ctrl+S untuk simpan, Ctrl+N untuk baru, Esc untuk tutup. Di PyQt5, `QShortcut` + `QKeySequence` mewujudkannya tanpa menyentuh event handler.

1. Membuat Shortcut Global

Setiap shortcut dibuat dengan satu baris: `QShortcut(QKeySequence("Ctrl+N"), self).activated.connect(action)` Skrip mendaftarkan berbagai shortcut: Ctrl+N (New), Ctrl+S (Save), Ctrl+O (Open), Ctrl+Q (Quit), Ctrl+Z (Undo), Ctrl+Y (Redo), F1 (Help), F5 (Refresh), dan Esc (Close).

2. Feedback Visual

Setiap shortcut yang dipicu memperbarui label status di bawah jendela—misalnya "Shortcut: Ctrl+S - Save File". Ini membantu pengguna belajar shortcut secara interaktif.

3. Undo/Redo Bawaan QTextEdit

`QTextEdit` sudah mendukung undo/redo secara bawaan. Cukup hubungkan shortcut Ctrl+Z ke `text_edit.undo()` dan Ctrl+Y ke `text_edit.redo()`. Tidak perlu implementasi manual.

4. Daftar Shortcut di UI

Tabel referensi ditampilkan di bawah text editor, menunjukkan semua shortcut yang tersedia. Ini adalah praktik UX yang baik—pengguna tidak perlu menghafal atau membuka dokumentasi terpisah.

Pelajaran Penting

- `QShortcut` lebih bersih daripada menangkap `keyPressEvent` secara manual.
- Gunakan `QKeySequence` standar agar shortcut konsisten lintas platform.
- Selalu sediakan referensi shortcut yang terlihat di UI.

42.3 Hasil Tampilan

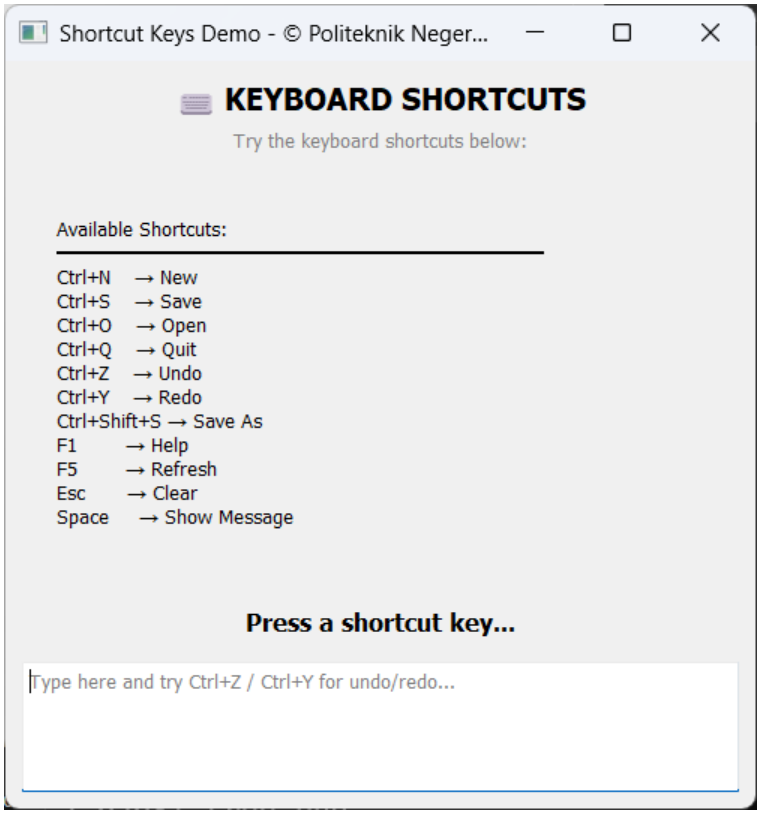

Figure 42.1: Shortcut Keys

Chapter 43

Internationalization (i18n)

43.1 Kode Program

```python
# BAB 43: Internationalization (i18n)
# ====================================
# Multi-language support

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLabel, QPushButton, QLineEdit,
                            QComboBox, QVBoxLayout, QWidget,
                            QFormLayout, QGroupBox)
from PyQt5.QtCore import Qt, QTranslator, QLocale

class Translator:
    """Simple translator class"""

    translations = {
        "en": {
            "title": "Internationalization Demo - Politeknik Negeri Bandung M Rizqi S",
            "language": "Language:",
            "welcome": "Welcome to the App!",
            "name": "Name:",
            "email": "Email:",
            "submit": "Submit",
            "cancel": "Cancel",
            "greeting": "Hello, {name}!",
            "info": "This app supports multiple languages."
        },
        "id": {
            "title": "Demo Internasionalisasi",
            "language": "Bahasa:",
            "welcome": "Selamat datang di Aplikasi!",
            "name": "Nama:",
            "email": "Email:",
            "submit": "Kirim",
            "cancel": "Batal",
            "greeting": "Halo, {name}!",
            "info": "Aplikasi ini mendukung banyak bahasa."
        },
        "ja": {
            "title": "   ",
            "language": "  ",
            "welcome": "     ",
            "name": "  ",
```

```
            "email": "   ",
            "submit": " ",
            "cancel": "   ",
            "greeting": "   {name  }",
            "info": "           "
        },
        "zh": {
            "title": "   ",
            "language": "  ",
            "welcome": "    ",
            "name": "  ",
            "email": "  ",
            "submit": " ",
            "cancel": " ",
            "greeting": "  {name }",
            "info": "     "
        }
    }

    def __init__(self, lang="en"):
        self.lang = lang

    def set_language(self, lang):
        self.lang = lang

    def tr(self, key, **kwargs):
        text = self.translations.get(self.lang, {}).get(key, key)
        if kwargs:
            return text.format(**kwargs)
        return text


class I18nDemo(QMainWindow):
    def __init__(self):
        super().__init__()

        self.translator = Translator("en")

        self.setWindowTitle("Internationalization Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 450, 400)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        self.header = QLabel()
        self.header.setStyleSheet("font-size: 20px; font-weight: bold;")
        self.header.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.header)

        # Language selector
        lang_layout = QVBoxLayout()

        self.lang_label = QLabel()
        lang_layout.addWidget(self.lang_label)

        self.lang_combo = QComboBox()
        self.lang_combo.addItem("🇬🇧 English", "en")
        self.lang_combo.addItem("🇮🇩 Bahasa Indonesia", "id")
        self.lang_combo.addItem("🇯🇵  ", "ja")
        self.lang_combo.addItem("🇨🇳  ", "zh")
```

```
        self.lang_combo.currentIndexChanged.connect(self.change_language)
        lang_layout.addWidget(self.lang_combo)

        layout.addLayout(lang_layout)

        # Welcome message
        self.welcome = QLabel()
        self.welcome.setStyleSheet("font-size: 18px; padding: 10px; background: #f0f0f0;
            border-radius: 5px;")
        self.welcome.setAlignment(Qt.AlignCenter)
        self.welcome.setWordWrap(True)
        layout.addWidget(self.welcome)

        # Form
        form_group = QGroupBox()
        form_layout = QFormLayout()

        self.name_label = QLabel()
        self.name_input = QLineEdit()
        self.name_input.setPlaceholderText("John Doe")
        form_layout.addRow(self.name_label, self.name_input)

        self.email_label = QLabel()
        self.email_input = QLineEdit()
        self.email_input.setPlaceholderText("john@example.com")
        form_layout.addRow(self.email_label, self.email_input)

        form_group.setLayout(form_layout)
        layout.addWidget(form_group)

        # Buttons
        btn_layout = QVBoxLayout()

        self.btn_submit = QPushButton()
        self.btn_submit.setStyleSheet("background-color: #27ae60; color: white; padding: 10px;")
        self.btn_submit.clicked.connect(self.submit)
        btn_layout.addWidget(self.btn_submit)

        self.btn_cancel = QPushButton()
        self.btn_cancel.clicked.connect(self.cancel)
        btn_layout.addWidget(self.btn_cancel)

        layout.addLayout(btn_layout)

        # Result
        self.result = QLabel()
        self.result.setStyleSheet("font-size: 14px; padding: 10px;")
        self.result.setAlignment(Qt.AlignCenter)
        self.result.setWordWrap(True)
        layout.addWidget(self.result)

        # Info
        self.info = QLabel()
        self.info.setStyleSheet("color: gray; font-size: 12px;")
        self.info.setAlignment(Qt.AlignCenter)
        self.info.setWordWrap(True)
        layout.addWidget(self.info)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

```

```
        # Initial translation
        self.update_ui()

    def change_language(self, index):
        lang = self.lang_combo.currentData()
        self.translator.set_language(lang)
        self.update_ui()

    def update_ui(self):
        """Update all UI text based on current language"""
        t = self.translator.tr

        self.setWindowTitle(t("title"))
        self.header.setText("🌍 " + t("title"))
        self.lang_label.setText(t("language"))
        self.welcome.setText(t("welcome"))
        self.name_label.setText(t("name"))
        self.email_label.setText(t("email"))
        self.btn_submit.setText(t("submit"))
        self.btn_cancel.setText(t("cancel"))
        self.info.setText(t("info"))

        # Clear result when changing language
        self.result.clear()

    def submit(self):
        name = self.name_input.text() or "User"
        greeting = self.translator.tr("greeting", name=name)
        self.result.setText(greeting)
        self.result.setStyleSheet("font-size: 14px; padding: 10px; color: #27ae60;")

    def cancel(self):
        self.name_input.clear()
        self.email_input.clear()
        self.result.clear()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = I18nDemo()
    window.show()
    sys.exit(app.exec_())
```

43.2 Penjelasan Kode

Aplikasi global harus bisa berbicara dalam bahasa pengguna. Internationalization (i18n) adalah proses memisahkan teks UI dari kode agar mudah diterjemahkan.

1. Kelas Translator

Kelas `Translator` menyimpan dictionary terjemahan untuk empat bahasa: English, Indonesian, Japanese, dan Chinese. Setiap key (misalnya "greeting", "name_label") dipetakan ke teks dalam bahasa masing-masing. Metode `get(key)` mengembalikan teks sesuai bahasa aktif.

2. Combo Box Pemilih Bahasa

`QComboBox` di bagian atas menampilkan daftar bahasa. Signal `currentTextChanged` dihubungkan ke `change_language()` yang mengubah bahasa aktif di translator, lalu memanggil `update_ui()`.

3. Update UI Dinamis

`update_ui()` mengambil teks terjemahan untuk setiap widget dan memperbarui label, placeholder, tombol, dan judul jendela. Semua terjadi tanpa restart aplikasi—pengguna langsung melihat perubahan.

4. Komponen yang Diterjemahkan

Skrip menerjemahkan: judul jendela, label sambutan, label formulir (nama, email, pesan), placeholder text, tombol (Submit, Clear, About), dialog About, dan bahkan status bar. Cakupan lengkap ini menunjukkan bagaimana i18n seharusnya diterapkan.

Pelajaran Penting

• Pisahkan semua string UI ke sistem terjemahan—jangan hardcode teks di widget.
• Dictionary-based translator sederhana namun efektif untuk aplikasi kecil-menengah.
• Untuk proyek besar, pertimbangkan Qt Linguist (`.ts`/`.qm` files) yang mendukung workflow penerjemah profesional.

43.3 Hasil Tampilan

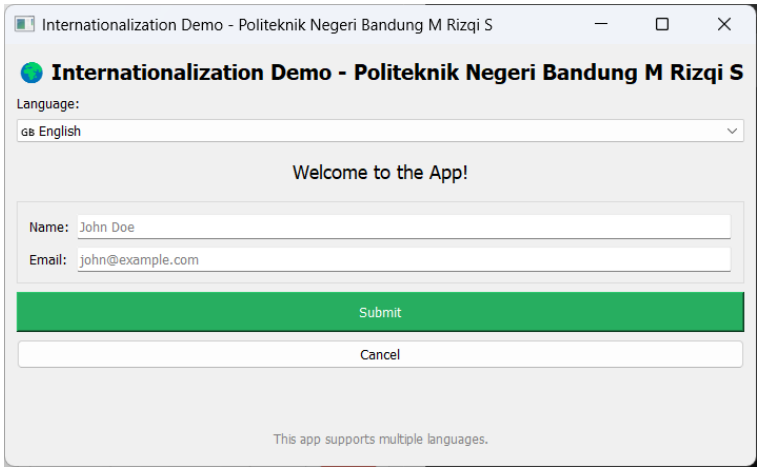

Figure 43.1: i18n

Chapter 44

Database SQLite

44.1 Kode Program

```python
# BAB 44: Database SQLite
# ========================
# SQLite database dengan PyQt5

import sys
import sqlite3
import os
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QTableWidget, QTableWidgetItem,
                            QLineEdit, QPushButton, QLabel,
                            QVBoxLayout, QHBoxLayout, QWidget,
                            QMessageBox, QHeaderView, QComboBox)
from PyQt5.QtCore import Qt

class DatabaseDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Database SQLite Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 700, 500)

        # Database setup
        self.db_path = os.path.join(os.path.dirname(__file__), "demo.db")
        self.setup_database()

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("🗄️ DATABASE SQLITE")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Search & Filter
        search_layout = QHBoxLayout()

        search_layout.addWidget(QLabel("Search:"))
        self.search = QLineEdit()
        self.search.setPlaceholderText("Search by name...")
        self.search.textChanged.connect(self.search_records)
        search_layout.addWidget(self.search)

```

```
        search_layout.addWidget(QLabel("Filter:"))
        self.filter_combo = QComboBox()
        self.filter_combo.addItem("All", "")
        self.filter_combo.addItem("A-E", "ae")
        self.filter_combo.addItem("F-J", "fj")
        self.filter_combo.addItem("K-O", "ko")
        self.filter_combo.addItem("P-T", "pt")
        self.filter_combo.addItem("U-Z", "uz")
        self.filter_combo.currentIndexChanged.connect(self.filter_records)
        search_layout.addWidget(self.filter_combo)

        layout.addLayout(search_layout)

        # Input form
        form_layout = QHBoxLayout()

        form_layout.addWidget(QLabel("Name:"))
        self.name_input = QLineEdit()
        self.name_input.setPlaceholderText("Name")
        form_layout.addWidget(self.name_input)

        form_layout.addWidget(QLabel("Email:"))
        self.email_input = QLineEdit()
        self.email_input.setPlaceholderText("Email")
        form_layout.addWidget(self.email_input)

        form_layout.addWidget(QLabel("Age:"))
        self.age_input = QLineEdit()
        self.age_input.setPlaceholderText("Age")
        self.age_input.setFixedWidth(50)
        form_layout.addWidget(self.age_input)

        btn_add = QPushButton("Add")
        btn_add.setStyleSheet("background-color: #27ae60; color: white;")
        btn_add.clicked.connect(self.add_record)
        form_layout.addWidget(btn_add)

        layout.addLayout(form_layout)

        # Table
        self.table = QTableWidget()
        self.table.setColumnCount(5)
        self.table.setHorizontalHeaderLabels(["ID", "Name", "Email", "Age", "Actions"])
        self.table.horizontalHeader().setSectionResizeMode(0, QHeaderView.ResizeToContents)
        self.table.horizontalHeader().setSectionResizeMode(1, QHeaderView.Stretch)
        self.table.horizontalHeader().setSectionResizeMode(2, QHeaderView.Stretch)
        self.table.horizontalHeader().setSectionResizeMode(3, QHeaderView.ResizeToContents)
        self.table.horizontalHeader().setSectionResizeMode(4, QHeaderView.ResizeToContents)
        layout.addWidget(self.table)

        # Stats
        self.stats = QLabel()
        self.stats.setStyleSheet("padding: 5px;")
        layout.addWidget(self.stats)

        # Buttons
        btn_layout = QHBoxLayout()

        btn_refresh = QPushButton("🔄 Refresh")
        btn_refresh.clicked.connect(self.load_records)
        btn_layout.addWidget(btn_refresh)
```

```
        btn_clear = QPushButton("🗑️ Clear All")
        btn_clear.setStyleSheet("background-color: #e74c3c; color: white;")
        btn_clear.clicked.connect(self.clear_all)
        btn_layout.addWidget(btn_clear)

        btn_backup = QPushButton("💾 Backup to JSON")
        btn_backup.clicked.connect(self.backup_to_json)
        btn_layout.addWidget(btn_backup)

        layout.addLayout(btn_layout)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

        # Load initial data
        self.load_records()

    def setup_database(self):
        """Create database and table if not exists"""
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute('''
            CREATE TABLE IF NOT EXISTS users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                email TEXT,
                age INTEGER
            )
        ''')

        conn.commit()
        conn.close()

    def load_records(self):
        """Load all records from database"""
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute("SELECT * FROM users ORDER BY id DESC")
        records = cursor.fetchall()

        conn.close()

        self.display_records(records)

    def display_records(self, records):
        """Display records in table"""
        self.table.setRowCount(len(records))

        for row, record in enumerate(records):
            id_, name, email, age = record

            self.table.setItem(row, 0, QTableWidgetItem(str(id_)))
            self.table.setItem(row, 1, QTableWidgetItem(name))
            self.table.setItem(row, 2, QTableWidgetItem(email or ""))
            self.table.setItem(row, 3, QTableWidgetItem(str(age) if age else ""))

            # Action buttons
            action_widget = QWidget()
```

```
            action_layout = QHBoxLayout()
            action_layout.setContentsMargins(0, 0, 0, 0)

            btn_edit = QPushButton("Edit")
            btn_edit.setStyleSheet("background-color: #3498db; color: white; padding: 2px 8px;")
            btn_edit.clicked.connect(lambda checked, r=row: self.edit_record(r))
            action_layout.addWidget(btn_edit)

            btn_delete = QPushButton("Del")
            btn_delete.setStyleSheet("background-color: #e74c3c; color: white; padding: 2px 8px;")
            btn_delete.clicked.connect(lambda checked, r=row: self.delete_record(r))
            action_layout.addWidget(btn_delete)

            action_widget.setLayout(action_layout)
            self.table.setCellWidget(row, 4, action_widget)

        self.stats.setText(f"Total records: {len(records)}")

    def add_record(self):
        """Add new record to database"""
        name = self.name_input.text().strip()
        email = self.email_input.text().strip()
        age = self.age_input.text().strip()

        if not name:
            QMessageBox.warning(self, "Error", "Name is required!")
            return

        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute(
            "INSERT INTO users (name, email, age) VALUES (?, ?, ?)",
            (name, email, int(age) if age else None)
        )

        conn.commit()
        conn.close()

        # Clear inputs
        self.name_input.clear()
        self.email_input.clear()
        self.age_input.clear()

        # Refresh
        self.load_records()

    def edit_record(self, row):
        """Edit record in database"""
        id_ = self.table.item(row, 0).text()
        current_name = self.table.item(row, 1).text()
        current_email = self.table.item(row, 2).text()
        current_age = self.table.item(row, 3).text()

        # Populate inputs
        self.name_input.setText(current_name)
        self.email_input.setText(current_email)
        self.age_input.setText(current_age)

        # Delete old record
        conn = sqlite3.connect(self.db_path)
```

```
        cursor = conn.cursor()
        cursor.execute("DELETE FROM users WHERE id = ?", (id_,))
        conn.commit()
        conn.close()

        # Focus on name input
        self.name_input.setFocus()
        self.name_input.selectAll()

        self.load_records()

    def delete_record(self, row):
        """Delete record from database"""
        id_ = self.table.item(row, 0).text()
        name = self.table.item(row, 1).text()

        reply = QMessageBox.question(
            self, "Confirm Delete",
            f"Delete '{name}'?",
            QMessageBox.Yes | QMessageBox.No
        )

        if reply == QMessageBox.Yes:
            conn = sqlite3.connect(self.db_path)
            cursor = conn.cursor()
            cursor.execute("DELETE FROM users WHERE id = ?", (id_,))
            conn.commit()
            conn.close()

            self.load_records()

    def search_records(self, text):
        """Search records by name"""
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute(
            "SELECT * FROM users WHERE name LIKE ? ORDER BY id DESC",
            (f"%{text}%",)
        )
        records = cursor.fetchall()
        conn.close()

        self.display_records(records)

    def filter_records(self):
        """Filter records by first letter"""
        filter_val = self.filter_combo.currentData()

        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        if filter_val == "":
            cursor.execute("SELECT * FROM users ORDER BY id DESC")
        elif filter_val == "ae":
            cursor.execute(
                "SELECT * FROM users WHERE name LIKE '[A-E]%' ORDER BY id DESC"
            )
        elif filter_val == "fj":
            cursor.execute(
                "SELECT * FROM users WHERE name LIKE '[F-J]%' ORDER BY id DESC"
```

```
            )
        elif filter_val == "ko":
            cursor.execute(
                "SELECT * FROM users WHERE name LIKE '[K-O]%' ORDER BY id DESC"
            )
        elif filter_val == "pt":
            cursor.execute(
                "SELECT * FROM users WHERE name LIKE '[P-T]%' ORDER BY id DESC"
            )
        elif filter_val == "uz":
            cursor.execute(
                "SELECT * FROM users WHERE name LIKE '[U-Z]%' ORDER BY id DESC"
            )

        records = cursor.fetchall()
        conn.close()

        self.display_records(records)

    def clear_all(self):
        """Clear all records"""
        reply = QMessageBox.question(
            self, "Confirm Clear",
            "Delete ALL records?",
            QMessageBox.Yes | QMessageBox.No
        )

        if reply == QMessageBox.Yes:
            conn = sqlite3.connect(self.db_path)
            cursor = conn.cursor()
            cursor.execute("DELETE FROM users")
            conn.commit()
            conn.close()

            self.load_records()

    def backup_to_json(self):
        """Backup database to JSON file"""
        import json

        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM users")
        records = cursor.fetchall()
        conn.close()

        data = [
            {"id": r[0], "name": r[1], "email": r[2], "age": r[3]}
            for r in records
        ]

        backup_path = os.path.join(os.path.dirname(__file__), "backup.json")
        with open(backup_path, 'w') as f:
            json.dump(data, f, indent=2)

        QMessageBox.information(self, "Backup", f"Backed up {len(records)} records to backup.json")


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = DatabaseDemo()
```

```
    window.show()
    sys.exit(app.exec_())
```

44.2 Penjelasan Kode

Sampai bab sebelumnya, data disimpan di memori atau file JSON. Untuk data terstruktur yang butuh query kompleks, database adalah jawabannya. SQLite hadir tanpa instalasi—satu file `.db` sudah cukup.

1. Koneksi dan Pembuatan Tabel

`sqlite3.connect("users.db")` membuat (atau membuka) database. `CREATE TABLE IF NOT EXISTS` memastikan tabel `users` tersedia dengan kolom id, name, email, phone, dan category. Koneksi dibuka sekali di konstruktor dan ditutup di `closeEvent`.

2. CRUD Lengkap

- **Create**: `add_record()` mengeksekusi `INSERT INTO` dengan parameter dari form input.
- **Read**: `load_data()` menjalankan `SELECT *` dan mengisi `QTableWidget` baris per baris.
- **Update**: `edit_record()` memuat data ke form, menghapus record lama, dan menunggu pengguna menyimpan versi baru.
- **Delete**: `delete_record()` menghapus record berdasarkan ID setelah konfirmasi.

3. Pencarian dan Filter

Input pencarian memfilter record berdasarkan nama atau email menggunakan `LIKE %keyword%`. Combo box kategori memfilter berdasarkan kategori. Keduanya memanggil `load_data()` ulang dengan parameter query berbeda.

4. Tombol Aksi di Dalam Tabel

Setiap baris memiliki tombol Edit dan Delete yang dibuat secara dinamis saat `load_data()`. Lambda mengirim ID record ke metode yang sesuai. Pendekatan ini lebih intuitif daripada memilih baris lalu menekan tombol di toolbar.

5. Backup ke JSON

Tombol Backup mengekspor seluruh tabel ke file JSON—jaring pengaman jika database corrupt atau perlu migrasi.

Pelajaran Penting

- SQLite ideal untuk aplikasi desktop single-user—tanpa server, tanpa konfigurasi.
- Selalu gunakan parameterized queries (`?`) untuk mencegah SQL injection.
- Tutup koneksi database di `closeEvent` untuk menghindari data corruption.

44.3 Hasil Tampilan

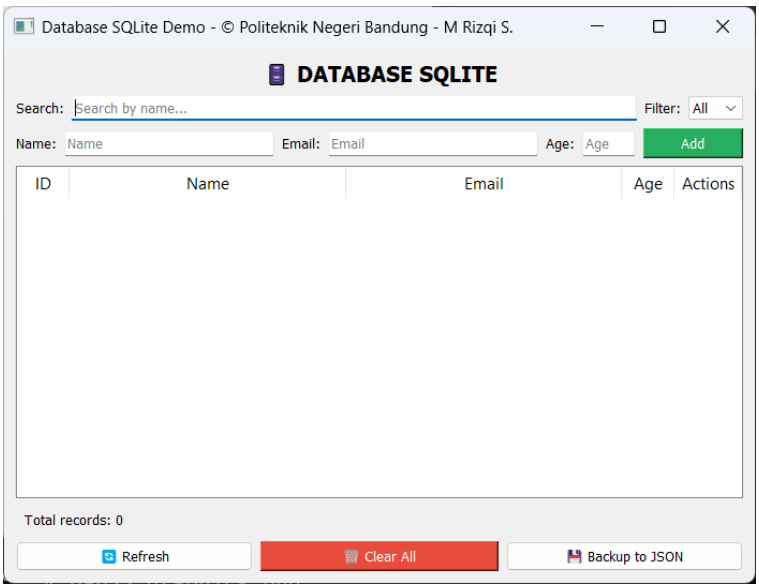

Figure 44.1: Database SQLite

Chapter 45

Network Request

45.1 Kode Program

```python
# BAB 45: Network Request
# ========================
# HTTP requests dengan QNetworkAccessManager

import sys
import json
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QTextEdit, QLineEdit, QPushButton,
                            QLabel, QVBoxLayout, QHBoxLayout,
                            QWidget, QComboBox, QProgressBar)
from PyQt5.QtNetwork import QNetworkAccessManager, QNetworkRequest
from PyQt5.QtCore import QUrl, Qt

class NetworkDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Network Request Demo - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 600, 500)

        # Network manager
        self.network_manager = QNetworkAccessManager()
        self.network_manager.finished.connect(self.on_response)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("🌐 NETWORK REQUEST")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Method selector
        method_layout = QHBoxLayout()

        method_layout.addWidget(QLabel("Method:"))
        self.method = QComboBox()
        self.method.addItems(["GET", "POST", "PUT", "DELETE"])
        method_layout.addWidget(self.method)

        layout.addLayout(method_layout)

```

```
        # URL input
        url_layout = QHBoxLayout()

        url_layout.addWidget(QLabel("URL:"))
        self.url_input = QLineEdit()
        self.url_input.setPlaceholderText("https://jsonplaceholder.typicode.com/posts/1")
        self.url_input.setText("https://jsonplaceholder.typicode.com/posts/1")
        url_layout.addWidget(self.url_input)

        btn_send = QPushButton("Send")
        btn_send.setStyleSheet("background-color: #27ae60; color: white;")
        btn_send.clicked.connect(self.send_request)
        url_layout.addWidget(btn_send)

        layout.addLayout(url_layout)

        # Request body (for POST/PUT)
        layout.addWidget(QLabel("Request Body (JSON):"))
        self.body_input = QTextEdit()
        self.body_input.setPlaceholderText('{"title": "Test", "body": "Content", "userId": 1}')
        self.body_input.setMaximumHeight(100)
        self.body_input.setText('{\n "title": "Test Post",\n "body": "This is a test",\n "userId":
            1\n}')
        layout.addWidget(self.body_input)

        # Progress
        self.progress = QProgressBar()
        self.progress.setVisible(False)
        layout.addWidget(self.progress)

        # Status
        self.status = QLabel("Ready")
        self.status.setStyleSheet("color: gray;")
        layout.addWidget(self.status)

        # Response
        layout.addWidget(QLabel("Response:"))
        self.response = QTextEdit()
        self.response.setReadOnly(True)
        layout.addWidget(self.response)

        # Quick API buttons
        quick_layout = QHBoxLayout()

        btn_posts = QPushButton("Get Posts")
        btn_posts.clicked.connect(lambda:
            self.quick_request("https://jsonplaceholder.typicode.com/posts"))
        quick_layout.addWidget(btn_posts)

        btn_users = QPushButton("Get Users")
        btn_users.clicked.connect(lambda:
            self.quick_request("https://jsonplaceholder.typicode.com/users"))
        quick_layout.addWidget(btn_users)

        btn_comments = QPushButton("Get Comments")
        btn_comments.clicked.connect(lambda:
            self.quick_request("https://jsonplaceholder.typicode.com/comments"))
        quick_layout.addWidget(btn_comments)

        layout.addLayout(quick_layout)

```

```
        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def send_request(self):
        url = self.url_input.text().strip()

        if not url:
            self.status.setText("Error: URL is required")
            self.status.setStyleSheet("color: red;")
            return

        method = self.method.currentText()

        self.status.setText(f"Sending {method} request...")
        self.status.setStyleSheet("color: blue;")
        self.progress.setVisible(True)
        self.progress.setRange(0, 0) # Indeterminate

        request = QNetworkRequest(QUrl(url))
        request.setHeader(QNetworkRequest.ContentTypeHeader, "application/json")

        if method == "GET":
            self.network_manager.get(request)
        elif method == "POST":
            body = self.body_input.toPlainText().encode()
            self.network_manager.post(request, body)
        elif method == "PUT":
            body = self.body_input.toPlainText().encode()
            self.network_manager.put(request, body)
        elif method == "DELETE":
            self.network_manager.deleteResource(request)

    def quick_request(self, url):
        self.url_input.setText(url)
        self.method.setCurrentText("GET")
        self.send_request()

    def on_response(self, reply):
        self.progress.setVisible(False)

        # Get status code
        status_code = reply.attribute(QNetworkRequest.HttpStatusCodeAttribute)

        # Get response body
        data = reply.readAll().data().decode()

        # Try to format JSON
        try:
            parsed = json.loads(data)
            formatted = json.dumps(parsed, indent=2)
        except:
            formatted = data

        # Display
        self.response.setPlainText(formatted)

        if reply.error():
            self.status.setText(f"Error: {reply.errorString()}")
            self.status.setStyleSheet("color: red;")
        else:
            self.status.setText(f"Success! Status: {status_code}")
```

```
            self.status.setStyleSheet("color: green;")

        reply.deleteLater()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = NetworkDemo()
    window.show()
    sys.exit(app.exec_())
```

45.2 Penjelasan Kode

Aplikasi modern sering berkomunikasi dengan server—mengambil data cuaca, mengirim formulir, atau mengakses API. PyQt5 menyediakan `QNetworkAccessManager` yang bekerja secara asinkron sehingga UI tidak membeku saat menunggu respons.

1. QNetworkAccessManager

Objek ini adalah pusat semua request HTTP. Cukup buat satu instance dan gunakan metode `get()`, `post()`, `put()`, atau `deleteResource()` sesuai kebutuhan. Signal `finished` dihubungkan ke `on_response()` yang memproses respons.

2. Form Input

UI menyediakan `QLineEdit` untuk URL, `QComboBox` untuk memilih metode HTTP (GET, POST, PUT, DELETE), dan `QTextEdit` untuk request body (digunakan oleh POST dan PUT). Tombol Send Request memicu `send_request()`.

3. Mengirim Request

`send_request()` membuat `QNetworkRequest` dari URL, menambahkan header `Content-Type: application/json`, lalu memanggil metode yang sesuai. Untuk POST dan PUT, body dikirim sebagai `QByteArray` berisi JSON.

4. Menampilkan Respons

`on_response(reply)` membaca status code dan body. Jika berhasil, JSON di-parse dan ditampilkan dengan indentasi rapi di `QTextEdit` respons. Jika gagal, pesan error ditampilkan. Progress bar dan label status memberikan feedback selama proses.

Pelajaran Penting

- `QNetworkAccessManager` bersifat asinkron—tidak memblokir UI.
- Selalu tangani error jaringan (timeout, server down) agar aplikasi tidak crash.
- Gunakan `json.dumps()` untuk mengirim dan `json.loads()` untuk menerima data JSON.

45.3 Hasil Tampilan

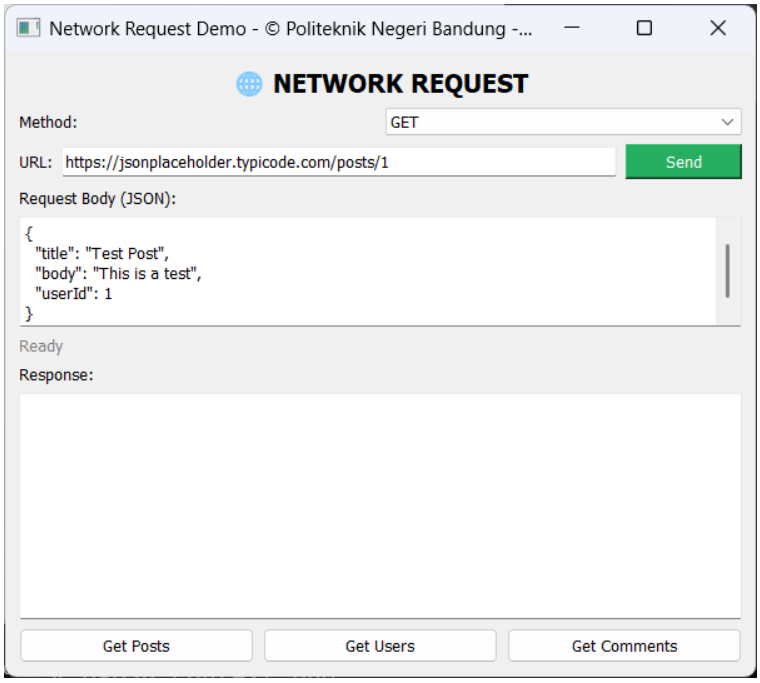

Figure 45.1: Network Request

Chapter 46

Animation

46.1 Kode Program

```python
# BAB 46: Animation
# ==================
# Animasi dengan QPropertyAnimation

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QPushButton, QLabel, QWidget,
                            QVBoxLayout, QHBoxLayout)
from PyQt5.QtCore import (QPropertyAnimation, QPoint, QSize,
                         QRect, QEasingCurve, pyqtProperty, Qt)
from PyQt5.QtGui import QColor


class AnimatedWidget(QWidget):
    """Widget dengan animatable color property"""

    def __init__(self):
        super().__init__()
        self._color = QColor("#3498db")
        self.setFixedSize(100, 100)

    def get_color(self):
        return self._color

    def set_color(self, color):
        self._color = QColor(color)
        self.update()

    color = pyqtProperty(QColor, get_color, set_color)

    def paintEvent(self, event):
        from PyQt5.QtGui import QPainter, QBrush
        painter = QPainter(self)
        painter.setBrush(QBrush(self._color))
        painter.setPen(Qt.NoPen)
        painter.drawEllipse(self.rect())


class AnimationDemo(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Animation Demo - © Politeknik Negeri Bandung - M Rizqi S.")
```

```
        self.setGeometry(100, 100, 600, 500)

        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("🎬 ANIMATION")
        header.setStyleSheet("font-size: 20px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Animated widget
        self.animated = AnimatedWidget()

        # Position animation
        layout.addWidget(QLabel("\n1. Position Animation:"))

        btn_layout1 = QHBoxLayout()

        btn_left = QPushButton("← Move Left")
        btn_left.clicked.connect(self.move_left)
        btn_layout1.addWidget(btn_left)

        btn_right = QPushButton("Move Right →")
        btn_right.clicked.connect(self.move_right)
        btn_layout1.addWidget(btn_right)

        layout.addLayout(btn_layout1)

        # Container for animated widget
        self.container = QWidget()
        self.container.setFixedHeight(150)
        self.container.setStyleSheet("background-color: #f0f0f0; border-radius: 10px;")

        container_layout = QHBoxLayout()
        container_layout.addStretch()
        container_layout.addWidget(self.animated)
        container_layout.addStretch()

        self.container.setLayout(container_layout)
        layout.addWidget(self.container)

        # Size animation
        layout.addWidget(QLabel("\n2. Size Animation:"))

        btn_layout2 = QHBoxLayout()

        btn_grow = QPushButton("Grow  ")
        btn_grow.clicked.connect(self.grow)
        btn_layout2.addWidget(btn_grow)

        btn_shrink = QPushButton("Shrink  ")
        btn_shrink.clicked.connect(self.shrink)
        btn_layout2.addWidget(btn_shrink)

        layout.addLayout(btn_layout2)

        # Color animation
        layout.addWidget(QLabel("\n3. Color Animation:"))

        btn_layout3 = QHBoxLayout()
```

```
        btn_red = QPushButton("Red")
        btn_red.setStyleSheet("background-color: #e74c3c; color: white;")
        btn_red.clicked.connect(lambda: self.change_color("#e74c3c"))
        btn_layout3.addWidget(btn_red)

        btn_green = QPushButton("Green")
        btn_green.setStyleSheet("background-color: #27ae60; color: white;")
        btn_green.clicked.connect(lambda: self.change_color("#27ae60"))
        btn_layout3.addWidget(btn_green)

        btn_blue = QPushButton("Blue")
        btn_blue.setStyleSheet("background-color: #3498db; color: white;")
        btn_blue.clicked.connect(lambda: self.change_color("#3498db"))
        btn_layout3.addWidget(btn_blue)

        btn_purple = QPushButton("Purple")
        btn_purple.setStyleSheet("background-color: #9b59b6; color: white;")
        btn_purple.clicked.connect(lambda: self.change_color("#9b59b6"))
        btn_layout3.addWidget(btn_purple)

        layout.addLayout(btn_layout3)

        # Easing curve demo
        layout.addWidget(QLabel("\n4. Easing Curve Demo:"))

        self.easing_combo = QHBoxLayout()

        btn_linear = QPushButton("Linear")
        btn_linear.clicked.connect(lambda: self.demo_easing(QEasingCurve.Linear))
        self.easing_combo.addWidget(btn_linear)

        btn_inout = QPushButton("InOutQuad")
        btn_inout.clicked.connect(lambda: self.demo_easing(QEasingCurve.InOutQuad))
        self.easing_combo.addWidget(btn_inout)

        btn_bounce = QPushButton("Bounce")
        btn_bounce.clicked.connect(lambda: self.demo_easing(QEasingCurve.OutBounce))
        self.easing_combo.addWidget(btn_bounce)

        btn_elastic = QPushButton("Elastic")
        btn_elastic.clicked.connect(lambda: self.demo_easing(QEasingCurve.OutElastic))
        self.easing_combo.addWidget(btn_elastic)

        layout.addLayout(self.easing_combo)

        # Combo animation
        layout.addWidget(QLabel("\n5. Combo Animation:"))

        btn_combo = QPushButton("🎉 Run Combo Animation!")
        btn_combo.setStyleSheet("padding: 15px; font-size: 14px;")
        btn_combo.clicked.connect(self.combo_animation)
        layout.addWidget(btn_combo)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def move_left(self):
        self.anim = QPropertyAnimation(self.animated, b"geometry")
        self.anim.setDuration(1000)
        self.anim.setStartValue(self.animated.geometry())
```

```
        current = self.animated.geometry()
        self.anim.setEndValue(QRect(10, current.y(), current.width(), current.height()))

        self.anim.setEasingCurve(QEasingCurve.OutCubic)
        self.anim.start()

    def move_right(self):
        self.anim = QPropertyAnimation(self.animated, b"geometry")
        self.anim.setDuration(1000)
        self.anim.setStartValue(self.animated.geometry())

        current = self.animated.geometry()
        self.anim.setEndValue(QRect(400, current.y(), current.width(), current.height()))

        self.anim.setEasingCurve(QEasingCurve.OutCubic)
        self.anim.start()

    def grow(self):
        self.anim = QPropertyAnimation(self.animated, b"geometry")
        self.anim.setDuration(500)
        self.anim.setStartValue(self.animated.geometry())

        current = self.animated.geometry()
        self.anim.setEndValue(QRect(current.x(), current.y(), 150, 150))

        self.anim.setEasingCurve(QEasingCurve.OutElastic)
        self.anim.start()

    def shrink(self):
        self.anim = QPropertyAnimation(self.animated, b"geometry")
        self.anim.setDuration(500)
        self.anim.setStartValue(self.animated.geometry())

        current = self.animated.geometry()
        self.anim.setEndValue(QRect(current.x(), current.y(), 50, 50))

        self.anim.setEasingCurve(QEasingCurve.OutElastic)
        self.anim.start()

    def change_color(self, color):
        self.anim = QPropertyAnimation(self.animated, b"color")
        self.anim.setDuration(1000)
        self.anim.setStartValue(self.animated.get_color())
        self.anim.setEndValue(QColor(color))

        self.anim.setEasingCurve(QEasingCurve.OutCubic)
        self.anim.start()

    def demo_easing(self, curve):
        self.anim = QPropertyAnimation(self.animated, b"geometry")
        self.anim.setDuration(1500)
        self.anim.setStartValue(QRect(10, 25, 100, 100))
        self.anim.setEndValue(QRect(400, 25, 100, 100))

        self.anim.setEasingCurve(curve)

        # Loop back
        self.anim.finished.connect(lambda: self.loop_back(curve))

        self.anim.start()
```

```
    def loop_back(self, curve):
        self.anim2 = QPropertyAnimation(self.animated, b"geometry")
        self.anim2.setDuration(1500)
        self.anim2.setStartValue(QRect(400, 25, 100, 100))
        self.anim2.setEndValue(QRect(10, 25, 100, 100))

        self.anim2.setEasingCurve(curve)
        self.anim2.start()

    def combo_animation(self):
        from PyQt5.QtCore import QSequentialAnimationGroup, QParallelAnimationGroup

        group = QSequentialAnimationGroup()

        # Move left
        anim1 = QPropertyAnimation(self.animated, b"geometry")
        anim1.setDuration(500)
        anim1.setStartValue(self.animated.geometry())
        anim1.setEndValue(QRect(10, 25, 100, 100))
        anim1.setEasingCurve(QEasingCurve.OutCubic)
        group.addAnimation(anim1)

        # Grow
        anim2 = QPropertyAnimation(self.animated, b"geometry")
        anim2.setDuration(300)
        anim2.setStartValue(QRect(10, 25, 100, 100))
        anim2.setEndValue(QRect(10, 0, 150, 150))
        anim2.setEasingCurve(QEasingCurve.OutElastic)
        group.addAnimation(anim2)

        # Change color
        anim3 = QPropertyAnimation(self.animated, b"color")
        anim3.setDuration(300)
        anim3.setStartValue(self.animated.get_color())
        anim3.setEndValue(QColor("#e74c3c"))
        group.addAnimation(anim3)

        # Shrink
        anim4 = QPropertyAnimation(self.animated, b"geometry")
        anim4.setDuration(300)
        anim4.setStartValue(QRect(10, 0, 150, 150))
        anim4.setEndValue(QRect(10, 25, 100, 100))
        group.addAnimation(anim4)

        # Move right with bounce
        anim5 = QPropertyAnimation(self.animated, b"geometry")
        anim5.setDuration(1000)
        anim5.setStartValue(QRect(10, 25, 100, 100))
        anim5.setEndValue(QRect(400, 25, 100, 100))
        anim5.setEasingCurve(QEasingCurve.OutBounce)
        group.addAnimation(anim5)

        # Change color back
        anim6 = QPropertyAnimation(self.animated, b"color")
        anim6.setDuration(500)
        anim6.setStartValue(QColor("#e74c3c"))
        anim6.setEndValue(QColor("#3498db"))
        group.addAnimation(anim6)

        group.start()
```

```python


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = AnimationDemo()
    window.show()
    sys.exit(app.exec_())
```

46.2 Penjelasan Kode

Animasi membuat transisi UI terasa hidup—tombol bergeser, panel membesar, warna berubah gradual. Qt menyediakan `QPropertyAnimation` yang menganimasi properti widget apa pun.

1. Animasi Posisi

`QPropertyAnimation(widget, b"geometry")` menggerakkan widget dari satu posisi ke posisi lain. `setStartValue()` dan `setEndValue()` menerima `QRect` yang mendefinisikan posisi dan ukuran awal/akhir. `setDuration()` menentukan durasi dalam milidetik.

2. Animasi Ukuran

Dengan properti `geometry` yang sama, animasi bisa memperbesar atau memperkecil widget. Skrip mendemonstrasikan tombol yang membesar saat diklik, lalu kembali ke ukuran semula.

3. Animasi Warna Kustom

Warna bukan properti bawaan `QWidget`. Skrip mendefinisikan `pyqtProperty` bernama "color" yang mengubah `background-color` via stylesheet. `QPropertyAnimation` kemudian menginterpolasi warna dari merah ke biru secara mulus.

4. Easing Curves

`QEasingCurve` mengontrol percepatan animasi: `Linear` (konstan), `InOutCubic` (lambat-cepat-lambat), `OutElastic` (memantul elastis), `OutBounce` (memantul seperti bola). Skrip menyediakan combo box untuk memilih kurva secara interaktif.

5. Animasi Gabungan

`QSequentialAnimationGroup` menjalankan animasi berurutan, `QParallelAnimationGroup` menjalankannya bersamaan. Kombinasi keduanya menciptakan koreografi UI yang kompleks.

Pelajaran Penting

- `QPropertyAnimation` bekerja pada properti Qt apa pun—termasuk yang Anda definisikan sendiri.
- Easing curves memberi karakter pada animasi; pilih yang sesuai konteks (bounce untuk fun, cubic untuk profesional).
- Gunakan animasi secukupnya—terlalu banyak animasi justru mengganggu produktivitas.

46.3 Hasil Tampilan

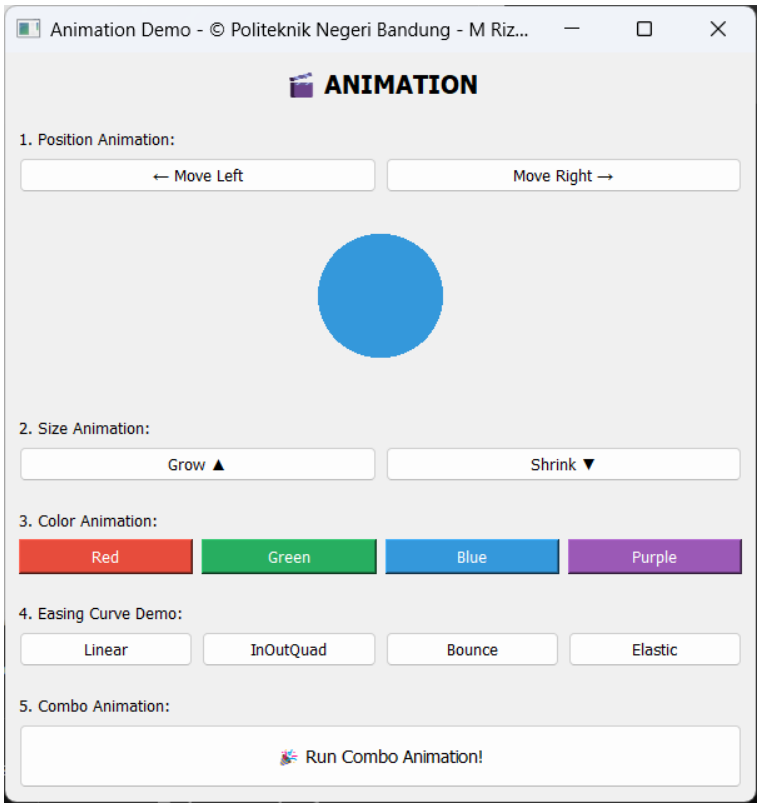

Figure 46.1: Animation

Chapter 47

Proyek Akhir: Address Book

47.1 Kode Program

```
# BAB 47: Proyek Akhir - Address Book
# ====================================
# Aplikasi buku alamat lengkap dengan database

import sys
import sqlite3
import os
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QTableWidget, QTableWidgetItem,
                            QLineEdit, QPushButton, QLabel,
                            QVBoxLayout, QHBoxLayout, QWidget,
                            QMessageBox, QHeaderView, QDialog,
                            QFormLayout, QComboBox, QDialogButtonBox)
from PyQt5.QtCore import Qt

class ContactDialog(QDialog):
    """Dialog untuk tambah/edit kontak"""

    def __init__(self, parent=None, contact=None):
        super().__init__(parent)
        self.setWindowTitle("Add Contact" if not contact else "Edit Contact")
        self.setFixedSize(400, 300)

        self.contact = contact
        self.result_data = None

        layout = QVBoxLayout()

        form = QFormLayout()

        self.name = QLineEdit()
        self.name.setPlaceholderText("Full name")
        form.addRow("Name*:", self.name)

        self.email = QLineEdit()
        self.email.setPlaceholderText("email@example.com")
        form.addRow("Email:", self.email)

        self.phone = QLineEdit()
        self.phone.setPlaceholderText("08xxxxxxxxxx")
        form.addRow("Phone:", self.phone)

```

```
        self.address = QLineEdit()
        self.address.setPlaceholderText("Address")
        form.addRow("Address:", self.address)

        self.city = QLineEdit()
        self.city.setPlaceholderText("City")
        form.addRow("City:", self.city)

        self.category = QComboBox()
        self.category.addItems(["Family", "Friend", "Work", "Business", "Other"])
        form.addRow("Category:", self.category)

        layout.addLayout(form)

        # Buttons
        buttons = QDialogButtonBox(
            QDialogButtonBox.Save | QDialogButtonBox.Cancel
        )
        buttons.accepted.connect(self.save)
        buttons.rejected.connect(self.reject)
        layout.addWidget(buttons)

        # Fill if editing
        if contact:
            self.name.setText(contact[1])
            self.email.setText(contact[2] or "")
            self.phone.setText(contact[3] or "")
            self.address.setText(contact[4] or "")
            self.city.setText(contact[5] or "")
            index = self.category.findText(contact[6] or "Other")
            if index >= 0:
                self.category.setCurrentIndex(index)

        self.setLayout(layout)

    def save(self):
        if not self.name.text().strip():
            QMessageBox.warning(self, "Error", "Name is required!")
            return

        self.result_data = {
            "name": self.name.text().strip(),
            "email": self.email.text().strip(),
            "phone": self.phone.text().strip(),
            "address": self.address.text().strip(),
            "city": self.city.text().strip(),
            "category": self.category.currentText()
        }
        self.accept()


class AddressBook(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("📚 Address Book - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 800, 600)

        # Database
        self.db_path = os.path.join(os.path.dirname(__file__), "contacts.db")
        self.setup_database()

```

```
        self.setStyleSheet("""
            QMainWindow {
                background-color: #f5f5f5;
            }
            QTableWidget {
                background-color: white;
                border: 1px solid #ddd;
                gridline-color: #eee;
            }
            QTableWidget::item {
                padding: 5px;
            }
            QTableWidget::item:selected {
                background-color: #3498db;
                color: white;
            }
            QLineEdit {
                padding: 8px;
                border: 1px solid #ddd;
                border-radius: 5px;
            }
            QPushButton {
                padding: 8px 15px;
                border-radius: 5px;
                font-weight: bold;
            }
        """)

        self.setup_ui()
        self.load_contacts()

    def setup_database(self):
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute('''
            CREATE TABLE IF NOT EXISTS contacts (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                email TEXT,
                phone TEXT,
                address TEXT,
                city TEXT,
                category TEXT,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        ''')

        conn.commit()
        conn.close()

    def setup_ui(self):
        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("📚 ADDRESS BOOK")
        header.setStyleSheet("font-size: 24px; font-weight: bold; padding: 10px;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

```

```
        # Search and filter
        search_layout = QHBoxLayout()

        search_layout.addWidget(QLabel("Search:"))
        self.search = QLineEdit()
        self.search.setPlaceholderText("Search by name, email, phone...")
        self.search.textChanged.connect(self.search_contacts)
        search_layout.addWidget(self.search)

        search_layout.addWidget(QLabel("Category:"))
        self.filter_category = QComboBox()
        self.filter_category.addItem("All", "")
        self.filter_category.addItems(["Family", "Friend", "Work", "Business", "Other"])
        self.filter_category.currentIndexChanged.connect(self.filter_contacts)
        search_layout.addWidget(self.filter_category)

        layout.addLayout(search_layout)

        # Table
        self.table = QTableWidget()
        self.table.setColumnCount(7)
        self.table.setHorizontalHeaderLabels(
            ["ID", "Name", "Email", "Phone", "City", "Category", "Actions"]
        )

        # Set column widths
        header = self.table.horizontalHeader()
        header.setSectionResizeMode(0, QHeaderView.ResizeToContents)
        header.setSectionResizeMode(1, QHeaderView.Stretch)
        header.setSectionResizeMode(2, QHeaderView.Stretch)
        header.setSectionResizeMode(3, QHeaderView.ResizeToContents)
        header.setSectionResizeMode(4, QHeaderView.ResizeToContents)
        header.setSectionResizeMode(5, QHeaderView.ResizeToContents)
        header.setSectionResizeMode(6, QHeaderView.ResizeToContents)

        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        self.table.doubleClicked.connect(self.edit_contact_dialog)

        layout.addWidget(self.table)

        # Stats
        self.stats = QLabel("Total: 0 contacts")
        self.stats.setStyleSheet("padding: 5px;")
        layout.addWidget(self.stats)

        # Buttons
        btn_layout = QHBoxLayout()

        btn_add = QPushButton("➕ Add Contact")
        btn_add.setStyleSheet("background-color: #27ae60; color: white;")
        btn_add.clicked.connect(self.add_contact_dialog)
        btn_layout.addWidget(btn_add)

        btn_edit = QPushButton("✏️ Edit")
        btn_edit.clicked.connect(self.edit_contact_dialog)
        btn_layout.addWidget(btn_edit)

        btn_delete = QPushButton("🗑️ Delete")
        btn_delete.setStyleSheet("background-color: #e74c3c; color: white;")
        btn_delete.clicked.connect(self.delete_contact)
        btn_layout.addWidget(btn_delete)
```

```
        btn_layout.addStretch()

        btn_export = QPushButton("📤 Export CSV")
        btn_export.clicked.connect(self.export_csv)
        btn_layout.addWidget(btn_export)

        btn_import = QPushButton("📥 Import CSV")
        btn_import.clicked.connect(self.import_csv)
        btn_layout.addWidget(btn_import)

        layout.addLayout(btn_layout)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def load_contacts(self):
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute("SELECT * FROM contacts ORDER BY name")
        contacts = cursor.fetchall()
        conn.close()

        self.display_contacts(contacts)

    def display_contacts(self, contacts):
        self.table.setRowCount(len(contacts))

        for row, contact in enumerate(contacts):
            id_, name, email, phone, address, city, category, created = contact

            self.table.setItem(row, 0, QTableWidgetItem(str(id_)))
            self.table.setItem(row, 1, QTableWidgetItem(name))
            self.table.setItem(row, 2, QTableWidgetItem(email or ""))
            self.table.setItem(row, 3, QTableWidgetItem(phone or ""))
            self.table.setItem(row, 4, QTableWidgetItem(city or ""))
            self.table.setItem(row, 5, QTableWidgetItem(category or ""))

            # Action buttons
            action_widget = QWidget()
            action_layout = QHBoxLayout()
            action_layout.setContentsMargins(2, 2, 2, 2)

            btn_view = QPushButton("👁️")
            btn_view.setFixedWidth(30)
            btn_view.clicked.connect(lambda checked, c=contact: self.view_contact(c))
            action_layout.addWidget(btn_view)

            btn_call = QPushButton("📞")
            btn_call.setFixedWidth(30)
            btn_call.clicked.connect(lambda checked, p=phone: self.call_contact(p))
            action_layout.addWidget(btn_call)

            action_widget.setLayout(action_layout)
            self.table.setCellWidget(row, 6, action_widget)

        self.stats.setText(f"Total: {len(contacts)} contacts")

    def add_contact_dialog(self):
        dialog = ContactDialog(self)
```

```
        if dialog.exec_():
            self.add_contact(dialog.result_data)

    def add_contact(self, data):
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute('''
            INSERT INTO contacts (name, email, phone, address, city, category)
            VALUES (?, ?, ?, ?, ?, ?)
        ''', (data["name"], data["email"], data["phone"],
              data["address"], data["city"], data["category"]))

        conn.commit()
        conn.close()

        self.load_contacts()

    def edit_contact_dialog(self):
        row = self.table.currentRow()
        if row < 0:
            QMessageBox.warning(self, "Warning", "Please select a contact!")
            return

        id_ = self.table.item(row, 0).text()

        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM contacts WHERE id = ?", (id_,))
        contact = cursor.fetchone()
        conn.close()

        dialog = ContactDialog(self, contact)
        if dialog.exec_():
            self.update_contact(id_, dialog.result_data)

    def update_contact(self, id_, data):
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute('''
            UPDATE contacts SET name=?, email=?, phone=?, address=?, city=?, category=?
            WHERE id=?
        ''', (data["name"], data["email"], data["phone"],
              data["address"], data["city"], data["category"], id_))

        conn.commit()
        conn.close()

        self.load_contacts()

    def delete_contact(self):
        row = self.table.currentRow()
        if row < 0:
            QMessageBox.warning(self, "Warning", "Please select a contact!")
            return

        id_ = self.table.item(row, 0).text()
        name = self.table.item(row, 1).text()

        reply = QMessageBox.question(
```

```
            self, "Confirm Delete",
            f"Delete contact '{name}'?",
            QMessageBox.Yes | QMessageBox.No
        )

        if reply == QMessageBox.Yes:
            conn = sqlite3.connect(self.db_path)
            cursor = conn.cursor()
            cursor.execute("DELETE FROM contacts WHERE id = ?", (id_,))
            conn.commit()
            conn.close()

            self.load_contacts()

    def view_contact(self, contact):
        id_, name, email, phone, address, city, category, created = contact

        info = f"""
Name: {name}
Email: {email or '-'}
Phone: {phone or '-'}
Address: {address or '-'}
City: {city or '-'}
Category: {category or '-'}
Created: {created}
        """

        QMessageBox.information(self, "Contact Details", info.strip())

    def call_contact(self, phone):
        if phone:
            QMessageBox.information(self, "Call", f"Calling {phone}...")
        else:
            QMessageBox.warning(self, "Error", "No phone number!")

    def search_contacts(self, text):
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        cursor.execute('''
            SELECT * FROM contacts
            WHERE name LIKE ? OR email LIKE ? OR phone LIKE ?
            ORDER BY name
        ''', (f"%{text}%", f"%{text}%", f"%{text}%"))

        contacts = cursor.fetchall()
        conn.close()

        self.display_contacts(contacts)

    def filter_contacts(self):
        category = self.filter_category.currentData()

        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()

        if category:
            cursor.execute(
                "SELECT * FROM contacts WHERE category = ? ORDER BY name",
                (category,)
            )
```

```
        else:
            cursor.execute("SELECT * FROM contacts ORDER BY name")

        contacts = cursor.fetchall()
        conn.close()

        self.display_contacts(contacts)

    def export_csv(self):
        file_path, _ = QFileDialog.getSaveFileName(
            self, "Export CSV", "contacts.csv", "CSV Files (*.csv)"
        )

        if file_path:
            conn = sqlite3.connect(self.db_path)
            cursor = conn.cursor()
            cursor.execute("SELECT * FROM contacts")
            contacts = cursor.fetchall()
            conn.close()

            with open(file_path, 'w') as f:
                f.write("ID,Name,Email,Phone,Address,City,Category,Created\n")
                for c in contacts:
                    f.write(','.join(str(x or '') for x in c) + '\n')

            QMessageBox.information(self, "Export", f"Exported {len(contacts)} contacts!")

    def import_csv(self):
        file_path, _ = QFileDialog.getOpenFileName(
            self, "Import CSV", "", "CSV Files (*.csv)"
        )

        if file_path:
            import csv

            with open(file_path, 'r') as f:
                reader = csv.DictReader(f)

                conn = sqlite3.connect(self.db_path)
                cursor = conn.cursor()

                count = 0
                for row in reader:
                    cursor.execute('''
                        INSERT INTO contacts (name, email, phone, address, city, category)
                        VALUES (?, ?, ?, ?, ?, ?)
                    ''', (
                        row.get('Name', ''),
                        row.get('Email', ''),
                        row.get('Phone', ''),
                        row.get('Address', ''),
                        row.get('City', ''),
                        row.get('Category', '')
                    ))
                    count += 1

                conn.commit()
                conn.close()

            self.load_contacts()
            QMessageBox.information(self, "Import", f"Imported {count} contacts!")
```

```python


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = AddressBook()
    window.show()
    sys.exit(app.exec_())
```

47.2 Penjelasan Kode

Proyek ini mengintegrasikan hampir semua yang telah dipelajari: database SQLite, tabel interaktif, dialog kustom, pencarian, filter, dan ekspor/impor CSV.

1. Database Kontak

Tabel `contacts` menyimpan nama, telepon, email, alamat, dan kategori. Metode `init_database()` membuat tabel jika belum ada. Semua operasi CRUD menggunakan parameterized queries.

2. ContactDialog: Dialog Kustom

Alih-alih mengedit langsung di tabel, proyek ini membuat `QDialog` khusus dengan form lengkap: nama, telepon, email, alamat (`QTextEdit`), dan kategori (`QComboBox`). Dialog ini digunakan baik untuk menambah maupun mengedit kontak—cukup isi field saat mode edit.

3. Tabel dengan Tombol Aksi

`QTableWidget` menampilkan semua kontak. Setiap baris memiliki tombol View, Edit, dan Delete yang dibuat dinamis. View membuka dialog detail (read-only), Edit membuka `ContactDialog` yang terisi data, Delete menghapus setelah konfirmasi.

4. Pencarian dan Filter Kategori

Input pencarian memfilter berdasarkan nama/email/telepon. Combo box kategori memfilter berdasarkan kategori (Teman, Keluarga, Kerja, Lainnya). Keduanya bisa dikombinasikan.

5. Ekspor dan Impor CSV

Export menulis header dan seluruh data kontak ke file CSV. Import membaca CSV, memasukkan setiap baris ke database, dan me-refresh tabel. Format CSV dipilih karena kompatibel dengan Excel dan aplikasi lain.

Pelajaran Penting

- Proyek nyata menggabungkan banyak konsep kecil—database, dialog, tabel, file I/O.
- Dialog kustom (`QDialog`) lebih fleksibel daripada `QInputDialog` untuk form kompleks.
- CSV adalah format pertukaran data paling universal—hampir semua aplikasi bisa membacanya.

47.3 Hasil Tampilan

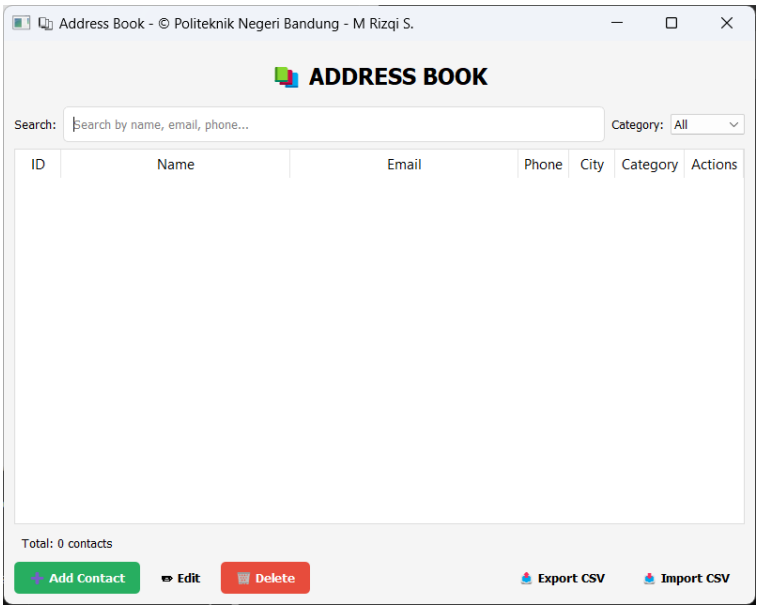

Figure 47.1: Address Book

Chapter 48

Proyek Akhir: File Manager

48.1 Kode Program

```python
# BAB 48: Proyek Akhir - File Manager Mini
# ========================================
# Simple file manager dengan tree dan list view

import sys
import os
import shutil
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QTreeView, QListView, QListWidget,
                            QListWidgetItem, QLabel, QPushButton,
                            QLineEdit, QVBoxLayout, QHBoxLayout,
                            QWidget, QSplitter, QMessageBox,
                            QMenu, QAction, QFileDialog, QFileIconProvider,
                            QInputDialog)
from PyQt5.QtCore import Qt, QDir, QFileInfo, QSize
from PyQt5.QtGui import QIcon


class FileManager(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("📁 File Manager Mini - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 900, 600)

        self.current_path = QDir.homePath()
        self.icon_provider = QFileIconProvider()

        self.setStyleSheet("""
            QMainWindow {
                background-color: #f5f5f5;
            }
            QListWidget {
                background-color: white;
                border: 1px solid #ddd;
            }
            QListWidget::item {
                padding: 5px;
            }
            QListWidget::item:selected {
                background-color: #3498db;
                color: white;
            }
```

```
            QLineEdit {
                padding: 8px;
                border: 1px solid #ddd;
                border-radius: 5px;
            }
            QPushButton {
                padding: 8px 15px;
                border-radius: 5px;
            }
        """)

        self.setup_ui()
        self.load_directory(self.current_path)

    def setup_ui(self):
        widget = QWidget()
        layout = QVBoxLayout()

        # Toolbar
        toolbar = QHBoxLayout()

        btn_back = QPushButton("⬅️ Back")
        btn_back.clicked.connect(self.go_back)
        toolbar.addWidget(btn_back)

        btn_up = QPushButton("⬆️ Up")
        btn_up.clicked.connect(self.go_up)
        toolbar.addWidget(btn_up)

        btn_home = QPushButton("🏠 Home")
        btn_home.clicked.connect(self.go_home)
        toolbar.addWidget(btn_home)

        btn_refresh = QPushButton("🔄 Refresh")
        btn_refresh.clicked.connect(self.refresh)
        toolbar.addWidget(btn_refresh)

        toolbar.addStretch()

        btn_new_folder = QPushButton("📁 New Folder")
        btn_new_folder.clicked.connect(self.new_folder)
        toolbar.addWidget(btn_new_folder)

        btn_new_file = QPushButton("📄 New File")
        btn_new_file.clicked.connect(self.new_file)
        toolbar.addWidget(btn_new_file)

        layout.addLayout(toolbar)

        # Path bar
        path_layout = QHBoxLayout()

        path_layout.addWidget(QLabel("Path:"))
        self.path_input = QLineEdit()
        self.path_input.returnPressed.connect(self.navigate_to_path)
        path_layout.addWidget(self.path_input)

        btn_go = QPushButton("Go")
        btn_go.clicked.connect(self.navigate_to_path)
        path_layout.addWidget(btn_go)

```

```
        layout.addLayout(path_layout)

        # Main content (splitter)
        splitter = QSplitter(Qt.Horizontal)

        # Left: Quick access
        left_panel = QWidget()
        left_layout = QVBoxLayout()

        left_layout.addWidget(QLabel("Quick Access:"))

        self.quick_access = QListWidget()
        self.quick_access.setFixedWidth(150)

        quick_items = [
            ("🏠 Home", QDir.homePath()),
            ("📄 Documents", QDir.homePath() + "/Documents"),
            ("Downloads", QDir.homePath() + "/Downloads"),
            ("Desktop", QDir.homePath() + "/Desktop"),
            ("📁 Root", "/"),
        ]

        for name, path in quick_items:
            item = QListWidgetItem(name)
            item.setData(Qt.UserRole, path)
            self.quick_access.addItem(item)

        self.quick_access.itemClicked.connect(self.quick_access_click)
        left_layout.addWidget(self.quick_access)

        left_panel.setLayout(left_layout)
        splitter.addWidget(left_panel)

        # Right: File list
        right_panel = QWidget()
        right_layout = QVBoxLayout()

        right_layout.addWidget(QLabel("Contents:"))

        self.file_list = QListWidget()
        self.file_list.setViewMode(QListWidget.IconMode)
        self.file_list.setIconSize(QSize(48, 48))
        self.file_list.setResizeMode(QListWidget.Adjust)
        self.file_list.itemDoubleClicked.connect(self.item_double_click)
        self.file_list.setContextMenuPolicy(Qt.CustomContextMenu)
        self.file_list.customContextMenuRequested.connect(self.show_context_menu)
        right_layout.addWidget(self.file_list)

        right_panel.setLayout(right_layout)
        splitter.addWidget(right_panel)

        # Set splitter sizes
        splitter.setSizes([150, 750])

        layout.addWidget(splitter)

        # Status bar
        self.status = QLabel("Ready")
        self.status.setStyleSheet("padding: 5px;")
        layout.addWidget(self.status)

```

```
        widget.setLayout(layout)
        self.setCentralWidget(widget)

        self.path_history = []

    def load_directory(self, path):
        if not os.path.exists(path):
            QMessageBox.warning(self, "Error", f"Directory not found: {path}")
            return

        self.current_path = os.path.abspath(path)
        self.path_input.setText(self.current_path)

        self.file_list.clear()

        try:
            entries = os.listdir(self.current_path)

            # Sort: folders first, then files
            folders = []
            files = []

            for entry in entries:
                full_path = os.path.join(self.current_path, entry)
                if os.path.isdir(full_path):
                    folders.append(entry)
                else:
                    files.append(entry)

            folders.sort(key=str.lower)
            files.sort(key=str.lower)

            # Add folders
            for folder in folders:
                full_path = os.path.join(self.current_path, folder)
                item = QListWidgetItem(f"📁 {folder}")
                item.setData(Qt.UserRole, full_path)
                item.setData(Qt.UserRole + 1, "folder")
                icon = self.icon_provider.icon(QFileInfo(full_path))
                item.setIcon(icon)
                self.file_list.addItem(item)

            # Add files
            for file in files:
                full_path = os.path.join(self.current_path, file)
                item = QListWidgetItem(file)
                item.setData(Qt.UserRole, full_path)
                item.setData(Qt.UserRole + 1, "file")
                icon = self.icon_provider.icon(QFileInfo(full_path))
                item.setIcon(icon)
                self.file_list.addItem(item)

            total = len(folders) + len(files)
            self.status.setText(f"{len(folders)} folders, {len(files)} files | {self.current_path}")

        except PermissionError:
            QMessageBox.warning(self, "Error", "Permission denied!")
            self.status.setText("Permission denied!")

    def navigate_to_path(self):
        path = self.path_input.text()
```

```
        if os.path.exists(path):
            self.path_history.append(self.current_path)
            self.load_directory(path)
        else:
            QMessageBox.warning(self, "Error", "Path not found!")

    def go_back(self):
        if self.path_history:
            path = self.path_history.pop()
            self.load_directory(path)

    def go_up(self):
        parent = os.path.dirname(self.current_path)
        if parent and parent != self.current_path:
            self.path_history.append(self.current_path)
            self.load_directory(parent)

    def go_home(self):
        self.path_history.append(self.current_path)
        self.load_directory(QDir.homePath())

    def refresh(self):
        self.load_directory(self.current_path)

    def quick_access_click(self, item):
        path = item.data(Qt.UserRole)
        if os.path.exists(path):
            self.path_history.append(self.current_path)
            self.load_directory(path)

    def item_double_click(self, item):
        path = item.data(Qt.UserRole)
        item_type = item.data(Qt.UserRole + 1)

        if item_type == "folder":
            self.path_history.append(self.current_path)
            self.load_directory(path)
        else:
            # Open file with system default
            import subprocess
            if sys.platform == 'win32':
                os.startfile(path)
            elif sys.platform == 'darwin':
                subprocess.run(['open', path])
            else:
                subprocess.run(['xdg-open', path])

    def show_context_menu(self, position):
        item = self.file_list.itemAt(position)

        menu = QMenu()

        if item:
            path = item.data(Qt.UserRole)

            action_open = QAction("Open", self)
            action_open.triggered.connect(lambda: self.item_double_click(item))
            menu.addAction(action_open)

            menu.addSeparator()

```

```
            action_copy = QAction("Copy", self)
            action_copy.triggered.connect(lambda: self.copy_item(path))
            menu.addAction(action_copy)

            action_cut = QAction("Cut", self)
            action_cut.triggered.connect(lambda: self.cut_item(path))
            menu.addAction(action_cut)

            action_rename = QAction("Rename", self)
            action_rename.triggered.connect(lambda: self.rename_item(path))
            menu.addAction(action_rename)

            menu.addSeparator()

            action_delete = QAction("Delete", self)
            action_delete.triggered.connect(lambda: self.delete_item(path))
            menu.addAction(action_delete)
        else:
            # Context menu for empty space
            action_new_folder = QAction("New Folder", self)
            action_new_folder.triggered.connect(self.new_folder)
            menu.addAction(action_new_folder)

            action_new_file = QAction("New File", self)
            action_new_file.triggered.connect(self.new_file)
            menu.addAction(action_new_file)

            menu.addSeparator()

            action_refresh = QAction("Refresh", self)
            action_refresh.triggered.connect(self.refresh)
            menu.addAction(action_refresh)

        menu.exec_(self.file_list.mapToGlobal(position))

    def new_folder(self):
        name, ok = QInputDialog.getText(self, "New Folder", "Folder name:")
        if ok and name:
            path = os.path.join(self.current_path, name)
            try:
                os.mkdir(path)
                self.refresh()
            except Exception as e:
                QMessageBox.warning(self, "Error", str(e))

    def new_file(self):
        name, ok = QInputDialog.getText(self, "New File", "File name:")
        if ok and name:
            path = os.path.join(self.current_path, name)
            try:
                open(path, 'w').close()
                self.refresh()
            except Exception as e:
                QMessageBox.warning(self, "Error", str(e))

    def copy_item(self, path):
        self.clipboard = ("copy", path)
        self.status.setText(f"Copied: {os.path.basename(path)}")

    def cut_item(self, path):
        self.clipboard = ("cut", path)
```

```
        self.status.setText(f"Cut: {os.path.basename(path)}")

    def paste(self):
        if hasattr(self, 'clipboard'):
            operation, src = self.clipboard
            name = os.path.basename(src)
            dst = os.path.join(self.current_path, name)

            try:
                if operation == "copy":
                    if os.path.isdir(src):
                        shutil.copytree(src, dst)
                    else:
                        shutil.copy2(src, dst)
                else: # cut
                    shutil.move(src, dst)

                self.refresh()
                self.status.setText(f"Pasted: {name}")
            except Exception as e:
                QMessageBox.warning(self, "Error", str(e))

    def rename_item(self, path):
        old_name = os.path.basename(path)
        new_name, ok = QInputDialog.getText(self, "Rename", "New name:", text=old_name)

        if ok and new_name and new_name != old_name:
            new_path = os.path.join(os.path.dirname(path), new_name)
            try:
                os.rename(path, new_path)
                self.refresh()
            except Exception as e:
                QMessageBox.warning(self, "Error", str(e))

    def delete_item(self, path):
        name = os.path.basename(path)

        reply = QMessageBox.question(
            self, "Confirm Delete",
            f"Delete '{name}'?",
            QMessageBox.Yes | QMessageBox.No
        )

        if reply == QMessageBox.Yes:
            try:
                if os.path.isdir(path):
                    shutil.rmtree(path)
                else:
                    os.remove(path)
                self.refresh()
                self.status.setText(f"Deleted: {name}")
            except Exception as e:
                QMessageBox.warning(self, "Error", str(e))


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = FileManager()
    window.show()
    sys.exit(app.exec_())
```

48.2 Penjelasan Kode

File manager mini ini menunjukkan bagaimana PyQt5 bisa berinteraksi langsung dengan sistem operasi—menavigasi folder, membuat file, menyalin, dan menghapus.

1. Navigasi Direktori

`QLineEdit` di atas menampilkan path saat ini. Pengguna bisa mengetik path langsung atau menggunakan tombol navigasi: Back (riwayat), Up (parent folder), Home (direktori home), dan Refresh. Double-klik pada folder di list membukanya; double-klik pada file membukanya dengan aplikasi default OS.

2. Menampilkan Isi Direktori

`load_directory()` membaca isi folder menggunakan `os.listdir()`, membedakan file dan folder, lalu menampilkannya di `QListWidget` lengkap dengan ikon dari `QFileIconProvider`. Folder ditampilkan lebih dulu, diikuti file—konvensi yang familiar bagi pengguna.

3. Operasi File via Context Menu

Klik kanan memunculkan menu konteks dengan aksi: New Folder, New File, Rename, Copy, Cut, Paste, dan Delete. Copy dan Cut menyimpan path ke variabel `clipboard_path`. Paste menggunakan `shutil.copy2()` atau `shutil.move()` tergantung mode.

4. Keamanan

Setiap aksi destruktif (Delete, overwrite) meminta konfirmasi via `QMessageBox.question`. Operasi file dibungkus `try-except` untuk menangani error izin akses atau file yang sedang digunakan.

Pelajaran Penting

- Modul `os` dan `shutil` Python menyediakan semua operasi file yang dibutuhkan.
- `QFileIconProvider` memberikan ikon native OS untuk setiap tipe file.
- Selalu tangani error I/O dan minta konfirmasi sebelum aksi destruktif.

48.3 Hasil Tampilan

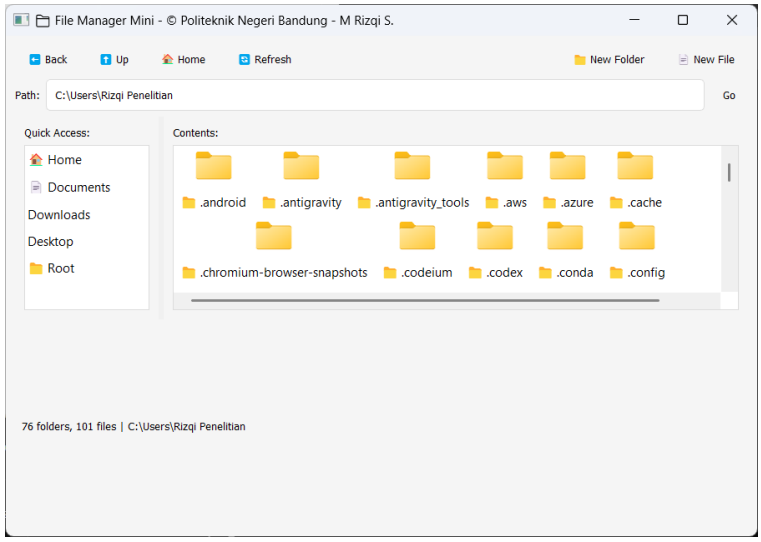

Figure 48.1: File Manager

Chapter 49

Proyek Akhir: Unit Converter

49.1 Kode Program

```python
# BAB 49: Proyek Akhir - Unit Converter
# ======================================
# Aplikasi konversi satuan lengkap

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLineEdit, QLabel, QComboBox,
                            QPushButton, QVBoxLayout, QHBoxLayout,
                            QWidget, QTabWidget, QFormLayout,
                            QGroupBox)
from PyQt5.QtCore import Qt

class UnitConverter(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("🔄 Unit Converter - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 500, 450)

        self.setStyleSheet("""
            QMainWindow {
                background-color: #f5f5f5;
            }
            QLineEdit {
                padding: 10px;
                font-size: 16px;
                border: 2px solid #ddd;
                border-radius: 8px;
            }
            QLineEdit:focus {
                border-color: #3498db;
            }
            QComboBox {
                padding: 10px;
                font-size: 14px;
                border: 2px solid #ddd;
                border-radius: 8px;
            }
            QPushButton {
                padding: 12px;
                font-size: 14px;
                font-weight: bold;
                border-radius: 8px;
```

```
            }
            QLabel#result {
                font-size: 24px;
                font-weight: bold;
                color: #27ae60;
            }
        """)

        self.setup_ui()

    def setup_ui(self):
        widget = QWidget()
        layout = QVBoxLayout()

        # Header
        header = QLabel("🔄 UNIT CONVERTER")
        header.setStyleSheet("font-size: 24px; font-weight: bold;")
        header.setAlignment(Qt.AlignCenter)
        layout.addWidget(header)

        # Tabs for different categories
        tabs = QTabWidget()

        # Length tab
        tabs.addTab(self.create_converter_tab(
            "Length",
            ["m", "km", "cm", "mm", "mile", "yard", "foot", "inch"],
            {
                "m": 1, "km": 1000, "cm": 0.01, "mm": 0.001,
                "mile": 1609.344, "yard": 0.9144, "foot": 0.3048, "inch": 0.0254
            }
        ), "📏 Length")

        # Weight tab
        tabs.addTab(self.create_converter_tab(
            "Weight",
            ["kg", "g", "mg", "lb", "oz", "ton"],
            {
                "kg": 1, "g": 0.001, "mg": 0.000001,
                "lb": 0.453592, "oz": 0.0283495, "ton": 1000
            }
        ), "⚖️ Weight")

        # Temperature tab
        tabs.addTab(self.create_temperature_tab(), "🌡️ Temperature")

        # Volume tab
        tabs.addTab(self.create_converter_tab(
            "Volume",
            ["L", "mL", "gal", "qt", "pt", "cup", "fl oz"],
            {
                "L": 1, "mL": 0.001, "gal": 3.78541,
                "qt": 0.946353, "pt": 0.473176, "cup": 0.236588, "fl oz": 0.0295735
            }
        ), "🧪 Volume")

        # Area tab
        tabs.addTab(self.create_converter_tab(
            "Area",
            ["m²", "km²", "ha", "acre", "ft²", "yd²"],
            {
```

```
            "m²": 1, "km²": 1000000, "ha": 10000,
            "acre": 4046.86, "ft²": 0.092903, "yd²": 0.836127
        }
    ), "📐 Area")

    # Speed tab
    tabs.addTab(self.create_converter_tab(
        "Speed",
        ["m/s", "km/h", "mph", "knot", "ft/s"],
        {
            "m/s": 1, "km/h": 0.277778, "mph": 0.44704,
            "knot": 0.514444, "ft/s": 0.3048
        }
    ), "🚀 Speed")

    # Time tab
    tabs.addTab(self.create_converter_tab(
        "Time",
        ["s", "ms", "min", "h", "day", "week", "year"],
        {
            "s": 1, "ms": 0.001, "min": 60,
            "h": 3600, "day": 86400, "week": 604800, "year": 31536000
        }
    ), "⏱️ Time")

    # Data tab
    tabs.addTab(self.create_converter_tab(
        "Data",
        ["B", "KB", "MB", "GB", "TB", "bit"],
        {
            "B": 1, "KB": 1024, "MB": 1048576,
            "GB": 1073741824, "TB": 1099511627776, "bit": 0.125
        }
    ), "💾 Data")

    layout.addWidget(tabs)

    widget.setLayout(layout)
    self.setCentralWidget(widget)

def create_converter_tab(self, category, units, conversions):
    """Create a standard converter tab"""
    widget = QWidget()
    layout = QVBoxLayout()

    # Input section
    input_group = QGroupBox("Input")
    input_layout = QFormLayout()

    # Value input
    self.value_input = QLineEdit()
    self.value_input.setPlaceholderText("Enter value")
    self.value_input.textChanged.connect(lambda: self.convert(category))
    input_layout.addRow("Value:", self.value_input)

    # From unit
    self.from_unit = QComboBox()
    self.from_unit.addItems(units)
    self.from_unit.currentIndexChanged.connect(lambda: self.convert(category))
    input_layout.addRow("From:", self.from_unit)

```

```
        input_group.setLayout(input_layout)
        layout.addWidget(input_group)

        # Swap button
        btn_swap = QPushButton("🔄 Swap Units")
        btn_swap.setStyleSheet("background-color: #3498db; color: white;")
        btn_swap.clicked.connect(self.swap_units)
        layout.addWidget(btn_swap)

        # Output section
        output_group = QGroupBox("Output")
        output_layout = QFormLayout()

        # To unit
        self.to_unit = QComboBox()
        self.to_unit.addItems(units)
        self.to_unit.setCurrentIndex(1) # Default to second unit
        self.to_unit.currentIndexChanged.connect(lambda: self.convert(category))
        output_layout.addRow("To:", self.to_unit)

        # Result
        self.result = QLabel("0")
        self.result.setObjectName("result")
        output_layout.addRow("Result:", self.result)

        output_group.setLayout(output_layout)
        layout.addWidget(output_group)

        # Formula
        self.formula = QLabel("Formula: -")
        self.formula.setStyleSheet("color: gray; font-size: 12px;")
        layout.addWidget(self.formula)

        layout.addStretch()

        # Store conversions
        self.conversions = conversions

        widget.setLayout(layout)
        return widget

    def create_temperature_tab(self):
        """Special tab for temperature (different formula)"""
        widget = QWidget()
        layout = QVBoxLayout()

        # Input section
        input_group = QGroupBox("Input")
        input_layout = QFormLayout()

        self.temp_value = QLineEdit()
        self.temp_value.setPlaceholderText("Enter temperature")
        self.temp_value.textChanged.connect(self.convert_temperature)
        input_layout.addRow("Value:", self.temp_value)

        self.temp_from = QComboBox()
        self.temp_from.addItems(["Celsius", "Fahrenheit", "Kelvin"])
        self.temp_from.currentIndexChanged.connect(self.convert_temperature)
        input_layout.addRow("From:", self.temp_from)

        input_group.setLayout(input_layout)
```

```
        layout.addWidget(input_group)

        # Swap button
        btn_swap = QPushButton("🔄 Swap Units")
        btn_swap.setStyleSheet("background-color: #3498db; color: white;")
        btn_swap.clicked.connect(self.swap_temperature)
        layout.addWidget(btn_swap)

        # Output section
        output_group = QGroupBox("Output")
        output_layout = QFormLayout()

        self.temp_to = QComboBox()
        self.temp_to.addItems(["Celsius", "Fahrenheit", "Kelvin"])
        self.temp_to.setCurrentIndex(1)
        self.temp_to.currentIndexChanged.connect(self.convert_temperature)
        output_layout.addRow("To:", self.temp_to)

        self.temp_result = QLabel("0")
        self.temp_result.setObjectName("result")
        output_layout.addRow("Result:", self.temp_result)

        output_group.setLayout(output_layout)
        layout.addWidget(output_group)

        # Formula
        self.temp_formula = QLabel("Formula: -")
        self.temp_formula.setStyleSheet("color: gray; font-size: 12px;")
        layout.addWidget(self.temp_formula)

        layout.addStretch()

        widget.setLayout(layout)
        return widget

    def convert(self, category):
        """Convert units"""
        try:
            value = float(self.value_input.text())
            from_unit = self.from_unit.currentText()
            to_unit = self.to_unit.currentText()

            # Convert to base unit, then to target
            base_value = value * self.conversions[from_unit]
            result = base_value / self.conversions[to_unit]

            # Format result
            if abs(result) >= 1000 or (abs(result) < 0.01 and result != 0):
                self.result.setText(f"{result:.6e}")
            else:
                self.result.setText(f"{result:.6f}".rstrip('0').rstrip('.'))

            # Show formula
            if from_unit == to_unit:
                self.formula.setText("Formula: Same unit")
            else:
                ratio = self.conversions[from_unit] / self.conversions[to_unit]
                self.formula.setText(f"Formula: {value} {from_unit} × {ratio:.6f} = {result:.6f}
                    {to_unit}")

        except ValueError:
```

```
            self.result.setText("0")
            self.formula.setText("Formula: -")

    def convert_temperature(self):
        """Convert temperature with special formulas"""
        try:
            value = float(self.temp_value.text())
            from_unit = self.temp_from.currentText()
            to_unit = self.temp_to.currentText()

            # Convert to Celsius first
            if from_unit == "Celsius":
                celsius = value
            elif from_unit == "Fahrenheit":
                celsius = (value - 32) * 5/9
            else: # Kelvin
                celsius = value - 273.15

            # Convert from Celsius to target
            if to_unit == "Celsius":
                result = celsius
            elif to_unit == "Fahrenheit":
                result = celsius * 9/5 + 32
            else: # Kelvin
                result = celsius + 273.15

            # Format result
            self.temp_result.setText(f"{result:.2f}".rstrip('0').rstrip('.'))

            # Formula
            self.temp_formula.setText(f"Formula: {value} {from_unit} →{result:.2f} {to_unit}")

        except ValueError:
            self.temp_result.setText("0")
            self.temp_formula.setText("Formula: -")

    def swap_units(self):
        """Swap from and to units"""
        from_idx = self.from_unit.currentIndex()
        to_idx = self.to_unit.currentIndex()

        self.from_unit.setCurrentIndex(to_idx)
        self.to_unit.setCurrentIndex(from_idx)

    def swap_temperature(self):
        """Swap temperature units"""
        from_idx = self.temp_from.currentIndex()
        to_idx = self.temp_to.currentIndex()

        self.temp_from.setCurrentIndex(to_idx)
        self.temp_to.setCurrentIndex(from_idx)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = UnitConverter()
    window.show()
    sys.exit(app.exec_())
```

49.2 Penjelasan Kode

Konversi satuan adalah kebutuhan sehari-hari—meter ke kaki, kilogram ke pon, Celsius ke Fahrenheit. Proyek ini membangun konverter universal dengan delapan kategori.

1. Delapan Tab Kategori

`QTabWidget` menyediakan tab untuk: Panjang, Berat, Suhu, Volume, Area, Kecepatan, Waktu, dan Data. Setiap tab dibuat oleh `create_converter_tab()` yang menerima dictionary unit dan faktor konversi.

2. Logika Konversi: Faktor Basis

Setiap unit dikonversi ke unit basis terlebih dahulu (misalnya semua panjang ke meter), lalu dari basis ke unit tujuan. Rumusnya: `result = value * from_factor / to_factor`. Pendekatan ini menghindari matriks konversi N×N—cukup N faktor saja.

3. Kasus Khusus: Suhu

Suhu tidak bisa dikonversi dengan faktor sederhana karena ada offset (Celsius ke Fahrenheit: $\times 9/5 + 32$). Tab suhu memiliki logika tersendiri dengan rumus eksplisit untuk setiap kombinasi C, F, K, dan R.

4. Tampilan Formula

Di bawah hasil, label formula menampilkan rumus yang digunakan—misalnya "100 cm × 0.01 = 1.0 m". Ini membantu pengguna memahami proses konversi, bukan sekadar melihat angka akhir.

5. Tombol Swap

Tombol pertukaran menukar unit "dari" dan "ke" tanpa mengubah nilai input—fitur kecil tapi sangat nyaman.

Pelajaran Penting

- Konversi berbasis faktor basis adalah pola desain yang skalabel—mudah menambah unit baru.
- Kasus khusus (suhu) harus ditangani terpisah jika formula tidak linear.
- Menampilkan formula meningkatkan transparansi dan kepercayaan pengguna terhadap hasil.

49.3 Hasil Tampilan

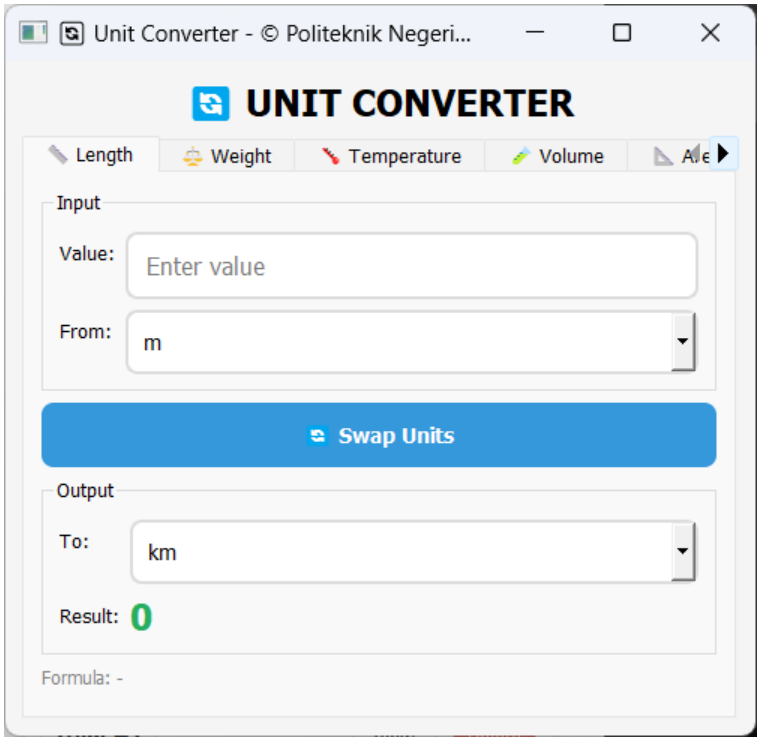

Figure 49.1: Unit Converter

Chapter 50

Proyek Akhir: Pomodoro Timer

50.1 Kode Program

```python
# BAB 50: Proyek Akhir - Pomodoro Timer
# ======================================
# Aplikasi produktivitas dengan teknik Pomodoro

import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow,
                            QLabel, QPushButton, QVBoxLayout,
                            QHBoxLayout, QWidget, QSpinBox,
                            QComboBox, QListWidget, QListWidgetItem,
                            QGroupBox, QProgressBar, QMessageBox)
from PyQt5.QtCore import Qt, QTimer, QDateTime
from PyQt5.QtGui import QFont
import winsound # For sound on Windows (optional)


class PomodoroTimer(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("🍅 Pomodoro Timer - © Politeknik Negeri Bandung - M Rizqi S.")
        self.setGeometry(100, 100, 450, 600)

        # Timer state
        self.timer = QTimer()
        self.timer.timeout.connect(self.update_timer)

        self.time_remaining = 25 * 60 # 25 minutes in seconds
        self.is_running = False
        self.is_break = False
        self.pomodoro_count = 0
        self.current_task = ""

        # Settings
        self.work_duration = 25 # minutes
        self.short_break = 5
        self.long_break = 15
        self.pomodoros_until_long_break = 4

        self.setStyleSheet("""
            QMainWindow {
                background-color: #2c3e50;
            }
            QLabel {
```

```
            color: white;
        }
        QPushButton {
            padding: 15px;
            font-size: 16px;
            font-weight: bold;
            border-radius: 10px;
            border: none;
        }
        QProgressBar {
            border: 2px solid #34495e;
            border-radius: 10px;
            text-align: center;
            font-weight: bold;
            color: white;
        }
        QProgressBar::chunk {
            background-color: #e74c3c;
            border-radius: 8px;
        }
        QSpinBox, QComboBox {
            padding: 10px;
            font-size: 14px;
            border-radius: 8px;
            border: 2px solid #34495e;
            background-color: #34495e;
            color: white;
        }
        QListWidget {
            background-color: #34495e;
            border: 2px solid #34495e;
            border-radius: 10px;
            color: white;
        }
        QListWidget::item {
            padding: 10px;
        }
        QListWidget::item:selected {
            background-color: #e74c3c;
        }
    """)

    self.setup_ui()

def setup_ui(self):
    widget = QWidget()
    layout = QVBoxLayout()

    # Header
    header = QLabel("🍅 POMODORO TIMER")
    header.setStyleSheet("font-size: 28px; font-weight: bold;")
    header.setAlignment(Qt.AlignCenter)
    layout.addWidget(header)

    # Status
    self.status = QLabel("Ready to Focus!")
    self.status.setStyleSheet("font-size: 18px; color: #ecf0f1;")
    self.status.setAlignment(Qt.AlignCenter)
    layout.addWidget(self.status)

    # Timer display
```

```
        self.timer_display = QLabel("25:00")
        self.timer_display.setStyleSheet("""
            font-size: 72px;
            font-weight: bold;
            color: #ecf0f1;
            background-color: #34495e;
            border-radius: 20px;
            padding: 20px;
        """)
        self.timer_display.setAlignment(Qt.AlignCenter)
        layout.addWidget(self.timer_display)

        # Progress bar
        self.progress = QProgressBar()
        self.progress.setMaximum(100)
        self.progress.setValue(0)
        self.progress.setFixedHeight(25)
        layout.addWidget(self.progress)

        # Control buttons
        btn_layout = QHBoxLayout()

        self.btn_start = QPushButton("▶️ Start")
        self.btn_start.setStyleSheet("background-color: #27ae60; color: white;")
        self.btn_start.clicked.connect(self.start_timer)
        btn_layout.addWidget(self.btn_start)

        self.btn_pause = QPushButton("⏸️ Pause")
        self.btn_pause.setStyleSheet("background-color: #f39c12; color: white;")
        self.btn_pause.clicked.connect(self.pause_timer)
        self.btn_pause.setEnabled(False)
        btn_layout.addWidget(self.btn_pause)

        self.btn_reset = QPushButton("⏹️ Reset")
        self.btn_reset.setStyleSheet("background-color: #e74c3c; color: white;")
        self.btn_reset.clicked.connect(self.reset_timer)
        btn_layout.addWidget(self.btn_reset)

        layout.addLayout(btn_layout)

        # Skip button
        btn_skip = QPushButton("⏭️ Skip to Break")
        btn_skip.clicked.connect(self.skip_to_break)
        layout.addWidget(btn_skip)

        # Stats
        stats_group = QGroupBox("📊 Today's Stats")
        stats_group.setStyleSheet("QGroupBox { color: white; font-weight: bold; }")
        stats_layout = QHBoxLayout()

        self.pomodoro_label = QLabel("🍅 0 Pomodoros")
        self.pomodoro_label.setStyleSheet("font-size: 16px;")
        stats_layout.addWidget(self.pomodoro_label)

        self.time_label = QLabel("⏱️ 0 min focused")
        self.time_label.setStyleSheet("font-size: 16px;")
        stats_layout.addWidget(self.time_label)

        stats_group.setLayout(stats_layout)
        layout.addWidget(stats_group)

```

```
        # Tasks
        task_group = QGroupBox("📋 Current Task")
        task_group.setStyleSheet("QGroupBox { color: white; font-weight: bold; }")
        task_layout = QVBoxLayout()

        task_input_layout = QHBoxLayout()

        from PyQt5.QtWidgets import QLineEdit
        self.task_input = QLineEdit()
        self.task_input.setPlaceholderText("What are you working on?")
        self.task_input.setStyleSheet("""
            QLineEdit {
                padding: 10px;
                border-radius: 8px;
                border: 2px solid #34495e;
                background-color: #34495e;
                color: white;
            }
        """)
        self.task_input.returnPressed.connect(self.add_task)
        task_input_layout.addWidget(self.task_input)

        btn_add = QPushButton("+")
        btn_add.setFixedWidth(50)
        btn_add.clicked.connect(self.add_task)
        task_input_layout.addWidget(btn_add)

        task_layout.addLayout(task_input_layout)

        self.task_list = QListWidget()
        self.task_list.setMaximumHeight(100)
        self.task_list.itemClicked.connect(self.select_task)
        task_layout.addWidget(self.task_list)

        task_group.setLayout(task_layout)
        layout.addWidget(task_group)

        # Settings
        settings_group = QGroupBox("⚙️ Settings")
        settings_group.setStyleSheet("QGroupBox { color: white; font-weight: bold; }")
        settings_layout = QVBoxLayout()

        duration_layout = QHBoxLayout()

        duration_layout.addWidget(QLabel("Work:"))
        self.spin_work = QSpinBox()
        self.spin_work.setRange(1, 60)
        self.spin_work.setValue(25)
        self.spin_work.setSuffix(" min")
        self.spin_work.valueChanged.connect(self.update_settings)
        duration_layout.addWidget(self.spin_work)

        duration_layout.addWidget(QLabel("Short Break:"))
        self.spin_short = QSpinBox()
        self.spin_short.setRange(1, 30)
        self.spin_short.setValue(5)
        self.spin_short.setSuffix(" min")
        duration_layout.addWidget(self.spin_short)

        duration_layout.addWidget(QLabel("Long Break:"))
        self.spin_long = QSpinBox()
```

```
        self.spin_long.setRange(1, 60)
        self.spin_long.setValue(15)
        self.spin_long.setSuffix(" min")
        duration_layout.addWidget(self.spin_long)

        settings_layout.addLayout(duration_layout)

        settings_group.setLayout(settings_layout)
        layout.addWidget(settings_group)

        widget.setLayout(layout)
        self.setCentralWidget(widget)

    def format_time(self, seconds):
        """Format seconds to MM:SS"""
        mins = seconds // 60
        secs = seconds % 60
        return f"{mins:02d}:{secs:02d}"

    def update_timer(self):
        """Update timer every second"""
        if self.time_remaining > 0:
            self.time_remaining -= 1
            self.timer_display.setText(self.format_time(self.time_remaining))

            # Update progress
            total = self.work_duration * 60 if not self.is_break else self.get_break_duration() * 60
            progress = int((1 - self.time_remaining / total) * 100)
            self.progress.setValue(progress)

        else:
            # Timer finished
            self.timer.stop()
            self.is_running = False

            if self.is_break:
                # Break finished, start work
                self.is_break = False
                self.time_remaining = self.work_duration * 60
                self.status.setText("⏰ Break over! Back to work!")
                self.timer_display.setStyleSheet("""
                    font-size: 72px;
                    font-weight: bold;
                    color: #ecf0f1;
                    background-color: #e74c3c;
                    border-radius: 20px;
                    padding: 20px;
                """)
            else:
                # Work finished, start break
                self.pomodoro_count += 1
                self.update_stats()

                self.is_break = True
                self.time_remaining = self.get_break_duration() * 60

                if self.pomodoro_count % self.pomodoros_until_long_break == 0:
                    self.status.setText("🎉 Long break time! You've earned it!")
                else:
                    self.status.setText(f"✅ Pomodoro #{self.pomodoro_count} complete!")

```

```
            self.timer_display.setStyleSheet("""
                font-size: 72px;
                font-weight: bold;
                color: #ecf0f1;
                background-color: #27ae60;
                border-radius: 20px;
                padding: 20px;
            """)

        self.timer_display.setText(self.format_time(self.time_remaining))
        self.progress.setValue(0)

        # Play sound (Windows only)
        try:
            winsound.Beep(1000, 500)
        except:
            pass

        # Ask to continue
        reply = QMessageBox.question(
            self, "Timer Complete",
            "Start " + ("break" if not self.is_break else "next pomodoro") + "?",
            QMessageBox.Yes | QMessageBox.No
        )

        if reply == QMessageBox.Yes:
            self.start_timer()
        else:
            self.btn_start.setEnabled(True)
            self.btn_pause.setEnabled(False)

    def get_break_duration(self):
        """Get break duration based on pomodoro count"""
        if self.pomodoro_count % self.pomodoros_until_long_break == 0:
            return self.long_break
        return self.short_break

    def start_timer(self):
        """Start or resume the timer"""
        if not self.is_running:
            self.is_running = True
            self.timer.start(1000) # Update every second

            self.btn_start.setEnabled(False)
            self.btn_pause.setEnabled(True)

            if not self.is_break:
                self.status.setText(f"🔥 Focusing on: {self.current_task or 'Task'}")
                self.timer_display.setStyleSheet("""
                    font-size: 72px;
                    font-weight: bold;
                    color: #ecf0f1;
                    background-color: #e74c3c;
                    border-radius: 20px;
                    padding: 20px;
                """)
            else:
                self.status.setText("☕ Taking a break...")
                self.timer_display.setStyleSheet("""
                    font-size: 72px;
                    font-weight: bold;
```

```
                color: #ecf0f1;
                background-color: #27ae60;
                border-radius: 20px;
                padding: 20px;
            """)

    def pause_timer(self):
        """Pause the timer"""
        if self.is_running:
            self.is_running = False
            self.timer.stop()

            self.btn_start.setEnabled(True)
            self.btn_pause.setEnabled(False)
            self.status.setText("⏸️ Paused")

    def reset_timer(self):
        """Reset the timer"""
        self.timer.stop()
        self.is_running = False
        self.is_break = False
        self.time_remaining = self.work_duration * 60

        self.timer_display.setText(self.format_time(self.time_remaining))
        self.progress.setValue(0)
        self.status.setText("Ready to Focus!")

        self.btn_start.setEnabled(True)
        self.btn_pause.setEnabled(False)

        self.timer_display.setStyleSheet("""
            font-size: 72px;
            font-weight: bold;
            color: #ecf0f1;
            background-color: #34495e;
            border-radius: 20px;
            padding: 20px;
        """)

    def skip_to_break(self):
        """Skip to break"""
        if not self.is_break:
            self.timer.stop()
            self.is_break = True
            self.time_remaining = self.get_break_duration() * 60
            self.timer_display.setText(self.format_time(self.time_remaining))
            self.status.setText("☕ Taking a break...")
            self.progress.setValue(0)

            self.timer_display.setStyleSheet("""
                font-size: 72px;
                font-weight: bold;
                color: #ecf0f1;
                background-color: #27ae60;
                border-radius: 20px;
                padding: 20px;
            """)

    def update_settings(self):
        """Update settings from spinboxes"""
        self.work_duration = self.spin_work.value()
```

```
        self.short_break = self.spin_short.value()
        self.long_break = self.spin_long.value()

        if not self.is_running and not self.is_break:
            self.time_remaining = self.work_duration * 60
            self.timer_display.setText(self.format_time(self.time_remaining))

    def update_stats(self):
        """Update statistics display"""
        self.pomodoro_label.setText(f"🍅 {self.pomodoro_count} Pomodoros")

        total_minutes = self.pomodoro_count * self.work_duration
        if total_minutes >= 60:
            hours = total_minutes // 60
            mins = total_minutes % 60
            self.time_label.setText(f"⏱️ {hours}h {mins}m focused")
        else:
            self.time_label.setText(f"⏱️ {total_minutes} min focused")

    def add_task(self):
        """Add a task to the list"""
        task = self.task_input.text().strip()
        if task:
            self.task_list.addItem(task)
            self.task_input.clear()

            # Select first task if none selected
            if self.task_list.count() == 1:
                self.task_list.setCurrentRow(0)
                self.current_task = task

    def select_task(self, item):
        """Select a task to focus on"""
        self.current_task = item.text()
        if self.is_running and not self.is_break:
            self.status.setText(f"🔥 Focusing on: {self.current_task}")

    def closeEvent(self, event):
        """Handle window close"""
        if self.is_running:
            reply = QMessageBox.question(
                self, "Timer Running",
                "Timer is still running. Exit anyway?",
                QMessageBox.Yes | QMessageBox.No
            )

            if reply == QMessageBox.No:
                event.ignore()
                return

        event.accept()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = PomodoroTimer()
    window.show()
    sys.exit(app.exec_())
```

50.2 Penjelasan Kode

Pomodoro Technique membagi kerja menjadi sesi 25 menit diselingi istirahat pendek. Proyek penutup ini membangun timer Pomodoro lengkap—menggabungkan `QTimer`, progress bar, manajemen tugas, dan notifikasi.

1. State Machine Sederhana

Aplikasi memiliki tiga state: Work (kerja), Short Break (istirahat pendek), dan Long Break (istirahat panjang setelah 4 sesi). Variabel `current_state` menentukan durasi dan warna tampilan. Setelah satu sesi selesai, state otomatis berpindah.

2. Countdown dengan QTimer

`QTimer` berjalan setiap 1000 ms. Setiap tick mengurangi `remaining_seconds` dan memperbarui label waktu (format MM:SS) serta progress bar. Saat mencapai nol, notifikasi berbunyi dan state berpindah.

3. Manajemen Tugas

Di bagian bawah, `QListWidget` menyimpan daftar tugas. Pengguna bisa menambah tugas baru, menandai selesai, dan menghapus. Tugas yang sedang dikerjakan di-highlight—memberikan konteks pada sesi Pomodoro.

4. Statistik

Label menampilkan jumlah Pomodoro yang telah diselesaikan dan total waktu kerja. Data ini memotivasi pengguna melihat produktivitas mereka secara kuantitatif.

5. Pengaturan Durasi

`QSpinBox` memungkinkan pengguna mengatur durasi kerja, istirahat pendek, dan istirahat panjang. Perubahan langsung diterapkan pada sesi berikutnya.

6. Notifikasi

Saat sesi berakhir, `QApplication.beep()` berbunyi dan `QMessageBox` muncul memberitahu pengguna. Jika system tray tersedia, notifikasi balloon juga bisa dikirim.

Pelajaran Penting

- State machine sederhana (variabel + if/elif) cukup untuk mengelola alur aplikasi kecil.
- Progress bar visual membuat pengguna merasakan waktu berjalan—lebih efektif dari angka saja.
- Proyek ini menggabungkan timer, list, settings, dan notifikasi—ringkasan sempurna dari seluruh modul.

50.3 Hasil Tampilan

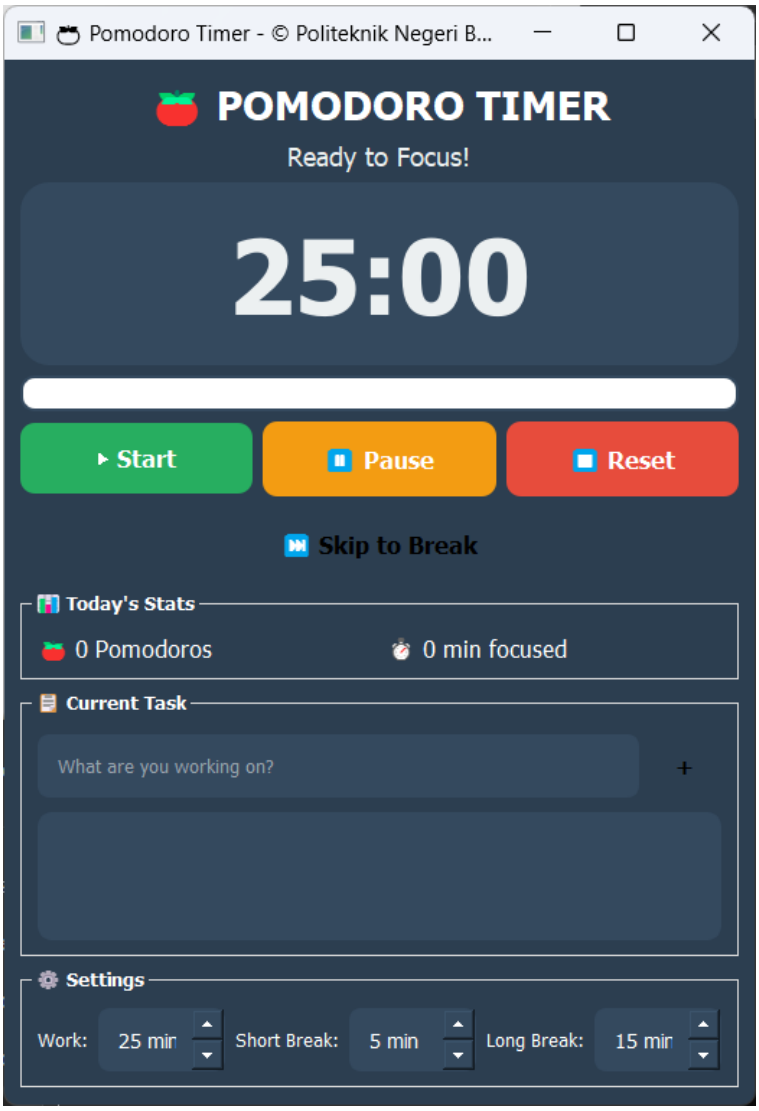

Figure 50.1: Pomodoro Timer